



**Creo® Parametric
VB API User's Guide**
12.4.2.0

Copyright © 2025 PTC Inc. and/or Its Subsidiary Companies. All Rights Reserved.

Copyright for PTC software products is with PTC Inc. and its subsidiary companies (collectively “PTC”), and their respective licensors. This software is provided under written license or other agreement, contains valuable trade secrets and proprietary information, and is protected by the copyright laws of the United States and other countries. It may not be copied or distributed in any form or medium, disclosed to third parties, or used in any manner not provided for in the applicable agreement except with written prior approval from PTC. More information regarding third party copyrights and trademarks and a list of PTC’s registered copyrights, trademarks, and patents can be viewed here: <https://www.ptc.com/support/go/copyright-and-trademarks>

User and training guides and related documentation from PTC are also subject to the copyright laws of the United States and other countries and are provided under a license agreement that restricts copying, disclosure, and use of such documentation. PTC hereby grants to the licensed software user the right to make copies of product documentation and guides in printed form, but only for internal/personal use and in accordance with the license agreement under which the applicable software is licensed. Any copy made shall include the PTC copyright notice and any other proprietary notice provided by PTC. Note that training materials may not be copied without the express written consent of PTC. This documentation may not be disclosed, transferred, modified, or reduced to any form, including electronic media, or transmitted or made publicly available by any means without the prior written consent of PTC and no authorization is granted to make copies for such purposes.

UNITED STATES GOVERNMENT RIGHTS

PTC software products and software documentation are “commercial items” as that term is defined at 48 C.F.R. 2.101. Pursuant to Federal Acquisition Regulation (FAR) 12.212 (a)-(b) (Computer Software) (MAY 2014) for civilian agencies or the Defense Federal Acquisition Regulation Supplement (DFARS) at 227.7202-1(a) (Policy) and 227.7202-3 (a) (Rights in commercial computer software or commercial computer software documentation) (FEB 2014) for the Department of Defense, PTC software products and software documentation are provided to the U.S. Government under the PTC commercial license agreement. Use, duplication or disclosure by the U.S. Government is subject solely to the terms and conditions set forth in the applicable PTC software license agreement.

PTC Inc., 121 Seaport Blvd, Boston, MA 02210 USA

Contents

Release Notes	4
Technical Summary of Changes for Creo 12.4.0.0	5
Get Started	6
Overview of the VB API	7
User's Guide	26
VB API Fundamentals:Controlling Creo Parametric	27
Session Objects	34
Selection	49
Ribbon Tabs, Groups, and Menu Items	55
Menus, Commands, and Pop-up Menus	63
Models	72
Drawings	88
Solid	138
Solid Bodies	155
Windows and Views	156
ModelItem	166
Features	170
Datum Features	186
Geometry Evaluation	194
Dimensions and Parameters	206
Relations	215
Assemblies and Components	220
Family Tables	231
Action Listeners	235
Interface	243
Simplified Representations	279
Task Based Application Libraries	286
Graphics	291
External Data	296
Windchill Connectivity APIs	301
Sample Applications	319
Geometry Traversal	324
Geometry Representations	327
Index	340

1

Release Notes

Technical Summary of Changes for Creo 12.4.0.0.....	5
---	---

Technical Summary of Changes for Creo 12.4.0.0

This section describes the critical and miscellaneous technical changes in Creo 12.4.0.0 and VB API. It also lists the new and superseded functions for this release.

New Functions

This section describes new functions for VB API for Creo 12.4.0.0.

Features

New Function	Description
IpfcFeature.GetPatternByType()	Returns the pattern of the feature.
IpfcFeature.PatternStatus	Returns the pattern status of the feature.
IpfcFeature.GroupStatus	Returns the group status of the feature.
IpfcFeature.GroupPatternStatus	Returns the group pattern status of the feature.

Evaluation of Surfaces

New Function	Description
IpfcSurfXYZData.FirstDerivative	Returns the first partial derivatives of X, Y, and Z, with respect to the u and v parameters.
IpfcSurfXYZData.SecondDerivative	Returns the second partial derivatives of X, Y, and Z, with respect to the u and v parameters.

2

Get Started

Overview of the VB API.....	7
-----------------------------	---

Overview of the VB API

Get Started

The VB API for Creo Parametric is an asynchronous application that can be used from any COM-enabled application including Visual Basic.NET (VB.NET), Visual Basic for Applications (VBA), and external Internet Explorer instances using scripting.

Setting Up a VB Application

For your application to communicate with Creo Parametric, you must set the `PRO_COMM_MSG_EXE` environment variable to the full path of the executable, `pro_comm_msg.exe`. Typically, the path to the executable is `<creo_loadpoint>\<datecode>\Common Files\<machine type>\obj\pro_comm_msg.exe`, where machine type is `i486_nt` for 32-bit Windows and `x86e_win64` for 64-bit Windows installations.

Set `PRO_COMM_MSG_EXE` as:

1. Click **Start > Settings > Control Panel**.
2. Click **System**. The **System Properties** windows opens.
3. In the **Advanced** tab, click the **Environment Variables** button.
4. Add `PRO_COMM_MSG_EXE` to **System variables**.

Registering the COM Server

To register the COM server, run the `vb_api_register.bat` file located at `<creo_loadpoint>/Parametric/bin`.

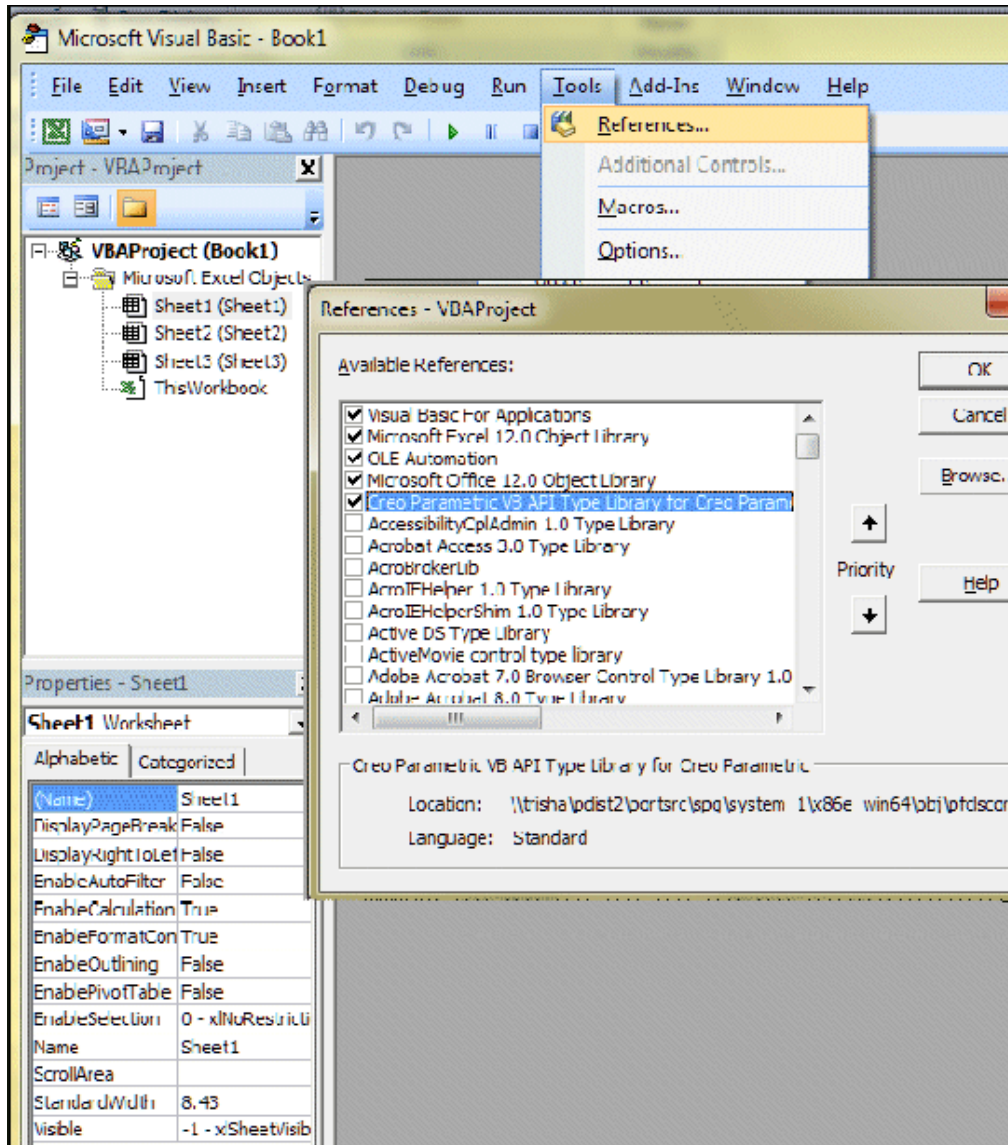
To unregister the COM server, run the `vb_api_unregister.bat` file located at `<creo_loadpoint>/Parametric/bin`.

After the COM server is registered with the system, whenever an application tries to access the types contained in this server the server starts automatically. By default, Windows starts services such as `pfclscom.exe` in the Windows system directory (`c:\winnt\system_32`). Because the server will also start new sessions of Creo Parametric from the process working directory, you may want to control the server run directory. You can configure the server to start in a specific directory by setting the system environment variable `PFCLS_START_DIR` to any existing directory on your computer.

Setting Project References for the VB API

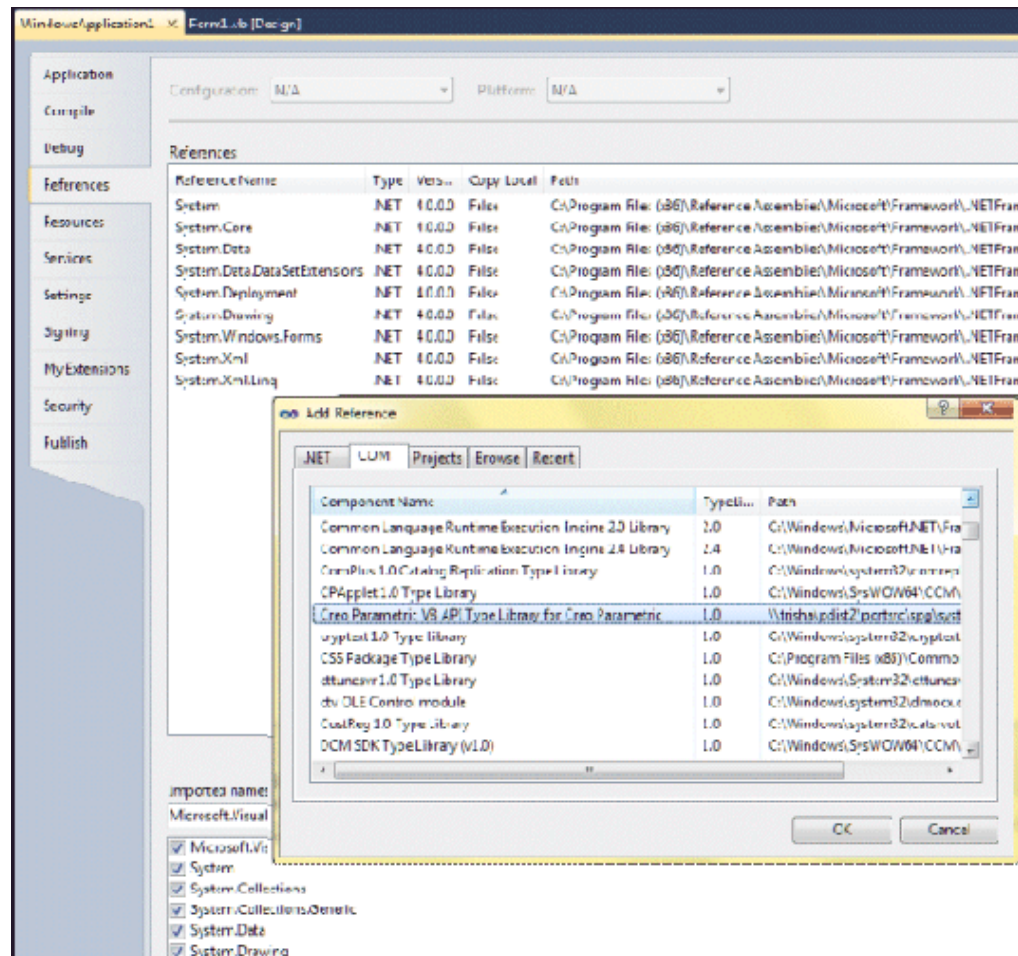
Set the reference to Creo Parametric VB API Type Library for Creo Parametric through your project. In the VBA environment set this reference as follows:

1. Click **Tools ► References**.
2. Check the box for **Creo Parametric VB API Type Library for Creo Parametric** as shown in the following figure.



In the VB.NET environment, set this reference as follows:

1. Click **Project ► Properties ► Add Reference ► COM**.
2. Select **Creo Parametric VB API Type Library for Creo Parametric** as shown in the following figure.



Visual Basic.NET Applications

You can use the VB API for Creo Parametric to:

- Create a VB.NET form capable of starting or connecting to Creo Parametric non-graphically, accepting user inputs and driving model modifications or deliverables.
- Create a VB.NET application that may or may not have its own User Interface (UI). The application should be able to establish one or more Creo Parametric UI or event listeners in session, and process those events using VB.NET code.

Visual Basic for Applications

The VB APIs provide support for accessing Creo Parametric from Visual Basic-enabled products such as Microsoft Excel, Microsoft Word, or Microsoft Access. The COM interface is provided to control Creo Parametric asynchronously and use the PFC API's to access its properties.

You can also access data from OLE objects embedded in Creo Parametric. The OLE objects can include VB code that can be used to drive the model from which the object is contained.

Limitations of the VB API

The asynchronous COM server has the following limitations:

- API calls to Creo Parametric should be made only from a single thread. Other threads can process non Creo Parametric data and set data to be seen by the Creo Parametric thread, but only one thread can communicate with Creo Parametric.
- Only one active connection can be made to a single Creo Parametric session at one time.

Object Types

The VB API is made up of a number of classes in many modules. The following are the main class types:

- **Creo Parametric-Related Classes**—Contain unique methods and properties that are directly related to the functions in Creo Parametric. See the section [Creo Parametric-Related Classes on page 11](#) for more information.
- **Compact Data Classes**—Classes containing data needed as arguments to some VB methods. See the section, [Compact Data Classes on page 12](#), for additional information.
- **Union Classes**—Classes with a potential to contain multiple types of values. See the section [Unions on page 13](#) for additional information.
- **Sequence Classes**—Expandable arrays of objects or primitive data types. See the section [Sequences on page 14](#) for more information.
- **Array Classes**—Arrays that are limited to a certain size. See the section [Arrays on page 15](#) for more information.
- **Enumeration Classes**—Enumerated types, which list a restricted and valid set of options for the property. See the section [Enumeration Classes on page 15](#) for more information.
- **Module-Level Classes**—Contain static methods used to initialize certain VB objects. See the section [Module-Level Classes on page 16](#) for more information.
- **ActionListener Classes**—Enable you to specify code that will run only if certain events in Creo Parametric take place. See the section [Action Listeners on page 235](#) for more information.

Each class shares specific rules regarding initialization, attributes, methods, inheritance, or exceptions. The following sections describe these classes in detail.

Creo Parametric - Related Classes

The `Creo Parametric-Related Classes` contain methods that directly manipulate objects in Creo Parametric. Examples of these objects include models, features, and parameters.

Initialization

You cannot construct one of these objects using the keyword `New`. Instead, you should obtain the handle to a Creo Parametric-related object by creating or listing that object with a method on the parent object in the hierarchy.

For example, `IpfcBaseSession.CurrentModel` returns a `IpfcModel` object set to the current model and `IpfcParameterOwner.CreateParam()` returns a newly created parameter object for manipulation.

Properties

Properties within Creo Parametric-related objects are directly accessible. Some attributes that have been designated as read-only can be accessed but not modified by the VB API.

Methods

You must invoke methods from the object in question and first initialize that object. For example, the following calls are illegal:

```
Dim window as pfcls.IpfcWindow;
window.Activate();           ' The window has not yet
                              ' been initialized.
Repaint();                   ' There is no invoking object.
```

The following calls are legal:

```
Dim window As Pfcls.IpfcWindow
Dim session as pfcls.IpfcSession
Dim asyncConnection as pfcls.IpfcAsyncConnection
Dim Casync as New pfcls.CCpfAsyncConnection

asyncConnection = Casync.Connect (DBNull.Value, DBNull.Value,
DBNull.Value, DBNull.Value)
session = asyncConnection.Session;
window = session.CurrentWindow;    ' You have initialized
                                   ' the window object.

window.Activate()
window.Repaint()
```

Inheritance

Many Creo Parametric-related objects inherit methods from other interfaces. In VB.NET and VBA, you must have an object of the correct type for the compiler and IDE to resolve the methods you wish to call. For example, an `IpfcComponentFeat` object could use the methods and properties as follows:

- `IpfcObject`
- `IpfcChild`
- `IpfcActionSource`
- `IpfcModelItem`
- `IpfcFeature`
- `IpfcComponentFeat`

The following are the approaches to using an object's inherited methods:

1. You can code the method call directly even though it is not available in Intellisense.

```
Dim componentFeat as pfcls.IpfcComponentFeat
MsgBox ("Feature number: " & componentFeat.Number);
```

Note

This works in VB.NET but is likely to result in a compilation error in VBA.

2. You can create another object of the appropriate type and assign it the object handle, and then call the required method.

```
Dim componentFeat as pfcls.IpfcComponentFeat
Dim feat as pfcls.IpfcFeature

feat = componentFeat
MsgBox ("Feature number: " & feat.Number);
```

Compact Data Classes

Compact data classes are data-only classes. They are used for arguments and return values for some VB API methods. They do not represent actual objects in Creo Parametric. Other than a difference in how they are initialized, compact data classes have similar requirements to Creo Parametric-related classes.

Initialization

You can create these compact data objects using a designated `Create` method which resides on the CC version of the compact class. You instantiate the CC class object with the keyword `New`.

For example,

```
'Class object, owns Create()  
Dim tableCellCreate As New pfcls.CCpfctTableCell  
Dim tableCell As pfcls.IpfctTableCell  
Set tableCell = tableCellCreate.Create(1, 1)
```

Unions

Unions are classes containing potentially several different value types. Every union has a discriminator property with the predefined name, `discr`. This property returns a value identifying the type of data that the union object holds. For each union member, a separate property is used to access the different data types. It is illegal to attempt to read any property except the one that matches the value returned from the discriminator. However, any property that switches the discriminator to the new value type can be modified.

The following is an example of a VB API union:

```
Interface IpfctParamValue  
-----  
    Description  
    This class describes the value of the parameter.  
-----  
    Union Discriminant  
    Property discr as IpfctParamValueType [readonly]  
    Returns the union discriminant value.  
-----  
    Property Summary  
    Property BoolValue as Boolean  
        If the parameter type is PARAM_BOOLEAN, this is a Boolean  
value.  
    Property DoubleValue as Double  
        If the parameter type is PARAM_DOUBLE, this is a double value.  
    Property IntValue as Long  
        If the parameter type is PARAM_INTEGER, this is an integer  
value.  
    Property NoteId as Long  
        If the parameter type is PARAM_NOTE, this is a note identifier.  
    Property StringValue as String  
        If the parameter type is PARAM_STRING, this is a string value.
```

Sequences

Sequences are expandable arrays of primitive data types or objects in the VB API. All sequence classes have the same methods for adding to and accessing the array. Sequence classes are typically identified by a plural name, or the suffix `seq`.

Initialization

You can create instances of these classes directly by instantiating the appropriate class object:

Properties

The read-only `Count` attribute identifies how many members are currently in the sequence. You may also access members of the sequence using the `Item` property or directly:

```
Dim model as IpfcModel  
model = models (0)
```

Methods

Sequence objects always contain the same methods. Use the following methods to access the contents of the sequence:

- `Append()` —Adds a new item to the end of the array
- `Clear()` —Removes all items from the array
- `Insert()` —Inserts a new item at any location of the array
- `InsertSeq()` —Inserts the contents of a sequence of items at any location of the array
- `Set()` —Assigns one item in the array to the input item
- `Remove()` —Removes a range of items from the array

Inheritance

Sequence classes do not inherit from any other VB API classes. Therefore, you cannot use sequence objects as arguments where any other type of VB API object is expected, including other types of sequences. For example, if you have a list of `IpfcModelItems` that happen to be features, you cannot use the sequence as if it were a sequence of `IpfcFeatures`.

To construct the array of features, you must insert each member of the `IpfcModelItems` list into the new `IpfcFeatures` list.

Arrays

Arrays are groups of primitive types or objects of a specified size. An array can be one- or two- dimensional. The online reference documentation indicates the exact size of each array class.

Initialization

You can create instances of these classes directly by instantiating the appropriate class object:

Properties

You may read members of the sequence using the `Item` property or directly as an array:

```
Dim point as IpfcPoint3D
Dim matrix as IpfcMatrix3D
MsgBox ("Y value of point: " & point.Item (1))
MsgBox ("(2, 2) value of matrix: " & matrix (2, 2))
```

Methods

Array objects contain only the `Set` method, which assigns one item in the array to the input item.

Enumeration Classes

In the VB API, an enumeration class defines a limited number of values that correspond to the members of the enumeration. Each value represents an appropriate type and may be accessed by name. In the `EpfcFeatureType` enumeration class, the value `EpfcFEATTYPE_HOLE` represents a Hole feature in Creo Parametric. Enumeration classes in the VB API generally have names of the form `EpfcXYZType` or `EpfcXYZStatus`.

Initialization

You can directly refer to instance of this class:

```
Dim type as EpfcFeatureType
type = EpfcFeatureType.EpfcFEATTYPE_HOLE
```

Attributes

An enumeration class is made up of constant integer properties. The names of these properties are all uppercase and describe what the attribute represents. For example:

-
- `EpfcPARAM_INTEGER`—A value in the `EpfcParamValueType` enumeration class that is used to indicate that a parameter stores an integer value.
 - `EpfcITEM_FEATURE`—An value in the `EpfcModelItemType` enumeration class that is used to indicate that a model item is a feature.

An enumeration class always has an integer value named `<type>_nil`, which is one more than the highest acceptable numerical value for that enumeration class.

Module-Level Classes

Some modules in the VB API have one class that contains special functions used to create and access some of the other classes in the package. These module classes have the naming convention, CM+ the name of the module, for example `CMpfcSelect`.

Initialization

You can create instances of these classes directly by instantiating the appropriate class object:

```
Dim mSelect as New CMpfcSelect
```

Methods

Module-level classes contain only static methods used for initializing certain VB API objects.

Action Listeners

Action Listeners notify you of events in Creo Parametric. They are also the basis for customization of the Creo Parametric User Interface. ActionListeners are not supported from VBA.

Initialization

In VB.NET, you can create and assign an ActionListener class as follows.

Create a class implementing the listener in question. It should define all the inherited methods, even if you want to only execute code for a few of the listener methods. Those other methods should be implemented with an empty body.

The class should also implement the interface `IpfcActionListener`, which has no methods.

The class should also implement `ICIPClientObject`. This method defines the object type to the CIP code in the server. This method returns a String which is the name of the listener type interface, for example,

```

IpfcSessionActionListener.
Private Class ModelEventListener
    Implements IpfcModelEventActionListener
    Implements ICIPClientObject
    Implements IpfcActionListener

    Public Function GetClientInterfaceName() As String _
        Implements ICIPClientObject.GetClientInterfaceName
        GetClientInterfaceName =
"IpfcModelEventActionListener"
    End Function
'=====
=====
'Function      :   OnAfterModelCopy
'Purpose       :   This method is executed after successfully
'               :   copying a model.
'=====
=====
    Public Sub OnAfterModelCopy(ByVal _FromMdl As
        pfcls.IpfcModelDescriptor, ByVal _ToMdl As
        pfcls.IpfcModelDescriptor) Implements
        pfcls.IpfcModelEventActionListener.OnAfterModelCopy
'Method Body
    End Sub
'=====
=====
'Function      :   OnAfterModelRename
'Purpose       :   This method is executed after successfully
'               :   renaming a model.
'=====
=====
    Public Sub OnAfterModelRename(ByVal _FromMdl As
        pfcls.IpfcModelDescriptor, ByVal _ToMdl As
        pfcls.IpfcModelDescriptor) Implements
        pfcls.IpfcModelEventActionListener.OnAfterModelRename
'Method Body

    End Sub

    Public Sub OnAfterModelCopyAll(ByVal _FromMdl As
        pfcls.IpfcModelDescriptor, ByVal _ToMdl As
        pfcls.IpfcModelDescriptor) Implements
        pfcls.IpfcModelEventActionListener.OnAfterModelCopyAll

    End Sub

```

```

Public Sub OnAfterModelDelete(ByVal _Descr As
    pfcls.IpfcModelDescriptor) Implements
    pfcls.IpfcModelEventActionListener.OnAfterModelDelete

End Sub

Public Sub OnAfterModelErase(ByVal _Descr As
    pfcls.IpfcModelDescriptor) Implements
    pfcls.IpfcModelEventActionListener.OnAfterModelErase

End Sub

End Class

```

Exceptions

Action listeners cause methods to be called outside of your application start and stop methods. Therefore, you must include exception-handling code inside the `IpfcActionListener` implementation if you want to respond to exceptions. In some methods called before an event, propagating a `IpfcXCancelProEAction` exception out of your method will cancel the impending event.

Programming Considerations

The items in this section introduce programming tips and techniques used for programming with the VB API.

Application Hierarchy

The rules of object orientation require a certain hierarchy of object creation when you start a VB application. The application must iterate down to the level of the object you want to access. For example, to list all the datum axes contained in the hole features in all models in session, do the following:

1. Use the method `CCpfcAsyncConnection.Connect()` to connect to an existing session of Creo Parametric.


```

Dim connection As IpfcAsyncConnection
Dim classAsyncConnection As New CCpfcAsyncConnection
connection = classAsyncConnection.Connect (DBNull.Value,
    DBNull.Value, DBNull.Value, DBNull.Value)

```
2. Get a handle to the session of Creo Parametric for the current active connection:


```

Dim session As IpfcBaseSession
session = connection.Session

```
3. Get the models that are loaded in the session:


```

Dim models As IpfcModels

```

-
- ```
models = session.ListModels()
```
4. Get the handle to the first model in the list:  

```
Dim model As IpfcModel
model = models[0]
```
  5. Get the feature model items in each model:  

```
Dim items As IpfcModelItems
items = model.ListItems (EpfcModelItemType.EpfcITEM_FEATURE)
```
  6. Filter out the features of type hole:  

```
if (feature.FeatType = EpfcFeatureType.EpfcFEATTYPE_HOLE) then
```
  7. Get the subitems in each feature that are axes:  

```
Dim axes As IpfcModelItems
axes = feature.ListSubItems (EpfcModelItemType.EpfcITEM_AXIS)
```

## Optional Arguments and Tags

Many methods in the VB API are shown in the online documentation as having optional arguments.

For example, the `IpfcModelItemOwner.ListItems()` method takes an optional *Type* argument.

```
IpfcModelItems ListItems (Type as IpfcModelItemType [optional]);
```

In VB.Net, you can pass the keyword `Nothing` in place of any such optional argument. In VBA, use `Null` in place of any such optional argument. The VB API methods that take optional arguments provide default handling for `Nothing` parameters which is described in the online documentation.



### Note

You can only pass `Nothing` in place of arguments that are shown in the documentation to be optional.

---

## Optional Returns for the VB API Methods

Some methods in the VB API have an optional return. Usually these correspond to lookup methods that may or may not find an object to return. For example, the `IpfcBaseSession.GetModel()` method returns an optional model:

```
Function GetModel (Name as String, Type as IpfcModelType) as
IpfcModel [optional]
```

The VB API might return `Nothing` in certain cases where these methods are called. You must use appropriate value checks in your application code to handle these situations.

## Parent-Child Relationships between the VB API Objects

Some VB API objects inherit from either the interface `IpfcParent` or `IpfcChild`. These interfaces are used to maintain a relationship between the two objects. This has nothing to do with object-oriented inheritance, but rather, refers to the relationship between the items in Creo Parametric. In the VB API, the Child is owned by the Parent.

Property Introduced:

- **`IpfcChild.DBParent`**

The `IpfcChild.DBParent` property returns the owner of the child object. The application developer must know the expected type of the parent in order to use it in later calls. The following table lists parent/child relationships in the VB API.

| Parent                   | Child                                |
|--------------------------|--------------------------------------|
| <code>IpfcSession</code> | <code>IpfcModel</code>               |
| <code>IpfcSession</code> | <code>IpfcWindow</code>              |
| <code>IpfcModel</code>   | <code>IpfcModelItem</code>           |
| <code>IpfcSolid</code>   | <code>IpfcFeature</code>             |
| <code>IpfcModel</code>   | <code>IpfcParameter</code>           |
| <code>IpfcModel</code>   | <code>IpfcExternalDataAccess</code>  |
| <code>IpfcPart</code>    | <code>IpfcMaterial</code>            |
| <code>IpfcModel</code>   | <code>IpfcView</code>                |
| <code>IpfcModel2D</code> | <code>IpfcView2D</code>              |
| <code>IpfcSolid</code>   | <code>IpfcXSection</code>            |
| <code>IpfcSession</code> | <code>IpfcDll (Creo TOOLKIT)</code>  |
| <code>IpfcSession</code> | <code>IpfcJLinkApplication ()</code> |

## Run-Time Type Identification in the VB API

The VB API and Visual Basic provide several methods to identify the type of an object.

Many classes provide read access to a type enumerated class. For example, the `IpfcFeature` class has a `IpfcFeature.FeatType` property, returning a `pfcFeatureType` enumeration value representing the type of the feature. Based upon the type, a user can recognize that the `IpfcFeature` object is actually a particular subtype, such as `IpfcComponentFeat`, which is an assembly component.

## Support for Embedded OLE Objects

OLE objects, when activated by the user, can include VB code that can be used to drive the model from which the object is contained. You can use the method `IpfcAsyncConnection.GetConnectionId()` to get the connection ID

---

of the Creo Parametric session that launched the process containing the OLE object. Refer to the section [Connecting to a Creo Parametric Process](#) on page 30 for information about getting the connection ID.

The user application code embedded in the OLE object passes the connection ID string to `CCpfcConnectionId.Create` and `IpfcAsyncConnection.ConnectById()` to establish the connection. The code may then obtain the owner model of the OLE object by retrieving the current model from the session using standard PFC APIs.

For example,

```
Dim ls As New pfcls.CCpfcAsyncConnection
Dim aC As pfcls.IpfcAsyncConnection
Dim cId As New pfcls.CCpfcConnectionId
Dim id As pfcls.IpfcConnectionId
Dim session As pfcls.IpfcBaseSession
Dim model As pfcls.IpfcModel

Set id = cId.Create(connectionId)
Set aC = ls.ConnectById(id, DBNull.Value, DBNull.Value)

Set session = aC.Session
Set model = session.CurrentModel
```

## Exceptions

All PFC methods that fail may throw exceptions as `System.Runtime.InteropServices.COMException`.

The type of the exception can be obtained from the `Message` property of this exception.

```
Try
 session.SetConfigOption("no_way", "no_how")
Catch ex As Exception
 MsgBox(ex.Message) 'Will show IpfcXToolkitNotFound
End Try
```

The `Description` property returns the full exception description as `[Exception type]; [additional details]`. The exception type is the module and exception name, for example, `IpfcXToolkitCheckoutConflict`.

The additional details include information that was contained in the exception when it was thrown by the PFC layer, such as conflict descriptions for exceptions caused by server operations and error details for exceptions generated during drawing creation.

## PFC Exceptions

The PFC exceptions are thrown by the classes that make up the VB API's public interface. The following table describes these exceptions.

| Exception                                   | Purpose                                                                                                                                                                                                  |
|---------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>IpfcXBadExternalData</code>           | An attempt to read contents of an external data object that has been terminated.                                                                                                                         |
| <code>IpfcXBadGetArgValue</code>            | Indicates attempt to read the wrong type of data from the <code>IpfcArgValue</code> union.                                                                                                               |
| <code>IpfcXBadGetExternalData</code>        | Indicates attempt to read the wrong type of data from the <code>IpfcExternalData</code> union.                                                                                                           |
| <code>IpfcXBadGetParamValue</code>          | Indicates attempt to read the wrong type of data from the <code>IpfcParamValue</code> union.                                                                                                             |
| <code>IpfcXBadOutlineExcludeType</code>     | Indicates an invalid type of item was passed to the outline calculation method.                                                                                                                          |
| <code>IpfcXCancelProEAction</code>          | This exception type will not be thrown by VB API methods, but you may instantiate and throw this from certain <code>ActionListener</code> methods to cancel the corresponding action in Creo Parametric. |
| <code>IpfcXCannotAccess</code>              | The contents of a VB API object cannot be accessed in this situation.                                                                                                                                    |
| <code>IpfcXEmptyString</code>               | An empty string was passed to a method that does not accept this type of input.                                                                                                                          |
| <code>IpfcXInvalidEnumValue</code>          | Indicates an invalid value for a specified enumeration class.                                                                                                                                            |
| <code>IpfcXInvalidFileName</code>           | Indicates a file name passed to a method was incorrectly structured.                                                                                                                                     |
| <code>IpfcXInvalidFileType</code>           | Indicates a model descriptor contained an invalid file type for a requested operation.                                                                                                                   |
| <code>IpfcXInvalidModelItem</code>          | Indicates that the item requested to be used is no longer usable (for example, it may have been deleted).                                                                                                |
| <code>IpfcXInvalidSelection</code>          | Indicates that the <code>IpfcSelection</code> passed is invalid or is missing a needed piece of information. For example, its component path, drawing view, or parameters.                               |
| <code>IpfcXJLinkApplicationException</code> | Contains the details when an attempt to call code in an external application failed due to an exception.                                                                                                 |
| <code>IpfcXJLinkApplicationInactive</code>  | Unable to operate on the requested <code>IpfcJLinkApplication</code> object because it has been shut down.                                                                                               |
| <code>IpfcXJLinkTaskNotFound</code>         | Indicates that the task with the given name could not be found and run.                                                                                                                                  |
| <code>IpfcXModelNotInSession</code>         | Indicates that the model is no longer in session; it may have been erased or deleted.                                                                                                                    |
| <code>IpfcXNegativeNumber</code>            | Numeric argument was negative.                                                                                                                                                                           |
| <code>IpfcXNumberTooLarge</code>            | Numeric argument was too large.                                                                                                                                                                          |
| <code>IpfcXProEWasNotConnected</code>       | The Creo Parametric session is not available so the operation failed.                                                                                                                                    |
| <code>IpfcXSequenceTooLong</code>           | Sequence argument was too long.                                                                                                                                                                          |
| <code>IpfcXStringTooLong</code>             | String argument was too long.                                                                                                                                                                            |
| <code>IpfcXUnimplemented</code>             | Indicates unimplemented method.                                                                                                                                                                          |
| <code>IpfcXUnknownModelExtension</code>     | Indicates that a file extension does not match a known Creo Parametric model type.                                                                                                                       |

## Creo TOOLKIT Errors

The `IpfcXToolkitError` exception types provide access to error codes from Creo TOOLKIT functions that the VB API uses internally and to the names of the functions returning such errors. `IpfcXToolkitError` is the exception you are most likely to encounter because the VB API is built on top of Creo TOOLKIT. The following table lists the `IpfcXToolkitError` types method and shows the corresponding Creo TOOLKIT constant that indicates the cause of the error.

| <b>IpfcXToolkitError Child Class</b>        | <b>Creo TOOLKIT Error</b>           | <b>#</b> |
|---------------------------------------------|-------------------------------------|----------|
| <code>IpfcXToolkitGeneralError</code>       | <code>PRO_TK_GENERAL_ERROR</code>   | -1       |
| <code>IpfcXToolkitBadInputs</code>          | <code>PRO_TK_BAD_INPUTS</code>      | -2       |
| <code>IpfcXToolkitUserAbort</code>          | <code>PRO_TK_USER_ABORT</code>      | -3       |
| <code>IpfcXToolkitNotFound</code>           | <code>PRO_TK_E_NOT_FOUND</code>     | -4       |
| <code>IpfcXToolkitFound</code>              | <code>PRO_TK_E_FOUND</code>         | -5       |
| <code>IpfcXToolkitLineTooLong</code>        | <code>PRO_TK_LINE_TOO_LONG</code>   | -6       |
| <code>IpfcXToolkitContinue</code>           | <code>PRO_TK_CONTINUE</code>        | -7       |
| <code>IpfcXToolkitBadContext</code>         | <code>PRO_TK_BAD_CONTEXT</code>     | -8       |
| <code>IpfcXToolkitNotImplemented</code>     | <code>PRO_TK_NOT_IMPLEMENTED</code> | -9       |
| <code>IpfcXToolkitOutOfMemory</code>        | <code>PRO_TK_OUT_OF_MEMORY</code>   | -10      |
| <code>IpfcXToolkitCommError</code>          | <code>PRO_TK_COMM_ERROR</code>      | -11      |
| <code>IpfcXToolkitNoChange</code>           | <code>PRO_TK_NO_CHANGE</code>       | -12      |
| <code>IpfcXToolkitSuppressedParents</code>  | <code>PRO_TK_SUPP_PARENTS</code>    | -13      |
| <code>IpfcXToolkitPickAbove</code>          | <code>PRO_TK_PICK_ABOVE</code>      | -14      |
| <code>IpfcXToolkitInvalidDir</code>         | <code>PRO_TK_INVALID_DIR</code>     | -15      |
| <code>IpfcXToolkitInvalidFile</code>        | <code>PRO_TK_INVALID_FILE</code>    | -16      |
| <code>IpfcXToolkitCantWrite</code>          | <code>PRO_TK_CANT_WRITE</code>      | -17      |
| <code>IpfcXToolkitInvalidType</code>        | <code>PRO_TK_INVALID_TYPE</code>    | -18      |
| <code>IpfcXToolkitInvalidPtr</code>         | <code>PRO_TK_INVALID_PTR</code>     | -19      |
| <code>IpfcXToolkitUnavailableSection</code> | <code>PRO_TK_UNAV_SEC</code>        | -20      |
| <code>IpfcXToolkitInvalidMatrix</code>      | <code>PRO_TK_INVALID_MATRIX</code>  | -21      |
| <code>IpfcXToolkitInvalidName</code>        | <code>PRO_TK_INVALID_NAME</code>    | -22      |
| <code>IpfcXToolkitNotExist</code>           | <code>PRO_TK_NOT_EXIST</code>       | -23      |
| <code>IpfcXToolkitCantOpen</code>           | <code>PRO_TK_CANT_OPEN</code>       | -24      |
| <code>IpfcXToolkitAbort</code>              | <code>PRO_TK_ABORT</code>           | -25      |
| <code>IpfcXToolkitNotValid</code>           | <code>PRO_TK_NOT_VALID</code>       | -26      |
| <code>IpfcXToolkitInvalidItem</code>        | <code>PRO_TK_INVALID_ITEM</code>    | -27      |
| <code>IpfcXToolkitMsgNotFound</code>        | <code>PRO_TK_MSG_NOT_FOUND</code>   | -28      |
| <code>IpfcXToolkitMsgNoTrans</code>         | <code>PRO_TK_MSG_NO_TRANS</code>    | -29      |
| <code>XToolkitMsgFmtError</code>            | <code>PRO_TK_MSG_FMT_ERROR</code>   | -30      |
| <code>IpfcXToolkitMsgUserQuit</code>        | <code>PRO_TK_MSG_USER_QUIT</code>   | -31      |
| <code>IpfcXToolkitMsgTooLong</code>         | <code>PRO_TK_MSG_TOO_LONG</code>    | -32      |

| <b>IpfcXToolkitError Child Class</b> | <b>Creo TOOLKIT Error</b>     | <b>#</b> |
|--------------------------------------|-------------------------------|----------|
| IpfcXToolkitCantAccess               | PRO_TK_CANT_ACCESS            | -33      |
| IpfcXToolkitObsoleteFunc             | PRO_TK_OBSOLETE_FUNC          | -34      |
| IpfcXToolkitNoCoordSystem            | PRO_TK_NO_COORD_SYSTEM        | -35      |
| IpfcXToolkitAmbiguous                | PRO_TK_E_AMBIGUOUS            | -36      |
| IpfcXToolkitDeadLock                 | PRO_TK_E_DEADLOCK             | -37      |
| IpfcXToolkitBusy                     | PRO_TK_E_BUSY                 | -38      |
| IpfcXToolkitInUse                    | PRO_TK_E_IN_USE               | -39      |
| IpfcXToolkitNoLicense                | PRO_TK_NO_LICENSE             | -40      |
| IpfcXToolkitBsplUnsuitableDegree     | PRO_TK_BSPL_UNSUITABLE_DEGREE | -41      |
| IpfcXToolkitBsplNonStdEndKnots       | PRO_TK_BSPL_NON_STD_END_KNOTS | -42      |
| IpfcXToolkitBsplMultiInnerKnots      | PRO_TK_BSPL_MULTI_INNER_KNOTS | -43      |
| IpfcXToolkitBadSrfCrv                | PRO_TK_BAD_SRF_CRV            | -44      |
| IpfcXToolkitEmpty                    | PRO_TK_EMPTY                  | -45      |
| IpfcXToolkitBadDimAttach             | PRO_TK_BAD_DIM_ATTACH         | -46      |
| IpfcXToolkitNotDisplayed             | PRO_TK_NOT_DISPLAYED          | -47      |
| IpfcXToolkitCantModify               | PRO_TK_CANT_MODIFY            | -48      |
| IpfcXToolkitCheckoutConflict         | PRO_TK_CHECKOUT_CONFLICT      | -49      |
| IpfcXToolkitCreateViewBadSheet       | PRO_TK_CRE_VIEW_BAD_SHEET     | -50      |
| IpfcXToolkitCreateViewBadModel       | PRO_TK_CRE_VIEW_BAD_MODEL     | -51      |
| IpfcXToolkitCreateViewBadParent      | PRO_TK_CRE_VIEW_BAD_PARENT    | -52      |
| IpfcXToolkitCreateViewBadType        | PRO_TK_CRE_VIEW_BAD_TYPE      | -53      |
| IpfcXToolkitCreateViewBadExplode     | PRO_TK_CRE_VIEW_BAD_EXPLODE   | -54      |
| IpfcXToolkitUnattachedFeats          | PRO_TK_UNATTACHED_FEATS       | -55      |
| IpfcXToolkitRegenerateAgain          | PRO_TK_REGEN_AGAIN            | -56      |
| IpfcXToolkitDrawingCreateErrors      | PRO_TK_DWGCREATE_ERRORS       | -57      |
| IpfcXToolkitUnsupported              | PRO_TK_UNSUPPORTED            | -58      |
| IpfcXToolkitNoPermission             | PRO_TK_NO_PERMISSION          | -59      |
| IpfcXToolkitAuthenticationFailure    | PRO_TK_AUTHENTICATION_FAILURE | -60      |
| IpfcXToolkitMultibodyUnsupported     | PRO_TK_MULTIBODY_UNSUPPORTED  | -69      |
| IpfcXToolkitAppNoLi                  | PRO_TK_APP_NO_LICENSE         | -92      |



| <b>IpfcXToolkitError Child Class</b> | <b>Creo TOOLKIT Error</b>   | <b>#</b> |
|--------------------------------------|-----------------------------|----------|
| cense                                |                             |          |
| IpfcXToolkitAppExcess Callbacks      | PRO_TK_APP_XS_CALLBACKS     | -93      |
| IpfcXToolkitAppStartup Failed        | PRO_TK_APP_STARTUP_FAIL     | -94      |
| IpfcXToolkitAppInitializationFailed  | PRO_TK_APP_INIT_FAIL        | -95      |
| IpfcXToolkitAppVersion Mismatch      | PRO_TK_APP_VERSION_MISMATCH | -96      |
| IpfcXToolkitAppCommunicationFailure  | PRO_TK_APP_COMM_FAILURE     | -97      |
| IpfcXToolkitAppNewVersion            | PRO_TK_APP_NEW_VERSION      | -98      |

The exception `IpfcXProdevError` represents a general error that occurred while executing a Pro/DEVELOP function and is equivalent to an `IpfcXToolkitGeneralError` exception. (PTC does not recommend the use of Pro/DEVELOP functions.)

The `IpfcXExternalDataError` exception types and its children are thrown from External Data methods. See the section on [External Data on page 297](#) for more information.

# 3

## User's Guide

|                                                      |     |
|------------------------------------------------------|-----|
| VB API Fundamentals:Controlling Creo Parametric..... | 27  |
| Session Objects .....                                | 34  |
| Selection .....                                      | 49  |
| Ribbon Tabs, Groups, and Menu Items .....            | 55  |
| Menus, Commands, and Pop-up Menus.....               | 63  |
| Models .....                                         | 72  |
| Drawings .....                                       | 88  |
| Solid.....                                           | 138 |
| Solid Bodies.....                                    | 155 |
| Windows and Views .....                              | 156 |
| ModelItem.....                                       | 166 |
| Features .....                                       | 170 |
| Datum Features .....                                 | 186 |
| Geometry Evaluation .....                            | 194 |
| Dimensions and Parameters .....                      | 206 |
| Relations .....                                      | 215 |
| Assemblies and Components.....                       | 220 |
| Family Tables.....                                   | 231 |
| Action Listeners .....                               | 235 |
| Interface .....                                      | 243 |
| Simplified Representations.....                      | 279 |
| Task Based Application Libraries .....               | 286 |
| Graphics.....                                        | 291 |
| External Data .....                                  | 296 |
| Windchill Connectivity APIs.....                     | 301 |
| Sample Applications .....                            | 319 |
| Geometry Traversal.....                              | 324 |
| Geometry Representations .....                       | 327 |

---

# VB API Fundamentals:Controlling Creo Parametric

## VB API Fundamentals:Controlling Creo Parametric

This section explains how to use the VB API to establish a connection to Creo Parametric.

### Overview

Asynchronous mode is a multiprocess mode in which the VB API application and Creo Parametric can perform concurrent operations. The VB API application (containing its own `main()` method) is started independently of Creo Parametric and subsequently either starts or connects to a Creo Parametric process.

Depending on how your asynchronous application handles messages from Creo Parametric, your application can be classified as either `simple` or `full`. The following sections describe `simple` and `full` asynchronous mode.

### Simple Asynchronous Mode

A `simple` asynchronous application does not implement a way to handle requests from Creo Parametric. Therefore, the VB API cannot plant listeners to be notified when events happen in Creo Parametric. Consequently, Creo Parametric cannot invoke the methods that must be supplied when you add, for example, menu buttons to Creo Parametric.

Despite this limitation, a simple asynchronous mode application can be used to automate processes in Creo Parametric. The application may either start or connect to an existing Creo Parametric session, and may access Creo Parametric in interactive or in a non graphical, non interactive mode. When Creo Parametric is running with graphics, it is an interactive process available to the user.

When you design a VB API application to run in simple asynchronous mode, keep the following points in mind:

- The Creo Parametric process and the application perform operations concurrently.
- None of the application's listener methods can be invoked by Creo Parametric.

### Starting and Stopping Creo Parametric

The following methods are used to start and stop Creo Parametric when using the VB API applications.

Methods Introduced:

- **CCpfcAsyncConnection.Start()**

---

- **IpfcAsyncConnection.End()**

A VB application can spawn and connect to a Creo Parametric process with the method `CCpfcAsyncConnection.Start()`.

After this method returns the asynchronous connection object, the VB API application can call the Creo Parametric process using the appropriate APIs. In the interactive mode, you can also access the Creo Parametric session when it is running.

The asynchronous application is not terminated when Creo Parametric terminates. This is useful when the application needs to perform Creo Parametric operations intermittently, and therefore, must start and stop Creo Parametric more than once during a session.

The application can connect to or start only one Creo Parametric session at any time. If the VB API application spawns a second session, connection to the first session is lost.

To end any Creo Parametric process that the application is connected to, call the method `IpfcAsyncConnection.End()`.

## Setting Up a Noninteractive Session

You can spawn a Creo Parametric session that is both noninteractive and nongraphical. In asynchronous mode, include the following strings in the Creo Parametric start or connect call to `CCpfcAsyncConnection.Start()`:

- `-g:no_graphics`—Turn off the graphics display.
- `-i:rpc_input`—Causes Creo Parametric to expect input from your asynchronous application only.

---

### Note

Both of these arguments are required, but the order is not important.

---

The syntax of the call for a noninteractive, nongraphical session is as follows:

```
Dim aC as IpfcAsyncConnection
Dim ccAC as New CcpfcAsyncConnection
aC = ccAC.Start ("pro -g:no_graphics -i:rpc_input", <text_dir>);
```

where `pro` is the command to start Creo Parametric.

---

## Example Code for Visual Basic.NET

The sample code in the file `pfcAsynchronousModeExamples.vb` located at `<creo_vbapi_loadpoint>/vbapi_examples` demonstrates how to use the VB API to start Creo Parametric asynchronously, retrieve a Session and to open a model in Creo Parametric.

## Example Code for Visual Basic for Applications

This example demonstrates the VB API syntax in a macro written in Visual Basic for Applications, for example, as would be run by a button in a Microsoft Word document or Microsoft Excel spreadsheet. This example is identical to the previous example, except for the syntax.

```
Private Sub btnRun_Click()
 Dim asyncConnection As IpfcAsyncConnection
 Dim cAC As CCpfcAsyncConnection
 Dim session As IpfcBaseSession
 Dim descModel As IpfcModelDescriptor
 Dim descModelCreate As CCpfcModelDescriptor
 Dim model As IpfcModel
 Dim workDir As String
 Dim position As Integer
 On Error GoTo RunError

 '=====
```

First Argument : The path to the Creo Parametric executable along with command line options. -i and -g flags make Creo Parametric run in non-graphic, non-interactive mode

Second Argument: String path to menu and message files.

```
 '=====
```

Set cAC = New CCpfcAsyncConnection

Set asyncConnection = cAC.Start(txtExePath.Text + " -g:no\_graphics -i:rpc\_input", ".")

Set session = asyncConnection.session

```
 '=====
```

Get current directory

Set it as working directory

```
 '=====
```

workDir = ActiveWorkbook.FullName

position = InStrRev(workDir, "\")

workDir = Left(workDir, position)

session.ChangeDirectory (workDir)

```

'=====
=====
'VB api process calls and other processing to be done
'=====
=====
 Set descModelCreate = New CCpfcModelDescriptor
 Set descModel = descModelCreate.Create(EpfcModelType.EpfcMDL_
PART,
 "partModel.prt", dbnull)
 Set model = session.RetrieveModel(descModel)
'=====
=====
End the Creo Parametric session when done
'=====
=====
 If Not asyncConnection Is Nothing Then
 If asyncConnection.IsRunning Then
 asyncConnection.End
 End If
 End If

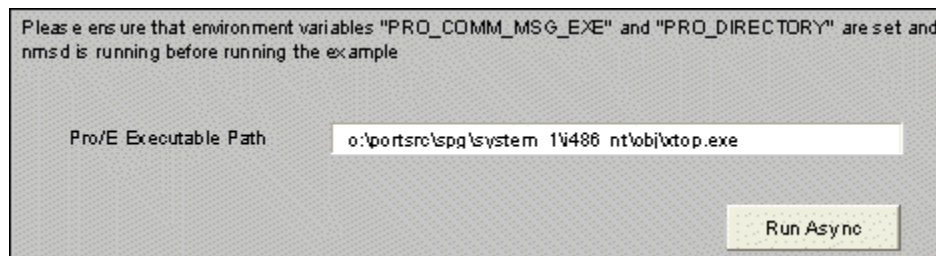
RunError:
If Err.Number <> 0 Then
 MsgBox "Process Failed : Unknown error occurred." + Chr(13) + _
 "Error No: " + CStr(Err.Number) + Chr(13) + _
 "Error: " + Err.Description, vbCritical, "Error"

 If Not asyncConnection Is Nothing Then
 If asyncConnection.IsRunning Then
 asyncConnection.End
 End If
 End If
End If

End Sub

```

The following figure displays the button in Microsoft Excel designed for the above application.



## Connecting to a Creo Parametric Process

Methods Introduced:

- 
- **CCpfcAsyncConnection.Connect()**
  - **CCpfcAsyncConnection.GetActiveConnection()**
  - **IpfcAsyncConnection.Disconnect()**

A simple asynchronous application can also connect to a Creo Parametric process that is already running on a local computer. The method `CCpfcAsyncConnection.Connect()` performs this connection. This method fails to connect if multiple Creo Parametric sessions are running. If several versions of Creo Parametric are running on the same computer, try to connect by specifying user and display parameters. However, if several versions of Creo Parametric are running in the same user and display parameters, the connection may not be possible.

`CCpfcAsyncConnection.GetActiveConnection()` returns the current connection to a Creo Parametric session.

To disconnect from a Creo Parametric process, call the method `IpfcAsyncConnection.Disconnect()`.

## Connecting Via Connection ID

Methods Introduced:

- **IpfcAsyncConnection.GetConnectionId()**
- **IpfcConnectionId.ExternalRep**
- **CCpfcConnectionId.Create()**
- **IpfcAsyncConnection.ConnectById()**

Each Creo Parametric process maintains a unique identity for communications purposes. Use this ID to reconnect to a Creo Parametric process.

The method `IpfcAsyncConnection.GetConnectionId()` returns a data structure containing the connection ID.

If the connection id must be passed to some other application the method `IpfcConnectionId.ExternalRep` provides the string external representation for the connection ID.

The method `CCpfcConnectionId.Create()` takes a string representation and creates a `ConnectionId` data object. The method `IpfcAsyncConnection.ConnectById()` connects to Creo Parametric at the specified connection ID.



### Note

Connection IDs are unique for each Creo Parametric process and are not maintained after you quit Creo Parametric.

---

---

## Status of a Creo Parametric Process

Method Introduced:

- **IpfcAsyncConnection.IsRunning()**

To find out whether a Creo Parametric process is running, use the method `IpfcAsyncConnection.IsRunning()`.

## Getting the Session Object

Method Introduced:

- **IpfcAsyncConnection.Session()**

The method `IpfcAsyncConnection.Session()` returns the session object representing the Creo Parametric session. Use this object to access the contents of the Creo Parametric session. See the [Session Objects on page 34](#) section for additional information.

## Full Asynchronous Mode

Full asynchronous mode is identical to the simple asynchronous mode except in the way the VB API application handles requests from Creo Parametric. In simple asynchronous mode, it is not possible to process these requests. In full asynchronous mode, the application implements a control loop that “listens” for messages from Creo Parametric. As a result, Creo Parametric can call functions in the application, including callback functions for menu buttons and notifications.



### Note

Using full asynchronous mode requires starting or connecting to Creo Parametric using the methods described in the previous sections. The difference is that the application must provide an event loop to process calls from menu buttons and listeners.

---

Methods Introduced:

- **IpfcAsyncConnection.EventProcess()**
- **IpfcAsyncConnection.WaitForEvents()**
- **IpfcAsyncConnection.InterruptEventProcessing()**
- **IpfcAsyncActionListener.OnTerminate()**

The control loop of an application running in full asynchronous mode must contain a call to the method `IpfcAsyncConnection.EventProcess()`, which takes no arguments. This method allows the application to respond to messages sent from Creo Parametric. For example, if the user selects a menu



---

button that is added by your application,

`IpfcAsyncConnection.EventProcess()` processes the call to your listener and returns when the call completes. For more information on listeners and adding menu buttons, see the [Session Objects on page 34](#) section.

The method `IpfcAsyncConnection.WaitForEvents()` provides an alternative to the development of an event processing loop in a full asynchronous mode application. Call this function to have the application wait in a loop for events to be passed from Creo Parametric. No other processing takes place while the application is waiting. The loop continues until `IpfcAsyncConnection.InterruptEventProcessing()` is called from a VB callback action, or until the application detects the termination of Creo Parametric.

It is often necessary for your full asynchronous application to be notified of the termination of the Creo Parametric process. In particular, your control loop need not continue to listen for Creo Parametric messages if Creo Parametric is no longer running.

An `AsyncConnection` object can be assigned an Action Listener to bind a termination action that is executed upon the termination of Creo Parametric. The method `IpfcAsyncActionListener.OnTerminate()` handles the termination that you must override. It sends a member of the class `IpfcTerminationStatus`, which is one of the following:

- `EpfcTERM_EXIT`—Normal exit (the user clicks **Exit** on the menu).
- `EpfcTERM_ABNORMAL`—Quit with error status.
- `EpfcTERM_SIGNAL`—Fatal signal raised.

Your application can interpret the termination type and take appropriate action. For more information on Action Listeners, see the [Action Listeners on page 235](#) section.

## Example Code

The sample code in the file `pfcAsynchronousModeExamples.vb` located at `<creo_vbapi_loadpoint>/vbapi_examples` is a fully asynchronous application. It follows the procedure for a full asynchronous application:

1. The application establishes listeners for Creo Parametric events, in this case, the menu button and the termination listener.
2. The application goes into a control loop calling **EventProcess** which allows the application to respond to the Creo Parametric events.

## Message and Menu File

```


VB-Async
```

---

```
VB-Async
#
#
USER#Async#App
Async Button
#
#
USER#Async#Help
Button added via Async Application
#
#
```

## Troubleshooting VB API Applications

### General Problems

#### **IpfcXToolkitNotFound exception on the first call to CCpfcAsyncConnection.Start on Windows**

Make sure your command is correct. If it is not a full path to a script or executable, make sure \$PATH is set correctly. Try full path in the command: if it works, then your \$PATH is incorrect.

#### **IpfcXToolkitGeneralError or IpfcXToolkitCommError on the first call to CCpfcAsyncConnection.Start or CCpfcAsyncConnection.Connect**

- Make sure the environment variable PRO\_COMM\_MSG\_EXE is set to the full path to pro\_comm\_msg, including <filename.exe>.
- Make sure the environment variable PRO\_DIRECTORY is set to the Creo Parametric installation directory.
- Make sure name service (nmsd) is running.

#### **CCpfcAsyncConnection.Start hangs, even though Creo Parametric already started**

Make sure name service (nmsd) is also started along with Creo Parametric. Open **Task Manager** and look for nmsd.exe in the process listing.

## Session Objects

### Session Objects

This section describes how to program on the session level using the VB API.

---

## Overview of Session Objects

The Creo Parametric `Session` object (contained in the class `IpfcSession`) is the highest level object in the VB API. Any program that accesses data from Creo Parametric must first get a handle to the `IpfcSession` object before accessing more specific data.

The `IpfcSession` object contains methods to perform the following operations:

- Accessing models and windows (described in the Models and Windows sections).
- Working with the Creo Parametric user interface.
- Allowing interactive selection of items within the session.
- Accessing global settings such as line styles, colors, and configuration options.

The following sections describe these operations in detail. Refer to the section "Controlling Creo Parametric" for more information on how to connect to a Creo Parametric session.

## Compatibility of Deprecated Methods

In a release cycle, some methods are deprecated, and new methods are added. The deprecated methods are supported in the current release, and then obsoleted in a future release. You can either choose to let the deprecated methods work in the current release, or allow only new methods to work. Use the methods explained in this section along with the compatibility value to work with either deprecated or new methods for the current release.

Methods Introduced:

- **`IpfcAppInfo.GetCompatibility()`**
- **`IpfcAppInfo.SetCompatibility()`**
- **`CCpfcAppInfo.Create()`**
- **`IpfcSession.GetAppInfo()`**
- **`IpfcSession.SetAppInfo()`**

The methods `IpfcAppInfo.GetCompatibility()` and `IpfcAppInfo.SetCompatibility()` get and set the compatibility value for the specified application using the enumerated data type `EpfcCreoCompatibility`. The valid values are:

- `EpfcCompatibilityUndefined`—Specifies that compatibility value is not set. The default compatibility value is used.
- `EpfcC3Compatible`—Specifies that the methods deprecated in the VB API 4.0 are compatible and continue working in the VB API 4.0. By default the compatibility is set to `EpfcC3Compatible`.

- 
- `EpfcC4Compatible`—Specifies that the methods deprecated in the VB API 4.0 will not work in the VB API 4.0. If your application uses the deprecated methods, you must replace these methods with new methods and rebuild your applications.

---

 **Note**

If you rebuild or run the VB API applications from previous releases in the current release, the compatibility is set `EpfcC3Compatible` to current release. For example, if you rebuild a the VB API 3.0 application in release 4.0, the compatibility is set to `C3Compatible`. To set the compatibility to the current release, use the method `IpfcAppInfo.SetCompatibility()`.

---

The method `CCpfcAppInfo.Create()` creates a new instance of the `pfcAppInfo` object.

The methods `IpfcSession.GetAppInfo()` and `IpfcSession.SetAppInfo()` get and set the information for an application in terms of their compatibility value as a `pfcSession.AppInfo` object.

## Directories

Methods Introduced:

- **`IpfcBaseSession.GetCurrentDirectory()`**
- **`IpfcBaseSession.ChangeDirectory()`**

The method `IpfcBaseSession.GetCurrentDirectory()` returns the absolute path name for the current working directory of Creo Parametric.

The method `IpfcBaseSession.ChangeDirectory()` changes Creo Parametric to another working directory.

## File Handling

Methods Introduced:

- **`IpfcBaseSession.ListFiles()`**
- **`IpfcBaseSession.ListSubdirectories()`**

The method `IpfcBaseSession.ListFiles()` returns a list of files in a directory, given the directory path. You can filter the list to include only files of a particular type, as specified by the file extension.

---

Starting with Pro/ENGINEER Wildfire 5.0 M040, the method `IpfcBaseSession.ListFiles()` can also list instance objects when accessing Windchill workspaces or folders. A PDM location (for workspace or commonspace) must be passed as the directory path. The following options have been added in the `EpfcFileListOpt` enumerated type:

- `EpfcFILE_LIST_ALL`—Lists all the files. It may also include multiple versions of the same file.
- `EpfcFILE_LIST_LATEST`—Lists only the latest version of each file.
- `EpfcFILE_LIST_ALL_INST`—Same as the `EpfcFILE_LIST_ALL` option. It returns instances only for PDM locations.
- `EpfcFILE_LIST_LATEST_INST`—Same as the `EpfcFILE_LIST_LATEST` option. It returns instances only for PDM locations.

The method `IpfcBaseSession.ListSubdirectories()` returns the subdirectories in a given directory location.

## Configuration Options

Methods Introduced:

- **`IpfcBaseSession.GetConfigOptionValues()`**
- **`IpfcBaseSession.SetConfigOption()`**
- **`IpfcBaseSession.LoadConfigFile()`**

You can access configuration options programmatically using the methods described in this section.

Use the method `IpfcBaseSession.GetConfigOptionValues()` to retrieve the value of a specified configuration file option. Pass the *Name* of the configuration file option as the input to this method. The method returns an array of values that the configuration file option is set to. It returns a single value if the configuration file option is not a multi-valued option. The method returns a null if the specified configuration file option does not exist.

The method `IpfcBaseSession.SetConfigOption()` is used to set the value of a specified configuration file option. If the option is a multi-value option, it adds a new value to the array of values that already exist.

The method `IpfcBaseSession.LoadConfigFile()` loads an entire configuration file into Creo Parametric.

## Macros

Method Introduced:

- **`IpfcBaseSession.RunMacro()`**

---

The method `IpfcBaseSession.RunMacro()` runs a macro string. A VB API macro string is equivalent to a Creo Parametric mapkey minus the key sequence and the mapkey name. To generate a macro string, create a mapkey in Creo Parametric. Refer to the Creo Parametric online help for more information about creating a mapkey.

Copy the Value of the generated mapkey Option from the **Tools ► Options** dialog box. An example Value is as follows:

```
$F2 @MAPKEY_LABELtest;
~ Activate `main_dlg_cur` `ProCmdModelNew.file`;
~ Activate `new` `OK`;
```

The key sequence is `$F2`. The mapkey name is `@MAPKEY_LABELtest`. The remainder of the string following the first semicolon is the macro string that should be passed to the method `IpfcBaseSession.RunMacro()`.

In this case, it is as follows:

```
~ Activate `main_dlg_cur` `ProCmdModelNew.file`;
~ Activate `new` `OK`;
```

---

#### **Note**

Creating or editing the macro string manually is not supported as the mapkeys are not a supported scripting language. The syntax is not defined for users and is not guaranteed to remain constant across different datecodes of Creo Parametric.

---

Macros are executed from synchronous mode only when control returns to Creo Parametric from the VB API program. Macros are stored in reverse order (last in, first out).

Macros are executed as soon as they are registered. Macros are run in the same order that they are saved.

## Colors and Line Styles

Methods Introduced:

- **`IpfcBaseSession.SetStdColorFromRGB()`**
- **`IpfcBaseSession.GetRGBFromStdColor()`**
- **`IpfcBaseSession.SetTextColor()`**
- **`IpfcBaseSession.SetLineStyle()`**

These methods control the general display of a Creo Parametric session.

Use the method `IpfcBaseSession.SetStdColorFromRGB()` to customize any of the Creo Parametric standard colors.

---

To change the color of any text in the window, use the method `IpfcBaseSession.SetTextColor()`.

To change the appearance of nonsolid lines (for example, datums) use the method `IpfcBaseSession.SetLineStyle()`.

## Accessing the Creo Parametric Interface

The `IpfcSession` object has methods that work with the Creo Parametric interface. These methods provide access to the message window.

## The Text Message File

A text message file is where you define strings that are displayed in the Creo Parametric user interface. This includes the strings on the command buttons that you add to the Creo Parametric number, the help string that displays when the user's cursor is positioned over such a command button, and text strings that you display in the Message Window. You have the option of including a translation for each string in the text message file.

## Restrictions on the Text Message File

You must observe the following restrictions when you name your message file:

- The name of the file must be 30 characters or less, including the extension.
- The name of the file must contain lower case characters only.
- The file extension must be three characters.
- The version number must be in the range 1 to 9999.
- All message file names must be unique, and all message key strings must be unique across all applications that run with Creo Parametric. Duplicate message file names or message key strings can cause Creo Parametric to exhibit unexpected behavior. To avoid conflicts with the names of Creo Parametric or foreign application message files or message key strings, PTC recommends that you choose a prefix unique to your application, and prepend that prefix to each message file name and each message key string corresponding to that application



### Note

Message files are loaded into Creo Parametric only once during a session. If you make a change to the message file while Creo Parametric is running you must exit and restart Creo Parametric before the change will take effect.

---

---

## Contents of the Message File

The message file consists of groups of four lines, one group for each message you want to write. The four lines are as follows:

1. A string that acts as the identifier for the message. This keyword must be unique for all Creo Parametric messages.
2. The string that will be substituted for the identifier.  
This string can include placeholders for run-time information stored in a `stringseq` object (shown in Writing Messages to the Message Window).
3. The translation of the message into another language (can be blank).
4. An intentionally blank line reserved for future extensions.

## Writing a Message Using a Message Pop-up Dialog Box

Method Introduced:

- **`IpfcSession.UIShowMessageDialog()`**

The method `IpfcSession.UIShowMessageDialog()` displays the UI message dialog. The input arguments to the method are:

- *Message*—The message text to be displayed in the dialog.
- *Options*—An instance of the `IpfcMessageDialogOptions` containing other options for the resulting displayed message. If this is not supplied, the dialog will show a default message dialog with an **Info** classification and an **OK** button. If this is not to be null, create an instance of this options type with `CCpfcMessageDialogOptions.Create()`. You can set the following options:
  - *Buttons*—Specifies an array of buttons to include in `IpfcMessageDialogOptions.Buttons` to set this option.
  - *DefaultButton*—Specifies the identifier of the default button for the dialog box. This must match one of the available buttons. Use the method `IpfcMessageDialogOptions.DefaultButton` to set this option.
  - *DialogLabel*—The text to display as the title of the dialog box. If not supplied, the label will be the English string `Info`. Use the method `IpfcMessageDialogOptions.DialogLabel` to set this option.
  - *MessageDialogType*—The type of icon to be displayed with the dialog box (**Info**, **Prompt**, **Warning**, or **Error**). If not supplied, an **Info** icon is used. Use the method `MessageDialogOptions.MessageDialogType` to set this option.



---

## Accessing the Message Window

The following sections describe how to access the message window using the VB API. The topics are as follows:

- [Writing Messages to the Message Window on page 41](#)
- [Accessing the Message Window on page 41](#)

## Writing Messages to the Message Window

Methods Introduced:

- **IpfcSession.UIDisplayMessage()**
- **IpfcSession.UIDisplayLocalizedMessage()**
- **IpfcSession.UIClearMessage()**

These methods enable you to display program information on the screen.

The input arguments to the methods `IpfcSession.UIDisplayMessage()` and `IpfcSession.UIDisplayLocalizedMessage()` include the names of the message file, a message identifier, and (optionally) a `stringseq` object that contains upto 10 pieces of run-time information. For

`IpfcSession.UIDisplayMessage()`, the strings in the `stringseq` are identified as `%0s`, `%1s`, ... `%9s` based on their location in the sequence. For `IpfcSession.UIDisplayLocalizedMessage()`, the strings in the `stringseq` are identified as `%0w`, `%1w`, ... `%9w` based on their location in the sequence. To include other types of run-time data (such as integers or reals) you must first convert the data to strings and store it in the string sequence.

## Writing Messages to an Internal Buffer

Methods Introduced:

- **IpfcBaseSession.GetMessageContents()**
- **IpfcBaseSession.GetLocalizedMessageContents()**

The methods `IpfcBaseSession.GetMessageContents()` and `IpfcBaseSession.GetLocalizedMessageContents()` enable you to write a message to an internal buffer instead of the Creo Parametric message area.

These methods take the same input arguments and perform exactly the same argument substitution and translation as the `IpfcSession.UIDisplayMessage()` and `IpfcSession.UIDisplayLocalizedMessage()` methods described in the previous section.

---

## Message Classification

Messages displayed in the VB API include a symbol that identifies the message type. Every message type is identified by a classification that begins with the characters %C. A message classification requires that the message key line (line one in the message file) must be preceded by the classification code.



### Note

Any message key string used in the code should not contain the classification.

---

the VB API applications can now display any or all of the following message symbols:

- **Prompt**—This VB API message is preceded by a green arrow. The user must respond to this message type. Responding includes, specifying input information, accepting the default value offered, or canceling the application. If no action is taken, the progress of the application is halted. A response may either be textual or a selection. The classification for Prompt messages is %CP
- **Info**—This VB API message is preceded by a blue dot. Info message types contain information such as user requests or feedback from VB API or Creo Parametric. The classification for Info messages is %CI



### Note

Do not classify messages that display information regarding problems with an operation or process as Info. These types of messages must be classified as Warnings.

---

- **Warning**—This VB API message is preceded by a triangle containing an exclamation point. Warning message types contain information to alert users to situations that could potentially lead to an error during a later stage of the process. Examples of warnings could be a process restriction or a suspected data problem. A Warning will not prevent or interrupt a process. Also, a Warning should not be used to indicate a failed operation. Warnings must only caution a user that the completed operation may not have been performed in a completely desirable way. The classification for Warning messages is %CW
- **Error**—This VB API message is preceded by a broken square. An Error message informs the user that a required task was not completed successfully. Depending on the application, a failed task may or may not require intervention or correction before work can continue. Whenever possible

---

redress this situation by providing a path. The classification for Error messages is %CE

- **Critical**—This VB API message is preceded by a red X. A Critical message type informs the user of an extremely serious situation that is usually preceded by loss of user data. Options redressing this situation, if available, should be provided within the message. The classification for a Critical messages is %CC

## Example Code: Writing a Message

The sample code in the file `pfcSessionObjectsExamples.vb` located at `<creo_vbapi_loadpoint>/vbapi_examples` demonstrates how to write a message to the message window. The program uses the message file `mymessages.txt`.

## Reading Data from the Message Window

Methods Introduced:

- **IpfcSession.UIReadIntMessage()**
- **IpfcSession.UIReadRealMessage()**
- **IpfcSession.UIReadStringMessage()**

These methods enable a program to get data from the user. The methods obtain keyboard input from a text box in the Creo Parametric user interface.



### Note

When the user presses Esc or clicks **Cancel** in the Creo Parametric user interface, these methods throw the exception `IpfcXToolkitMsgUserQuit`.

---

The `IpfcSession.UIReadIntMessage()` and `IpfcSession.UIReadRealMessage()` methods contain optional arguments that can be used to limit the value of the data to a certain range.

The method `IpfcSession.UIReadStringMessage()` includes an optional Boolean argument that specifies whether to echo characters entered onto the screen. You would use this argument when prompting a user to enter a password.



### Note

A default value is displayed in the text box as input. When user presses the Enter key as input in the user interface, the default value is not passed to the VB API method; instead, the method returns a constant string `use_default_string`. When this string is returned, the application must interpret that the user wants to use the default value.

---

## Displaying Feature Parameters

Method Introduced:

- **`IpfcSession.Session.UIDisplayFeatureParams`**

The method `IpfcSession.UIDisplayFeatureParams()` forces Creo Parametric to show dimensions or other parameters stored on a specific feature. The displayed dimensions may then be interactively selected by the user.

## File Dialogs

Methods and Properties Introduced:

- **`IpfcSession.UIOpenFile()`**
- **`CCpfcFileOpenOptions.Create()`**
- **`IpfcFileOpenOptions.FilterString`**
- **`IpfcFileOpenOptions.PreselectedItem`**
- **`IpfcFileUIOptions.DefaultPath`**
- **`IpfcFileUIOptions.DialogLabel`**
- **`IpfcFileUIOptions.Shortcuts`**
- **`CCpfcFileOpenShortcut.Create()`**
- **`IpfcFileOpenShortcut.ShortcutName`**
- **`IpfcFileOpenShortcut.ShortcutPath`**
- **`IpfcSession.UISaveFile()`**
- **`CCpfcFileSaveOptions.Create()`**
- **`IpfcSession.UISelectDirectory()`**
- **`CCpfcDirectorySelectionOptions.Create()`**
- **`IpfcBaseSession.UIRegisterFileOpen`**
- **`CCpfcFileOpenRegisterOptions.Create`**

- 
- **IpfcFileOpenRegisterOptions.FileDescription**
  - **IpfcFileOpenRegisterOptions.FileType**
  - **IpfcFileOpenRegisterListener.FileOpenAccess**
  - **IpfcFileOpenRegisterListener.OnFileOpenRegister**
  - **IpfcBaseSession.UIRegisterFileSave**
  - **CCpfcFileSaveRegisterOptions.Create**
  - **IpfcFileSaveRegisterOptions.FileDescription**
  - **IpfcFileSaveRegisterOptions.FileType**
  - **IpfcFileSaveRegisterListener.FileSaveAccess**
  - **IpfcFileSaveRegisterListener.OnFileSaveRegister**

The method `IpfcSession.UIOpenFile()` opens the relevant dialog box for browsing directories and opening files. The method lets you specify several options through the input arguments `IpfcFileOpenOptions` and `IpfcFileUIOptions`.

Use the method `CCpfcFileOpenOptions.Create()` to create a new instance of the `IpfcFileOpenOptions` object. This object contains the following options:

- **FilterString**—Specifies the filter string for the type of file accepted by the dialog box. Multiple file types should be listed with wildcards and separated by commas, for example, `*.prt, *.asm, *.txt, *.avi`, and so on. Use the property `IpfcFileOpenOptions.FilterString` to set this option.
- **PreselectedItem**—Specifies the name of an item to preselect in the dialog box. Use the property `IpfcFileOpenOptions.PreselectedItem` to set this option.

The `IpfcFileUIOptions` object contains the following options:

- **DefaultPath**—Specifies the name of the path to be opened by default in the dialog box. Use the property `IpfcFileUIOptions.DefaultPath` to set this option.
- **DialogLabel**—Specifies the title of the dialog box. Use the property `IpfcFileUIOptions.DialogLabel` to set this option.
- **Shortcuts**—Specifies an array of file shortcuts of the type `IpfcFileOpenShortcut`. Create this object using the method `CCpfcFileOpenShortcut.Create()`. This object contains the following attributes:
  - **ShortcutName**—Specifies the name of shortcut path to be made available in the dialog box.

- `ShortcutPath`—Specifies the string for the shortcut path.

Use the property `IpfcFileUIOptions.Shortcuts` to set the array of file shortcuts.

The method `IpfcSession.UIOpenFile()` returns the file selected by you. The application must use other methods or techniques to perform the desired action on the file.

The method `IpfcSession.UISaveFile()` opens the relevant dialog box for saving a file. The method accepts options similar to `IpfcSession.UIOpenFile()` through the `IpfcFileSaveOptions` and `IpfcFileUIOptions` objects. Use the method `CCpfcFileSaveOptions.Create()` to create a new instance of the `IpfcFileSaveOptions` object. When using the **Save** dialog box, you can set the name to a non-existent file. The method `IpfcSession.UISaveFile()` returns the name of the file selected by you; the application must use other methods or techniques to perform the desired action on the file.

The method `IpfcSession.UISelectDirectory()` prompts the user to select a directory using the Creo Parametric dialog box for browsing directories. The method accepts options through the `IpfcDirectorySelectionOptions` object which is similar to the `IpfcFileUIOptions` object (described for the method `IpfcSession.UIOpenFile()`). Specify the default directory path, the title of the dialog box, and a set of shortcuts to other directories to start browsing. If the default path is specified as `NULL`, the current directory is used. Use the method `CCpfcDirectorySelectionOptions.Create()` to create a new instance of the `IpfcDirectorySelectionOptions` object. The method `IpfcSession.UISelectDirectory()` returns the selected directory path; the application must use other methods or techniques to perform other relevant tasks with this selected path.

The method `IpfcBaseSession.UIRegisterFileOpen()` registers a new file type in the **File ► Open** dialog box in Creo Parametric. This method takes the `IpfcFileOpenRegisterOptions` and `IpfcFileOpenRegisterListener` objects as its input arguments. These objects are as follows:

- `IpfcFileOpenRegisterOptions`—This object contains the options for registering an open operation. Use the method `CCpfcFileOpenRegisterOptions.Create` to create a new instance of the object. It contains the following options:
  - `FileDescription`—Specifies the short description of the file type to be opened. This description appears for the file type in the **File ► Open** dialog box. Use the property `IpfcFileOpenRegisterOptions.FileDescription` to modify this option.

- 
- **FileType**—Specifies the file type to be opened. The file type appears as the file extension in the **File ► Open** dialog box. Use the property `IpfcFileOpenRegisterOptions.FileType` to modify this option.
  - **IpfcFileOpenRegisterListener**—This object provides the action listener methods for the new file type to be registered. The method `Ipfc.FileOpenRegisterListener.FileOpenAccess` is called to determine whether the new file type can be opened using the **File ► Open** dialog box. The method `IpfcFileOpenRegisterListener.OnFileOpenRegister` is called on clicking **Open** for the newly registered file type.

The method `IpfcBaseSession.UIRegisterFileSave()` registers a new file type in the **File ► Save a Copy** dialog box in Creo Parametric. This method takes the `IpfcFileSaveRegisterOptions` and `IpfcFileSaveRegisterListener` objects as its input arguments. These objects are described as follows:

- **IpfcFileSaveRegisterOptions**—This object contains the options for registering a save operation. Use the method `CCpfcFileSaveRegisterOptions_Create()` to create a new instance of the object. It contains the following options:
  - **FileDescription**—Specifies the short description of the file type to be saved. This description appears for the file type in the **File ► Save a Copy** dialog box. Use the property `IpfcFileSaveRegisterOptions.FileDescription` to modify this option.
  - **FileType**—Specifies the file type to be saved. The file type appears as the file extension in the **File ► Save a Copy** dialog box. Use the property `IpfcFileSaveRegisterOptions.FileType` to modify this option.
- **IpfcFileSaveRegisterListener**—This object provides the action listener methods for the new file type to be registered. The method `IpfcFileSaveRegisterListener.FileSaveAccess()` is called to determine whether the new file type can be saved using the **File ► Save a Copy** dialog box. The method `IpfcFileSaveRegisterListener.OnFileSaveRegister` is called on clicking **OK** for the newly registered file type.

---

## Customizing the Creo Parametric Navigation Area

The Creo Parametric navigation area includes the Model and Layer Tree pane, Folder browser pane, and Favorites pane. The methods described in this section enable the VB API applications to add custom panes that contain Web pages to the Creo Parametric navigation area.

### Adding Custom Web Pages

To add custom Web pages to the navigation area, the the VB API application must:

1. Add a new pane to the navigation area.
2. Set an icon for this pane.
3. Set the URL of the location that will be displayed in the pane.

Methods Introduced:

- **IpfcSession.NavigatorPaneBrowserAdd()**
- **IpfcSession.NavigatorPaneBrowserIconSet()**
- **IpfcSession.NavigatorPaneBrowserURLSet()**

The method `IpfcSession.NavigatorPaneBrowserAdd()` adds a new pane that can display a Web page to the navigation area. The input parameters are:

- *PaneName*—Specify a unique name for the pane. Use this name in subsequent calls to `IpfcSession.NavigatorPaneBrowserIconSet()` and `IpfcSession.NavigatorPaneBrowserURLSet()`.
- *IconFileName*—Specify the name of the icon file, including the extension. A valid format for the icon file is the PTC-proprietary format used by Creo Parametric .BIF, .GIF, .JPG, or .PNG. The new pane is displayed with the icon image. If you specify the value as NULL, the default Creo Parametric icon is used.

The default search paths for finding the icons are:

- `<creo_loadpoint>\<datecode>\Common Files\text\resource`
- `<Application text dir>\resource`
- `<Application text dir>\<language>\resource`

The location of the application text directory is specified in the registry file.

- *URL*—Specify the URL of the location to be accessed from the pane.

Use the method `IpfcSession.NavigatorPaneBrowserIconSet()` to set or change the icon of a specified browser pane in the navigation area.



---

Use the method `IpfcSession.NavigatorPaneBrowserURLSet()` to change the URL of the page displayed in the browser pane in the navigation area.

## Example: Customizing the Creo Parametric Navigation Pane

The sample code in the file `pfcNavigatorPane.vb` located at `<creo_vbapi_loadpoint>/vbapi_examples` shows you how to customize the Creo Parametric navigation pane.

# Selection

## Selection

This section describes how to use Interactive Selection in the VB API.

## Interactive Selection

Methods and Properties Introduced:

- **`IpfcBaseSession.Select()`**
- **`CCpfcSelectionOptions.Create()`**
- **`IpfcSelectionOptions.MaxNumSels`**
- **`IpfcSelectionOptions.OptionKeywords`**

The method `IpfcBaseSession.Select()` activates the standard Creo Parametric menu structure for selecting objects and returns a `IpfcSelection` sequence that contains the objects the user selected. Using the *Options* argument, you can control the type of object that can be selected and the maximum number of selections.

In addition, you can pass in a `IpfcSelection` sequence to the method. The returned `IpfcSelection` sequence will contain the input sequence and any new objects.

The method `CCpfcSelectionOptions.Create()` and the property `IpfcSelectionOptions.OptionKeywords` take a *String* argument made up of one or more of the identifiers listed in the table below, separated by commas.

For example, to allow the selection of features and axes, the arguments would be *feature, axis*.

| Creo Parametric Database Item | String Identifier | ModelItemType               |
|-------------------------------|-------------------|-----------------------------|
| Datum point                   | <i>point</i>      | <code>EpfcITEM_POINT</code> |
| Datum axis                    | <i>axis</i>       | <code>EpfcITEM_AXIS</code>  |

| Creo Parametric Database Item | String Identifier | ModelItemType                          |
|-------------------------------|-------------------|----------------------------------------|
| Datum plane                   | <i>datum</i>      | EpfcITEM_SURFACE                       |
| Coordinate system datum       | <i>csys</i>       | EpfcITEM_COORD_SYS                     |
| Feature                       | <i>feature</i>    | EpfcITEM_FEATURE                       |
| Edge (solid or datum surface) | <i>edge</i>       | EpfcITEM_EDGE                          |
| Edge (solid only)             | <i>slledge</i>    | EpfcITEM_EDGE                          |
| Edge (datum surface only)     | <i>qlledge</i>    | EpfcITEM_EDGE                          |
| Datum curve                   | <i>curve</i>      | EpfcITEM_CURVE                         |
| Composite curve               | <i>comp_crv</i>   | EpfcITEM_CURVE                         |
| Surface (solid or quilt)      | <i>surface</i>    | EpfcITEM_SURFACE                       |
| Surface (solid)               | <i>sldface</i>    | EpfcITEM_SURFACE                       |
| Surface (datum surface)       | <i>qltface</i>    | EpfcITEM_SURFACE                       |
| Quilt                         | <i>dtmqlt</i>     | ITEM_QUILT                             |
| Dimension                     | <i>dimension</i>  | EpfcITEM_DIMENSION                     |
| Reference dimension           | <i>ref_dim</i>    | EpfcITEM_REF_DIMENSION                 |
| Integer parameter             | <i>ipar</i>       | EpfcITEM_DIMENSION                     |
| Part                          | <i>part</i>       | N/A                                    |
| Part or subassembly           | <i>prt_or_asm</i> | N/A                                    |
| Assembly component model      | <i>component</i>  | N/A                                    |
| Component or feature          | <i>membfeat</i>   | EpfcITEM_FEATURE                       |
| Detail symbol                 | <i>dtl_symbol</i> | EpfcITEM_DTL_SYM_INSTANCE              |
| Note                          | <i>any_note</i>   | EpfcITEM_NOTE, EpfcITEM_DTL_NOTE       |
| Draft entity                  | <i>draft_ent</i>  | EpfcITEM_DTL_ENTITY                    |
| Table                         | <i>dwg_table</i>  | EpfcITEM_TABLE                         |
| Table cell                    | <i>table_cell</i> | EpfcITEM_TABLE                         |
| Drawing view                  | <i>dwg_view</i>   | N/A                                    |
| Solid body                    | <i>3d_body</i>    | EpfcITEM_BODY                          |
| Datum Curve End               | <i>curve_end</i>  | EpfcITEM_CRV_START or EpfcITEM_CRV_END |

When you specify the maximum number of selections, the argument to `IpfcSelectionOptions.MaxNumSels` must be an Integer.

The default value assigned when creating a `IpfcSelectionOptions` object is -1, which allows any number of selections by the user.

## Accessing Selection Data

Properties Introduced:

- **IpfcSelection.SelModel**
- **IpfcSelection.SelItem**

- 
- **IpfcSelection.Path**
  - **IpfcSelection.Params**
  - **IpfcSelection.TParam**
  - **IpfcSelection.Point**
  - **IpfcSelection.Depth**
  - **IpfcSelection.SelView2D**
  - **IpfcSelection.SelTableCell**
  - **IpfcSelection.SelTableSegment**

These properties return objects and data that make up the selection object. Using the appropriate properties, you can access the following data:

- For a selected model or model item use `IpfcSelection.SelModel` or `IpfcSelection.SelItem`.
- For an assembly component use `IpfcSelection.Path`.
- For UV parameters of the selection point on a surface use `IpfcSelection.Params`.
- For the T parameter of the selection point on an edge or curve use `IpfcSelection.TParam`.
- For a three-dimensional point object that contains the selected point use `IpfcSelection.Point`.
- For selection depth, in screen coordinates use `IpfcSelection.Depth`.
- For the selected drawing view, if the selection was from a drawing, use `IpfcSelection.SelView2D`.
- For the selected table cell, if the selection was from a table, use `IpfcSelection.SelTableCell`.
- For the selected table segment, if the selection was from a table, use `IpfcSelection.SelTableSegment`.

## Controlling Selection Display

Methods Introduced:

- **IpfcSelection.Highlight()**
- **IpfcSelection.UnHighlight()**
- **IpfcSelection.Display()**

These methods cause a specific selection to be highlighted or dimmed on the screen using the color specified as an argument.

---

The method `IpfcSelection.Highlight()` highlights the selection in the current window. This highlight is the same as the one used by Creo Parametric when selecting an item—it just repaints the wire-frame display in the new color. The highlight is removed if you use the **View, Repaint** command or `IpfcWindow.Repaint()`; it is not removed if you use `IpfcWindow.Refresh()`.

The method `IpfcSelection.UnHighlight()` removes the highlight.

The method `IpfcSelection.Display()` causes a selected object to be displayed on the screen, even if it is suppressed or hidden.



#### Note

This is a one-time action and the next repaint will erase this display.

---

## Example Code: Using Interactive Selection

The sample code in the file `pfcSelectionExamples.vb` located at `<creo_vbapi_loadpoint>/vbapi_examples` demonstrates how to use the VB API to allow interactive selection.

## Programmatic Selection

The VB API provides methods whereby you can make your own Selection objects, without prompting the user. These Selections are required as inputs to some methods and can also be used to highlight certain objects on the screen.

Methods and Properties Introduced:

- **CMpfcSelect.CreateModelItemSelection()**
- **CMpfcSelect.CreateComponentSelection()**
- **CMpfcSelect.CreateModelSelection()**
- **IpfcSelection.SelItem**
- **IpfcSelection.SelTableCell**
- **IpfcSelection.SetSelView2D**

The method `pCMpfcSelect.CreateModelItemSelection()` creates a selection out of any model item object. It takes a `IpfcModelItem` and optionally a `IpfcComponentPath` object to identify which component in an assembly the Selection Object belongs to.

---

The method `CMpfcSelect.CreateComponentSelection()` creates a selection out of any component in an assembly. It takes a `IpfcComponentPath` object. For more information about `IpfcComponentPath` objects, see the section [Getting a Solid Object on page 138](#) in the [Solid on page 138](#) section.

Use the method `CMpfcSelect.CreateModelSelection()` to create a `IpfcSelection` object, based on a `IpfcModel` object.

Some VB API properties require more information to be set in the selection object. The methods allow you to set the following:

The selected item using the method `IpfcSelection.SetSelItem`.

The selected table cell using the method `IpfcSelection.SetSelTableCell`.

The selected drawing view using the method `IpfcSelection.SetSelView2D`.

## Selection Buffer

### Introduction to Selection Buffers

Selection is the process of choosing items on which you want to perform an operation. In Creo Parametric, before a feature tool is invoked, the user can select items to be used in a given tool's collectors. Collectors are like storage bins of the references of selected items. The location where preselected items are stored is called the selection buffer.

Depending on the situation, different selection buffers may be active at any one time. In Part and Assembly mode, Creo Parametric offers the default selection buffer, the Edit selection buffer, and other more specialized buffers. Other Creo Parametric modes offer different selection buffers.

In the default Part and Assembly buffer there are two levels at which selection is done:

- **First Level Selection**  
Provides access to higher-level objects such as features or components. You can make a second level selection only after you select the higher-level object.
- **Second Level Selection**  
Provides access to geometric objects such as edges and faces.



## Note

First-level and second-level objects are usually incompatible in the selection buffer.

---

The VB API allows access to the contents of the currently active selection buffer. The available functions allow your application to:

- Get the contents of the active selection buffer.
- Remove the contents of the active selection buffer.
- Add to the contents of the active selection buffer.

## Reading the Contents of the Selection Buffer

Properties Introduced:

- **IpfcSession.CurrentSelectionBuffer**
- **IpfcSelectionBuffer.Contents**

The property `IpfcSession.CurrentSelectionBuffer` returns the selection buffer object for the current active model in session. The selection buffer contains the items preselected by the user to be used by the selection tool and popup menus.

Use the property `IpfcSelectionBuffer.Contents` to access the contents of the current selection buffer. The method returns independent copies of the selections in the selection buffer (if the buffer is cleared, this array is still valid).

## Removing the Items of the Selection Buffer

Methods Introduced:

- **IpfcSelectionBuffer.RemoveSelection()**
- **IpfcSelectionBuffer.Clear()**

Use the method `IpfcSelectionBuffer.RemoveSelection()` to remove a specific selection from the selection buffer. The input argument is the *IndexToRemove* specifies the index where the item was found in the call to the property `IpfcSelectionBuffer.Contents`.

Use the method `IpfcSelectionBuffer.Clear()` to clear the currently active selection buffer of all contents. After the buffer is cleared, all contents are lost.

---

## Adding Items to the Selection Buffer

Method Introduced:

- **IpfcSelectionBuffer.AddSelection()**

Use the method `IpfcSelectionBuffer.AddSelection()` to add an item to the currently active selection buffer.



### Note

The selected item must refer to an item that is in the current model such as its owner, component path or drawing view.

---

This method may fail due to any of the following reasons:

- There is no current selection buffer active.
- The selection does not refer to the current model.
- The item is not currently displayed and so cannot be added to the buffer.
- The selection cannot be added to the buffer in combination with one or more objects that are already in the buffer. For example: geometry and features cannot be selected in the default buffer at the same time.

## Ribbon Tabs, Groups, and Menu Items

### Ribbon Tabs, Groups, and Menu Items

This section describes the VB API support for the Ribbon User Interface (UI). It also describes the impact of the ribbon user interface on legacy VB API applications and the procedure to place the commands, buttons, and menu items created by the legacy applications in the Creo Parametric ribbon user interface. Refer to the Creo Parametric Help for more information on the ribbon user interface and the procedure to customize the ribbon.

### Creating Ribbon Tabs, Groups, and Menu Items

Customizations to the ribbon user interface using the VB API applications are supported through the **Customize Ribbon** tab in the Creo Parametric **Creo parametric Options** dialog box. You can specify the user interface layout for a VB API application and save the layout definition in a ribbon definition file, `toolkitribbonui.rbn`. Set the configuration option `tk_enable_ribbon_custom_save` to `yes` before customizing the ribbon user interface using the the VB API application. When you run Creo Parametric, the

---

`toolkitribbonui.rbn` file is loaded along with the VB API application and the commands created by the VB API application appear in the ribbon user interface. Refer to the section [About the Ribbon Definition File on page 57](#) for more information on the `toolkitribbonui.rbn` file.

You can customize the ribbon user interface only for a particular mode in Creo Parametric. For example, if you customize the ribbon user interface and save it to the `toolkitribbonui.rbn` file in the Part mode, then on loading Creo Parametric the customized user interface will be visible only in the Part mode. To view a particular tab or group in all the modes, you must customize the ribbon user interface and save the `toolkitribbonui.rbn` file in each mode. Refer to the Creo Parametric Fundamentals Help for more information on customizing the ribbon.

---

 **Note**

You can add a new group to an existing tab or create a new tab using the **Customize Ribbon** tab in the **Creo Parametric Options** dialog box. You will not be able to modify the tabs or groups that are defined by Creo Parametric.

---

## Workflow to Add Menu Items to the Ribbon User Interface

---

 **Note**

The instructions explained below are applicable only if the application is implemented in full asynchronous mode. This is because applications in simple asynchronous mode cannot handle requests, that is, command callbacks, from Creo Parametric. Refer to the section on Asynchronous Mode, for more information.

---

Set the configuration option `tk_enable_ribbon_custom_save` to `yes` before customizing the ribbon user interface. The steps to add commands to the Creo Parametric ribbon user interface are as follows:

1. Create a VB API application with complete command definition, which includes specifying command label, help text, large icon name, and small icon name. Designate the command using `IpfcUICommand.Designate()`.
2. Start the VB API application and ensure that it has started or connected to Creo Parametric. The commands created by the VB API application will be loaded in Creo Parametric.
3. Click **File ► Options**. The **Creo Parametric Options** dialog box opens.
4. Click **Customize Ribbon**.



- 
5. In the **Customize the Ribbon** list, select a tab and create a new group in it or create a new tab and a group in it.
  6. In the **Choose commands from** list, select **TOOLKIT Commands**. The commands created by the VB API application are displayed.
  7. Click **Add** to add the commands to the new tab or group.
  8. Click **Import/Export ► Save the Auxilliary Application User Interface**. The changes are saved to the `toolkitribbonui.rbn` file. The `toolkitribbonui.rbn` file is saved in the text folder specified in the VB API application registry file. For more information refer to the section on [About the Ribbon Definition File on page 57](#).

---

 **Note**

The **Save the Auxilliary Application User Interface** button is enabled only if you set the configuration option `tk_enable_ribbon_custom_save` to `yes`.

---

9. Click **Apply**. The custom settings are saved to the `toolkitribbonui.rbn` file.
10. Reload the VB API application or restart Creo Parametric. The `toolkitribbonui.rbn` file will be loaded along with the VB API application.

If translated messages are available for the newly added tabs or groups, then Creo Parametric displays the translated strings by searching for the same string from the list of string based messages that are loaded. For more information refer to the section on [Localizing the Ribbon User Interface Created by the VB API Applications on page 61](#).

## About the Ribbon Definition File

A ribbon definition file is a file that is created through the **Customize Ribbon** interface in Creo Parametric. This file defines the containers, that is, Tabs, Group, or Cascade menus that are created by a particular VB API application. It contains information on whether to show an icon or label. It also contains the size of the icon to be used, that is, a large icon (32X32) or a small icon (16x16).

The ribbon user interface displays the commands referenced in the ribbon definition file only if the commands are loaded and are visible in that particular Creo Parametric mode. If translated messages are available for the newly added tabs or groups, then Creo Parametric displays the translated strings by searching for the same string from the list of string based messages that are loaded. For more information refer to the section on [Localizing the Ribbon User Interface Created by the VB API Applications on page 61](#).

---

Set the configuration option `tk_enable_ribbon_custom_save` to `yes` before customizing the ribbon user interface. To save the ribbon user interface layout definition to the `toolkitribbonui.rbn` file:

1. Click **File ► Options**. The **Creo Parametric Options** dialog box opens.
2. Click **Customize Ribbon**.
3. In the **Customize the Ribbon** list, select a tab and create a new group in it or create a new tab and a group in it.
4. In the **Choose commands from** list, select **TOOLKIT Commands**. The commands created by the VB API application are displayed.
5. Click **Add** to add the commands to the new tab or group.
6. Click **Import/Export ► Save the Auxilliary Application User Interface**. The modified layout is saved to the `toolkitribbonui.rbn` file located in the text folder within the VB API application directory, that is, `<application_dir>/text`

---

 **Note**

The **Save the Auxilliary Application User Interface** button is enabled only if you set the configuration option `tk_enable_ribbon_custom_save` to `yes`.

---

7. Click **OK**.

---

 **Note**

You cannot edit the `toolkitribbonui.rbn` file manually.

---

## To Specify the Path for the Ribbon Definition File

You can rename the `toolkitribbonui.rbn` to another filename with the `.rbn` extension. To enable the VB API application to read the ribbon definition file having a name other than `toolkitribbonui.rbn`, it must be available at the location `<application_dir>/text/ribbon`. The method introduced in this section enables you to load the ribbon definition file from within a VB API application.

method Introduced:

- **IpfcSession.RibbonDefinitionfileLoad**

---

The method `IpfcSession.RibbonDefinitionfileLoad()` loads a specified ribbon definition file from a default path into the Creo Parametric application. The input argument is as follows:

- *file\_name* - Specify the name of the ribbon definition file including its extension. The default search path for this file is:
  - The working directory from where Creo Parametric is loaded.
  - `<application_text_dir>/ribbon`
  - `<application_text_dir>/language/ribbon`

---

 **Note**

- ◆ The location of the application text directory is specified in the VB API registry file.
  - ◆ A VB API application can load a ribbon definition file only once. After the application has loaded the ribbon, calls made to the method `IpfcSession.RibbonDefinitionfileLoad()` to load other ribbon definition files are ignored.
- 

## Loading Multiple Applications Using the Ribbon Definition File

Creo Parametric supports loading of multiple `.rbn` files in the same session. You can develop multiple VB API applications that share the same tabs or groups and each application will have its own ribbon definition file. As each application is loaded, its `.rbn` file will be read and applied. When an application is unloaded, the containers and command created by its `.rbn` file will be removed.

For example, consider two VB API applications, namely, `pt_geardesign` and `pt_examples` that add commands to the same group on a tab on the Ribbon user interface. The application `pt_geardesign` adds a command **Pro/TOOLKIT Gear Design** to the **Advanced Modeling** group on the **Modeling** tab and the application `pt_examples` adds a command **TKPart** to the **Advanced Modeling** group on the **Model** tab. The ribbon definition file for each application will contain an instruction to create the **Advanced Modeling** group and if both the ribbon files are loaded, the group will be created only once and the two ribbon customizations will be merged into the same group.

That is, if both the applications are running in the same Creo Parametric session, then the commands, **Pro/TOOLKIT Gear Design** and **TKPart** will be available under the **Advanced Modeling** group on the **Model** tab.

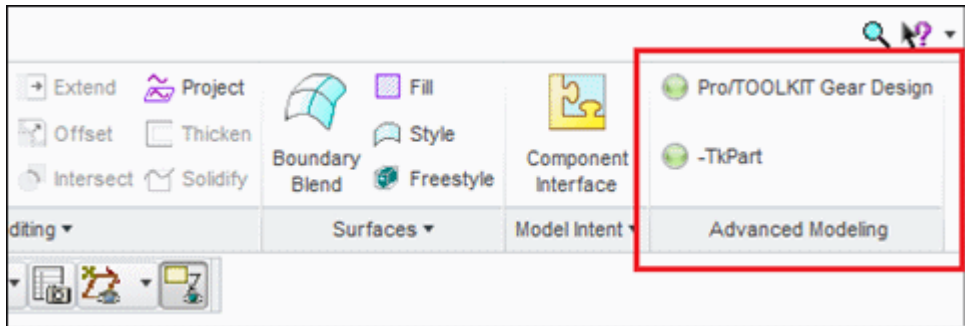
---

 **Note**

The order in which the commands will be displayed within the group will depend on the order of loading of the .rbn file for each application.

---

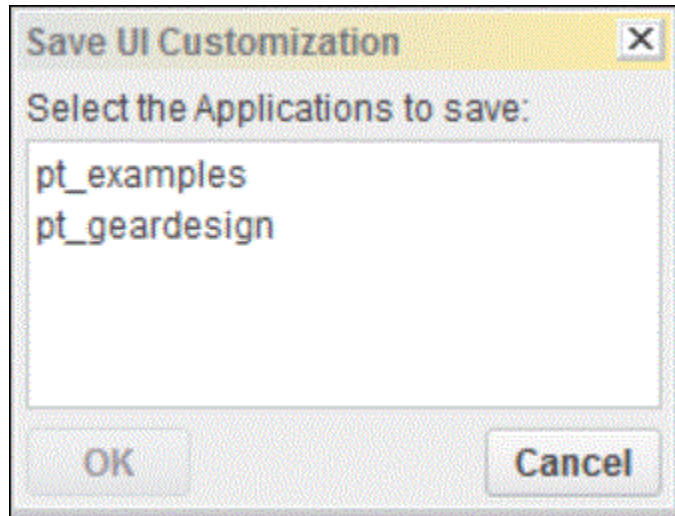
The following image displays commands added by two VB API applications to the same group.



To save the customization when multiple applications are loaded:

1. Click **File ► Options**. The **Creo Parametric Options** dialog box opens.
2. Click **Customize Ribbon**.
3. In the **Customize the Ribbon** list, select a tab and create a new group in it or create a new tab and a group in it.
4. In the **Choose commands from list**, select **TOOLKIT Commands**. The commands created by the VB API application are displayed.
5. Click **Add** to add the commands to the new tab or group.
6. Click **Import/Export ► Save the Auxilliary Application User Interface**. The **Save UI Customization** dialog box opens.

7. Select a VB API application and Click **Save**. The modified layout is saved to the `.rbn` file of the specified VB API application. The **Save UI Customization** dialog box is shown in the following image:



## Localizing the Ribbon User Interface Created by the VB API Applications

The labels for the custom tabs, groups, and cascade menus belonging to the VB API application can be translated in the languages supported by Creo Parametric. To display localized labels, specify the translated labels in the `ribbonui.txt` file and save this file at the location `<application_text_dir>/<language>`. For example, the text file for German locale must be saved at the location `<application_text_dir>/german/ribbonui.txt`.

Create a file containing translations for each of the languages in which the VB API application is localized. The Localized translation files must use the UTF-8 encoding with BOM character for the translated text to be displayed correctly in the user interface.

The format of the `ribbonui.txt` file is as shown below. Specify the following lines for each label entry in the file:

1. A hash sign (#) followed by the label, as specified in the ribbon definition file.
2. The label as specified in the ribbon definition file and as displayed in the ribbon user interface.
3. The translated label.
4. Add an empty line at the end of each label entry in the file.

---

For example, if the Creo Parametric application creates a tab with the name TK\_TAB having a group with the name TK\_GROUP, then the translated file will contain the following:

```
#TK_TAB
TK_TAB
<translation for TK_TAB>
<Empty_line>
#TK_GROUP
TK_GROUP
<translation for TK_GROUP>
<Empty_line>
```

## Support for Legacy the VB API Applications

The user interface for Creo Parametric 1.0 has been restructured to a ribbon user interface. This may affect the behavior of existing the VB API applications that were designed to add commands to specific Pro/ENGINEER menus or toolbars. These menus or toolbars or both have been redesigned in Creo Parametric 1.0. The commands added by the the VB API applications appear on the Creo Parametric ribbon in the **Home** tab under the **Pro/TK** group. When you open a model, the **Pro/TK** group is on the **Tools** tab.

You can also arrange the commands added by the the VB API applications under a new tab or an existing tab by customizing the ribbon using the **Customize Ribbon** tab in the **Creo Parametric Options** dialog box. For a list of all the commands added by the VB API applications, follow this procedure:

1. Click **File ► Options**. The **Creo Parametric Options** dialog box opens.
2. Click **Customize Ribbon**.
3. In the **Choose commands from list**, select **TOOLKIT Commands**. All the commands added by legacy applications are listed.



### Note

Commands that have not been designated will not have an icon or will have a generic icon.

---

Refer to the Creo Parametric Help for more information on customizing the Ribbon.

---

## Migration of Legacy VB API Applications

To migrate existing the VB API applications to the Creo Parametric Ribbon user interface without compiling the source code:

1. Load the the VB API applications in Creo Parametric so that the commands created in these applications are available in the **Customize Ribbon** user interface.
2. Modify the ribbon user interface layout and save the changes to the `toolkitribbonui.rbn`.
3. Copy the `toolkitribbonui.rbn` to the location `<application_text_dir>/ribbon`.
4. Reload the VB API application or restart Creo Parametric. The `.rbn` file will be loaded along with the VB API application and the commands will be visible in the ribbon user interface if its accessibility will be visible.

## Menus, Commands, and Pop-up Menus

### Menus, Commands, and Pop-up Menus

This section describes the methods provided by the VB API to create and modify menus, buttons, and pop-up menus in the Creo Parametric user interface.

Refer to the section [Ribbon Tabs, Groups, and Menu Items on page 55](#) for more information. Also, refer to the Creo Parametric Help for more information on customizing the ribbon user interface.

### Introduction

The VB API classes enable you to supplement the Creo Parametric ribbon user interface.

Once the VB API application is loaded, you can add a new group to an existing tab or create a new tab using the **Customize Ribbon** tab in the **Creo Parametric Options** dialog box in Creo Parametric. You will not be able to modify the groups that are defined by Creo Parametric. If the translated messages are available for the newly added tabs or groups, then Creo Parametric will use them by searching for the same string in the list of sting based messages loaded.

You can customize the ribbon user interface only for a particular mode in Creo Parametric. For example, if you customize the ribbon user interface and save it to the `toolkitribbonui.rbn` file in the Part mode, then on loading Creo Parametric the customized user interface will be visible only in the Part mode. To view a particular tab or group in all the modes, you must customize the ribbon user interface and save the `toolkitribbonui.rbn` file in each mode. Refer to the Creo Parametric Fundamentals Help for more information on customizing the ribbon.

---

## Menu Bar Definitions

- **Menu**—A menu, such as the **File** menu, or a sub-menu, such as the **Manage File** menu under the **File** menu.
- **Menu button**—A named item in a group or menu that is used to launch a set of instructions.
- **Pop-up menu**—A menu invoked by selection of an item in the Creo Parametric graphics window.
- **Command**—A procedure in Creo Parametric that may be activated from a button.

## Menus Buttons and Menus

The following methods enable you to add new menu buttons in any location on the Ribbon user interface.

Methods Introduced:

- **`IpfcSession.UICreateCommand()`**
- **`IpfcSession.UICreateMaxPriorityCommand()`**
- **`IpfcUICommandActionListener.OnCommand()`**

The method `IpfcSession.UICreateCommand()` creates a `IpfcUICommand` object that contains a `IpfcUICommandActionListener`. You should override the `IpfcUICommandActionListener.OnCommand()` method with the code that you want to execute when the user clicks a button.

The method `IpfcSession.UICreateMaxPriorityCommand()` creates a `IpfcUICommand` object having maximum command priority. The priority of the action refers to the level of precedence the added action takes over other Creo Parametric actions. Maximum priority actions dismiss all other actions except asynchronous actions.

Maximum command priority should be used only in commands that open and activate a new model in a window. Create all other commands using the method `IpfcSession.UICreateCommand()`.

The listener method `IpfcUICommandListener.OnCommand()` is called when the command is activated in Creo Parametric by pressing a button.

Designate the command using the function `IpfcUICommand.Designate()` and add a button to the ribbon user interface using the **Customize Ribbon** tab in the **Creo Parametric Options** dialog box. This operation binds the command to the button.



---

## Finding Creo Parametric Commands

This method enables you to find existing Creo Parametric commands in order to modify their behavior.

Method Introduced:

- **IpfcSession.UIGetCommand()**

The method `IpfcSession.UIGetCommand()` returns a `IpfcUICommand` object representing an existing Creo Parametric command. The method allows you to find the command ID for an existing command so that you can add an access function or bracket function to the command. You must know the name of the command in order to find its ID.

To find the name of an action command, click the corresponding icon on the ribbon user interface and then check the last entry in the trail file. For example, for the Save icon, the trail file will have the corresponding entry:

```
~ Command `ProCmdModelSave`
```

The action name for the Save icon is `ProCmdModelSave`. This string can be used as input to `IpfcSession.UIGetCommand()` to get the command ID.

You can determine a command ID string for an option without an icon by searching through the resource files located in the `<creo_loadpoint>\<datecode>\Common Files\text\resource` directory. If you search for the menu button name, the line will contain the corresponding action command for the button.

## Access Listeners for Commands

These methods allow you to apply an access listener to a command. The access listener determines whether or not the command is visible at the current time in the current session.

Methods Introduced:

- **IpfcActionSource.AddActionListener()**
- **IpfcUICommandAccessListener.OnCommandAccess()**

Use the method `IpfcActionSource.AddActionListener()` to register a new `IpfcUICommandAccessListener` on any command (created either by an application or Creo Parametric). This listener will be called when buttons based on the command might be shown.

The listener method

`IpfcUICommandAccessListener.OnCommandAccess()` allows you to impose an access function on a particular command. The method determines if the action or command should be available, unavailable, or hidden.

The potential return values are listed in the enumerated type `EpfcCommandAccess` and are as follows:

- `EpfcACCESS_REMOVE`—The button is not visible and if all of the menu buttons in the containing menu possess an access function returning `EpfcACCESS_REMOVE`, the containing menu will also be removed from the Creo Parametric user interface..
- `EpfcACCESS_INVISIBLE`—The button is not visible.
- `EpfcACCESS_UNAVAILABLE`—The button is visible, but gray and cannot be selected.
- `EpfcACCESS_DISALLOW`—The button shows as available, but the command will not be executed when it is chosen.
- `EpfcACCESS_AVAILABLE`—The button is not gray and can be selected by the user. This is the default value.

## Example 1: Command Access Listeners

The sample code in the file `pfcCommandExamples.vb` <creo\_vbapi\_loadpoint>/vbapi\_examples demonstrates the usage of the access listener method for a particular command. The `OnCommandAccess` function returns `EpfcACCESS_UNAVAILABLE` that disables button associated with the command, if the model is not present or if it is not of type PART. Else, the function returns `EpfcACCESS_AVAILABLE` that enables button associated with the command.

## Bracket Listeners for Commands

These methods allow you to apply a bracket listener to a command. The bracket listener is called before and after the command runs, which allows your application to provide custom logic to execute whenever the command is selected.

Methods Introduced:

- **`IpfcActionSource.AddActionListener()`**
- **`IpfcUICommandBracketListener.OnBeforeCommand()`**
- **`IpfcUICommandBracketListener.OnAfterCommand()`**

Use the method `IpfcActionSource.AddActionListener()` to register a new `IpfcUICommandBracketListener` on any command (created either by an application or Creo Parametric. This listener will be called when the command is selected by the user.

The listener methods

`IpfcUICommandBracketListener.OnBeforeCommand()` and `IpfcUICommandBracketListener.OnAfterCommand()` allow the creation of functions that will be called immediately before and after execution of a given command. These methods are used to add the business logic to the start or end (or both) of an existing Creo Parametric command.

---

The method `IpfcUICommandBracketListener.OnBeforeCommand()` could also be used to cancel an upcoming command. To do this, throw a `IpfcXCancelProEAction` exception from the body of the listener method using `CCpfcXCancelProEAction.Throw()`.

## Example 2: Bracket Listeners

The sample code in the file `pfcCommandExamples.vb` located at `<creo_vbapi_loadpoint>/vbapi_examples` demonstrates the usage of the bracket listener methods that are called before and after when the user tries to rename the model. If the model contains a certain parameter, the rename attempt will be aborted by this listener.

## Designating Commands

Using the VB API you can designate Creo Parametric commands. These commands can later appear in the Creo Parametric ribbon user interface.

To add a command you must:

1. Define or add the command to be initiated on clicking the icon in the VB API application.
2. Optionally designate an icon button to be used with the command.
3. Designate the command to appear in the **Customize Ribbon** tab in the **Creo Parametric Options** dialog box.

---

### Note

Refer to the section on [Ribbon Tabs, Groups, and Menu Items](#) on page 55 for more information. Also, refer to the Creo Parametric Parametric Help for more information on customizing the Ribbon User Interface.

---

4. Save the configuration in Creo Parametric so that changes to the ribbon user interface appear when a new session of Creo Parametric is started.

## Command Icons

Method Introduced:

- **IpfcUICommand.SetIcon()**

The method `IpfcUICommand.SetIcon()` allows you to designate an icon to be used with the command you created. The method adds the icon to the Creo Parametric command. Specify the name of the icon file, including the extension as the input argument for this method. A valid format for the icon file is a standard

---

.GIF, .JPG, or .PNG. PTC recommends using .PNG format. All icons in the Creo Parametric ribbon are either 16x16 (small) or 32x32 (large) size. The naming convention for the icons is as follows:

- Small icon—<iconname.ext>
- Large icon—<iconname\_large.ext>

---

 **Note**

- The legacy naming convention for icons <icon\_name\_icon\_size.ext> will not be supported in future releases of Creo Parametric. The icon size was added as a suffix to the name of the icon. For example, the legacy naming convention for small icons was iconname16X16.ext. It is recommended to use the standard naming conventions for icons, that is, iconname.ext or iconname\_large.ext.
  - While specifying the name of the icon file, do not specify the full path to the icon names.
- 

The application searches for the icon files in the following locations:

- <creo\_loadpoint>\<datecode>\Common Files\text\resource
- <Application text dir>\resource
- <Application text dir>\<language>\resource

The location of the application text directory is specified in the registry file.

Commands that do not have an icon assigned to them display the button label.

You may also use this method to assign a small icon to a button. The icon appears to the left of the button label.

## Designating the Command

Method Introduced:

- **IpfcUICommand.Designate()**

This method allows you designate the command as available in the **Customize Ribbon** tab in the **Creo Parametric Options** dialog of Creo Parametric. After a VB API application has used the method `IpfcUICommand.Designate()` on a command, can add the button associated with this command into the Creo Parametric ribbon user interface.

If this method is not called, the button will not be visible in the **Toolkit Commands** list in the **Customize Ribbon** tab in the **Creo Parametric Options** dialog of Creo Parametric.

---

The arguments to this method are:

- *Label*—The message string that refers to the icon label. This label (stored in the message file) identifies the text seen when the button is displayed. If the command is not assigned an icon, the button label string appears on the toolbar button by default.
- *Help*—The one-line Help for the icon. This label (stored in the message file) identifies the help line seen when the mouse moves over the icon.
- *Description*—The message appears in the **Customize Ribbon** tab in the **Creo Parametric Options** dialog box and also when **Description** is clicked in Creo Parametric.
- *MessageFile*—The message file name. All the labels including the one-line Help labels must be present in the message file.

---

 **Note**

This file must be in the directory <text\_path>/text or <text\_path>/text/<language>.

---

## Placing the Button

Once the button has been created using the methods discussed, place the button on the Creo Parametric ribbon user interface. Refer to the section on [Ribbon Tabs, Groups, and Menu Items on page 55](#) for more information. Also, refer to the Creo Parametric Help for more information on customizing the Ribbon User Interface.

## Pop-up Menus

Creo Parametric provides shortcut menus that contain frequently used commands appropriate to the currently selected items. You can access a shortcut menu by right-clicking a selected item. Shortcut menus are accessible in:

- Graphics window
- Model Tree
- Some dialog boxes
- Any area where you can perform an object-action operation by selecting an item and choosing a command to perform on the selected item.

The methods described in this section allow you to add menus to a graphics window pop-up menu.

---

## Adding a Pop-up Menu to the Graphics Window

You can activate different pop-up menus during a given session of Creo Parametric. Every time the Creo Parametric context changes when you open a different model type, enter different tools or special modes such as **Edit**, a different pop-up menu is created. When Creo Parametric moves to the next context, the pop-up menu may be destroyed.

As a result of this, the VB API applications must attach a button to the pop-up menu during initialization of the pop-up menu. The VB API application is notified each time a particular pop-up menu is created, which then allows the user to add to the pop-up menu.

Use the following procedure to add items to pop-up menus in the graphics window:

1. Obtain the name of the existing pop-up menus to which you want to add a new menu using the trail file.
2. Create commands for the new pop-up menu items.
3. Implement access listeners to provide visibility information for the items.
4. Add an action listener to the session to listen for pop-up menu initialization.
5. In the listener method, if the pop-up menu is the correct menu to which you wish to add the button, then add it.

The following sections describe each of these steps in detail. You can add push buttons and cascade menus to the pop-up menus. You can add pop-up menu items only in the active window. You cannot use this procedure to remove items from existing menus.

## Using the Trail File to Determine Existing Pop-up Menu Names

The trail file in Creo Parametric contains a comment that identifies the name of the pop-up menu if the configuration option, `auxapp_popup_menu_info` is set to `yes`.

For example, the pop-up menu, **Edit Properties**, has the following comment in the trail file:

```
~ Close `rmb_popup` `PopupMenu`
~ Activate `rmb_popup` `EditProperties`
!Command ProCmdEditPropertiesDtm was pushed from the software.
!Item was selected from popup menu 'popup_mnu_edit'
```

## Listening for Pop-up Menu Initialization

Methods Introduced:

- **IpfcActionSource.AddActionListener()**

- 
- **`IpfcActionSource.AddActionListenerWithType()`**
  - **`IpfcPopupmenuListener.OnPopupmenuCreate()`**

Use the method `IpfcActionSource.AddActionListener()` or `IpfcActionSource.AddActionListenerWithType()` to register a new `IpfcPopupmenuListener` to the session. This listener will be called when pop-up menus are initialized.

The method `IpfcPopupmenuListener.OnPopupmenuCreate()` is called after the pop-up menu is created internally in Creo Parametric and may be used to assign application-owned buttons to the pop-up menu.

## Accessing the Pop-up Menus

The property described in this section provides the name of the pop-up menus used to access these menus while using other methods.

Property Introduced:

- **`IpfcPopupmenu.Name`**

The property `IpfcPopupmenu.Name` returns the name of the pop-up menu.

## Adding Content to the Pop-up Menus

Methods Introduced:

- **`IpfcPopupmenu.AddButton()`**
- **`IpfcPopupmenu.AddMenu()`**

Use the method `IpfcPopupmenu.AddButton()` to add a new item to a pop-up menu. The input arguments are:

- *Command*—Specifies the command associated with the pop-up menu.
- *Options* – A `IpfcPopupmenuOptions` object containing other options for the method. The options that may be included are:
  - *PositionIndex*—Specifies the position in the pop-up menu at which to add the menu button. Pass null to add the button to the bottom of the menu. Use the property `IpfcPopupmenuOptions.PositionIndex` to set this option.
  - *Name*—Specifies the name of the added button. The button name is placed in the trail file when the user selects the menu button. Use the property `IpfcPopupmenuOptions.Name` to set this option.
  - *SetLabel*—Specifies the button label. This label identifies the text displayed when the button is displayed. Use the property `IpfcPopupmenuOptions.Label` to set this option.

- **Helptext**—Specifies the help message associated with the button. Use the property `IpfcPopupMenuOptions.Helptext` to set this option.

Use the method `IpfcPopupMenu.AddMenu()` to add a new cascade menu to an existing pop-up menu.

The argument for this method is a `IpfcPopupMenuOptions` object, whose members have the same purpose as described for newly added buttons. This method returns a new `IpfcPopupMenu` object to which you may add new buttons.

### Example 3: Creating a Pop-up Menu

The sample code in the file `pfcPopupMenuExamples.vb` located at `<creo_vbapi_loadpoint>/vbapi_examples` demonstrates the usage of UI functions to add a new model tree pop-up menu.

## Models

### Models

This section describes how to program on the model level using the VB API.

### Overview of Model Objects

Models can be any Creo Parametric file type, including parts, assemblies, drawings, sections, and notebook. The classes in the module `pfcModel` provide generic access to models, regardless of their type. The available methods enable you to do the following:

- Access information about a model.
- Open, copy, rename, and save a model.

### Getting a Model Object

Methods and Properties Introduced:

- **`IpfcFamilyTableRow.CreateInstance()`**
- **`IpfcSelection.SelModel`**
- **`IpfcBaseSession.GetModel()`**
- **`Ipfc.BaseSession.CurrentModel`**
- **`IpfcBaseSession.GetActiveModel()`**
- **`Ipfc.BaseSession.ListModels()`**
- **`IpfcBaseSession.GetByRelationId()`**



---

- **IpfcWindow.Model**

These methods get a model object that is already in session.

The property `IpfcSelection.SelModel` returns the model that was interactively selected.

The method `IpfcBaseSession.GetModel()` returns a model based on its name and type, whereas `IpfcBaseSession.GetByRelationId()` returns a model in an assembly that has the specified integer identifier.

The property `IpfcBaseSession.CurrentModel` returns the current active model.

The method `IpfcBaseSession.GetActiveModel()` returns the active Creo Parametric model.

Use the method `IpfcBaseSession.ListModels()` to return a sequence of all the models in session.

For more methods that return solid models, refer to the section [Solid on page 138](#).

## Model Descriptors

Methods and Properties Introduced:

- **CCpfcModelDescriptor.Create()**
- **IpfcModelDescriptor.CreateFromFileName()**
- **IpfcModelDescriptor.SetGenericName**
- **IpfcModelDescriptor.SetInstanceName**
- **IpfcModelDescriptor.SetType**
- **IpfcModelDescriptor.SetDevice**
- **IpfcModelDescriptor.SetPath**
- **IpfcModelDescriptor.FileVersion**
- **IpfcModelDescriptor.GetFullName**
- **IpfcModel.GetFullName**

Model descriptors are data objects used to describe a model file and its location in the system. The methods in the model descriptor enable you to set specific information that enables Creo Parametric to find the specific model you want.

The static utility method `CCpfcModelDescriptor.Create()` allows you to specify as data to be entered a model type, an instance name, and a generic name. The model descriptor constructs the full name of the model as a string, as follows:

```
String FullName = InstanceName+"<" + GenericName+">";
 // As long as the
 // generic name is
 // not an empty
```

---

```
// string ("")
```

If you want to load a model that is not a family table instance, pass an empty string as the generic name argument so that the full name of the model is constructed correctly. If the model is a family table interface, you should specify both the instance and generic names.

---

 **Note**

You are allowed to set other fields in the model descriptor object, but they may be ignored by some methods.

---

The static utility method

`IpfcModelDescriptor.CreateFromFileName()` allows you to create a new model descriptor from a given a file name. The file name is a string in the form `<name>.<extension>`.

## Retrieving Models

Methods Introduced:

- **`IpfcBaseSession.RetrieveModel()`**
- **`IpfcBaseSession.RetrieveModelWithOpts()`**
- **`IpfcBaseSession.OpenFile()`**
- **`IpfcSolid.HasRetrievalErrors()`**

These methods cause Creo Parametric to retrieve the model that corresponds to the `IpfcModelDescriptor` argument.

The method `IpfcBaseSession.RetrieveModel()` retrieves the specified model into the Creo Parametric session given its model descriptor from a standard directory. This method ignores the path argument specified in the model descriptor. But this method does not create a window for it, nor does it display the model anywhere.

The method `IpfcBaseSession.RetrieveModelWithOpts()` retrieves the specified model into the Creo Parametric session based on the path specified by the model descriptor. The path can be a disk path, a workspace path, or a commonspace path. The *Opts* argument (given by the `IpfcRetrieveModelOptions` object) provides the user with the option to specify simplified representations.

The method `IpfcBaseSession.OpenFile()` brings the model into memory, opens a new window for it (or uses the base window, if it is empty), and displays the model.



## Note

`IpfcBaseSession.OpenFile()` actually returns a handle to the window it has created.

The file version set by the `IpfcModelDescriptor.FileVersion` property is ignored by the `IpfcBaseSession.RetrieveModelWithOpts()` and `IpfcBaseSession.OpenFile()` methods. Instead, the latest file version of the model is used by these two methods.

To get a handle to the model you need, use the property `IpfcWindow.Model`.

The method `IpfcSolid.HasRetrievalErrors()` returns a true value if the features in the solid model were suppressed during the `RetrieveModel` or `OpenFile` operations. This method must be called immediately after the `IpfcBaseSession.RetrieveModel()` method or an equivalent retrieval method.

## Example Code: Retrieving a Model

The sample code in the file `pfcModelsExamples.vb` located at `<creo_vbapi_loadpoint>/vbapi_examples` demonstrates how to retrieve a model.

## Model Information

Methods and Properties Introduced:

- **`IpfcModel.FileName`**
- **`IpfcModel.CommonName`**
- **`IpfcModel.IsCommonNameModifiable()`**
- **`IpfcModel.FullName`**
- **`IpfcModel.GenericName`**
- **`IpfcModel.InstanceName`**
- **`IpfcModel.Origin`**
- **`IpfcModel.RelationId`**
- **`IpfcModel.Descr`**
- **`IpfcModel.Type`**
- **`IpfcModel.IsModified`**
- **`IpfcModel.Version`**
- **`IpfcModel.Revision`**

- 
- **IpfcModel.Branch**
  - **IpfcModel.ReleaseLevel**
  - **IpfcModel.VersionStamp**
  - **IpfcModel.ListDependencies()**
  - **IpfcModel.CleanupDependencies()**
  - **IpfcModel.ListDeclaredModels()**
  - **IpfcModel.CheckIsModifiable()**
  - **IpfcModel.CheckIsSaveAllowed()**

The property `IpfcModel.FileName` retrieves the model file name in the "name"."type" format.

The property `IpfcModel.CommonName` retrieves the common name for the model. This name is displayed for the model in Windchill PDMLink.

Use the method `IpfcModel.IsCommonNameModifiable()` to identify if the common name of the model can be modified. You can modify the name for models that are not yet owned by Windchill PDMLink, or in certain situations if the configuration option `let_proe_rename_pdm_objects` is set to `yes`.

The property `IpfcModel.FullName` retrieves the full name of the model in the instance <generic> format.

The property `IpfcModel.GenericName` retrieves the name of the generic model. If the model is not an instance, this name must be `NULL` or an empty string.

The property `IpfcModel.InstanceName` retrieves the name of the model. If the model is an instance, this method retrieves the instance name.

The property `IpfcModel.Origin` returns the complete path to the file from which the model was opened. This path can be a location on disk from a Windchill workspace, or from a downloaded URL.

The property `IpfcModel.RelationId` retrieves the relation identifier of the specified model. It can be `NULL`.

The property `IpfcModel.Descr` returns the descriptor for the specified model. Model descriptors can be used to represent models not currently in session.

---

 **Note**

From Pro/ENGINEER Wildfire 4.0 onwards, the properties `IpfcModel.FullName`, `IpfcModel.GenericName`, and `IpfcModel.Descr` throw an exception `IpfcXtoolkitCantOpen` if called on a model instance whose immediate generic is not in session. Handle this exception and typecast the model as `IpfcSolid`, which in turn can be typecast as `IpfcFamilyMember`, and use the method `IpfcFamilyMember.GetImmediateGenericInfo()` to get the model descriptor of the immediate generic model. The model descriptor can be used to derive the full name or generic name of the model. If you wish to switch off this behavior and continue to run legacy applications in the pre-Wildfire 4.0 mode, set the configuration option `retrieve_instance_dependencies` to `instance_and_generic_deps`.

---

The property `IpfcModel.Type` returns the type of model in the form of the `IpfcModelType` object. The types of models are as follows:

- `EpfcMDL_ASSEMBLY`—Specifies an assembly.
- `EpfcMDL_PART`—Specifies a part.
- `EpfcMDL_DRAWING`—Specifies a drawing.
- `EpfcMDL_2D_SECTION`—Specifies a 2D section.
- `EpfcMDL_LAYOUT`—Specifies a notebook.
- `EpfcMDL_DWG_FORMAT`—Specifies a drawing format.
- `EpfcMDL_MFG`—Specifies a manufacturing model.
- `EpfcMDL_REPORT`—Specifies a report.
- `EpfcMDL_MARKUP`—Specifies a drawing markup.
- `EpfcMDL_DIAGRAM`—Specifies a diagram
- `EpfcMDL_CE_SOLID`—Specifies a Layout model.

---

 **Note**

the VB API methods will only be able to read models of type `Layout`, but will not be able to pass `Layout` models as input to other methods. PTC recommends that you review all the VB API applications that use the object `IpfcModelType` and modify the code as appropriate to ensure that the applications work correctly.

---

---

The property `IpfcModel.IsModified` identifies whether the model has been modified since it was last saved.

The property `IpfcModel.Version` returns the version of the specified model from the PDM system. It can be `NULL`, if not set.

The property `IpfcModel.Revision` returns the revision number of the specified model from the PDM system. It can be `NULL`, if not set.

The property `IpfcModel.Branch` returns the branch of the specified model from the PDM system. It can be `NULL`, if not set.

The property `IpfcModel.ReleaseLevel` returns the release level of the specified model from the PDM system. It can be `NULL`, if not set.

The property `IpfcModel.VersionStamp` returns the version stamp of the specified model. The version stamp is a Creo Parametric specific identifier that changes with each change made to the model.

The method `IpfcModel.ListDependencies()` returns a list of the first-level dependencies for the specified model in the Creo Parametric workspace in the form of the `IpfcDependencies` object.

Use the method `IpfcModel.CleanupDependencies()` to clean the dependencies for an object in the Creo Parametric workspace.

---

 **Note**

Do not call the method `IpfcModel.CleanupDependencies()` during operations that alter the dependencies, such as, restructuring components and creating or redefining features.

---

The method `IpfcModel.ListDeclaredModels()` returns a list of all the first-level objects declared for the specified model.

The method `IpfcModel.CheckIsModifiable()` identifies if a given model can be modified without checking for any subordinate models. This method takes a boolean argument *ShowUI* that determines whether the Creo Parametric conflict resolution dialog box should be displayed to resolve conflicts, if detected. If this argument is false, then the conflict resolution dialog box is not displayed, and the model can be modified only if there are no conflicts that cannot be overridden, or are resolved by default resolution actions. For a generic model, if *ShowUI* is true, then all instances of the model are also checked.

The method `IpfcModel.CheckIsSaveAllowed()` identifies if a given model can be saved along with all of its subordinate models. The subordinate models can be saved based on their modification status and the value of the configuration option `save_objects`. This method also checks the current user interface context to identify if it is currently safe to save the model. Thus, calling

---

this method at different times might return different results. This method takes a boolean argument *ShowUI*. Refer to the previous method for more information on this argument.

## Model Operations

Methods and Property Introduced:

- **IpfcModel.Backup()**
- **IpfcModel.Copy()**
- **IpfcModel.CopyAndRetrieve()**
- **IpfcModel.Rename()**
- **IpfcModel.Save()**
- **IpfcModel.Erase()**
- **IpfcModel.EraseWithDependencies()**
- **IpfcModel.Delete()**
- **IpfcModel.Display()**
- **IpfcModel.CommonName**

These model operations duplicate most of the commands available in the Creo Parametric **File** menu.

The method `IpfcModel.Backup()` makes a backup of an object in memory to a disk in a specified directory.

The method `IpfcModel.Copy()` copies the specified model to another file.

The method `IpfcModel.CopyAndRetrieve()` copies the model to another name, and retrieves that new model into session.

The method `IpfcModel.Rename()` renames a specified model.

The method `IpfcModel.Save()` stores the specified model to a disk.

The method `IpfcModel.Erase()` erases the specified model from the session. Models used by other models cannot be erased until the models dependent upon them are erased.

The method `IpfcModel.EraseWithDependencies()` erases the specified model from the session and all the models on which the specified model depends from disk, if the dependencies are not needed by other items in session.

---

## Note

However, while erasing an active model, `IpfcModel.Erase()` and `IpfcModel.EraseWithDependencies()` only clear the graphic display immediately, they do not clear the data in the memory until the control returns to Creo Parametric from the the VB API application. Therefore, after calling them the control must be returned to Creo Parametric before calling any other function, otherwise the behavior of Creo Parametric may be unpredictable.

---

The method `IpfcModel.Delete()` removes the specified model from memory and disk.

The method `IpfcModel.Display()` displays the specified model. You must call this method if you create a new window for a model because the model will not be displayed in the window until you call `IpfcDisplay`.

The property `IpfcModel.CommonName` modifies the common name of the specified model. You can modify this name for models that are not yet owned by Windchill PDMLink, or in certain situations if the configuration option `let_proe_rename_pdm_objects` is set to yes.

## Running Creo ModelCHECK

Creo ModelCHECK is an integrated application that runs transparently within Creo Parametric. Creo ModelCHECK uses a configurable list of company design standards and best modeling practices. You can configure Creo ModelCHECK to run interactively or automatically when you regenerate or save a model.

Methods and Properties Introduced:

- **`IpfcBaseSession.ExecuteModelCheck()`**
- **`CCpfcModelCheckInstructions.Create()`**
- **`IpfcModelCheckInstructions.ConfigDir`**
- **`IpfcModelCheckInstructions.Mode`**
- **`IpfcModelCheckInstructions.OutputDir`**
- **`IpfcModelCheckInstructions.ShowInBrowser`**
- **`IpfcModelCheckResults.NumberOfErrors`**
- **`IpfcModelCheckResults.NumberOfWarnings`**
- **`IpfcModelCheckResults.WasModelSaved`**



---

You can run Creo ModelCHECK from an external application using the method `IpfcBaseSession.ExecuteModelCheck()`. This method takes the model *Model* on which you want to run Creo ModelCHECK and instructions in the form of the object `IpfcModelCheckInstructions` as its input parameters. This object contains the following parameters:

- `ConfigDir`—Specifies the location of the configuration files. If this parameter is set to `NULL`, the default Creo ModelCHECK configuration files are used.
- `Mode`—Specifies the mode in which you want to run Creo ModelCHECK. The modes are:
  - `MODELCHECK_GRAPHICS`—Interactive mode
  - `MODELCHECK_NO_GRAPHICS`—Batch mode
- `OutputDir`—Specifies the location for the reports. If you set this parameter to `NULL`, the default Creo ModelCHECK directory, as per `config_init.mc`, will be used.
- `ShowInBrowser`—Specifies if the results report should be displayed in the Web browser.

The method `CCpfcModelCheckInstructions.Create()` creates the `IpfcModelCheckInstructions` object containing the Creo ModelCHECK instructions described above.

Use the methods and properties

`IpfcModelCheckInstructions.ConfigDir`,  
`IpfcModelCheckInstructions.Mode`,  
`IpfcModelCheckInstructions.OutputDir`, and  
`IpfcModelCheckInstructions.ShowInBrowser` to modify the Creo ModelCHECK instructions.

The method `IpfcBaseSession.ExecuteModelCheck()` returns the results of the Creo ModelCHECK run in the form of the `IpfcModelCheckResults` object. This object contains the following parameters:

- `NumberOfErrors`—Specifies the number of errors detected.
- `NumberOfWarnings`—Specifies the number of warnings found.
- `WasModelSaved`—Specifies whether the model is saved with updates.

Use the properties `IpfcModelCheckResults.NumberOfErrors`, `IpfcModelCheckResults.NumberOfWarning`, and `IpfcModelCheckResults.WasModelSaved` to access the results obtained.

---

## Custom Checks

This section describes how to define custom checks in Creo ModelCHECK that users can run using the standard Creo ModelCHECK interface in Creo Parametric.

To define and register a custom check:

1. Set the CUSTMTK\_CHECKS\_FILE configuration option in the start configuration file to a text file that stores the check definition. For example:  
CUSTMTK\_CHECKS\_FILE text/custmtk\_checks.txt.
2. Set the contents of the CUSTMTK\_CHECKS\_FILE file to define the checks. Each check should list the following items:
  - DEF\_<checkname>—Specifies the name of the check. The format must be CHKTK\_<checkname>\_<mode>, where mode is PRT, ASM, or DRW.
  - TAB\_<checkname>—Specifies the tab category in the Creo ModelCHECK report under which the check is classified. Valid tab values are:
    - INFO
    - PARAMETER
    - LAYER
    - FEATURE
    - RELATION
    - DATUM
    - MISC
    - VDA
    - VIEWS
  - MSG\_<checkname>—Specifies the description of the check that appears in the lower part of the Creo ModelCHECK report when you select the name.
  - DSC\_<checkname>—Specifies the name of the check as it appears in the Creo ModelCHECK report table.
  - ERM\_<checkname>—If set to INFO, the check is considered an INFO check and the report table displays the text from the first item returned by the check, instead of a count of the items. Otherwise, this value must be included, but is ignored by Creo Parametric.

See [Example 1: Text File for Custom Checks](#) on page 86 for a sample custom checks text file.
3. Add the check and its values to the Creo ModelCHECK configuration file.
4. Register the Creo ModelCHECK check from the VB API application.

---

## Note

Other than the requirements listed above, the VB API custom checks do not have access to the rest of the values in the Creo ModelCHECK configuration files. All the custom settings specific to the check, such as start parameters, constants, and so on, must be supported by the user application and not Creo ModelCHECK.

---

## Registering Custom Checks

Methods and Properties Introduced:

- **IpfcBaseSession.RegisterCustomModelCheck**
- **CCpfcCustomCheckInstructions.Create**
- **IpfcCustomCheckInstructions.CheckName**
- **IpfcCustomCheckInstructions.CheckLabel**
- **IpfcCustomCheckInstructions.Listener**
- **IpfcCustomCheckInstructions.ActionButtonLabel**
- **IpfcCustomCheckInstructions.UpdateButtonLabel**

The method `IpfcBaseSession.RegisterCustomModelCheck` registers a custom check that can be included in any ModelCHECK run. This method takes the instructions in the form of the `IpfcCustomCheckInstructions` object as its input argument. This object contains the following parameters:

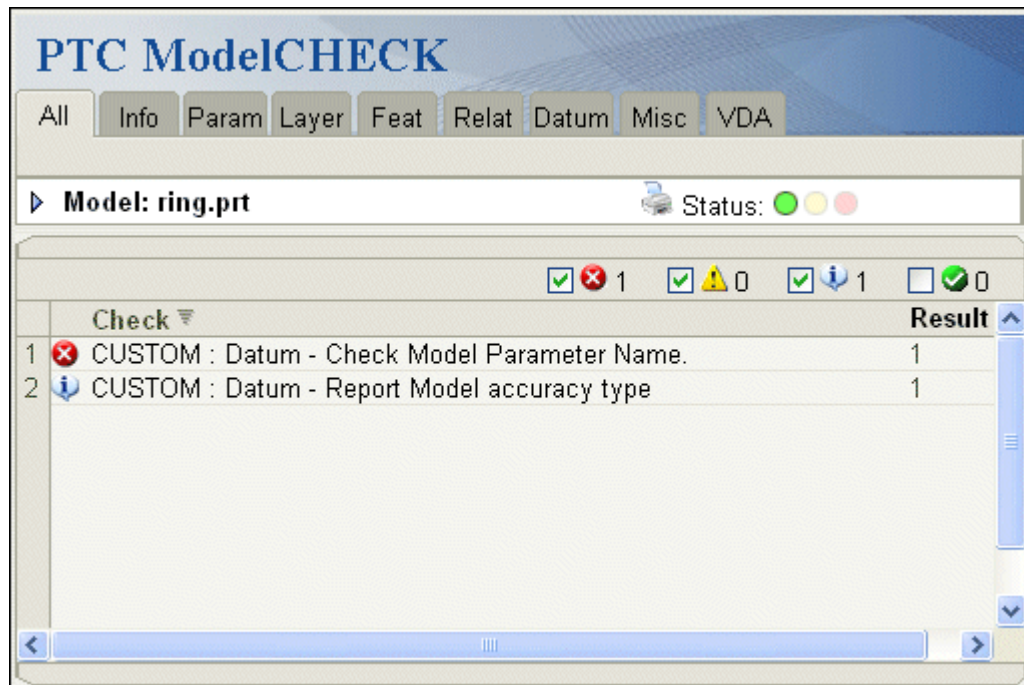
- *CheckName*—Specifies the name of the custom check.
- *CheckLabel*—Specifies the label of the custom check.
- *Listener*—Specifies the listener object containing the custom check methods. Refer to the section [Custom Check Listeners on page 84](#) for more information.
- *ActionButtonLabel*—Specifies the label for the action button. If you specify NULL for this parameter, this button is not shown.
- *UpdateButtonLabel*—Specifies the label for the update button. If you specify NULL for this parameter, this button is not shown.

The method `CCpfcCustomCheckInstructions.Create` creates the `IpfcCustomCheckInstructions` object containing the custom check instructions described above.

Use the properties `IpfcCustomCheckInstructions.SetCheckName`, `IpfcCustomCheckInstructions.SetCheckLabel`, `IpfcCustomCheckInstructions.SetListener`,

`IpfcCustomCheckInstructions.SetActionButtonLabel`, and `IpfcCustomCheckInstructions.SetUpdateButtonLabel` to modify the instructions.

The following figure illustrates how the results of some custom checks might be displayed in the ModelCHECK report.



## Custom Check Listeners

Methods and Properties Introduced:

- **`IpfcModelCheckCustomCheckListener.OnCustomCheck`**
- **`CCpfcCustomCheckResults_Create`**
- **`IpfcCustomCheckResults.ResultsCount`**
- **`IpfcCustomCheckResults.ResultsTable`**
- **`IpfcCustomCheckResults.ResultsUrl`**
- **`IpfcModelCheckCustomCheckListener.OnCustomCheckAction`**
- **`IpfcModelCheckCustomCheckListener.OnCustomCheckUpdate`**

The interface `IpfcModelCheckCustomCheckListener` provides the method signatures to implement a custom Creo ModelCHECK check.

Each listener method takes the following input arguments:

- *CheckName*—The name of the custom check as defined in the original call to the method `IpfcBaseSession.RegisterCustomModelCheck`

- 
- *Mdl*—The model being checked.

The application method that overrides

`IpfcModelCheckCustomCheckListener.OnCustomCheck` is used to evaluate a custom defined check. The user application runs the check on the specified model and returns the results in the form of the `IpfcCustomCheckResults` object

- *ResultsCount*—Specifies an integer indicating the number of errors found by the check. This value is displayed in the Creo ModelCHECK report generated.
- *ResultsTable*—Specifies a list of text descriptions of the problem encountered for each error or warning.
- *ResultsUrl*—Specifies the link to an application-owned page that provides information on the results of the custom check.

The method `IpfcIpfcCustomCheckResults.Create` creates the `IpfcCustomCheckResults` object containing the custom check results described above.

Use the properties `IpfcCustomCheckResults.SetResultsCount`, `IpfcCustomCheckResults.SetResultsTable`, and `IpfcCustomCheckResults.SetResultsUrl` listed above to modify the custom checks results obtained.

The method that overrides

`IpfcModelCheckCustomCheckListener.OnCustomCheckAction` is called when the custom check's **Action** button is pressed. The input supplied includes the text selected by the user from the custom check results.

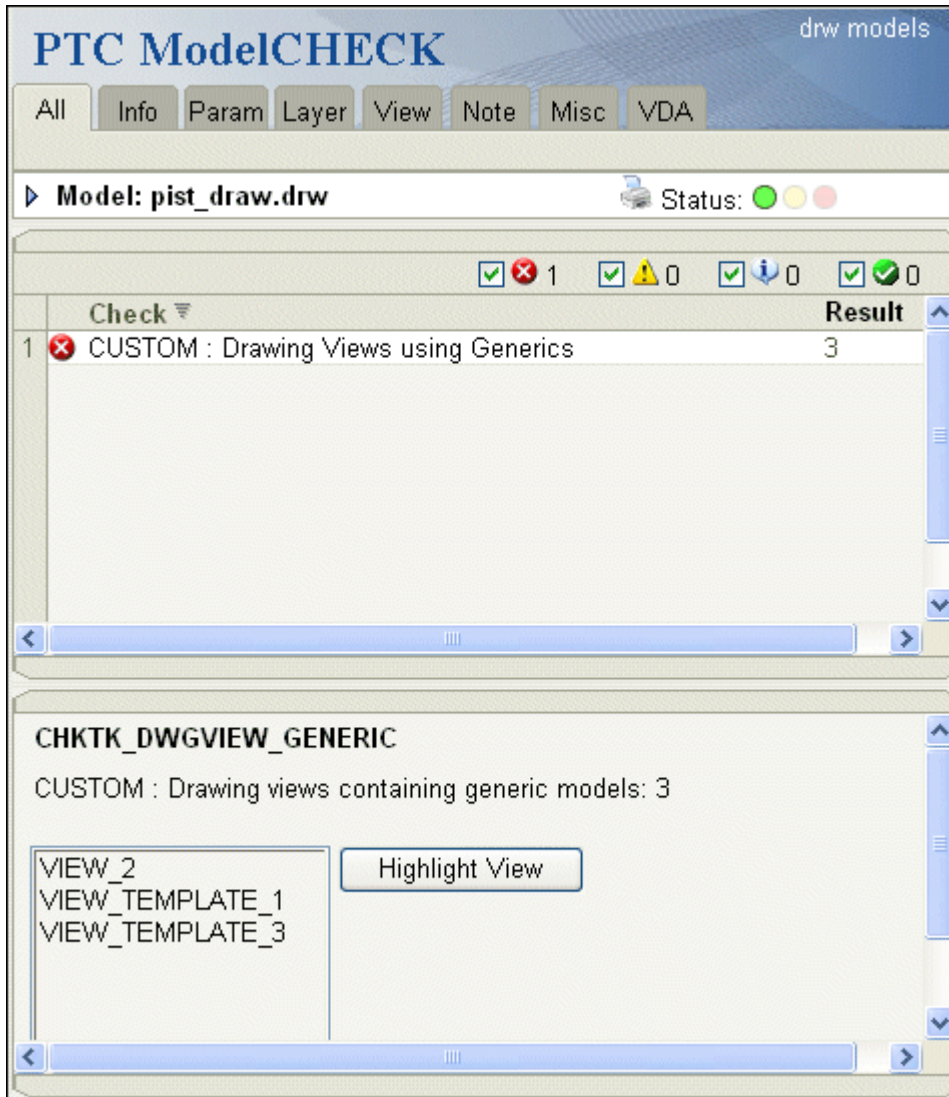
The method that overrides

`IpfcModelCheckCustomCheckListener.OnCustomCheckUpdate` is called when the custom check's **Update** button is pressed. The input supplied includes the text selected by the user from the custom check results.

Custom Creo ModelCHECK checks can have an **Action** button to highlight the problem, and possibly an **Update** button to fix it automatically.

The following figure displays the Creo ModelCHECK report with an **Action** button that invokes the

`IpfcModelCheckCustomCheckListener.OnCustomCheckAction`



## Example 1: Text File for Custom Checks

The following is the text file `custmtk_checks.txt` located at `<creo_vbapi_loadpoint>/vbapi_examples/models` for custom checks examples.

```
Custom TK Check File
def-name of check as registered

MDLPARAM_NAME
DEF_MDLPARAM_NAME CHKTK_MDLPARAM_NAME_PRT
TAB_MDLPARAM_NAME DATUM
MSG_MDLPARAM_NAME CUSTOM : Datum - Model Param Name
ERM_MDLPARAM_NAME CUSTOM : Datum - Invalid Parameter value.
DSC_MDLPARAM_NAME CUSTOM : Datum - Check Model Parameter Name.
```

```
MODEL_ACCURACY
DEF_MODEL_ACCURACY_CHKTK_MODEL_ACCURACY_PRT
TAB_MODEL_ACCURACY_DATUM
MSG_MODEL_ACCURACY_CUSTOM : Datum - Report Model accuracy type
ERM_MODEL_ACCURACY_CUSTOM : Datum - Report Model accuracy type
DSC_MODEL_ACCURACY_CUSTOM : Datum - Report Model accuracy type

DWGVIEW_GENERIC
DEF_DWGVIEW_GENERIC_CHKTK_DWGVIEW_GENERIC_DRW
TAB_DWGVIEW_GENERIC_VIEWS
MSG_DWGVIEW_GENERIC_CUSTOM : Drawing views containing generic
models:
ERM_DWGVIEW_GENERIC_N/A
DSC_DWGVIEW_GENERIC_CUSTOM : Drawing Views using Generics
```

## Example 2: Registering Custom Creo ModelCHECK Checks

The sample code in the file `pfcModelCheckExamples.vb` located at `<creo_vbapi_loadpoint>/vbapi_examples` demonstrates how to register custom Creo ModelCHECK checks using the VB API. The following custom checks are registered:

- `CHKTK_MDLPARAM_NAME`—Determines if the model has a parameter whose name is equal to the model name.
- `CHKTK_MODEL_ACCURACY`—Checks the type of accuracy defined for the model.
- `CHKTK_DWGVIEW_GENERIC`—Drawing mode check that identifies the drawing views that use generic models.

## Example 3: Implementing a Model Name Parameter Check

The sample code in the file `pfcModelCheckExamples.vb` located at `<creo_vbapi_loadpoint>/vbapi_examples` defines the custom Creo ModelCHECK check for the parameter name in a model. This check updates the parameter name to be equal to the model name, if the parameter exists with a different name, or creates the parameter with the model name if it does not exist.

## Example 4: Implementing a Model Accuracy Type Check

The sample code in the file `pfcModelCheckExamples.vb` located at `<creo_vbapi_loadpoint>/vbapi_examples` defines the custom Creo ModelCHECK check for the type of accuracy whether relative or absolute that has been set for a model. This check has a check listener method, but no action or update listener method since it is an info-only check.

---

## Example 5: Implementing a Check for Drawing Views Using Generic Models

The sample code in the file `pfcModelCheckExamples.vb` located at `<creo_vbapi_loadpoint>/vbapi_examples` defines the custom Creo ModelCHECK check for identifying drawing views using generic models. This check has a check listener method and an action listener method to highlight the views that use generic models.

## Example 6: Changes to the Creo ModelCHECK Configuration Files to enable Custom Checks

Lines added to the ModelCheck configuration file (`default_checks.mch`)

```
E Check the item. If not succeed, report as an error
W Check the item. If not succeed, report as a warning
N Do not check the item.
Y Check the item. If not succeed, do not report err or warn
```

```
CHKTK_MDLPARAM_NAME_PRT YNEW E E E E
Y
CHKTK_MODEL_ACCURACY_PRT YNEW Y Y Y Y
Y
CHKTK_DWGVIEW_GENERIC_DRW YNEW E E E E
Y
```

Lines added to the ModelCheck start file (`sample_start.mcs`)

```
CUSTOMTK_CHECKS_FILE customtk_checks.txt
```

# Drawings

## Drawings

This section describes how to program drawing functions using the VB API.

### Overview of Drawings in the VB API

This section describes the functions that deal with drawings. You can create drawings of all Creo Parametric models using the functions in the VB API. You can annotate the drawing, manipulate dimensions, and use layers to manage the display of different items.

Unless otherwise specified, the VB API functions that operate on drawings use world units.



---

## Creating Drawings from Templates

Drawing templates simplify the process of creating a drawing using the VB API. Creo Parametric can create views, set the view display, create snap lines, and show the model dimensions based on the template. Use templates to:

- Define layout views
- Set view display
- Place notes
- Place symbols
- Define tables
- Show dimensions

Method Introduced:

- **IpfcBaseSession.CreateDrawingFromTemplate()**

Use the method `IpfcBaseSession.CreateDrawingFromTemplate()` to create a drawing from the drawing template and to return the created drawing. The attributes are:

- New drawing name
- Name of an existing template
- Name and type of the solid model to use while populating template views
- Sequence of options to create the drawing. The options are as follows:
  - `EpfcDRAWINGCREATE_DISPLAY_DRAWING`—display the new drawing.
  - `EpfcDRAWINGCREATE_SHOW_ERROR_DIALOG`—display the error dialog box.
  - `EpfcDRAWINGCREATE_WRITE_ERROR_FILE`—write the errors to a file.
  - `EpfcDRAWINGCREATE_PROMPT_UNKNOWN_PARAMS`—prompt the user on encountering unknown parameters

## Drawing Creation Errors

The exception `IpfcXToolkitDrawingCreateErrors` is thrown if an error is encountered when creating a drawing from a template. This exception contains a list of errors which occurred during drawing creation.

---

 **Note**

When this exception type is encountered, the drawing is actually created, but some of the contents failed to generate correctly.

---

The exception message will list the details for each error including its type, sheet number, view name, and (if applicable) item name. The types of errors are as follows:

- *Type*—The type of error as follows:
  - `EpfcDWGCREATE_ERR_SAVED_VIEW_DOESNT_EXIST`—Saved view does not exist.
  - `EpfcDWGCREATE_ERR_X_SEC_DOESNT_EXIST`—Specified cross section does not exist.
  - `EpfcDWGCREATE_ERR_EXPLODE_DOESNT_EXIST`—Exploded state did not exist.
  - `EpfcDWGCREATE_ERR_MODEL_NOT_EXPLODABLE`—Model cannot be exploded.
  - `EpfcDWGCREATE_ERR_SEC_NOT_PERP`—Cross section view not perpendicular to the given view.
  - `EpfcDWGCREATE_ERR_NO_RPT_REGIONS`—Repeat regions not available.
  - `EpfcDWGCREATE_ERR_FIRST_REGION_USED`—Repeat region was unable to use the region specified.
  - `EpfcDWGCREATE_ERR_NOT_PROCESS_ASSEM`—Model is not a process assembly view.
  - `EpfcDWGCREATE_ERR_NO_STEP_NUM`—The process step number does not exist.
  - `EpfcDWGCREATE_ERR_TEMPLATE_USED`—The template does not exist.
  - `EpfcDWGCREATE_ERR_NO_PARENT_VIEW_FOR_PROJ`—There is no possible parent view for this projected view.
  - `EpfcDWGCREATE_ERR_CANT_GET_PROJ_PARENT`—Could not get the projected parent for a drawing view.
  - `EpfcDWGCREATE_ERR_SEC_NOT_PARALLEL`—The designated cross section was not parallel to the created view.
  - `EpfcDWGCREATE_ERR_SIMP_REP_DOESNT_EXIST`—The designated simplified representation does not exist.

---

### Example: Drawing Creation from a Template

The sample code in the file `pfcDrawingExamples.vb` located at `<creo_vbapi_loadpoint>/vbapi_examples` creates a new drawing using a predefined template.

## Obtaining Drawing Models

This section describes how to obtain drawing models.

Methods Introduced:

- **`IpfcBaseSession.RetrieveModel()`**
- **`IpfcBaseSession.GetModel()`**
- **`IpfcBaseSession.GetModelFromDescr()`**
- **`IpfcBaseSession.ListModels()`**
- **`IpfcBaseSession.ListModelsByType()`**

The method `IpfcBaseSession.RetrieveModel()` retrieves the drawing specified by the model descriptor. Model descriptors are data objects used to describe a model file and its location in the system. The method returns the retrieved drawing.

The method `IpfcBaseSession.GetModel()` returns a drawing based on its name and type, whereas `IpfcBaseSession.GetModelFromDescr()` returns a drawing specified by the model descriptor. The model must be in session.

Use the method `IpfcBaseSession.ListModels()` to return a sequence of all the drawings in session.

## Drawing Information

Methods and Property Introduced:

- **`IpfcModel2D.ListModels()`**
- **`IpfcModel2D.GetCurrentSolid()`**
- **`IpfcModel2D.ListSimplifiedReps()`**
- **`IpfcModel2D.TextHeight`**

The method `IpfcModel2D.ListModels()` returns a list of all the solid models used in the drawing.

The method `IpfcModel2D.GetCurrentSolid()` returns the current solid model of the drawing.

The method `IpfcModel2D.ListSimplifiedReps()` returns the simplified representations of a solid model that are assigned to the drawing.

---

The property `IpfcModel2D.TextHeight` returns the text height of the drawing.

## Drawing Operations

Methods Introduced:

- **`IpfcModel2D.AddModel()`**
- **`IpfcModel2D.DeleteModel()`**
- **`IpfcModel2D.ReplaceModel()`**
- **`IpfcModel2D.SetCurrentSolid()`**
- **`IpfcModel2D.AddSimplifiedRep()`**
- **`IpfcModel2D.DeleteSimplifiedRep()`**
- **`IpfcModel2D.Regenerate()`**
- **`IpfcModel2D.CreateDrawingDimension()`**
- **`IpfcModel2D.CreateView()`**

The method `IpfcModel2D.AddModel()` adds a new solid model to the drawing.

The method `IpfcModel2D.DeleteModel()` removes a model from the drawing. The model to be deleted should not appear in any of the drawing views.

The method `IpfcModel2D.ReplaceModel()` replaces a model in the drawing with a related model (the relationship should be by family table or interchange assembly). It allows you to replace models that are shown in drawing views and regenerates the view.

The method `IpfcModel2D.SetCurrentSolid()` assigns the current solid model for the drawing. Before calling this method, the solid model must be assigned to the drawing using the method `IpfcModel2D.AddModel()`. To see the changes to parameters and fields reflecting the change of the current solid model, regenerate the drawing using the method `IpfcSheetOwner.RegenerateSheet()`.

The method `IpfcModel2D.AddSimplifiedRep()` associates the drawing with the simplified representation of an assembly

The method `IpfcModel2D.DeleteSimplifiedRep()` removes the association of the drawing with an assembly simplified representation. The simplified representation to be deleted should not appear in any of the drawing views.

Use the property `IpfcModel2D.Regenerate()` to regenerate the drawing draft entities and appearance.

---

The method `IpfcModel2D.CreateDrawingDimension()` creates a new drawing dimension based on the data object that contains information about the location of the dimension. This method returns the created dimension. Refer to the section [Drawing Dimensions on page 101](#).

The method `IpfcModel2D.CreateView()` creates a new drawing view based on the data object that contains information about how to create the view. The method returns the created drawing view. Refer to the section [Creating Drawing Views on page 96](#).

### Example: Replace Drawing Model Solid with its Generic

The sample code in the file `pfcDrawingExamples.vb` located at `<creo_vbapi_loadpoint>/vbapi_examples` replaces all solid model instances in a drawing with its generic. Models are not replaced if the generic model is already present in the drawing.

## Drawing Sheets

A drawing sheet is represented by its number. Drawing sheets in the VB API are identified by the same sheet numbers seen by a Creo Parametric user.



### Note

These identifiers may change if the sheets are moved as a consequence of adding, removing or reordering sheets.

---

## Drawing Sheet Information

Methods and Properties Introduced

- `IpfcSheetOwner.GetSheetTransform()`
- `IpfcSheetOwner.GetSheetInfo()`
- `IpfcSheetOwner.GetSheetScale()`
- `IpfcSheetOwner.GetSheetFormat()`
- `IpfcSheetOwner.GetSheetFormatDescr()`
- `IpfcSheetOwner.GetSheetBackgroundView()`
- `IpfcSheetOwner.NumberOfSheets`
- `IpfcSheetOwner.CurrentSheetNumber`
- `IpfcSheetOwner.GetSheetUnits()`

Superseded Method:

---

- **IpfcSheetOwner.GetSheetData()**

The method `IpfcSheetOwner.GetSheetTransform()` returns the transformation matrix for the sheet specified by the sheet number. This transformation matrix includes the scaling needed to convert screen coordinates to drawing coordinates (which use the designated drawing units).

The method `IpfcSheetOwner.GetSheetInfo()` returns sheet data including the size, orientation, and units of the sheet specified by the sheet number.

The method `IpfcSheetOwner.GetSheetData()` and the interface `IpfcSheetData` have been deprecated. Use the method `IpfcSheetOwner.GetSheetInfo()` and the interface `IpfcSheetInfo` instead.

The method `IpfcSheetOwner.GetSheetScale()` returns the scale of the drawing on a particular sheet based on the drawing model used to measure the scale. If no models are used in the drawing then the default scale value is 1.0.

The method `IpfcSheetOwner.GetSheetFormat()` returns the drawing format used for the sheet specified by the sheet number. It returns a null value if no format is assigned to the sheet.

The method `IpfcSheetOwner.GetSheetFormatDescr()` returns the model descriptor of the drawing format used for the specified drawing sheet.

The method `IpfcSheetOwner.GetSheetBackgroundView()` returns the view object representing the background view of the sheet specified by the sheet number.

The property `IpfcSheetOwner.NumberOfSheets` returns the number of sheets in the model.

The property `IpfcSheetOwner.CurrentSheetNumber` returns the current sheet number in the model.

---

 **Note**

The sheet numbers range from 1 to n, where n is the number of sheets.

---

The method `IpfcSheetOwner.GetSheetUnits()` returns the units used by the sheet specified by the sheet number.

## Drawing Sheet Operations

Methods Introduced:

- **IpfcSheetOwner.AddSheet()**
- **IpfcSheetOwner.DeleteSheet()**

- 
- **IpfcSheetOwner.ReorderSheet()**
  - **IpfcSheetOwner.RegenerateSheet()**
  - **IpfcSheetOwner.SetSheetScale()**
  - **IpfcSheetOwner.SetSheetFormat()**

The method `IpfcSheetOwner.AddSheet()` adds a new sheet to the model and returns the number of the new sheet.

The method `IpfcSheetOwner.DeleteSheet()` removes the sheet specified by the sheet number from the model.

Use the method `IpfcSheetOwner.ReorderSheet()` to reorder the sheet from a specified sheet number to a new sheet number.



#### **Note**

The sheet number of other affected sheets also changes due to reordering or deletion.

---

The method `IpfcSheetOwner.RegenerateSheet()` regenerates the sheet specified by the sheet number.



#### **Note**

You can regenerate a sheet only if it is displayed.

---

Use the method `IpfcSheetOwner.SetSheetScale()` to set the scale of a model on the sheet based on the drawing model to scale and the scale to be used. Pass the value of the *DrawingModel* parameter as null to select the current drawing model.

Use the method `IpfcSheetOwner.SetSheetFormat()` to apply the specified format to a drawing sheet based on the drawing format, sheet number of the format, and the drawing model.

The sheet number of the format is specified by the *FormatSheetNumber* parameter. This number ranges from 1 to the number of sheets in the format. Pass the value of this parameter as null to use the first format sheet.

The drawing model is specified by the *DrawingModel* parameter. Pass the value of this parameter as null to select the current drawing model.

---

## Example: Listing Drawing Sheets

The sample code in the file `pfcDrawingExamples.vb` located at `<creo_vbapi_loadpoint>/vbapi_examples` shows how to list the sheets in the current drawing. The information is placed in an external browser window.

## Drawing Views

A drawing view is represented by the interface `IpfcView2D`. All model views in the drawing are associative, that is, if you change a dimensional value in one view, the system updates other drawing views accordingly. The model automatically reflects any dimensional changes that you make to a drawing. In addition, corresponding drawings also reflect any changes that you make to a model such as the addition or deletion of features and dimensional changes.

## Creating Drawing Views

Method Introduced:

- **`IpfcModel2D.CreateView()`**

The method `IpfcModel2D.CreateView()` creates a new view in the drawing. Before calling this method, the drawing must be displayed in a window.

The interface `IpfcView2DCreateInstructions` contains details on how to create the view. The types of drawing views supported for creation are:

- `EpfcDRAWVIEW_GENERAL`—General drawing views
- `EpfcDRAWVIEW_PROJECTION`—Projected drawing views

### General Drawing Views

The interface `IpfcGeneralViewCreateInstructions` contains details on how to create general drawing views.

Methods and Properties Introduced:

- **`CCpfcGeneralViewCreateInstructions.Create()`**
- **`IpfcGeneralViewCreateInstructions.ViewModel`**
- **`IpfcGeneralViewCreateInstructions.Location`**
- **`IpfcGeneralViewCreateInstructions.SheetNumber`**
- **`IpfcGeneralViewCreateInstructions.Orientation`**
- **`IpfcGeneralViewCreateInstructions.Exploded`**
- **`IpfcGeneralViewCreateInstructions.Scale`**
- **`IpfcGeneralViewCreateInstructions.ViewScale`**



---

The method `CCpfcGeneralViewCreateInstructions.Create()` creates the `IpfcGeneralViewCreateInstructions` data object used for creating general drawing views.

Use the property `IpfcGeneralViewCreateInstructions.ViewModel` to assign the solid model to display in the created general drawing view.

Use the property

`IpfcGeneralViewCreateInstructions.SheetNumber` to assign the location in a drawing sheet to place the created general drawing view.

Use the property

`IpfcGeneralViewCreateInstructions.SheetNumber` to set the number of the drawing sheet in which the general drawing view is created.

The property `IpfcGeneralViewCreateInstructions.Orientation` assigns the orientation of the model in the general drawing view in the form of the `IpfcTransform3D` data object. The transformation matrix must only consist of the rotation to be applied to the model. It must not consist of any displacement or scale components. If necessary, set the displacement to `{0, 0, 0}` using the method `IpfcTransform3D.SetOrigin`, and remove any scaling factor by normalizing the matrix.

Use the property `IpfcGeneralViewCreateInstructions.Exploded` to set the created general drawing view to be an exploded view.

Use the property `IpfcGeneralViewCreateInstructions.Scale` to assign a scale to the created general drawing view. This value is optional, if not assigned, the default drawing scale is used.

Use the property `IpfcGeneralViewCreateInstructions.ViewScale` to assign a scale to the created general drawing view. This value is optional, if not assigned, the default drawing scale is used.

## Projected Drawing Views

The interface `IpfcProjectionViewCreateInstructions` contains details on how to create general drawing views.

Methods and Properties Introduced:

- **`CCpfcProjectionViewCreateInstructions.Create()`**
- **`IpfcProjectionViewCreateInstructions.ParentView`**
- **`IpfcProjectionViewCreateInstructions.Location`**
- **`IpfcProjectionViewCreateInstructions.Exploded`**

The method `CCpfcProjectionViewCreateInstructions.Create()` creates the `IpfcProjectionViewCreateInstructions` data object used for creating projected drawing views.

---

Use the property

`IpfcProjectionViewCreateInstructions.ParentView` to assign the parent view for the projected drawing view.

Use the property

`IpfcProjectionViewCreateInstructions.Location` to assign the location of the projected drawing view. This location determines how the drawing view will be oriented.

Use the property

`IpfcProjectionViewCreateInstructions.Exploded` to set the created projected drawing view to be an exploded view.

### Example: Creating Drawing Views

The sample code in the file `pfcDrawingExamples.vb` located at `<creo_weblink_loadpoint>/weblinkexamples/jscript<creo_vbapi_loadpoint>/vbapi_examples` adds a new sheet to a drawing and creates three views of a selected model.

## Obtaining Drawing Views

Methods and Property Introduced:

- **`IpfcSelection.SelView2D`**
- **`IpfcpfcModel2D.List2DViews()`**
- **`IpfcModel2D.GetViewByName()`**
- **`IpfcModel2D.GetViewDisplaying()`**
- **`IpfcSheetOwner.GetSheetBackgroundView()`**

The property `IpfcSelection.SelView2D` returns the selected drawing view (if the user selected an item from a drawing view). It returns a null value if the selection does not contain a drawing view.

The method `IpfcpfcModel2D.List2DViews()` lists and returns the drawing views found. This method does not include the drawing sheet background views returned by the method `IpfcSheetOwner.GetSheetBackgroundView()`.

The method `IpfcModel2D.GetViewByName()` returns the drawing view based on the name. This method returns a null value if the specified view does not exist.

The method `IpfcModel2D.GetViewDisplaying()` returns the drawing view that displays a dimension. This method returns a null value if the dimension is not displayed in the drawing.

---

 **Note**

This method works for solid and drawing dimensions.

---

The method `IpfcSheetOwner.GetSheetBackgroundView()` returns the drawing sheet background views.

## Drawing View Information

Methods and Properties Introduced:

- **`IpfcChild.DBParent`**
- **`IpfcView2D.GetSheetNumber()`**
- **`IpfcView2D.IsBackground`**
- **`IpfcView2D.GetModel()`**
- **`IpfcView2D.Scale()`**
- **`IpfcView2D.GetIsScaleUserdefined()`**
- **`IpfcView2D.Outline`**
- **`IpfcView2D.GetLayerDisplayStatus()`**
- **`IpfcView2D.IsViewdisplayLayerDependent`**
- **`IpfcView2D.Display`**
- **`IpfcView2D.GetTransform()`**
- **`IpfcView2D.Name`**
- **`IpfcView2D.GetSimpRep()`**

The inherited property `IpfcChild.DBParent`, when called on a `IpfcView2D` object, provides the drawing model which owns the specified drawing view. The return value of the method can be downcast to a `IpfcModel2D` object.

---

 **Note**

The property `IpfcChild.OId` is reserved for internal use.

---

The method `IpfcView2D.GetSheetNumber()` returns the sheet number of the sheet that contains the drawing view.

The property `IpfcView2D.IsBackground` returns a value that indicates whether the view is a background view or a model view.

---

The method `IpfcView2D.GetModel()` returns the solid model displayed in the drawing view.

The property `IpfcView2D.Scale()` returns the scale of the drawing view.

The method `IpfcView2D.GetIsScaleUserdefined()` specifies if the drawing has a user-defined scale.

The property `IpfcView2D.Outline` returns the position of the view in the sheet in world units.

The method `IpfcView2D.GetLayerDisplayStatus()` returns the display status of the specified layer in the drawing view.

The property `IpfcView2D.Display` returns an output structure that describes the display settings of the drawing view. The fields in the structure are as follows:

- *Style*—Whether to display as wireframe, hidden lines, no hidden lines, or shaded
- *TangentStyle*—Linestyle used for tangent edges
- *CableStyle*—Linestyle used to display cables
- *RemoveQuiltHiddenLines*—Whether or not to apply hidden-line-removal to quilts
- *ShowConceptModel*—Whether or not to display the skeleton
- *ShowWeldXSection*—Whether or not to include welds in the cross-section

The method `IpfcView2D.GetTransform()` returns a matrix that describes the transform between 3D solid coordinates and 2D world units for that drawing view. The transformation matrix is a combination of the following factors:

- The location of the view origin with respect to the drawing origin.
- The scale of the view units with respect to the drawing units
- The rotation of the model with respect to the drawing coordinate system.

The property `IpfcView2D.Name` returns the name of the specified view in the drawing.

The simplified representations of assembly and part can be used as drawing models to create general views. Use the method `IpfcView2D.GetSimpRep()` to retrieve the simplified representation for the specified view in the drawing.

## Example: Listing the Views in a Drawing

The sample code in the file `pfcDrawingExamples.vb` located at `<creo_vbapi_loadpoint>/vbapi_examples` creates an information window about all the views in a drawing. The information is placed in an external browser window.

---

## Drawing Views Operations

Methods Introduced:

- **IpfcView2D.Translate()**
- **IpfcView2D.Delete()**
- **IpfcView2D.Regenerate()**
- **IpfcView2D.SetLayerDisplayStatus()**

The method `IpfcView2D.Translate()` moves the drawing view by the specified transformation vector.

The method `IpfcView2D.Delete()` deletes a specified drawing view. Set the *DeleteChildren* parameter to `true` to delete the children of the view. Set this parameter to `false` or `null` to prevent deletion of the view if it has children.

The method `IpfcView2D.Regenerate()` erases the displayed view of the current object, regenerates the view from the current drawing, and redisplay the view.

The method `IpfcView2D.SetLayerDisplayStatus()` sets the display status for the layer in the drawing view.

## Drawing Dimensions

This section describes the VB API methods that give access to the types of dimensions that can be created in the drawing mode. They do not apply to dimensions created in the solid mode, either those created automatically as a result of feature creation, or reference dimension created in a solid. A drawing dimension or a reference dimension shown in a drawing is represented by the interface `IpfcDimension2D`.

## Obtaining Drawing Dimensions

Methods and Property Introduced:

- **IpfcModelItemOwner.ListItems()**
- **IpfcModelItemOwner.GetItemById()**
- **IpfcSelection.SellItem**

The method `IpfcModelItemOwner.ListItems()` returns a list of drawing dimensions specified by the parameter *Type* or returns `null` if no drawing dimensions of the specified type are found. This method lists only those dimensions created in the drawing.

The values of the parameter *Type* for the drawing dimensions are:

- `ITEM_DIMENSION`—Dimension

- `ITEM_REF_DIMENSION`—Reference dimension

Set the parameter *Type* to the type of drawing dimension to retrieve. If this parameter is set to null, then all the dimensions in the drawing are listed.

The method `IpfcModelItemOwner.GetItemById()` returns a drawing dimension based on the type and the integer identifier. The method returns only those dimensions created in the drawing. It returns a null if a drawing dimension with the specified attributes is not found.

The property `IpfcSelection.SelectedItem` returns the value of the selected drawing dimension.

## Creating Drawing Dimensions

Methods Introduced:

- `CCpfcDrawingDimCreateInstructions.Create()`
- `IpfcModel2D.CreateDrawingDimension()`
- `CCpfcEmptyDimensionSense.Create()`
- `CCCpfcPointDimensionSense.Create()`
- `CCpfcSplinePointDimensionSense.Create()`
- `CCpfcTangentIndexDimensionSense.Create()`
- `CCpfcLinAOCTangentDimensionSense.Create()`
- `CCpfcAngleDimensionSense.Create()`
- `CCpfcPointToAngleDimensionSense.Create()`

The method `CCpfcDrawingDimCreateInstructions.Create()` creates an instructions object that describes how to create a drawing dimension using the method `IpfcModel2D.CreateDrawingDimension()`.

The parameters of the instruction object are:

- *Attachments*—The entities that the dimension is attached to. The selections should include the drawing model view.
- *IsRefDimension*—True if the dimension is a reference dimension, otherwise null or false.
- *OrientationHint*—Describes the orientation of the dimensions in cases where this cannot be deduced from the attachments themselves.
- *Senses*—Gives more information about how the dimension attaches to the entity, i.e., to what part of the entity and in what direction the dimension runs. The types of dimension senses are as follows:

- `EpfcDIMSENSE_NONE`
- `EpfcDIMSENSE_POINT`

- `EpfcDIMSENSE_SPLINE_PT`
- `EpfcDIMSENSE_TANGENT_INDEX`
- `EpfcDIMSENSE_LINEAR_TO_ARC_OR_CIRCLE_TANGENT`
- `EpfcDIMSENSE_ANGLE`
- `EpfcDIMSENSE_POINT_TO_ANGLE`

- *TextLocation*—The location of the dimension text, in world units.

The method `IpfcModel2D.CreateDrawingDimension()` creates a dimension in the drawing based on the instructions data object that contains information needed to place the dimension. It takes as input an array of `pfcSelection` objects and an array of `pfcDimensionSense` structures that describe the required attachments. The method returns the created drawing dimension.

The method `CCpfcEmptyDimensionSense.Create()` creates a new dimension sense associated with the type `DIMSENSE_NONE`. The sense field is set to *Type*. In this case no information such as location or direction is needed to describe the attachment points. For example, if there is a single attachment which is a straight line, the dimension is the length of the straight line. If the attachments are two parallel lines, the dimension is the distance between them.

The method `CCCpfcPointDimensionSense.Create()` creates a new dimension sense associated with the type `DIMSENSE_POINT` which specifies the part of the entity to which the dimension is attached. The sense field is set to the value of the parameter *PointType*.

The possible values of *PointType* are:

- `EpfcDIMPOINT_END1`—The first end of the entity
- `EpfcDIMPOINT_END2`—The second end of the entity
- `EpfcDIMPOINT_CENTER`—The center of an arc or circle
- `EpfcDIMPOINT_NONE`—No information such as location or direction of the attachment is specified. This is similar to setting the *PointType* to `DIMSENSE_NONE`.
- `EpfcDIMPOINT_MIDPOINT`—The mid point of the entity

The method `CCpfcSplinePointDimensionSense.Create()` creates a dimension sense associated with the type `DIMSENSE_SPLINE_PT`. This means that the attachment is to a point on a spline. The sense field is set to *SplinePointIndex* i.e., the index of the spline point.

The method `CCpfcTangentIndexDimensionSense.Create()` creates a new dimension sense associated with the type `DIMSENSE_TANGENT_INDEX`. The attachment is to a tangent of the entity, which is an arc or a circle. The sense field is set to *TangentIndex*, i.e., the index of the tangent of the entity.

---

The method `CCpfcLinAOCTangentDimensionSense.Create()` creates a new dimension sense associated with the type `DIMSENSE_LINEAR_TO_ARC_OR_CIRCLE_TANGENT`. The dimension is the perpendicular distance between the a line and a tangent to an arc or a circle that is parallel to the line. The sense field is set to the value of the parameter *TangentType*.

The possible values of *TangentType* are:

- `EpfcDIMLINAOCTANGENT_LEFT0`—The tangent is to the left of the line, and is on the same side, of the center of the arc or circle, as the line.
- `EpfcDIMLINAOCTANGENT_RIGHT0`—The tangent is to the right of the line, and is on the same side, of the center of the arc or circle, as the line.
- `EpfcDIMLINAOCTANGENT_LEFT1`—The tangent is to the left of the line, and is on the opposite side of the line.
- `EpfcDIMLINAOCTANGENT_RIGHT1`— The tangent is to the right of the line, and is on the opposite side of the line.

The method `CCpfcAngleDimensionSense.Create()` creates a new dimension sense associated with the type `DIMSENSE_ANGLE`. The dimension is the angle between two straight entities. The sense field is set to the value of the parameter *AngleOptions*.

The possible values of *AngleOptions* are:

- `IsFirst`—Is set to `TRUE` if the angle dimension starts from the specified entity in a counterclockwise direction. Is set to `FALSE` if the dimension ends at the specified entity. The value is `TRUE` for one entity and `FALSE` for the other entity forming the angle.
- `ShouldFlip`—If the value of `ShouldFlip` is `FALSE`, and the direction of the specified entity is away from the vertex of the angle, then the dimension attaches directly to the entity. If the direction of the entity is away from the vertex of the angle, then the dimension is attached to the a witness line. The witness line is in line with the entity but in the direction opposite to the vertex of the angle. If the value of `ShouldFlip` is `TRUE` then the above cases are reversed.

The method `CCpfcPointToAngleDimensionSense.Create()` creates a new dimension sense associated with the type `DIMSENSE_POINT_TO_ANGLE`. The dimension is the angle between a line entity and the tangent to a curved entity. The curve attachment is of the type `DIMSENSE_POINT_TO_ANGLE` and the line attachment is of the type `DIMSENSE_POINT`. In this case both the angle and the angle\_sense fields must be set. The field sense shows which end of the curve the dimension is attached to and the field angle\_sense shows the direction in which the dimension rotates and to which side of the tangent it attaches.



---

## Drawing Dimensions Information

Methods and Properties Introduced:

- **`IpfcDimension2D.IsAssociative`**
- **`IpfcDimension2D.GetIsReference()`**
- **`IpfcDimension2D.IsDisplayed`**
- **`IpfcDimension2D.GetAttachmentPoints()`**
- **`IpfcDimension2D.GetDimensionSenses()`**
- **`IpfcDimension2D.GetOrientationHint()`**
- **`IpfcDimension2D.GetBaselineDimension()`**
- **`IpfcDimension2D.Location`**
- **`IpfcDimension2D.GetView()`**
- **`IpfcDimension2D.GetTolerance()`**
- **`IpfcDimension2D.IsToleranceDisplayed`**

The property `IpfcDimension2D.IsAssociative` returns whether the dimension or reference dimension in a drawing is associative.

The method `IpfcDimension2D.GetIsReference()` determines whether the drawing dimension is a reference dimension.

The property `IpfcDimension2D.IsDisplayed` determines whether the dimension will be displayed in the drawing.

The method `IpfcDimension2D.GetAttachmentPoints()` returns a sequence of attachment points. The dimension senses array returned by the method `IpfcDimension2D.GetDimensionSenses()` gives more information on how these attachments are interpreted.

The method `IpfcDimension2D.GetDimensionSenses()` returns a sequence of dimension senses, describing how the dimension is attached to each attachment returned by the method

`IpfcDimension2D.GetAttachmentPoints()`.

The method `IpfcDimension2D.GetOrientationHint()` returns the orientation hint for placing the drawing dimensions. The orientation hint determines how Creo Parametric will orient the dimension with respect to the attachment points.



### Note

This methods described above are applicable only for dimensions created in the drawing mode. It does not support dimensions created at intersection points of entities.

---

---

The method `IpfcDimension2D.GetBaselineDimension()` returns an ordinate baseline drawing dimension. It returns a null value if the dimension is not an ordinate dimension.

---

 **Note**

The method updates the display of the dimension only if it is currently displayed.

---

The property `IpfcDimension2D.Location` returns the placement location of the dimension.

The method `IpfcDimension2D.GetView()` returns the drawing view in which the dimension is displayed. This method applies to dimensions stored in the solid or in the drawing.

The method `IpfcDimTolerance` object. A null value indicates a nominal tolerance.

Use the property `IpfcDimension2D.IsToleranceDisplayed` determines whether or not the dimension's tolerance is displayed in the drawing.

## Drawing Dimensions Operations

Methods Introduced:

- **`IpfcDimension2D.ConvertToLinear()`**
- **`IpfcDimension2D.ConvertToOrdinate()`**
- **`IpfcDimension2D.ConvertToBaseline()`**
- **`IpfcDimension2D.SwitchView()`**
- **`IpfcDimension2D.SetTolerance()`**
- **`IpfcDimension2D.EraseFromModel2D()`**
- **`IpfcModel2D.SetViewDisplaying()`**

The method `IpfcDimension2D.ConvertToLinear()` converts an ordinate drawing dimension to a linear drawing dimension. The drawing containing the dimension must be displayed.

The method `IpfcDimension2D.ConvertToOrdinate()` converts a linear drawing dimension to an ordinate baseline dimension.

The method `IpfcDimension2D.ConvertToBaseline()` converts a location on a linear drawing dimension to an ordinate baseline dimension. The method returns the newly created baseline dimension.

---

## Note

The method updates the display of the dimension only if it is currently displayed.

---

The method `IpfcDimension2D.SwitchView()` changes the view where a dimension created in the drawing is displayed.

The method `IpfcDimension2D.SetTolerance()` assigns the upper and lower tolerance limits of the drawing dimension.

The method `IpfcDimension2D.EraseFromModel2D()` permanently erases the dimension from the drawing.

The method `IpfcModel2D.SetViewDisplaying()` changes the view where a dimension created in a solid model is displayed.

### Example: Command Creation of Dimensions from Model Datum Points

The sample code in the file `pfcDrawingExamples.vb` located at `<creo_vbapi_loadpoint>/vbapi_examples` shows a command which creates vertical and horizontal ordinate dimensions from each datum point in a model in a drawing view to a selected coordinate system datum.

## Drawing Tables

A drawing table in the VB API is represented by the `IpfcTable`. It is a child of the `IpfcModelItem` interface.

Some drawing table methods operate on specific rows or columns. The row and column numbers in the VB API begin with 1 and range up to the total number of rows or columns in the table. Some drawing table methods operate on specific table cells. The interface `IpfcTableCell` is used to represent a drawing table cell.

## Creating Drawing Cells

Method Introduced:

- **CCpfcTableCell.Create()**

The method `CCpfcTableCell.Create()` creates the `IpfcTableCell` object representing a cell in the drawing table.

---

Some drawing table methods operate on specific drawing segment. A multisegmented drawing table contains 2 or more areas in the drawing. Inserting or deleting rows in one segment of the table can affect the contents of other segments. Table segments are numbered beginning with 0. If the table has only a single segment, use 0 as the segment id in the relevant methods.

## Selecting Drawing Tables and Cells

Methods and Properties Introduced:

- **`IpfcBaseSession.Select()`**
- **`IpfcSelection.SelItem`**
- **`IpfcSelection.SelTableCell`**
- **`IpfcSelection.SelTableSegment`**

Tables may be selected using the method `IpfcBaseSession.Select()`. Pass the filter `dwg_table` to select an entire table and the filter `table_cell` to prompt the user to select a particular table cell.

The property `IpfcSelection.SelItem` returns the selected table handle. It is a model item that can be cast to a `IpfcTable` object.

The property `IpfcSelection.SelTableCell` returns the row and column indices of the selected table cell.

The property `IpfcSelection.SelTableSegment` returns the table segment identifier for the selected table cell. If the table consists of a single segment, this method returns the identifier 0.

## Creating Drawing Tables

Methods Introduced:

- **`CCpfcTableCreateInstructions.Create()`**
- **`IpfcTableOwner.CreateTable()`**

The method `CCpfcTableCreateInstructions.Create()` creates the `IpfcTableCreateInstructions` data object that describes how to construct a new table using the method `IpfcTableOwner.CreateTable()`.

The parameters of the instructions data object are:

- *Origin*—This parameter stores a three dimensional point specifying the location of the table origin. The origin is the position of the top left corner of the table.
- *RowHeights*—Specifies the height of each row of the table.
- *ColumnData*—Specifies the width of each column of the table and its justification.

- 
- *SizeTypes*—Indicates the scale used to measure the column width and row height of the table.

The method `IpfcTableOwner.CreateTable()` creates a table in the drawing specified by the `IpfcTableCreateInstructions` data object.

## Retrieving Drawing Tables

Methods Introduced

- **`CCpfcTableRetrieveInstructions.Create()`**
- **`IpfcTableRetrieveInstructions.FileName`**
- **`IpfcTableRetrieveInstructions.Path`**
- **`IpfcTableRetrieveInstructions.Version`**
- **`IpfcTableRetrieveInstructions.Position`**
- **`IpfcTableRetrieveInstructions.ReferenceSolid`**
- **`IpfcTableRetrieveInstructions.ReferenceRep`**
- **`IpfcTableOwner.RetrieveTable()`**
- **`IpfcTableOwner.RetrieveTableByOrigin()`**

The method `IpfcTableOwner.RetrieveTable()` retrieves a table specified by the `pfcTableRetrieveInstructions` data object from a file on the disk. It returns the retrieved table. The data object contains information on the table to retrieve and is returned by the method `CCpfcTableRetrieveInstructions.Create()`.

The method `CCpfcTableRetrieveInstructions.Create()` creates the `IpfcTableRetrieveInstructions` data object that describes how to retrieve a drawing table using the methods `IpfcTableOwner.RetrieveTable()` and `IpfcTableOwner.RetrieveTableByOrigin()`. The method returns the created instructions data object.

The parameters of the instruction object are:

- *FileName*—Name of the file containing the drawing table.
- *Position*—Coordinates of the point on the drawing sheet, where the retrieved table must be placed. You must specify the value in screen coordinates.

You can also set the parameters for `IpfcTableRetrieveInstructions` data object using the following property:

- `IpfcTableRetrieveInstructions.FileName`—Sets the name of the drawing table. You must not specify the extension.

- `IpfcTableRetrieveInstructions.Path`—Sets the path to the drawing table file. The path must be specified relative to the working directory.
- `IpfcTableRetrieveInstructions.Version`—Sets the version of the drawing table that must be retrieved. If you specify `NULL`, the latest version of the drawing table is retrieved.
- `IpfcTableRetrieveInstructions.Position`—Sets the coordinates of the point on the drawing sheet, where the table must be placed. You must specify the value in screen coordinates.
- `IpfcTableRetrieveInstructions.ReferenceSolid`—Sets the model from which data must be copied into the drawing table. If this argument is passed as `NULL`, an empty table is created.
- `IpfcTableRetrieveInstructions.ReferenceRep`—Sets the handle to the simplified representation in a solid, from which data must be copied into the drawing table. If this argument is passed as `NULL`, and the argument *solid* is not `NULL`, then data from the solid model is copied into the drawing table

The method `IpfcTableOwner.RetrieveTable()` retrieves a table specified by the `IpfcTableRetrieveInstructions` data object from a file on the disk. It returns the retrieved table. The upper-left corner of the table is placed on the drawing sheet at the position specified by the `IpfcTableRetrieveInstructions` data object.

The method `IpfcTableOwner.RetrieveTableByOrigin()` also retrieves a table specified by the `IpfcTableRetrieveInstructions` data object from a file on the disk. The origin of the table is placed on the drawing sheet at the position specified by the `IpfcTableRetrieveInstructions` data object. Tables can be created with different origins by specifying the option **Direction**, in the **Insert Table** dialog box.

## Drawing Tables Information

Methods Introduced:

- `IpfcTableOwner.ListTables()`
- `IpfcTableOwner.GetTable()`
- `IpfcTable.GetRowCount()`
- `IpfcTable.GetColumnCount()`
- `IpfcTable.CheckIfIsFromFormat()`
- `IpfcTable.GetRowSize()`
- `IpfcTable.GetColumnSize()`
- `IpfcTable.GetText()`

---

- **IpfcTable.GetCellNote()**

The method `IpfcTableOwner.ListTables()` returns a sequence of tables found in the model.

The method `IpfcTableOwner.GetTable()` returns a table specified by the table identifier in the model. It returns a null value if the table is not found.

The method `IpfcTable.GetRowCount()` returns the number of rows in the table.

The method `IpfcTable.GetColumnCount()` returns the number of columns in the table.

The method `IpfcTable.CheckIfIsFromFormat()` checks if the drawing table was created using the format. The method returns a true value if the table was created by applying the drawing format.

The method `IpfcTable.GetRowSize()` returns the height of the drawing table row specified by the segment identifier and the row number.

The method `IpfcTable.GetColumnSize()` returns the width of the drawing table column specified by the segment identifier and the column number.

The method `IpfcTable.GetText()` returns the sequence of text in a drawing table cell. Set the value of the parameter *Mode* to `DWGTABLE_NORMAL` to get the text as displayed on the screen. Set it to `DWGTABLE_FULL` to get symbolic text, which includes the names of parameter references in the table text.

The method `IpfcTable.GetCellNote()` returns the detail note item contained in the table cell.

## Drawing Tables Operations

Methods Introduced:

- **IpfcTable.Erase()**
- **IpfcTable.Display()**
- **IpfcTable.RotateClockwise()**
- **IpfcTable.InsertRow()**
- **IpfcTable.InsertColumn()**
- **IpfcTable.MergeRegion()**
- **IpfcTable.SubdivideRegion()**
- **IpfcTable.DeleteRow()**
- **IpfcTable.DeleteColumn()**
- **IpfcTable.SetText()**
- **IpfcTableOwner.DeleteTable()**

---

The method `IpfcTable.Erase()` erases the specified table temporarily from the display. It still exists in the drawing. The erased table can be displayed again using the method `IpfcTable.Display()`. The table will also be redisplayed by a window repaint or a regeneration of the drawing. Use these methods to hide a table from the display while you are making multiple changes to the table.

The method `IpfcTable.RotateClockwise()` rotates a table clockwise by the specified amount of rotation.

The method `IpfcTable.InsertRow()` inserts a new row in the drawing table. Set the value of the parameter *RowHeight* to specify the height of the row. Set the value of the parameter *InsertAfterRow* to specify the row number after which the new row has to be inserted. Specify 0 to insert a new first row.

The method `IpfcTable.InsertColumn()` inserts a new column in the drawing table. Set the value of the parameter *ColumnWidth* to specify the width of the column. Set the value of the parameter *InsertAfterColumn* to specify the column number after which the new column has to be inserted. Specify 0 to insert a new first column.

The method `IpfcTable.MergeRegion()` merges table cells within a specified range of rows and columns to form a single cell. The range is a rectangular region specified by the table cell on the upper left of the region and the table cell on the lower right of the region.

The method `IpfcTable.SubdivideRegion()` removes merges from a region of table cells that were previously merged. The region to remove merges is specified by the table cell on the upper left of the region and the table cell on the lower right of the region.

The methods `IpfcTable.DeleteRow()` and `IpfcTable.DeleteColumn()` delete any specified row or column from the table. The methods also remove the text from the affected cells.

The method `IpfcTable.SetText()` sets text in the table cell.

Use the method `IpfcTableOwner.DeleteTable()` to delete a specified drawing table from the model permanently. The deleted table cannot be displayed again.



#### **Note**

Many of the above methods provide a parameter *Repaint*. If this is set to true the table will be repainted after the change. If set to false or null Creo Parametric will delay the repaint, allowing you to perform several operations before showing changes on the screen.

---



---

### Example: Creation of a Table Listing Datum Points

The sample code in the file `pfcDrawingExamples.vb` located at `<creo_vbapi_loadpoint>/vbapi_examples` creates a drawing table that lists the datum points in a model shown in a drawing view.

## Drawing Table Segments

Drawing tables can be constructed with one or more segments. Each segment can be independently placed. The segments are specified by an integer identifier starting with 0.

Methods and Property Introduced:

- **`IpfcSelection.SelTableSegment`**
- **`IpfcTable.GetSegmentCount()`**
- **`IpfcTable.GetSegmentSheet()`**
- **`IpfcTable.MoveSegment()`**
- **`IpfcTable.GetInfo()`**

The property `IpfcSelection.SelTableSegment` returns the value of the segment identifier of the selected table segment. It returns a null value if the selection does not contain a segment identifier.

The method `IpfcTable.GetSegmentCount()` returns the number of segments in the table.

The method `IpfcTable.GetSegmentSheet()` determines the sheet number that contains a specified drawing table segment.

The method `IpfcTable.MoveSegment()` moves a drawing table segment to a new location. Pass the coordinates of the target position in the format `x, y, z=0`.



### Note

Set the value of the parameter *Repaint* to true to repaint the drawing with the changes. Set it to false or null to delay the repaint.

---

To get information about a drawing table pass the value of the segment identifier as input to the method `IpfcTable.GetInfo()`. The method returns the table information including the rotation, row and column information, and the 3D outline.

## Repeat Regions

Methods Introduced:

- 
- **IpfcTable.IsCommentCell()**
  - **IpfcTable.GetCellComponentModel()**
  - **IpfcTable.GetCellReferenceModel()**
  - **IpfcTable.GetCellTopModel()**
  - **IpfcTableOwner.UpdateTables()**

The methods `IpfcTable.IsCommentCell()`, `IpfcTable.GetCellComponentModel()`, `IpfcTable.GetCellReferenceModel()`, `IpfcTable.GetCellTopModel()`, and `IpfcTableOwner.UpdateTables()` apply to repeat regions in drawing tables.

The method `IpfcTable.IsCommentCell()` tells you whether a cell in a repeat region contains a comment.

The method `IpfcTable.GetCellComponentModel()` returns the path to the assembly component model that is being referenced by a cell in a repeat region of a drawing table. It does not return a valid path if the cell attribute is set to NO DUPLICATE or NO DUPLICATE/LEVEL.

The method `IpfcTable.GetCellReferenceModel()` returns the reference component that is being referred to by a cell in a repeat region of a drawing table, even if cell attribute is set to NO DUPLICATE or NO DUPLICATE/LEVEL.

The method `IpfcTable.GetCellTopModel()` returns the top model that is being referred to by a cell in a repeat region of a drawing table, even if cell attribute is set to NO DUPLICATE or NO DUPLICATE/LEVEL.

Use the method `IpfcTableOwner.UpdateTables()` to update the repeat regions in all the tables to account for changes to the model. It is equivalent to the command **Table, Repeat Region, Update**.

## Detail Items

The methods described in this section operate on detail items.

In the VB API you can create, delete and modify detail items, control their display, and query what detail items are present in the drawing. The types of detail items available are:

- **Draft Entities**—Contain graphical items created in Creo Parametric. The items are as follows:
  - Arc
  - Ellipse
  - Line

- 
- Point
  - Polygon
  - Spline
  - Notes—Textual annotations
  - Symbol Definitions—Contained in the drawing’s symbol gallery.
  - Symbol Instances—Instances of a symbol placed in a drawing.
  - Draft Groups—Groups of detail items that contain notes, symbol instances, and draft entities.
  - OLE objects—Object Linking and Embedding (OLE) objects embedded in the Creo Parametric drawing file.

## Listing Detail Items

Methods Introduced:

- **`IpfcModelItemOwner.ListItems()`**
- **`IpfcDetailItemOwner.ListDetailItems()`**
- **`IpfcModelItemOwner.GetItemById()`**
- **`IpfcDetailItemOwner.CreateDetailItem()`**

The method `IpfcModelItemOwner.ListItems()` returns a list of detail items specified by the parameter *Type* or returns null if no detail items of the specified type are found.

The values of the parameter *Type* for detail items are:

- `EpfcITEM_DTL_ENTITY`—Detail Entity
- `EpfcITEM_DTL_NOTE`—Detail Note
- `EpfcITEM_DTL_GROUP`—Draft Group
- `EpfcITEM_DTL_SYM_DEFINITION`—Detail Symbol Definition
- `EpfcITEM_DTL_SYM_INSTANCE`—Detail Symbol Instance
- `EpfcITEM_DTL_OLE_OBJECT`—Drawing embedded OLE object

If this parameter is set to null, then all the model items in the drawing are listed.

If the model has multiple bodies, the method

`IpfcModelItemOwner.ListItems()` returns the exception `IpfcXToolkitMultibodyUnsupported`.

The method `IpfcDetailItemOwner.ListDetailItems()` also lists the detail items in the model. Pass the type of the detail item and the sheet number that contains the specified detail items.

---

Set the input parameter *Type* to the type of detail item to be listed. Set it to null to return all the detail items. The input parameter *SheetNumber* determines the sheet that contains the specified detail item. Pass null to search all the sheets. This argument is ignored if the parameter *Type* is set to `EpfcDETAIL_SYM_DEFINITION`.

The method returns a sequence of detail items and returns a null if no items matching the input values are found.

The method `IpfcModelItemOwner.GetItemById()` returns a detail item based on the type of the detail item and its integer identifier. The method returns a null if a detail item with the specified attributes is not found.

## Creating a Detail Item

Methods Introduced:

- **`IpfcDetailItemOwner.CreateDetailItem()`**
- **`CCpfcDetailGroupInstructions.Create()`**

The method `IpfcDetailItemOwner.CreateDetailItem()` creates a new detail item based on the instruction data object that describes the type and content of the new detail item. The instructions data object is returned by the method `CCpfcDetailGroupInstructions.Create()`. The method returns the newly created detail item.

## Detail Entities

A detail entity in the VB API is represented by the interface `IpfcDetailEntityItem`. It is a child of the `IpfcDetailItem` interface.

The interface `IpfcDetailEntityInstructions` contains specific information used to describe a detail entity item.

## Instructions

Methods and Properties Introduced:

- **`CCpfcDetailEntityInstructions.Create()`**
- **`IpfcDetailEntityInstructions.Geometry`**
- **`IpfcDetailEntityInstructions.IsConstruction`**
- **`IpfcDetailEntityInstructions.Color`**
- **`IpfcDetailEntityInstructions.FontName`**
- **`IpfcDetailEntityInstructions.Width`**
- **`IpfcDetailEntityInstructions.View`**

---

The method `CCpfcDetailEntityInstructions.Create()` creates an instructions object that describes how to construct a detail entity, for use in the methods `IpfcDetailItemOwner.CreateDetailItem()`, `IpfcDetailSymbolDefItem.CreateDetailItem()`, and `IpfcDetailEntityItem.Modify()`.

The instructions object is created based on the curve geometry and the drawing view associated with the entity. The curve geometry describes the trajectory of the detail entity in world units. The drawing view can be a model view returned by the method `IpfcModel2D.List2DViews()` or a drawing sheet background view returned by the method

`IpfcSheetOwner.GetSheetBackgroundView()`. The background view indicates that the entity is not associated with a particular model view.

The method returns the created instructions object.



#### Note

Changes to the values of a `IpfcDetailEntityInstructions` object do not take effect until that instructions object is used to modify the entity using `IpfcDetailEntityItem.Modify()`.

---

The property `IpfcDetailEntityInstructions.Geometry` returns the geometry of the detail entity item.

The property `IpfcDetailEntityInstructions.IsConstruction` returns a value that specifies whether the entity is a construction entity.

The property `IpfcDetailEntityInstructions.Color` returns the color of the detail entity item.

The property `IpfcDetailEntityInstructions.FontName` returns the line style used to draw the entity. The method returns a null value if the default line style is used.

The property `IpfcDetailEntityInstructions.Width` returns the value of the width of the entity line. The method returns a null value if the default line width is used.

The property `IpfcDetailEntityInstructions.View` returns the drawing view associated with the entity. The view can either be a model view or a drawing sheet background view.

#### Example: Create a Draft Line with Predefined Color

The sample code in the file `pfcDrawingExamples.vb` located at `<creo_vbapi_loadpoint>/vbapi_examples` shows a utility that creates a draft line in one of the colors predefined in Creo Parametric.

---

## Detail Entities Information

Method and Property Introduced:

- **`IpfcDetailEntityItem.GetInstructions()`**
- **`IpfcDetailEntityItem.SymbolDef`**

The method `IpfcDetailEntityItem.GetInstructions()` returns the instructions data object that is used to construct the detail entity item.

The property `IpfcDetailEntityItem.SymbolDef` returns the symbol definition that contains the entity. This property returns a null value if the entity is not a part of a symbol definition.

## Detail Entities Operations

Methods Introduced:

- **`IpfcDetailEntityItem.Draw()`**
- **`IpfcDetailEntityItem.Erase()`**
- **`IpfcDetailEntityItem.Modify()`**

The method `IpfcDetailEntityItem.Draw()` temporarily draws a detail entity item, so that it is removed during the next draft regeneration.

The method `IpfcDetailEntityItem.Erase()` undraws a detail entity item temporarily, so that it is redrawn during the next draft regeneration.

The method `IpfcDetailEntityItem.Modify()` modifies the definition of an entity item using the specified instructions data object.

## OLE Objects

An object linking and embedding (OLE) object is an external file, such as a document, graphics file, or video file that is created using an external application and which can be inserted into another application, such as Creo Parametric. You can create and insert supported OLE objects into a two-dimensional Creo Parametric file, such as a drawing, report, format file, notebook, or diagram. The functions described in this section enable you to identify and access OLE objects embedded in drawings.

Properties Introduced:

- **`IpfcDetailOLEObject.ApplicationType`**
- **`IpfcDetailOLEObject.Outline`**
- **`IpfcDetailOLEObject.Path`**
- **`IpfcDetailOLEObject.Sheet`**

The property `IpfcDetailOLEObject.ApplicationType` returns the type of the OLE object as a string, for example, Microsoft Word Document.

---

The property `IpfcDetailOLEObject.Outline` returns the extent of the OLE object embedded in the drawing.

The property `IpfcDetailOLEObject.Path` returns the path to the external file for each OLE object, if it is linked to an external file.

The property `IpfcDetailOLEObject.Sheet` returns the sheet number for the OLE object.

## Detail Notes

A detail note in the VB API is represented by the interface `IpfcDetailNoteItem`. It is a child of the `IpfcDetailItem` interface .

The interface `IpfcDetailNoteInstructions` contains specific information that describes a detail note.

The interface `IpfcDetailNoteInstructions` and all the methods under this interface are deprecated.

## Instructions

Methods Introduced:

- **`CCpfcDetailNoteInstructions.Create()`**
- **`IpfcNoteItem.GetTextLines`**
- **`IpfcNoteItem.SetTextLines`**
- **`IpfcNoteItem.IsDisplayed`**
- **`IpfcNoteItem.SetIsDisplayed`**
- **`IpfcNoteItem.IsReadOnly`**
- **`IpfcNoteItem.SetReadOnly`**
- **`IpfcNoteItem.GetNoteTextStyle`**
- **`IpfcNoteItem.SetNoteTextStyle`**
- **`IpfcAnnotationTextStyle.IsTextMirrored`**
- **`IpfcAnnotationTextStyle.MirrorText`**
- **`IpfcAnnotationTextStyle.GetHorizontalJustification`**
- **`IpfcAnnotationTextStyle.SetHorizontalJustification`**
- **`IpfcAnnotationTextStyle.GetVerticalJustification`**
- **`IpfcAnnotationTextStyle.SetVerticalJustification`**
- **`IpfcAnnotationTextStyle.GetColor`**
- **`IpfcAnnotationTextStyle.SetColor`**
- **`IpfcNoteItem.GetElbowLength`**
- **`IpfcNoteItem.SetElbow`**

- **IpfcNoteItem.SetOnItemAttachment**
- **IpfcNoteItem.SetOffsetAttachment**
- **IpfcNoteItem.SetFreeAttachment**
- **IpfcNoteItem.GetAttachment**
- **IpfcAnnotationTextStyle.GetAngle**
- **IpfcAnnotationTextStyle.SetAngle**

Superseded Methods:

- **IpfcNoteInstructions.GetTextLines**
- **IpfcNoteInstructions.SetTextLines**
- **IpfcNoteInstructions.GetIsDisplayed**
- **IpfcNoteInstructions.SetIsDisplayed**
- **IpfcNoteInstructions.GetIsReadOnly**
- **IpfcNoteInstructions.SetIsReadOnly**
- **IpfcNoteInstructions.GetIsMirrored**
- **IpfcNoteInstructions.SetIsMirrored**
- **IpfcNoteInstructions.GetHorizontal**
- **IpfcNoteInstructions.SetHorizontal**
- **IpfcNoteInstructions.GetVertical**
- **IpfcNoteInstructions.SetVertical**
- **IpfcNoteInstructions.GetColor**
- **IpfcNoteInstructions.SetColor**
- **IpfcNoteInstructions.GetLeader**
- **IpfcNoteInstructions.SetLeader**
- **IpfcNoteInstructions.GetTextAngle**
- **IpfcNoteInstructions.SetTextAngle**

The method `IpfcNoteInstructions.Create` creates a data object that describes how a detail note item should be constructed when passed to the methods `IpfcItemOwner.CreateDetailItem`, `IpfcSymbolDefItem.CreateDetailItem`, or `IpfcNoteItem.Modify`. The parameter *inTextLines* specifies the sequence of text line data objects that describe the contents of the note.



---

## **Note**

Changes to the values of a `IpfcNoteInstructions` object do not take effect until that instructions object is used to modify the note using `IpfcNoteItem.Modify`.

For creating a detail note item, the method `IpfcItemOwner.CreateDetailItem` is deprecated, as it uses the deprecated interface `IpfcNoteInstructions`. Use the methods `IpfcItemOwner.CreateFreeNote`, `IpfcItemOwner.CreateOffsetNote`, `IpfcItemOwner.CreateOnItemNote`, or `IpfcItemOwner.CreateLeaderNote` instead.

---

The method `IpfcNoteItem.GetTextLines` returns the description of text line contents in the note.

The method `IpfcNoteItem.SetTextLines` sets the description of the text line contents in the note.

The method `IpfcNoteItem.IsDisplayed` checks if note data is displayed. This is useful for notes whose owner is not displayed in session.

The method `IpfcNoteItem.SetIsDisplayed` sets note data displayed. This is useful for notes whose owner is not displayed in session.

The method `IpfcNoteItem.IsReadOnly` determines whether the note can be edited by the user, while the method `IpfcNoteItem.SetReadOnly` toggles the read only status of the note.

The method `IpfcNoteItem.GetNoteTextStyle` and `IpfcNoteItem.SetNoteTextStyle` get and set the textstyle of note.

The methods `IpfcAnnotationTextStyle.IsTextMirrored` checks if the text style is mirrored.

The methods `IpfcAnnotationTextStyle.MirrorText` sets the mirroring of the text style. specifies the option to mirror the text style. Specify the input argument *Mirror* to true to mirror the text style.

The methods

`IpfcAnnotationTextStyle.GetHorizontalJustification` and `IpfcAnnotationTextStyle.SetHorizontalJustification` get and set the horizontal justification of the text style using the enumerated data type `IpfcHorizontalJustification`. The valid values are:

- `IpfcH_JUSTIFY_LEFT`—Aligns the text style object to the left.
- `IpfcH_JUSTIFY_CENTER`—Aligns the text style object in the center.

- `IpfcH_JUSTIFY_RIGHT`—Aligns the text style object to the right.
- `IpfcH_JUSTIFY_DEFAULT`—Aligns the text using the default justification. The justification selected for the first note becomes the default for all successive notes added during the current session.

The methods

`IpfcAnnotationTextStyle.GetVerticalJustification` and `IpfcAnnotationTextStyle.SetVerticalJustification` get and set the vertical justification of the text style using the enumerated data type `IpfcVerticalJustification`. The valid values are:

- `IpfcV_JUSTIFY_TOP`—Aligns the text style object to the top.
- `IpfcV_JUSTIFY_MIDDLE`—Aligns the text style object to the middle.
- `IpfcV_JUSTIFY_BOTTOM`—Aligns the text style object to the bottom.
- `IpfcV_JUSTIFY_DEFAULT`—Aligns the text using the default justification. The justification selected for the first note becomes the default for all successive notes added during the current session.

The methods `IpfcAnnotationTextStyle.GetColor` and `IpfcAnnotationTextStyle.SetColor` retrieve and set the color of the text style as a `IpfcColorRGB` object.

The method `IpfcNoteItem.GetElbowLength` returns length of note leader elbow.

The method `IpfcNoteItem.SetElbow` sets elbow to leader note.

The method `IpfcNoteItem.SetOnItemAttachment` sets on item attachment information of note.

The method `IpfcNoteItem.SetOffsetAttachment` sets offset attachment information of note.

The method `IpfcNoteItem.SetFreeAttachment` sets free attachment information of note.

The method `IpfcNoteItem.GetAttachment` returns the attachment information of note.

The methods `IpfcAnnotationTextStyle.GetAngle` and `IpfcAnnotationTextStyle.SetAngle` get and set the angle of the text style.

#### Example: Create Drawing Note at Specified Location with Leader to Surface and Surface Name

The sample code in the file `pfcdrawingexamples.vb` located at `<creo_vbapi_loadpoint>/vbapi_examples` creates a drawing note at a specified location, with a leader attached to a solid surface, and displays the name of the surface.

---

## Detail Notes Information

Methods and Property Introduced:

- **`IpfcDetailNoteItem.GetInstructions()`**
- **`IpfcDetailNoteItem.SymbolDef`**
- **`IpfcDetailNoteItem.GetLineEnvelope()`**
- **`IpfcDetailNoteItem.GetModelReference()`**

The method `IpfcDetailNoteItem.GetInstructions()` returns an instructions data object that describes how to construct the detail note item. This method takes a `ProBoolean` argument, *GiveParametersAsNames*, which determines whether symbolic representations of parameters and drawing properties in the note text should be displayed, or the actual text seen by the user should be displayed.



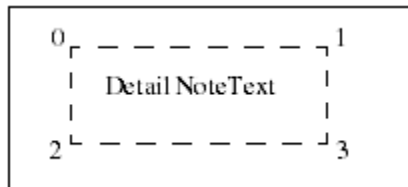
### Note

Creo Parametric does not resolve and replace symbolic callouts for notes which are not displayed. Therefore, if the note is not displayed or is hidden in a layer, the text retrieved may contain symbolic callouts, even when *GiveParametersAsNames* is false.

---

The property `IpfcDetailNoteItem.SymbolDef` returns the symbol definition that contains the note. The method returns a null value if the note is not a part of a symbol definition.

The method `IpfcDetailNoteItem.GetLineEnvelope()` determines the screen coordinates of the envelope around the detail note. This envelope is defined by four points. The following figure illustrates how the point order is determined.



The ordering of the points is maintained even if the notes are mirrored or are at an angle.

The method `IpfcDetailNoteItem.GetModelReference()` returns the model referenced by the parameterized text in a note. The model is referenced based on the line number and the text index where the parameterized text appears.

## Details Notes Operations

Methods Introduced:

- **IpfcDetailNoteItem.Draw()**
- **IpfcDetailNoteItem.Show()**
- **IpfcDetailNoteItem.Erase()**
- **IpfcDetailNoteItem.Remove()**
- **IpfcDetailNoteItem.KeepArrowTypeAsIs()**
- **IpfcDetailNoteItem.Modify()**

The method `IpfcDetailNoteItem.Draw()` temporarily draws a detail note item, so that it is removed during the next draft regeneration.

The method `IpfcDetailNoteItem.Show()` displays the note item, such that it is repainted during the next draft regeneration.

The method `IpfcDetailNoteItem.Erase()` undraws a detail note item temporarily, so that it is redrawn during the next draft regeneration.

The method `IpfcDetailNoteItem.Remove()` undraws a detail note item permanently, so that it is not redrawn during the next draft regeneration.

The method `IpfcDetailNoteItem.KeepArrowTypeAsIs()` allows you to keep arrow type of the leader note as it is, after a note is modified. You must call this method before the method `IpfcDetailNoteItem.Modify()` is called.

The method `IpfcDetailNoteItem.Modify()` modifies the definition of an existing detail note item based on the instructions object that describes the new detail note item.

## Detail Groups

A detail group in the VB API is represented by the interface `IpfcDetailGroupItem`. It is a child of the `IpfcDetailItem` interface.

The interface `IpfcDetailGroupInstructions` contains information used to describe a detail group item.

## Instructions

Method and Properties Introduced:

- **CCpfcDetailGroupInstructions.Create()**
- **IpfcDetailGroupInstructions.Name**
- **IpfcDetailGroupInstructions.Elements**
- **IpfcDetailGroupInstructions.IsDisplayed**

---

The method `CCpfcDetailGroupInstructions.Create()` creates an instruction data object that describes how to construct a detail group for use in `pfcDetailItemOwner.CreateDetailItem()` and `IpfcDetailGroupItem.Modify()`.

---

 **Note**

Changes to the values of a `IpfcDetailGroupInstructions` object do not take effect until that instructions object is used to modify the group using `IpfcDetailGroupItem.Modify()`.

---

The property `IpfcDetailGroupInstructions.Name` returns the name of the detail group.

The property `IpfcDetailGroupInstructions.Elements` returns the sequence of the detail items (notes, groups and entities) contained in the group.

The property `IpfcDetailGroupInstructions.IsDisplayed` returns whether the detail group is displayed in the drawing.

## Detail Groups Information

Method Introduced:

- **`IpfcDetailGroupItem.GetInstructions()`**

The method `IpfcDetailGroupItem.GetInstructions()` gets a data object that describes how to construct a detail group item. The method returns the data object describing the detail group item.

## Detail Groups Operations

Methods Introduced:

- **`IpfcDetailGroupItem.Draw()`**
- **`IpfcDetailGroupItem.Erase()`**
- **`IpfcDetailGroupItem.Modify()`**

The method `IpfcDetailGroupItem.Draw()` temporarily draws a detail group item, so that it is removed during the next draft generation.

The method `IpfcDetailGroupItem.Erase()` temporarily undraws a detail group item, so that it is redrawn during the next draft generation.

The method `IpfcDetailGroupItem.Modify()` changes the definition of a detail group item based on the data object that describes how to construct a detail group item.

---

## Example: Create New Group of Items

The sample code in the file `pfcDrawingExamples.vb` located at `<creo_vbapi_loadpoint>/vbapi_examples` creates a group from a set of selected detail items.

## Detail Symbols

### Detail Symbol Definitions

A detail symbol definition in the VB API is represented by the interface `IpfcDetailSymbolDefItem`. It is a child of the `IpfcDetailItem` interface.

The interface `IpfcDetailSymbolDefInstructions` contains information that describes a symbol definition. It can be used when creating symbol definition entities or while accessing existing symbol definition entities.

#### Instructions

Methods and Properties Introduced:

- **`CCpfcDetailSymbolDefInstructions.Create()`**
- **`IpfcDetailSymbolDefInstructions.SymbolHeight`**
- **`IpfcDetailSymbolDefInstructions.HasElbow`**
- **`IpfcDetailSymbolDefInstructions.IsTextAngleFixed`**
- **`IpfcDetailSymbolDefInstructions.ScaledHeight`**
- **`IpfcDetailSymbolDefInstructions.Attachments`**
- **`IpfcDetailSymbolDefInstructions.FullPath`**
- **`IpfcDetailSymbolDefInstructions.Reference`**

The method `CCpfcDetailSymbolDefInstructions.Create()` creates an instruction data object that describes how to create a symbol definition based on the path and name of the symbol definition. The instructions object is passed to the methods `pfcDetailItemOwner.CreateDetailItem()` and `pfcDetailSymbolDefItem.Modify()`.

---

#### Note

Changes to the values of a `IpfcDetailSymbolDefInstructions` object do not take effect until that instructions object is used to modify the definition using the method `IpfcDetailSymbolDefItem.Modify()`.

---

---

The property `IpfcDetailSymbolDefInstructions.SymbolHeight` returns the value of the height type for the symbol definition. The symbol definition height options are as follows:

- `EpfcSYMDEF_FIXED`—Symbol height is fixed.
- `EpfcSYMDEF_VARIABLE`—Symbol height is variable.
- `EpfcSYMDEF_RELATIVE_TO_TEXT`—Symbol height is determined relative to the text height.

The property `IpfcDetailSymbolDefInstructions.HasElbow` determines whether the symbol definition includes an elbow.

The property

`IpfcDetailSymbolDefInstructions.IsTextAngleFixed` returns whether the text of the angle is fixed.

The property `IpfcDetailSymbolDefInstructions.ScaledHeight` returns the height of the symbol definition in inches.

The `IpfcDetailSymbolDefInstructions.Attachments` returns the value of the sequence of the possible instance attachment points for the symbol definition.

The property `IpfcDetailSymbolDefInstructions.FullPath` returns the value of the complete path of the symbol definition file.

The property `IpfcDetailSymbolDefInstructions.Reference` returns the text reference information for the symbol definition. It returns a null value if the text reference is not used. The text reference identifies the text item used for a symbol definition which has a height type of `SYMDEF_TEXT_RELATED`.

## Detail Symbol Definitions Information

Methods Introduced:

- **`IpfcDetailSymbolDefItem.ListDetailItems()`**
- **`IpfcDetailSymbolDefItem.GetInstructions()`**

The method `IpfcDetailSymbolDefItem.ListDetailItems()` lists the detail items in the symbol definition based on the type of the detail item.

The method `IpfcDetailSymbolDefItem.GetInstructions()` returns an instruction data object that describes how to construct the symbol definition.

## Detail Symbol Definitions Operations

Methods Introduced:

- **`IpfcDetailSymbolDefItem.CreateDetailItem()`**
- **`IpfcDetailSymbolDefItem.Modify()`**

---

The method `IpfcDetailSymbolDefItem.CreateDetailItem()` creates a detail item in the symbol definition based on the instructions data object. The method returns the detail item in the symbol definition.

The method `IpfcDetailSymbolDefItem.Modify()` modifies a symbol definition based on the instructions data object that contains information about the modifications to be made to the symbol definition.

## Retrieving Symbol Definitions

Methods Introduced:

- **`IpfcDetailItemOwner.RetrieveSymbolDefItem()`**

From Creo 4.0 F000 onwards, the method `IpfcDetailItemOwner.RetrieveSymbolDefinition()` has been deprecated. Use the method `IpfcDetailItemOwner.RetrieveSymbolDefItem` instead.

Creo Parametric symbols exist in two different areas: the user-defined area and the system symbols area.

The method `IpfcDetailItemOwner.RetrieveSymbolDefItem()` retrieves a symbol definition from the user-defined location designated by the configuration option `pro_symbol_dir`. The symbol definition should have been previously saved to a file using Creo Parametric.

The method `IpfcDetailItemOwner.RetrieveSymbolDefItem()` also retrieves a symbol definition from the system directory. The system area contains symbols provided by Creo Parametric with the Detail module (such as the Welding Symbols Library).

The input parameters of this method are:

- *FileName*—Name of the symbol definition file.
- *Source*—Source of the symbol definition file. The input parameter *Source* is defined by the enumerated type `pfcDetail.DetailSymbolDefItemSource`. The valid values which are supported are listed below:
  - `DTLSYMDEF_SRC_SYSTEM`—Specifies the system symbol definition directory.
  - `DTLSYMDEF_SRC_PATH`—Specifies the absolute path to a directory containing the symbol definition.
- *FilePath*—Path to the symbol definition file. It is relative to the path specified by the option `pro_symbol_dir` in the configuration file. A null value indicates that the function should search the current directory.
- *Version*—Numerical version of the symbol definition file. A null value retrieves the latest version.



- *UpdateUnconditionally*—True if Creo should update existing instances of this symbol definition, or false to quit the operation if the definition exists in the model.

The method returns the retrieved symbol definition.

#### Example : Create Symbol Definition

The sample code in the file `pfcDrawingExamples.vb` located at `<creo_vbapi_loadpoint>/vbapi_examples` creates a symbol definition which contains four line entities forming a box, a note at the middle of the box, and a free attachment.

## Detail Symbol Instances

A detail symbol instance in the VB API is represented by the interface `IpfcDetailSymbolInstItem`. It is a child of the `IpfcDetailItem` interface .

The interface `IpfcDetailSymbolInstInstructions` contains information that describes a symbol instance. It can be used when creating symbol instances and while accessing existing groups.

## Instructions

Methods and Properties Introduced:

- **`CCpfcDetailSymbolInstInstructions.Create()`**
- **`IpfcDetailSymbolInstInstructions.IsDisplayed`**
- **`IpfcDetailSymbolInstInstructions.Color`**
- **`IpfcDetailSymbolInstInstructions.SymbolDef`**
- **`IpfcDetailSymbolInstInstructions.AttachOnDefType`**
- **`IpfcDetailSymbolInstInstructions.DefAttachment`**
- **`IpfcDetailSymbolInstInstructions.InstAttachment`**
- **`IpfcDetailSymbolInstInstructions.Angle`**
- **`IpfcDetailSymbolInstInstructions.ScaledHeight`**
- **`IpfcDetailSymbolInstInstructions.TextValues`**
- **`IpfcDetailSymbolInstInstructions.CurrentTransform`**
- **`IpfcDetailSymbolInstInstructions.SetGroups()`**

The method `CCpfcDetailSymbolInstInstructions.Create()` creates a data object that contains information about the placement of a symbol instance.

---

## Note

Changes to the values of a `IpfcDetailSymbolInstInstructions` object do not take effect until that instructions object is used to modify the instance using `IpfcDetailSymbolInstItem.Modify()`.

---

The property `IpfcDetailSymbolInstInstructions.IsDisplayed` returns a value that specifies whether the instance of the symbol is displayed.

The property `IpfcDetailSymbolInstInstructions.Color` returns the color of the detail symbol instance. A null value indicates that the default drawing color is used.

The property `IpfcDetailSymbolInstInstructions.SymbolDef` returns the symbol definition used for the instance.

The property

`IpfcDetailSymbolInstInstructions.AttachOnDefType` returns the attachment type of the instance. The method returns a null value if the attachment represents a free attachment. The attachment options are as follows:

- `EpfcSYMDEFATTACH_FREE`—Attachment on a free point.
- `EpfcSYMDEFATTACH_LEFT_LEADER`—Attachment via a leader on the left side of the symbol.
- `EpfcSYMDEFATTACH_RIGHT_LEADER`—Attachment via a leader on the right side of the symbol.
- `EpfcSYMDEFATTACH_RADIAL_LEADER`—Attachment via a leader at a radial location.
- `EpfcSYMDEFATTACH_ON_ITEM`—Attachment on an item in the symbol definition.
- `EpfcSYMDEFATTACH_NORMAL_TO_ITEM`—Attachment normal to an item in the symbol definition.

The property `IpfcDetailSymbolInstInstructions.DefAttachment` returns the value that represents the way in which the instance is attached to the symbol definition.

The property

`IpfcDetailSymbolInstInstructions.InstAttachment` returns the value of the attachment of the instance that includes location and leader information.

The property `IpfcDetailSymbolInstInstructions.Angle` returns the value of the angle at which the instance is placed. The method returns a null value if the value of the angle is 0 degrees.

---

The property `IpfcDetailSymbolInstInstructions.ScaledHeight` returns the height of the symbol instance in the owner drawing or model coordinates. This value is consistent with the height value shown for a symbol instance in the **Properties** dialog box in the Creo Parametric User Interface.

---

 **Note**

The scaled height obtained using the above property is partially based on the properties of the symbol definition assigned using the property `IpfcDetailSymbolInstInstructions.SymbolDef`. Changing the symbol definition may change the calculated value for the scaled height.

---

The property `IpfcDetailSymbolInstInstructions.TextValues` returns the sequence of variant text values used while placing the symbol instance.

The property

`IpfcDetailSymbolInstInstructions.CurrentTransform` returns the coordinate transformation matrix to place the symbol instance.

The method

`IpfcDetailSymbolInstInstructions.SetGroups()` `IpfcDetailSymbolGroupOption`

- `EpfcDETAIL_SYMBOL_GROUP_INTERACTIVE`—Symbol groups are interactively selected for display. This is the default value in the GRAPHICS mode.
- `EpfcDETAIL_SYMBOL_GROUP_ALL`—All non-exclusive symbol groups are included for display.
- `EpfcDETAIL_SYMBOL_GROUP_NONE`—None of the non-exclusive symbol groups are included for display.
- `EpfcDETAIL_SYMBOL_GROUP_CUSTOM`—Symbol groups specified by the application are displayed.

Refer to the section [Detail Symbol Groups on page 133](#) for more information on detail symbol groups.

## Detail Symbol Instances Information

Method Introduced:

- **`IpfcDetailSymbolInstItem.GetInstructions()`**

The method `IpfcDetailSymbolInstItem.GetInstructions()` returns an instructions data object that describes how to construct a symbol instance. This method takes a `ProBoolean` argument, *GiveParametersAsNames*,

---

which determines whether symbolic representations of parameters and drawing properties in the symbol instance should be displayed, or the actual text seen by the user should be displayed.

## Detail Symbol Instances Operations

Methods Introduced:

- **`IpfcDetailSymbolInstItem.Draw()`**
- **`IpfcDetailSymbolInstItem.Erase()`**
- **`IpfcDetailSymbolInstItem.Show()`**
- **`IpfcDetailSymbolInstItem.Remove()`**
- **`IpfcDetailSymbolInstItem.Modify()`**

The method `IpfcDetailSymbolInstItem.Draw()` draws a symbol instance temporarily to be removed on the next draft regeneration.

The method `IpfcDetailSymbolInstItem.Erase()` undraws a symbol instance temporarily from the display to be redrawn on the next draft generation.

The method `IpfcDetailSymbolInstItem.Show()` displays a symbol instance to be repainted on the next draft regeneration.

The method `IpfcDetailSymbolInstItem.Remove()` deletes a symbol instance permanently.

The method `IpfcDetailSymbolInstItem.Modify()` modifies a symbol instance based on the instructions data object that contains information about the modifications to be made to the symbol instance.

### Example: Create a Free Instance of Symbol Definition

The sample code in the file `pfcDrawingExamples.vb` located at `<creo_vbapi_loadpoint>/vbapi_examples` creates a free instance of a symbol definition.

### Example: Create a Free Instance of a Symbol Definition with drawing unit heights, variable text and groups

The sample code in the file `pfcDrawingExamples.vb` located at `<creo_vbapi_loadpoint>/vbapi_examples` creates a free instance of a symbol definition with drawing unit heights, variable text and groups.

---

## Detail Symbol Groups

A detail symbol group in the VB API is represented by the interface `IpfcDetailSymbolGroup`. It is a child of the `IpfcObject` interface. A detail symbol group is accessible only as a part of the contents of a detail symbol definition or instance.

The interface `IpfcDetailSymbolGroupInstructions` contains information that describes a symbol group. It can be used when creating new symbol groups, or while accessing or modifying existing groups.

### Instructions

Methods and Properties Introduced:

- **`CCpfcDetailSymbolGroupInstructions.Create()`**
- **`IpfcDetailSymbolGroupInstructions.Items`**
- **`IpfcDetailSymbolGroupInstructions.Name`**

The method `CCpfcDetailSymbolGroupInstructions.Create()` creates the `IpfcDetailSymbolGroupInstructions` data object that stores the name of the symbol group and the list of detail items to be included in the symbol group.



#### Note

Changes to the values of the `IpfcDetailSymbolGroupInstructions` data object do not take effect until this object is used to modify the instance using the method `IpfcDetailSymbolGroup.Modify()`.

---

The property `IpfcDetailSymbolGroupInstructions.Items` returns the list of detail items included in the symbol group.

The property `IpfcDetailSymbolGroupInstructions.Name` returns the name of the symbol group.

### Detail Symbol Group Information

Methods Introduced:

- **`IpfcDetailSymbolGroup.GetInstructions()`**
- **`IpfcDetailSymbolGroup.ParentGroup`**
- **`IpfcDetailSymbolGroup.ParentDefinition`**
- **`IpfcDetailSymbolGroup.ListChildren()`**
- **`IpfcDetailSymbolDefItem.ListSubgroups()`**

- **IpfcDetailSymbolDefItem.IsSubgroupLevelExclusive()**
- **IpfcDetailSymbolDefItem.IsSubgroupLevelExclusive()**

The method `IpfcDetailSymbolGroup.GetInstructions()` returns the `IpfcDetailSymbolGroupInstructions` data object that describes how to construct a symbol group.

The method `IpfcDetailSymbolGroup.ParentGroup` returns the parent symbol group to which a given symbol group belongs.

The method `IpfcDetailSymbolGroup.ParentDefinition` returns the symbol definition of a given symbol group.

The method `IpfcDetailSymbolGroup.ListChildren()` lists the subgroups of a given symbol group.

The method `IpfcDetailSymbolDefItem.ListSubgroups()` lists the subgroups of a given symbol group stored in the symbol definition at the indicated level.

The method

`IpfcDetailSymbolDefItem.IsSubgroupLevelExclusive()` identifies if the subgroups of a given symbol group stored in the symbol definition at the indicated level are exclusive or independent. If groups are exclusive, only one of the groups at this level can be active in the model at any time. If groups are independent, any number of groups can be active.

The method `IpfcDetailSymbolDefItem.ListSubgroups()` lists the symbol groups included in a symbol instance. The `IpfcSymbolGroupFilter` argument determines the types of symbol groups that can be listed. It takes the following values:

- `EpfcDTLSYMINST_ALL_GROUPS`—Retrieves all groups in the definition of the symbol instance.
- `EpfcDTLSYMINST_ACTIVE_GROUPS`—Retrieves only those groups that are actively shown in the symbol instance.
- `EpfcDTLSYMINST_INACTIVE_GROUPS`—Retrieves only those groups that are not shown in the symbol instance.

## Detail Symbol Group Operations

Methods Introduced:

- **IpfcDetailSymbolGroup.Delete()**
- **IpfcDetailSymbolGroup.Modify()**
- **IpfcDetailSymbolDefItem.CreateSubgroup()**
- **IpfcDetailSymbolDefItem.SetSubgroupLevelExclusive()**
- **IpfcDetailSymbolDefItem.SetSubgroupLevelIndependent()**

---

The method `IpfcDetailSymbolGroup.Delete()` deletes the specified symbol group from the symbol definition. This method does not delete the entities contained in the group.

The method `IpfcDetailSymbolGroup.Modify()` modifies the specified symbol group based on the `IpfcDetailSymbolGroupInstructions` data object that contains information about the modifications that can be made to the symbol group.

The method `IpfcDetailSymbolDefItem.CreateSubgroup()` creates a new subgroup in the symbol definition at the indicated level below the parent group.

The method `IpfcDetailSymbolDefItem.SetSubgroupLevelExclusive()` makes the subgroups of a symbol group exclusive at the indicated level in the symbol definition.

---

 **Note**

After you set the subgroups of a symbol group as exclusive, only one of the groups at the indicated level can be active in the model at any time.

---

The method `IpfcDetailSymbolDefItem.SetSubgroupLevelIndependent()` makes the subgroups of a symbol group independent at the indicated level in the symbol definition.

---

 **Note**

After you set the subgroups of a symbol group as independent, any number of groups at the indicated level can be active in the model at any time.

---

## Detail Attachments

A detail attachment in VB API is represented by the interface `IpfcAttachment`. It is used for the following tasks:

- The way in which a drawing note or a symbol instance is placed in a drawing.
- The way in which a leader on a drawing note or symbol instance is attached.

Method Introduced:

- **`IpfcAttachment.GetType()`**

---

The method `IpfcAttachment.GetType()` returns the `IpfcAttachmentType` object containing the types of detail attachments. The detail attachment types are as follows:

- `EpfcATTACH_FREE`—The attachment is at a free point possibly with respect to a given drawing view.
- `EpfcATTACH_PARAMETRIC`—The attachment is to a point on a surface or an edge of a solid.
- `EpfcATTACH_OFFSET`—The attachment is offset to another drawing view, to a model item, or to a 3D model annotation.
- `EpfcATTACH_TYPE_UNSUPPORTED`—The attachment is to an item that cannot be represented in PFC at the current time. However, you can still retrieve the location of the attachment.

## Free Attachment

The `EpfcATTACH_FREE` detail attachment type is represented by the interface `IpfcFreeAttachment`. It is a child of the `IpfcAttachment` interface.

Properties Introduced:

- **`IpfcFreeAttachment.AttachmentPoint`**
- **`IpfcFreeAttachment.View`**

The property `IpfcFreeAttachment.AttachmentPoint` returns the attachment point. This location is in screen coordinates for drawing items, symbol instances and surface finishes on flat-to-screen annotation planes, and in model coordinates for symbols and surface finishes on 3D model annotation planes.

The method `IpfcFreeAttachment.View` returns the drawing view to which the attachment is related. The attachment point is relative to the drawing view, that is the attachment point moves when the drawing view is moved. This method returns a `NULL` value, if the detail attachment is not related to a drawing view, but is placed at the specified location in the drawing sheet, or if the attachment is offset to a model item or to a 3D model annotation.

## Parametric Attachment

The `EpfcATTACH_PARAMETRIC` detail attachment type is represented by the interface `IpfcParametricAttachment`. It is a child of the `IpfcAttachment` interface.

Property Introduced:

- **`IpfcParametricAttachment.AttachedGeometry`**



---

The property `IpfcParametricAttachment.AttachedGeometry` returns the `IpfcSelection` object representing the item to which the detail attachment is attached. This includes the drawing view in which the attachment is made.

## Offset Attachment

The `EpfcATTACH_OFFSET` detail attachment type is represented by the interface `IpfcOffsetAttachment`. It is a child of the `IpfcAttachment` interface .

Properties Introduced:

- **`IpfcOffsetAttachment.AttachedGeometry`**
- **`IpfcOffsetAttachment.AttachmentPoint`**

The property `IpfcOffsetAttachment.AttachedGeometry` returns the `IpfcSelection` object representing the item to which the detail attachment is attached. This includes the drawing view where the attachment is made, if the offset reference is in a model.

The property `IpfcOffsetAttachment.AttachmentPoint` returns the attachment point. This location is in screen coordinates for drawing items, symbol instances and surface finishes on flat-to-screen annotation planes, and in model coordinates for symbols and surface finishes on 3D model annotation planes. The distance from the attachment point to the location of the item to which the detail attachment is attached is saved as the offset distance.

## Unsupported Attachment

The `EpfcATTACH_TYPE_UNSUPPORTED` detail attachment type is represented by the interface `IpfcUnsupportedAttachment`. It is a child of the `IpfcAttachment` interface .

Property Introduced:

- **`IpfcUnsupportedAttachment.AttachmentPoint`**

The property `IpfcUnsupportedAttachment.AttachmentPoint` returns the attachment point. This location is in screen coordinates for drawing items, symbol instances and surface finishes on flat-to-screen annotation planes, and in model coordinates for symbols and surface finishes on 3D model annotation planes.

---

# Solid

## Solid

Most of the objects and methods in the VB API are used with solid models (parts and assemblies). Because solid objects inherit from the interface `IpfcModel`, you can use any of the `IpfcModel` methods on any `IpfcSolid`, `IpfcPart`, or `IpfcAssembly` object.

## Getting a Solid Object

Methods and Properties Introduced:

- **`IpfcBaseSession.CreatePart()`**
- **`IpfcBaseSession.CreateAssembly()`**
- **`IpfcComponentPath.Root`**
- **`IpfcComponentPath.Leaf`**
- **`IpfcMFG.GetSolid()`**

The methods `IpfcBaseSession.CreatePart()` and `IpfcBaseSession.CreateAssembly()` create new solid models with the names you specify.

The properties `IpfcComponentPath.Root` and `IpfcComponentPath.Leaf` specify the solid objects that make up the component path of an assembly component model. You can get a component path object from any component that has been interactively selected.

The method `IpfcMFG.GetSolid()` retrieves the storage solid in which the manufacturing model's features are placed. In order to create a UDF group in the manufacturing model, call the method `IpfcSolid.CreateUDFGroup()` on the storage solid.

## Solid Information

Properties Introduced:

- **`IpfcSolid.RelativeAccuracy`**
- **`IpfcSolid.AbsoluteAccuracy`**

You can set the relative and absolute accuracy of any solid model using these methods. Relative accuracy is relative to the size of the solid. For example, a relative accuracy of .01 specifies that the solid must be accurate to within 1/100 of its size. Absolute accuracy is measured in absolute units (inches, centimeters, and so on).

---

 **Note**

For a change in accuracy to take effect, you must regenerate the model.

---

## Solid Operations

Methods and Properties Introduced:

- **`IpfcSolid.Regenerate()`**
- **`CCpfcRegenInstructions.Create()`**
- **`IpfcRegenInstructions.AllowFixUI`**
- **`IpfcRegenInstructions.ForceRegen`**
- **`IpfcRegenInstructions.FromFeat`**
- **`IpfcRegenInstructions.RefreshModelTree`**
- **`IpfcRegenInstructions.ResumeExcludedComponents`**
- **`IpfcRegenInstructions.UpdateAssemblyOnly`**
- **`IpfcRegenInstructions.UpdateInstances`**
- **`IpfcRegenInstructions.ResolveModeRegen`**
- **`IpfcSolid.GeomOutline`**
- **`IpfcSolid.EvalOutline()`**
- **`IpfcSolid.IsSkeleton`**
- **`IpfcSolid.ListGroups()`**
- **`IpfcSolid.GetSurfaceSolidBody()`**
- **`IpfcSolid.GetEdgeSolidBody()`**

The method `IpfcSolid.Regenerate()` causes the solid model to regenerate according to the instructions provided in the form of the `IpfcRegenInstructions` object. Passing a null value for the instructions argument causes an automatic regeneration.

Pro/ENGINEER Wildfire 5.0 introduces the No-Resolve mode, wherein if a model and feature regeneration fails, failed features and children of failed features are created and regeneration of other features continues. However, VB API does not support regeneration in this mode. The method `IpfcSolid.Regenerate()` throws an exception `IpfcXToolkitBadContext`, if Creo Parametric is running in the No-Resolve mode. To continue with the Pro/ENGINEER Wildfire 4.0 behavior in the Resolve mode, set the configuration option `regen_failure_handling` to `resolve_mode` in the Creo Parametric session.

---

## Note

Setting the configuration option to switch to Resolve mode ensures the old behavior as long as you do not retrieve the models saved under the No-Resolve mode. To consistently preserve the old behavior, use Resolve mode from the beginning and throughout your Creo Parametric session.

---

The `IpfcRegenInstructions` object contains the following input parameters:

- *AllowFixUI*—Determines whether or not to activate the **Fix Model** user interface, if there is an error.  
Use the property `IpfcRegenInstructions.AllowFixUI` to modify this parameter.
- *ForceRegen*—Creo Parametric  
Use the property `IpfcRegenInstructions.ForceRegen` to modify this parameter.
- *FromFeat*—Not currently used. This parameter is reserved for future use.  
Use the property `IpfcRegenInstructions.FromFeat` to modify this parameter.
- *RefreshModelTree*—Creo Parametric **Model Tree**  
Use the property `IpfcRegenInstructions.RefreshModelTree` to modify this parameter.
- *ResumeExcludedComponents*—Creo Parametric  
Use the methodproperty  
`IpfcRegenInstructions.ResumeExcludedComponents` to modify this parameter.
- *UpdateAssemblyOnly*—Updates the placements of an assembly and all its sub-assemblies, and regenerates the assembly features and intersected parts. If the affected assembly is retrieved as a simplified representation, then the locations of the components are updated. If this attribute is false, the component locations are not updated, even if the simplified representation is retrieved. By default, it is false.  
Use the property `IpfcRegenInstructions.UpdateAssemblyOnly` to modify this parameter.
- *UpdateInstances*—Updates the instances of the solid model in memory. This may slow down the regeneration process. By default, this attribute is false.

---

Use the property `IpfcRegenInstructions.UpdateInstances` to modify this parameter.

- *ResolveModeRegen*—Allows regeneration of a solid in resolve mode. The regeneration behavior is controlled by temporarily overriding the default settings.

Use the property `IpfcRegenInstructions.ResolveModeRegen` to modify this parameter. By default, it is false. If you want to set your Creo Parametric application in resolve mode, you need to set the value of this parameter to true.

---

 **Note**

However, resolve mode will be deprecated in a future release of Creo Parametric. Hence, it is recommended to run the application without calling the property `IpfcRegenInstructions.ResolveModeRegen`.

---

The property `IpfcSolid.GeomOutline` returns the three-dimensional bounding box for the specified solid.

---

 **Note**

Do not use `IpfcSolid.GeomOutline` to calculate the outline of a solid as the dimensions of the boundary box could be slightly bigger than the outline dimensions of the geometry. Use `IpfcSolid.EvalOutline()` to compute an accurate outline of a solid.

---

The method `IpfcSolid.EvalOutline()` also returns a three-dimensional bounding box, but you can specify the coordinate system used to compute the extents of the solid object.

The property `IpfcSolid.IsSkeleton` determines whether the part model is a skeleton or a concept model. It returns a true value if the model is a skeleton, else it returns a false.

The method `IpfcSolid.ListGroups()` returns the list of groups (including UDFs) in the solid.

The method `IpfcSolid.GetSurfaceSolidBody()` returns the body to which the surface belongs.

The method `IpfcSolid.GetEdgeSolidBody()` returns the body to which the edge belongs.

---

## Solid Units

Each model has a basic system of units to ensure all material properties of that model are consistently measured and defined. All models are defined on the basis of the system of units. A part can have only one system of unit.

The following types of quantities govern the definition of units of measurement:

- **Basic Quantities**—The basic units and dimensions of the system of units. For example, consider the Centimeter Gram Second (CGS) system of unit. The basic quantities for this system of units are:
  - Length—cm
  - Mass—g
  - Force—dyne
  - Time—sec
  - Temperature—K
- **Derived Quantities**—The derived units are those that are derived from the basic quantities. For example, consider the Centimeter Gram Second (CGS) system of unit. The derived quantities for this system of unit are as follows:
  - Area—cm<sup>2</sup>
  - Volume—cm<sup>3</sup>
  - Velocity—cm/sec

In the VB API, individual units in the model are represented by the interface `pfcUnits.Unit`.

### Types of Unit Systems

The types of systems of units are as follows:

- **Pre-defined system of units**—This system of unit is provided by default.
- **Custom-defined system of units**—This system of unit is defined by the user only if the model does not contain standard metric or nonmetric units, or if the material file contains units that cannot be derived from the predefined system of units or both.

In Creo Parametric, the system of units are categorized as follows:

- **Mass Length Time (MLT)**—The following systems of units belong to this category:
  - CGS—Centimeter Gram Second
  - MKS—Meter Kilogram Second
  - mmKS—millimeter Kilogram Second

- 
- Force Length Time (FLT)—The following systems of units belong to this category:
    - Creo Parametric Default—Inch lbm Second. This is the default system followed by Creo Parametric.
    - FPS—Foot Pound Second
    - IPS—Inch Pound Second
    - mmNS—Millimeter Newton Second

In the VB API, the system of units followed by the model is represented by the interface `IpfcUnitSystem`.

### Accessing Individual Units

Methods and Properties Introduced:

- **`IpfcSolid.ListUnits()`**
- **`IpfcSolid.GetUnit()`**
- **`IpfcUnit.Name`**
- **`IpfcUnit.Expression`**
- **`IpfcUnit.Type`**
- **`IpfcUnit.IsStandard`**
- **`IpfcUnit.ReferenceUnit`**
- **`IpfcUnit.ConversionFactor`**
- **`IpfcUnitConversionFactor.Offset`**
- **`IpfcUnitConversionFactor.Scale`**

The method `IpfcSolid.ListUnits()` returns the list of units available to the specified model.

The method `IpfcSolid.GetUnit()` retrieves the unit, based on its name or expression for the specified model in the form of the `IpfcUnit` object.

The property `IpfcUnit.Name` returns the name of the unit.

The property `IpfcUnit.Expression` returns a user-friendly unit description in the form of the name (for example, `ksi`) for ordinary units and the expression (for example, `N/m^3`) for system-generated units.

The property `IpfcUnitSystem.Type` returns the type of quantity represented by the unit in terms of the `IpfcUnitType` object. The types of units are as follows:

- `EpfcUNIT_LENGTH`—Specifies length measurement units.
- `EpfcUNIT_MASS`—Specifies mass measurement units.

- `EpfcUNIT_FORCE`—Specifies force measurement units.
- `EpfcUNIT_TIME`—Specifies time measurement units.
- `EpfcUNIT_TEMPERATURE`—Specifies temperature measurement units.
- `EpfcUNIT_ANGLE`—Specifies angle measurement units.

The property `IpfcUnit.IsStandard` identifies whether the unit is system-defined (if the property *IsStandard* is set to true) or user-defined (if the property *IsStandard* is set to false).

The property `IpfcUnit.ReferenceUnit` returns a reference unit (one of the available system units) in terms of the `IpfcUnit` object.

The property `IpfcUnit.ConversionFactor` identifies the relation of the unit to its reference unit in terms of the `IpfcUnitConversionFactor` object. The unit conversion factors are as follows:

- `Offset`—Specifies the offset value applied to the values in the reference unit.
- `Scale`—Specifies the scale applied to the values in the reference unit to get the value in the actual unit.

Example - Consider the formula to convert temperature from Centigrade to Fahrenheit

$$F = a + (C * b)$$

where

F is the temperature in Fahrenheit

C is the temperature in Centigrade

a = 32 (constant signifying the offset value)

b = 9/5 (ratio signifying the scale of the unit)

### Note

Creo Parametric scales the length dimensions of the model using the factors listed above. If the scale is modified, the model is regenerated. When you scale the model, the model units are not changed. Imported geometry cannot be scaled.

Use the properties `IpfcUnitConversionFactor.Offset` and `IpfcUnitConversionFactor.Scale` to retrieve the unit conversion factors listed above.

## Modifying Individual Units

Methods and Properties Introduced:

- **`IpfcUnit.Modify()`**
- **`IpfcUnit.Delete()`**



---

The method `IpfcUnit.Modify()` modifies the definition of a unit by applying a new conversion factor specified by the `IpfcUnitConversionFactor` object and a reference unit.

The method `IpfcUnit.Delete()` deletes the unit.



### Note

You can delete only custom units and not standard units.

---

## Creating a New Unit

Methods Introduced:

- **`IpfcSolid.CreateCustomUnit()`**
- **`CCpfcUnitConversionFactor.Create()`**

The method `IpfcSolid.CreateCustomUnit()` creates a custom unit based on the specified name, the conversion factor given by the `IpfcUnitConversionFactor` object, and a reference unit.

The method `CCpfcUnitConversionFactor.Create()` creates the `IpfcUnitConversionFactor` object containing the unit conversion factors.

## Accessing Systems of Units

Methods and Properties Introduced:

- **`IpfcSolid.ListUnitSystems()`**
- **`IpfcSolid.GetPrincipalUnits()`**
- **`IpfcUnitSystem.GetUnit()`**
- **`IpfcUnitSystem.Name`**
- **`IpfcUnitSystem.Type`**
- **`IpfcUnitSystem.IsStandard`**

The method `IpfcSolid.ListUnitSystems()` returns the list of unit systems available to the specified model.

The method `IpfcSolid.GetPrincipalUnits()` returns the system of units assigned to the specified model in the form of the `IpfcUnitSystem` object.

The method `IpfcUnitSystem.GetUnit()` retrieves the unit of a particular type used by the unit system.

The property `IpfcUnitSystem.Name` returns the name of the unit system.

---

The property `IpfcUnitSystem.Type` returns the type of the unit system in the form of the `IpfcUnitSystemType` object. The types of unit systems are as follows:

- `EpfcUNIT_SYSTEM_MASS_LENGTH_TIME`—Specifies the Mass Length Time (MLT) unit system.
- `EpfcUNIT_SYSTEM_FORCE_LENGTH_TIME`—Specifies the Force Length Time (FLT) unit system.

For more information on these unit systems listed above, refer to the section [Types of Unit Systems on page 142](#).

The property `IpfcUnitSystem.IsStandard` identifies whether the unit system is system-defined (if the property *IsStandard* is set to true) or user-defined (if the property *IsStandard* is set to false).

### Modifying Systems of Units

Method Introduced:

- **`IpfcUnitSystem.Delete()`**

The method `IpfcUnitSystem.Delete()` deletes a custom-defined system of units.

---

#### Note

You can delete only a custom-defined system of units and not a standard system of units.

---

### Creating a New System of Units

Method Introduced:

- **`IpfcSolid.CreateUnitSystem()`**

The method `IpfcSolid.CreateUnitSystem()` creates a new system of units in the model based on the specified name, the type of unit system given by the `IpfcUnitSystemType` object, and the types of units specified by the `IpfcUnits` sequence to use for each of the base measurement types (length, force or mass, and temperature).

### Conversion to a New Unit System

Methods and Properties Introduced:

- **`IpfcSolid.SetPrincipalUnits()`**
- **`CCpfcUnitConversionOptions.Create()`**

- **IpfcUnitConversionOptions.DimensionOption**
- **IpfcUnitConversionOptions.IgnoreParamUnits**

The method `IpfcSolid.SetPrincipalUnits()` changes the principal system of units assigned to the solid model based on the unit conversion options specified by the `IpfcUnitConversionOptions` object. The method `IpfcUnitConversionOptions.Create()` creates the `IpfcUnitConversionOptions` object containing the unit conversion options listed below.

The types of unit conversion options are as follows:

- **DimensionOption**—Use the option while converting the dimensions of the model.

Use the property `IpfcUnitConversionOptions.DimensionOption` to modify this option.

This option can be of the following types:

- `EpfcUNITCONVERT_SAME_DIMS`—Specifies that unit conversion occurs by interpreting the unit value in the new unit system. For example, 1 inch will equal to 1 millimeter.
- `EpfcUNITCONVERT_SAME_SIZE`—Specifies that unit conversion will occur by converting the unit value in the new unit system. For example, 1 inch will equal to 25.4 millimeters.
- **IgnoreParamUnits**—This boolean attribute determines whether or not ignore the parameter units. If it is null or true, parameter values and units do not change when the unit system is changed. If it is false, parameter units are converted according to the rule.

Use the property

`IpfcUnitConversionOptions.IgnoreParamUnits` to modify this attribute.

## Mass Properties

Method Introduced:

- **IpfcSolid.GetMassProperty()**
- **IpfcSolid.GetMassPropertyWithDensity()**
- **IpfcAssembly.GetMassPropertyByCompPath()**

The function `IpfcSolid.GetMassProperty()` provides information about the distribution of mass in the part or assembly. It can provide the information relative to a coordinate system datum, which you name, or the default one if you provide null as the name. It returns an object containing the following fields:

- The volume.
- The surface area.
- The density. The density value is 1.0, unless a material has been assigned.
- The mass.
- The center of gravity (COG).
- The inertia matrix.
- The inertia tensor.
- The inertia about the COG.
- The principal moments of inertia (the eigen values of the COG inertia).
- The principal axes (the eigenvectors of the COG inertia).

The property `IpfcSolid.GetMassPropertyWithDensity()` calculates the mass properties of a part or an assembly in the specified coordinate system. The input arguments follow:

- *CoordSysName*—Name of the coordinate system. If this is `Null`, the method uses the default coordinate system.
- *DensityOpt*—Value of the density flag specified using the enumerated data type `IpfcMPDensityUse` and the valid values are as follows:
  - `EpfcMP_DENSITY_DEFAULT`—Calculate the mass properties using the material density.
  - `EpfcMP_DENSITY_USE_ALWAYS`—Calculate the mass properties using the specified density, even if material has a defined density.
  - `EpfcMP_DENSITY_USE_IF_MISSING`—Calculate mass properties using specified density, even if material does not have a defined density.
- *density*—Density used while calculating mass properties depending on the value specified for the input argument *DensityOpt*.

The method `IpfcAssembly.GetMassPropertyByCompPath()` calculates the mass properties of a solid that is referenced by the specified coordinate system selection, using the respective component paths. The input arguments follow:

- *CompPath*—Component path of the solid. If this is null, the top assembly is referred.
- *CsysItem*—Coordinate system model item. If this is null, default coordinate system is referred.
- *CsysPath*—Component path of the coordinate system. If this is null, default coordinate system is referred.

---

## Example Code: Retrieving a Mass Property Object

The sample code in the file `pfcSolidExamples.vb` located at `<creo_vbapi_loadpoint>/vbapi_examples` retrieves a `MassProperty` object from a specified solid model. The solid's mass, volume, and center of gravity point are then printed.

## Annotations

Methods and Properties Introduced:

- **`IpfcNote.Lines`**
- **`IpfcNote.GetText()`**
- **`IpfcNote.URL`**
- **`IpfcNote.Display()`**
- **`IpfcNote.Delete()`**
- **`IpfcNote.GetOwner()`**

3D model notes are instance of `ModelItem` objects. They can be located and accessed using methods that locate model items in solid models, and downcast to the `Note` interface to use the methods in this section.

The property `IpfcNote.Lines` returns the text contained in the 3D model note.

The method `IpfcNote.GetText()` returns the text of the solid model note. If you set the parameter *GiveParametersAsNames* to `TRUE`, then the text displays the parameter callouts with ampersands (&). If you set the parameter to `FALSE`, then the text displays the parameter values with no callout information.

The property `IpfcNote.URL` returns the URL stored in the 3D model note.

The method `IpfcNote.Display()` forces the display of the model note.

The method `IpfcNote.Delete()` deletes a model note.

The method `IpfcNote.GetOwner()` returns the solid model owner of the note.

## Cross Sections

Methods Introduced:

- **`IpfcSolid.ListCrossSections()`**
- **`IpfcSolid.GetCrossSection()`**
- **`IpfcXSection.GetName()`**
- **`IpfcXSection.SetName()`**
- **`IpfcXSection.GetXSecType()`**

- 
- **IpfcXSection.Delete()**
  - **IpfcXSection.Display()**
  - **IpfcXSection.Regenerate()**

The method `IpfcSolid.ListCrossSections()` returns a sequence of cross section objects represented by the `Xsection` interface. The method `IpfcSolid.GetCrossSection()` searches for a cross section given its name.

The method `IpfcXSection.GetName()` returns the name of the cross section in Creo Parametric. The method `IpfcXSection.SetName()` modifies the cross section name.

The method `IpfcXSection.GetXSecType()` returns the type of cross section, that is planar or offset, and the type of item intersected by the cross section.

The method `IpfcXSection.Delete()` deletes a cross section.

The method `IpfcXSection.Display()` forces a display of the cross section in the window.

The method `IpfcXSection.Regenerate()` regenerates a cross section.

## Materials

the VB API enables you to programmatically access the material types and properties of parts. Using the methods and properties described in the following sections, you can perform the following actions:

- Create or delete materials
- Set the current material
- Access and modify the material types and properties

Methods and Properties Introduced:

- **IpfcMaterial.Save()**
- **IpfcMaterial.Delete()**
- **IpfcPart.CurrentMaterial**
- **IpfcPart.ListMaterials()**
- **IpfcPart.CreateMaterial()**
- **IpfcPart.RetrieveMaterial()**

The method `IpfcMaterial.Save()` writes to a material file that can be imported into any Creo Parametric part.

The method `IpfcMaterial.Delete()` removes material from the part.

---

The property `IpfcPart.CurrentMaterial` returns and sets the material assigned to the part.

---

 **Note**

By default, while assigning a material to a sheetmetal part, the property `IpfcPart.CurrentMaterial` modifies the values of the sheetmetal properties such as Y factor and bend table according to the material file definition. This modification triggers a regeneration and a modification of the developed length calculations of the sheetmetal part. However, you can avoid this behavior by setting the value of the configuration option `material_update_smt_bend_table` to `never_replace`.

The property `IpfcPart.CurrentMaterial` may change the model display, if the new material has a default appearance assigned to it.

The property may also change the family table, if the parameter `PTC_MATERIAL_NAME` is a part of the family table.

---

The method `IpfcPart.ListMaterials()` returns a list of the materials available in the part.

The method `IpfcPart.CreateMaterial()` creates a new empty material in the specified part.

The method `IpfcPart.RetrieveMaterial()` imports a material file into the part. The name of the file read can be as either:

- `<name>.mtl`—Specifies the new material file format.
- `<name>.mat`—Specifies the material file format prior to Pro/ENGINEER Wildfire 3.0.

If the material is not already in the part database, `IpfcPart.RetrieveMaterial()` adds the material to the database after reading the material file. If the material is already in the database, the function replaces the material properties in the database with those contained in the material file.

## Accessing Material Types

Properties Introduced:

- **`IpfcMaterial.StructuralMaterialType`**
- **`IpfcMaterial.ThermalMaterialType`**
- **`IpfcMaterial.SubType`**
- **`IpfcMaterial.PermittedSubTypes`**

---

- **IpfcMaterial.FluidMaterialType**

The property `IpfcMaterial.StructuralMaterialType` sets the material type for the structural properties of the material. The material types are as follows:

- `EpfcMTL_ISOTROPIC`—Specifies a material with an infinite number of planes of material symmetry, making the properties equal in all directions.
- `EpfcMTL_ORTHOTROPIC`—Specifies a material with symmetry relative to three mutually perpendicular planes.
- `EpfcMTL_TRANSVERSELY_ISOTROPIC`—Specifies a material with rotational symmetry about an axis. The properties are equal for all directions in the plane of isotropy.
- `EpfcMTL_FLUID`—Specifies a material with fluid properties.

The property `IpfcMaterial.ThermalMaterialType` sets the material type for the thermal properties of the material. The material types are as follows:

- `EpfcMTL_ISOTROPIC`—Specifies a material with an infinite number of planes of material symmetry, making the properties equal in all directions.
- `EpfcMTL_ORTHOTROPIC`—Specifies a material with symmetry relative to three mutually perpendicular planes.
- `EpfcMTL_TRANSVERSELY_ISOTROPIC`—Specifies a material with rotational symmetry about an axis. The properties are equal for all directions in the plane of isotropy.
- `EpfcMTL_FLUID`—Specifies a material with fluid properties.

The property `IpfcMaterial.SubType` sets the subtype for the `EpfcMTL_ISOTROPIC` material type.

Use the property `IpfcMaterial.PermittedSubTypes` to retrieve a list of the permitted string values for the material subtype.

The property `IpfcMaterial.FluidMaterialType` sets the material type for the fluid properties of the material. The material types are as follows:

- `EpfcMTL_ISOTROPIC`—Specifies a material with an infinite number of planes of material symmetry, making the properties equal in all directions.
- `EpfcMTL_ORTHOTROPIC`—Specifies a material with symmetry relative to three mutually perpendicular planes.
- `EpfcMTL_TRANSVERSELY_ISOTROPIC`—Specifies a material with rotational symmetry about an axis. The properties are equal for all directions in the plane of isotropy.
- `EpfcMTL_FLUID`—Specifies a material with fluid properties.



---

## Accessing Material Properties

The methods and properties listed in this section enable you to access material properties.

Methods and Properties Introduced:

- **CCpfcMaterialProperty.Create()**
- **IpfcMaterial.GetPropertyValue()**
- **IpfcMaterial.SetPropertyValue()**
- **IpfcMaterial.SetPropertyUnits()**
- **IpfcMaterial.RemoveProperty()**
- **IpfcMaterial.Description**
- **IpfcMaterial.FatigueType**
- **IpfcMaterial.PermittedFatigueTypes**
- **IpfcMaterial.FatigueMaterialType**
- **IpfcMaterial.PermittedFatigueMaterialTypes**
- **IpfcMaterial.FatigueMaterialFinish**
- **IpfcMaterial.PermittedFatigueMaterialFinishes**
- **IpfcMaterial.FailureCriterion**
- **IpfcMaterial.PermittedFailureCriteria**
- **IpfcMaterial.Hardness**
- **IpfcMaterial.HardnessType**
- **IpfcMaterial.Condition**
- **IpfcMaterial.BendTable**
- **IpfcMaterial.CrossHatchFile**
- **IpfcMaterial.MaterialModel**
- **IpfcMaterial.PermittedMaterialModels**
- **IpfcMaterial.ModelDefByTests**

The method `CCpfcMaterialProperty.Create()` creates a new instance of a material property object.

All numerical material properties are accessed using the same set of APIs. You must provide a property type to indicate the property you want to read or modify.

The method `IpfcMaterial.GetPropertyValue()` returns the value and the units of the material property.

Use the method `IpfcMaterial.SetPropertyValue()` to set the value and units of the material property. If the property type does not exist for the material, then this method creates it.

---

Use the method `IpfcMaterial.SetPropertyUnits()` to set the units of the material property.

Use the method `IpfcMaterial.RemoveProperty()` to remove the material property.

Material properties that are non-numeric can be accessed using the following properties.

The property `IpfcMaterial.Description` sets the description string for the material.

The property `IpfcMaterial.FatigueType` and sets the valid fatigue type for the material.

Use the property `IpfcMaterial.PermittedFatigueTypes` to get a list of the permitted string values for the fatigue type.

The property `IpfcMaterial.FatigueMaterialType` sets the class of material when determining the effect of the fatigue.

Use the property `IpfcMaterial.PermittedFatigueMaterialTypes` to retrieve a list of the permitted string values for the fatigue material type.

The property `IpfcMaterial.FatigueMaterialFinish` sets the type of surface finish for the fatigue material.

Use the property `IpfcMaterial.PermittedFatigueMaterialFinishes` to retrieve a list of permitted string values for the fatigue material finish.

The property `IpfcMaterial.FailureCriterion` sets the reduction factor for the failure strength of the material. This factor is used to reduce the endurance limit of the material to account for unmodeled stress concentrations, such as those found in welds.

Use the property `IpfcMaterial.PermittedFailureCriteria` to retrieve a list of permitted string values for the material failure criterion.

The property `IpfcMaterial.Hardness` sets the hardness for the specified material.

The property `IpfcMaterial.HardnessType` sets the hardness type for the specified material.

The property `IpfcMaterial.Condition` sets the condition for the specified material.

The property `IpfcMaterial.BendTable` sets the bend table for the specified material.

The property `IpfcMaterial.CrossHatchFile` sets the file containing the crosshatch pattern for the specified material.

---

The property `IpfcMaterial.MaterialModel` sets the type of hyperelastic isotropic material model .

Use the property `IpfcMaterial.PermittedMaterialModels` to retrieve a list of the permitted string values for the material model.

The property `IpfcMaterial.ModelDefByTests` determines whether the hyperelastic isotropic material model has been defined using experimental data for stress and strain.

### Accessing User-defined Material Properties

Materials permit assignment of user-defined parameters. These parameters allow you to place non-standard properties on a given material. Therefore `IpfcMaterial` is a child of `IpfcParameterOwner`, which provides access to user-defined parameters and properties of materials through the methods in that interface.

## Solid Bodies

### Solid Bodies

This section describes the VB API functions that access the functions of a Creo Parametric part with multiple solid bodies.

### Solid Body Information

The state of the body is derived from the features and geometry in which the body is created.

Methods introduced:

- **`IpfcSolidBody.GetBodyState()`**
- **`IpfcSolidBody.IsConstruction()`**
- **`IpfcSolidBody.ListSurfaces()`**
- **`IpfcSolid.GetDefaultBody()`**
- **`IpfcSolidBody.GetFeatures()`**
- **`IpfcSolidBody.GetOutline()`**
- **`IpfcSolidBody.IsSheetmetal()`**
- **`IpfcSolidBody.GetDensity()`**
- **`IpfcSolidBody.GetMassProperty()`**

---

The method `IpfcSolidBody.GetBodyState()` specifies the state of the body using the enumerated type `EpfcSolidBodyState`. The valid values are:

- `EpfcBODY_STATE_MISSING`
- `EpfcBODY_STATE_CONSUMED`
- `EpfcBODY_STATE_NO_CONTR_FEAT`
- `EpfcBODY_STATE_NO_GEOMETRY`
- `EpfcBODY_STATE_ACTIVE`

The method `IpfcSolidBody.IsConstruction()` checks if the body is a construction body. The method returns a `True` if solid body is a construction body.

The method `IpfcSolidBody.ListSurfaces()` lists all the surfaces in the specified body

The method `IpfcSolid.GetDefaultBody()` returns the default body in the specified solid.

The method `IpfcSolidBody.GetFeatures()` lists the features that are associated with the specified body.

Use the method `IpfcSolidBody.GetOutline()` to retrieve the regeneration outline of a solid body, with respect to the base coordinate system orientation. This outline defines the boundary box of the body.

The method `IpfcSolidBody.IsSheetmetal()` checks if the specified body is an active sheetmetal body.

The method `IpfcSolidBody.GetDensity()` determines the density of the body.

Use the method `IpfcSolidBody.GetMassProperty()` to retrieve the mass properties of a body in the specified coordinate system.

## Windows and Views

### Windows and Views

The VB API provides access to Creo Parametric windows and saved views. This section describes the methods that provide this access.

### Windows

This section describes the VB API methods that access `Window` objects. The topics are as follows:

- [Getting a Window Object on page 157](#)

- 
- [Window Operations on page 279](#)

Getting a Window Object

Methods and Property Introduced:

- **`IpfcBaseSession.CurrentWindow`**
- **`IpfcBaseSession.CreateModelWindow()`**
- **`IpfcModel.Display()`**
- **`IpfcBaseSession.ListWindows()`**
- **`IpfcBaseSession.GetWindow()`**
- **`IpfcBaseSession.OpenFile()`**
- **`IpfcBaseSession.GetModelWindow()`**

The property `IpfcBaseSession.CurrentWindow` provides access to the current active window in Creo Parametric.

The method `IpfcBaseSession.CreateModelWindow()` creates a new window that contains the model that was passed as an argument.



#### **Note**

You must call the method `IpfcModel.Display()` for the model geometry to be displayed in the window.

---

Use the method `IpfcBaseSession.ListWindows()` to get a list of all the current windows in session.

The method `IpfcBaseSession.GetWindow()` gets the handle to a window given its integer identifier.

The method `IpfcBaseSession.OpenFile()` returns the handle to a newly created window that contains the opened model.



#### **Note**

If a model is already open in a window the method returns a handle to the window.

---

The method `IpfcBaseSession.GetModelWindow()` returns the handle to the window that contains the opened model, if it is displayed.

---

## Window Operations

Methods and Properties Introduced:

- **IpfcWindow.Height**
- **IpfcWindow.Width**
- **IpfcWindow.XPos**
- **IpfcWindow.YPos**
- **IpfcWindow.GraphicsAreaHeight**
- **IpfcWindow.GraphicsAreaWidth**
- **IpfcWindow.Clear()**
- **IpfcWindow.Repaint()**
- **IpfcWindow.Refresh()**
- **IpfcWindow.Close()**
- **IpfcWindow.Activate()**
- **IpfcWindow.GetId**
- **IpfcBaseSession.FlushCurrentWindow()**

The properties `IpfcWindow.Height`, `IpfcWindow.Width`, `IpfcWindow.XPos`, and `IpfcWindow.YPos` retrieve the height, width, x-position, and y-position of the window respectively. The values of these parameters are normalized from 0 to 1.

The properties `IpfcWindow.GraphicsAreaHeight` and `IpfcWindow.GraphicsAreaWidth` retrieve the height and width of the Creo Parametric graphics area window without the border respectively. The values of these parameters are normalized from 0 to 1. For both the window and graphics area sizes, if the object occupies the whole screen, the window size returned is 1. For example, if the screen is 1024 pixels wide and the graphics area is 512 pixels, then the width of the graphics area window is returned as 0.5.

The method `IpfcWindow.Clear()` removes geometry from the window.

Both `IpfcWindow.Repaint()` and `IpfcWindow.Refresh()` repaint solid geometry. However, the `Refresh()` method does not remove highlights from the screen and is used primarily to remove temporary geometry entities from the screen.

Use the method `IpfcWindow.Close()` to close the window. If the current window is the original window created when Creo Parametric started, this method clears the window. Otherwise, it removes the window from the screen.

The method `IpfcWindow.Activate()` activates a window. This function is available only in the asynchronous mode.

---

The method `pfcWindow.Window.GetId` retrieves the ID of the Creo Parametric window.

The method `IpfcBaseSession.FlushCurrentWindow()` flushes the pending display commands on the current window.

---

 **Note**

It is recommended to call this method only after completing all the display operations. Excessive use of this method will cause major slow down of systems running on Windows Vista and Windows 7.

---

## Embedded Browser

Methods Introduced:

- **`IpfcWindow.GetURL()`**
- **`IpfcWindow.SetURL()`**
- **`IpfcWindow.GetBrowserSize()`**
- **`IpfcWindow.SetBrowserSize()`**

The methods `IpfcWindow.GetURL()` and `IpfcWindow.SetURL()` enables you to find and change the URL displayed in the embedded browser in the Creo Parametric window.

The methods `IpfcWindow.GetBrowserSize()` and `IpfcWindow.SetBrowserSize()` enables you to find and change the size of the embedded browser in the Creo Parametric window.

---

 **Note**

The methods `IpfcWindow.GetBrowserSize()` and `IpfcWindow.SetBrowserSize()` are not supported if the browser is open in a separate window.

---

## Views

This section describes the VB API methods that access `IpfcView` objects. The topics are as follows:

- [Getting a View Object on page 160](#)
- [View Operations on page 160](#)

---

## Getting a View Object

Methods Introduced:

- **`IpfcViewOwner.RetrieveView()`**
- **`IpfcViewOwner.GetView()`**
- **`IpfcViewOwner.ListViews()`**
- **`IpfcViewOwner.GetCurrentView()`**

Any solid model inherits from the interface `IpfcViewOwner`. This will enable you to use these methods on any solid object.

The method `IpfcViewOwner.RetrieveView()` sets the current view to the orientation previously saved with a specified name.

Use the method `IpfcViewOwner.GetView()` to get a handle to a named view without making any modifications.

The method `IpfcViewOwner.ListViews()` returns a list of all the views previously saved in the model.

From Creo Parametric 2.0 M120 onward, the method, `IpfcViewOwner.GetCurrentView()` has been deprecated. The method returns a view handle that represents the current orientation. Although this view does not have a name, you can use this view to find or modify the current orientation.

## View Operations

Methods and Properties Introduced:

- **`IpfcView.Name`**
- **`IpfcView.IsCurrent`**
- **`IpfcView.Reset()`**
- **`IpfcViewOwner.SaveView()`**

To get the name of a view given its identifier, use the property `IpfcView.Name`.

The property `IpfcView.IsCurrent` determines if the View object represents the current view.

The method `IpfcView.Reset()` restores the current view to the default view.

To store the current view under the specified name, call the method `IpfcViewOwner.SaveView()`.



---

## Coordinate Systems and Transformations

This section describes the various coordinate systems used by Creo Parametric and accessible from the VB API and how to transform from one coordinate system to another.

### Coordinate Systems

Creo Parametric and the VB API use the following coordinate systems:

- [Solid Coordinate System on page 161](#)
- [Screen Coordinate System on page 161](#)
- [Window Coordinate System on page 162](#)
- [Drawing Coordinate System on page 162](#)
- [Drawing View Coordinate System on page 162](#)
- [Assembly Coordinate System on page 162](#)
- [Datum Coordinate System on page 163](#)
- [Section Coordinate System on page 163](#)

The following sections describe each of these coordinate systems.

### Solid Coordinate System

The solid coordinate system is the three-dimensional, Cartesian coordinate system used to describe the geometry of a Creo Parametric solid model. In a part, the solid coordinate system describes the geometry of the surfaces and edges. In an assembly, the solid coordinate system also describes the locations and orientations of the assembly members.

You can visualize the solid coordinate system in Creo Parametric by creating a coordinate system datum with the option **Default**. Distances measured in solid coordinates correspond to the values of dimensions as seen by the Creo Parametric user.

Solid coordinates are used by the VB API for all the methods that look at geometry and most of the methods that draw three-dimensional graphics.

### Screen Coordinate System

The screen coordinate system is two-dimensional coordinate system that describes locations in a Creo Parametric window. This is an intermediate coordinate system after which the screen points are transformed to screen pixels. All the models are first mapped to the screen coordinate system. When the user zooms or pans the view, the screen coordinate system follows the display of the solid, so a particular point on the solid always maps to the same screen coordinate. The mapping changes only when the view orientation is changed.

---

Screen coordinates are used by some of the graphics methods, the mouse input methods, and all methods that draw graphics or manipulate items on a drawing.

### Window Coordinate System

The window coordinate system is similar to the screen coordinate system. After mapping the models to the screen coordinate system, they are mapped to the window coordinate before being drawn to screen pixels based on screen resolution. When pan or zoom values are applied to the coordinates in the screen coordinate system, they result in window coordinates. When an object is first displayed in a window, or the option **View ► Refit** is used, the screen and window coordinates are the same.

Window coordinates are needed only if you need to take account of zoom and pan—for example, to find out whether a point on the solid is visible in the window, or to draw two-dimensional text in a particular window location, regardless of pan and zoom.

### Drawing Coordinate System

The drawing coordinate system is a two-dimensional system that describes the location on a drawing relative to the bottom, left corner, and measured in drawing units. For example, on a U.S. letter-sized, landscape-format drawing sheet that uses inches, the top, right-corner is (11, 8.5) in drawing coordinates.

The VB API methods and properties that manipulate drawings generally use screen coordinates.

### Drawing View Coordinate System

The drawing view coordinate system is used to describe the locations of entities in a drawing view.

### Assembly Coordinate System

An assembly has its own coordinate system that describes the positions and orientations of the member parts, subassemblies, and the geometry of datum features created in the assembly.

When an assembly is retrieved into memory each member is also loaded and continues to use its own solid coordinate system to describe its geometry.

This is important when you are analyzing the geometry of a subassembly and want to extract or display the results relative to the coordinate system of the parent assembly.

---

## Datum Coordinate System

A coordinate system datum can be created anywhere in any part or assembly, and represents a user-defined coordinate system. It is often a requirement in a VB API application to describe geometry relative to such a datum.

## Section Coordinate System

Every sketch has a coordinate system used to locate entities in that sketch. Sketches used in features will use a coordinate system different from that of the solid model.

## Transformations

Methods and Properties Introduced:

- **`IpfcTransform3D.Invert()`**
- **`IpfcTransform3D.TransformPoint()`**
- **`IpfcTransform3D.TransformVector()`**
- **`IpfcTransform3D.Matrix`**
- **`IpfcTransform3D.GetOrigin()`**
- **`IpfcTransform3D.GetXAxis()`**
- **`IpfcTransform3D.GetYAxis`**
- **`IpfcTransform3D.GetZAxis()`**
- **`CMpfcBase.MakeMatrixOrthonormal()`**

All coordinate systems are treated in the VB API as if they were three-dimensional. Therefore, a point in any of the coordinate systems is always represented by the `IpfcPoint3D` class:

Vectors store the same data but are represented for clarity by the `IpfcVector3D` class.

Screen coordinates contain a z-value whose positive direction is outwards from the screen. The value of z is not generally important when specifying a screen location as an input to a method, but it is useful in other situations. For example, if you select a datum plane, you can find the direction of the plane by calculating the normal to the plane, transforming to screen coordinates, then looking at the sign of the z-coordinate.

A transformation between two coordinate systems is represented by the `IpfcTransform3D` class. This class contains a 4x4 matrix that combines the conventional 3x3 matrix that describes the relative orientation of the two systems, and the vector that describes the shift between them.

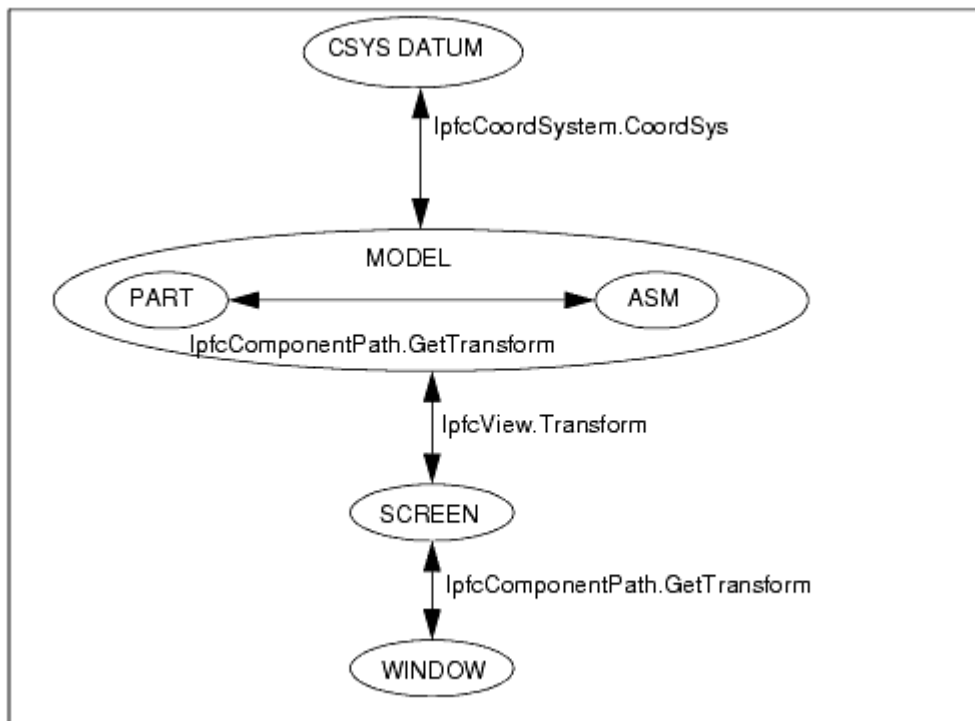
The 4x4 matrix used for transformations is as follows:

$$\begin{bmatrix} X' & Y' & Z' & 1 \end{bmatrix} = \begin{bmatrix} X & Y & Z & 1 \end{bmatrix} \begin{bmatrix} \dots & \dots & \dots & 0 \\ \dots & \dots & \dots & 0 \\ \dots & \dots & \dots & 0 \\ X_s & Y_s & Z_s & 1 \end{bmatrix}$$

The utility method `IpfcTransform3D.Invert()` inverts a transformation matrix so that it can be used to transform points in the opposite direction.

the VB API provides two utilities for performing coordinate transformations. The method `IpfcTransform3D.TransformPoint()` transforms a three-dimensional point and `IpfcTransform3D.TransformVector()` transforms a three-dimensional vector.

The following diagram summarizes the coordinate transformations needed when using the VB API and specifies the the VB API methods that provide the transformation matrix.



The method `CMpfcBase.MakeMatrixOrthonormal()` converts a non-orthonormal matrix to an orthonormal matrix with the specified scaling factor. The input arguments follow:

- *Matrix*—The matrix to be converted to orthonormal.
- *Scale*—Scale factor to be applied on the matrix.

---

## Transforming to Screen Coordinates

Methods and Properties Introduced:

- **IpfcView.Transform**
- **IpfcView.Rotate()**
- **IpfcViewOwner.GetCurrentViewTransform()**
- **IpfcViewOwner.SetCurrentViewTransform()**
- **IpfcViewOwner.CurrentViewRotate()**
- **IpfcTransform3D.Invert()**

The view matrix describes the transformation from solid to screen coordinates. The property `IpfcView.Transform` provides the view matrix for a model in the view.

To get and set the transformation matrix for a model in the current view, use the methods `IpfcViewOwner.GetCurrentViewTransform()` and `IpfcViewOwner.SetCurrentViewTransform()`.

The method `IpfcView.Rotate()` rotates an object, relative to the X, Y, or Z axis for the specified rotation angle.

To rotate an object in the current view, use the method `IpfcViewOwner.CurrentViewRotate()`.

To transform from screen to solid coordinates, invert the transformation matrix using the method `IpfcTransform3D.Invert()`.

## Transforming to Coordinate System Datum Coordinates

Property Introduced:

- **IpfcCoordSystem.CoordSys**

The property `IpfcCoordSystem.CoordSys` provides the location and orientation of the coordinate system datum in the coordinate system of the solid that contains it. The location is in terms of the directions of the three axes and the position of the origin.

## Transforming Window Coordinates

Properties Introduced

- **IpfcWindow.ScreenTransform**
- **IpfcScreenTransform.PanX**
- **IpfcScreenTransform.PanY**
- **IpfcScreenTransform.Zoom**

---

You can alter the pan and zoom of a window by using a Screen Transform object. This object contains three attributes. PanX and PanY represent the horizontal and vertical movement. Every increment of 1.0 moves the view point one screen width or height. Zoom represents a scaling factor for the view. This number must be greater than zero.

## Transforming Coordinates of an Assembly Member

Method Introduced:

- **IpfcComponentPath.GetTransform()**

The method `IpfcComponentPath.GetTransform()` provides the matrix for transforming from the solid coordinate system of the assembly member to the solid coordinates of the parent assembly, or the reverse.

### Example Code - Normalizing a Coordinate Transformation Matrix

The sample code in the file `pfcViewExamples.vb` located at `<creo_vbapi_loadpoint>/vbapi_examples` uses two methods to transfer the view transformation from one view to another..

# ModelItem

## ModelItem

This section describes the the VB API methods that enable you to access and manipulate `ModelItems`.

## Solid Geometry Traversal

Solid models are made up of 11 distinct types of `IpfcModelItem`, as follows:

- `IpfcFeature`
- `IpfcSurface`
- `IpfcEdge`
- `IpfcCurve` (datum curve)
- `IpfcAxis` (datum axis)
- `IpfcPoint` (datum point)
- `IpfcQuilt` (datum quilt)
- `IpfcLayer`
- `IpfcNote`
- `IpfcDimension`

- 
- `IpfcRefDimension`

Each model item is assigned a unique identification number that will never change. In addition, each model item can be assigned a string name. Layers, points, axes, dimensions, and reference dimensions are automatically assigned a name that can be changed.

## Getting ModelItem Objects

Methods and Properties Introduced:

- **`IpfcModelItemOwner.ListItems()`**
- **`IpfcFeature.ListSubItems()`**
- **`IpfcLayer.ListItems()`**
- **`IpfcModelItemOwner.GetItemById()`**
- **`IpfcModelItemOwner.GetItemByName()`**
- **`IpfcFamColModelItem.ReflItem`**
- **`IpfcSelection.SelItem`**

All models inherit from the interface `IpfcModelItemOwner`. The method `IpfcModelItemOwner.ListItems()` returns a sequence of `IpfcModelItems` contained in the model. You can specify which type of `IpfcModelItem` to collect by passing in one of the enumerated `EpfcModelItemType` values, or you can collect all `IpfcModelItems` by passing `null` as the model item type.

If the model has multiple bodies, the method `IpfcModelItemOwner.ListItems()` returns the exception `IpfcXToolkitMultibodyUnsupported`.



### Note

The part modeling features introduced in Creo Parametric 1.0 will be excluded from the list of features returned by the method `IpfcModelItemOwner.ListItems()` if the model item type is specified as `EpfcITEM_FEATURE`. For example edit round features, flexible modeling features, and so on will be excluded from the list.

---

The methods `IpfcFeature.ListSubItems()` and `IpfcLayer.ListItems()` produce similar results for specific features and layers. These methods return a list of subitems in the feature or items in the layer.

To access specific model items, call the method `IpfcModelItemOwner.GetItemById()`. This method enables you to access the model item by identifier.

---

To access specific model items, call the method

`IpfcModelItemOwner.GetItemByName()`. This method enables you to access the model item by name.

The property `IpfcFamColModelItem.RefItem` returns the dimension or feature used as a header for a family table.

The property `IpfcSelection.SelectedItem` returns the item selected interactively by the user.

## ModelItem Information

Methods and Properties Introduced:

- **`IpfcModelItem.GetName()`**
- **`IpfcModelItem.SetName()`**
- **`IpfcModelItem.Id`**
- **`IpfcModelItem.Type`**

Certain `IpfcModelItems` also have a string name that can be changed at any time. The methods `IpfcModelItem.GetName()` and `IpfcModelItem.SetName()` access this name.

The property `IpfcModelItem.Id` returns the unique integer identifier for the `IpfcModelItem`.

The `IpfcModelItem.Type` property returns an enumeration object that indicates the model item type of the specified `IpfcModelItem`. See the section [Solid Geometry Traversal on page 166](#) for the list of possible model item types.

## Duplicating ModelItems

Methods and Properties Introduced:

- **`IpfcBaseSession.AllowDuplicateModelItems()`**

You can control the creation of `ModelItems` more than twice for the same Creo Parametric item. The method

`IpfcBaseSession.AllowDuplicateModelItems()` allows you to turn ON or OFF the option to duplicate model items. By default, this option is OFF. To turn the option ON, set the boolean value to `FALSE`.



---

## Note

If this option is not handled properly on the application side, it can cause memory corruption. Thus, although you can turn ON and OFF this option as many times as you want, PTC recommends turning ON and OFF this option only once, right after the session is obtained.

---

## Layer Objects

In the VB API, layers are instances of `IpfcModelItem`. The following sections describe how to get layer objects and the operations you can perform on them.

### Getting Layer Objects

Method Introduced:

- **`IpfcModel.CreateLayer()`**

The method `IpfcModel.CreateLayer()` returns a new layer with the name you specify.

See the section [Getting ModelItem Objects on page 167](#) for other methods that can return layer objects.

### Layer Operations

Methods and Properties Introduced:

- **`IpfcLayer.Status`**
- **`IpfcLayer.ListItems()`**
- **`IpfcLayer.AddItem()`**
- **`IpfcLayer.RemoveItem()`**
- **`IpfcLayer.Delete()`**
- **`IpfcLayer.CountUnsupportedItems()`**

Superseded Method:

- **`IpfcLayer.HasUnsupportedItems()`**

The property `IpfcLayer.Status` enables you to access the display status of a layer. The corresponding enumeration class is `EpfcDisplayStatus` and the possible values are `Normal`, `Displayed`, `Blank`, or `Hidden`.

---

Use the methods `IpfcLayer.ListItems()`, `IpfcLayer.AddItem()`, and `IpfcLayer.RemoveItem()` to control the contents of a layer.

---

 **Note**

You cannot add the following items to a layer:

- `ITEM_SURFACE`,
- `ITEM_EDGE`,
- `ITEM_COORD_SYS`,
- `ITEM_AXIS`,
- `ITEM_SIMPREP`,
- `ITEM_DTL_SYM_DEFINITION`,
- `ITEM_DTL_OLE_OBJECT`,
- `ITEM_EXPLODED_STATE`.

For these items the method will throw the exception `IpfcXToolkitInvalidType`.

---

The method `IpfcLayer.Delete()` removes the layer (but not the items it contains) from the model.

The method `IpfcLayer.CountUnsupportedItems()` returns the number of item types not supported as a `IpfcModelItem` object in the specified layer. This method deprecates the method `IpfcLayer.HasUnsupportedItems()`.

## Features

### Features

All Creo Parametric solid models are made up of features. This section describes how to program on the feature level using the VB API.

### Access to Features

Methods and Properties Introduced:

- **`IpfcFeature.ListChildren()`**
- **`IpfcFeature.ListParents()`**
- **`IpfcFeature.GroupDirectHeader`**

- **IpfcFeatureGroup.GroupLeader**
- **IpfcFeaturePattern.PatternLeader**
- **IpfcFeaturePattern.ListMembers()**
- **IpfcSolid.ListFailedFeatures()**
- **IpfcSolid.ListFeaturesByType()**
- **IpfcSolid.GetFeatureById()**
- **IpfcBaseSession.QueryFeatureEdit()**

The methods `IpfcFeature.ListChildren()` and `IpfcFeature.ListParents()` return a sequence of features that contain all the children or parents of the specified feature.

The property `IpfcFeature.GroupDirectHeader` returns the direct header of the group.

To get the first feature in the specified group access the property `IpfcFeatureGroup.GroupLeader`.

The property `IpfcFeaturePattern.PatternLeader` and the method `IpfcFeaturePattern.ListMembers()` return features that make up the specified feature pattern. See the section [Feature Groups and Patterns on page 174](#) for more information on feature patterns.

The method `IpfcSolid.ListFailedFeatures()` returns a sequence that contains all the features that failed regeneration.

The method `IpfcSolid.ListFeaturesByType()` returns a sequence of features contained in the model. You can specify which type of feature to collect by passing in one of the `EpfcFeatureType` enumeration objects, or you can collect all features by passing `void null` as the type. If you list all features, the resulting sequence will include invisible features that Creo Parametric creates internally. Internal features are invisible features used internally for construction purposes. Use the method's *VisibleOnly* argument to exclude them. If the argument *VisibleOnly* is `True`, the function lists the public features only. If the argument is `False`, the function lists both public and internal features.

The method `IpfcSolid.GetFeatureById()` returns the feature object with the corresponding integer identifier.

A feature can be edited with the **Edit Definition** command in Creo Parametric. The method `IpfcBaseSession.QueryFeatureEdit()` returns a list of all the features that are currently being edited by the **Edit Definition** command.

## Feature Information

Properties Introduced:

- **IpfcFeature.FeatType**

- **`IpfcFeature.Status`**
- **`IpfcFeature.IsVisible`**
- **`IpfcFeature.IsReadonly`**
- **`IpfcFeature.IsEmbedded`**
- **`IpfcFeature.Number`**
- **`IpfcFeature.FeatTypeName`**
- **`IpfcFeature.FeatSubType`**
- **`IpfcRoundFeat.IsAutoRoundMember`**

The enumeration classes `EpfcFeatureType` and `EpfcFeatureStatus` provide information for a specified feature. The following properties specify this information:

- `IpfcFeature.FeatType`—Returns the type of a feature.
- `IpfcFeature.Status`—Returns whether the feature is suppressed, active, or failed regeneration.

The other properties that gather feature information include the following:

- `IpfcFeature.IsVisible`—Identifies whether the specified feature will be visible on the screen. The method distinguishes visible features from internal features. Internal features are invisible features used for construction purposes.
- `IpfcFeature.IsReadonly`—Identifies whether the specified feature can be modified.
- `IpfcFeature.IsEmbedded`—Specifies whether the specified feature is an embedded datum.
- `IpfcFeature.Number`—Returns the feature regeneration number. This method returns `void null` if the feature is suppressed.

The property `IpfcFeature.FeatTypeName` returns a string representation of the feature type.

The property `IpfcFeature.FeatSubType` returns a string representation of the feature subtype, for example, "Extrude" for a protrusion feature.

The property `IpfcRoundFeat.IsAutoRoundMember` determines whether the specified round feature is a member of an Auto Round feature.

## Feature Operations

Methods and Properties Introduced:

- **`IpfcSolid.ExecuteFeatureOps()`**
- **`IpfcFeature.CreateSuppressOp()`**

- 
- **IpfcSuppressOperation.Clip**
  - **IpfcSuppressOperation.AllowGroupMembers**
  - **IpfcSuppressOperation.AllowChildGroupMembers**
  - **IpfcFeature.CreateDeleteOp**
  - **IpfcDeleteOperation.Clip**
  - **IpfcDeleteOperation.AllowGroupMembers**
  - **IpfcDeleteOperation.AllowChildGroupMembers**
  - **IpfcDeleteOperation.KeepEmbeddedDatums**
  - **IpfcFeature.CreateResumeOp**
  - **IpfcResumeOperation.WithParents**
  - **IpfcFeature.CreateReorderBeforeOp**
  - **IpfcReorderBeforeOperation.BeforeFeat**
  - **IpfcFeature.CreateReorderAfterOp**
  - **IpfcReorderAfterOperation.AfterFeat**

The method `IpfcSolid.ExecuteFeatureOps()` causes a sequence of feature operations to run in order. Feature operations include suppressing, resuming, reordering, and deleting features. The optional `IpfcRegenInstructions` argument specifies whether the user will be allowed to fix the model if a regeneration failure occurs.

---

#### **Note**

Regenerating models in `resolve_mode` is not supported. As a result, the method `IpfcSolid.ExecuteFeatureOps()` is not supported anymore.

---

You can create an operation that will delete, suppress, reorder, or resume certain features using the methods in the class `IpfcFeature`. Each created operation must be passed as a member of the `IpfcFeatureOperations` object to the method `IpfcSolid.ExecuteFeatureOps()`.

Some of the operations have specific options that you can modify to control the behavior of the operation:

- **Clip**—Specifies whether to delete or suppress all features after the selected feature. By default, this option is false.  
Use the properties `IpfcDeleteOperation.Clip` and `IpfcSuppressOperation.Clip` to modify this option.
- **AllowGroupMembers**—If this option is set to true and if the feature to be deleted or suppressed is a member of a group, then the feature will be deleted

---

or suppressed out of the group. If this option is set to false, then the entire group containing the feature is deleted or suppressed. By default, this option is false. It can be set to true only if the option `Clip` is set to true.

Use the properties `IpfcSuppressOperation.AllowGroupMembers` and `IpfcDeleteOperation.AllowGroupMembers` to modify this option.

- `AllowChildGroupMembers`—If this option is set to true and if the children of the feature to be deleted or suppressed are members of a group, then the children of the feature will be individually deleted or suppressed out of the group. If this option is set to false, then the entire group containing the feature and its children is deleted or suppressed. By default, this option is false. It can be set to true only if the options `Clip` and `AllowGroupMembers` are set to true.

Use the properties `IpfcSuppressOperation.AllowChildGroupMembers` and `IpfcDeleteOperation.AllowChildGroupMembers` to modify this option.

- `KeepEmbeddedDatums`—Specifies whether to retain the embedded datums stored in a feature while deleting the feature. By default, this option is false.

Use the property `IpfcDeleteOperation.KeepEmbeddedDatums` to modify this option.

- `WithParents`—Specifies whether to resume the parents of the selected feature.

Use the property `IpfcResumeOperation.WithParents` to modify this option.

- `BeforeFeat`—Specifies the feature before which you want to reorder the features.

Use the property `IpfcReorderBeforeOperation.BeforeFeat` to modify this option.

- `AfterFeat`—Specifies the feature after which you want to reorder the features.

Use the property `IpfcReorderAfterOperation.AfterFeat` to modify this option.

## Feature Groups and Patterns

Patterns are treated as features in Creo Parametric. A feature type, `FEATTYPE_PATTERN_HEAD`, is used for the pattern header feature.

---

 **Note**

The pattern header feature is not treated as a leader or a member of the pattern by the methods described in the following section.

---

Methods and Properties Introduced:

- **IpfcFeature.Group**
- **IpfcFeature.Pattern**
- **IpfcFeature.GetPatternByType()**
- **IpfcFeature.PatternStatus**
- **IpfcFeature.GroupStatus**
- **IpfcFeature.GroupPatternStatus**
- **IpfcSolid.CreateLocalGroup**
- **IpfcFeatureGroup.Pattern**
- **IpfcFeatureGroup.GroupLeader()**
- **IpfcFeaturePattern.PatternLeader**
- **IpfcFeaturePattern.ListMembers()**
- **IpfcFeaturePattern.Delete()**

The property `IpfcFeature.Group` returns a handle to the local group that contains the specified feature.

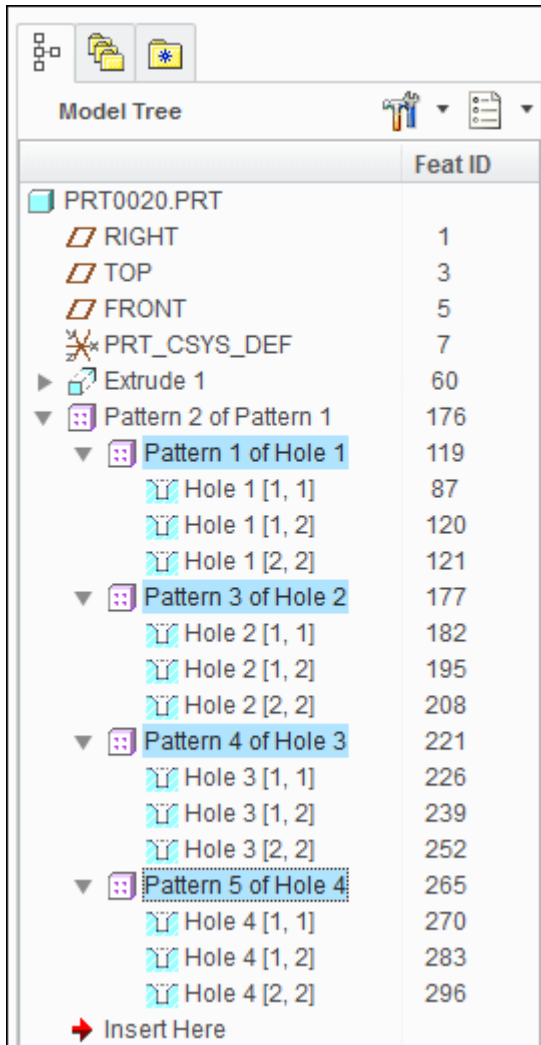
To get the first feature in the specified group call the property `IpfcFeatureGroup.GroupLeader`.

The property `IpfcFeaturePattern.PatternLeader` and the method `IpfcFeaturePattern.ListMembers()` return features that make up the specified feature pattern.

A pattern is composed of a pattern header feature and a number of member features. You can pattern only a single feature. To pattern several features, create a local group and pattern this group.

You can also create a pattern of pattern. This creates a multiple level pattern. From Creo Parametric 2.0 M170 onward, for a pattern of pattern, the method `IpfcFeaturePattern.ListMembers()` returns all the pattern header features created at the first level.

For example, consider a model where a pattern of pattern has been created. The model tree is as shown below:



|                        | Feat ID |
|------------------------|---------|
| PRT0020.PRT            |         |
| RIGHT                  | 1       |
| TOP                    | 3       |
| FRONT                  | 5       |
| PRT_CSYS_DEF           | 7       |
| Extrude 1              | 60      |
| Pattern 2 of Pattern 1 | 176     |
| Pattern 1 of Hole 1    | 119     |
| Hole 1 [1, 1]          | 87      |
| Hole 1 [1, 2]          | 120     |
| Hole 1 [2, 2]          | 121     |
| Pattern 3 of Hole 2    | 177     |
| Hole 2 [1, 1]          | 182     |
| Hole 2 [1, 2]          | 195     |
| Hole 2 [2, 2]          | 208     |
| Pattern 4 of Hole 3    | 221     |
| Hole 3 [1, 1]          | 226     |
| Hole 3 [1, 2]          | 239     |
| Hole 3 [2, 2]          | 252     |
| Pattern 5 of Hole 4    | 265     |
| Hole 4 [1, 1]          | 270     |
| Hole 4 [1, 2]          | 283     |
| Hole 4 [2, 2]          | 296     |
| Insert Here            |         |

The method `IpfcFeaturePattern.ListMembers()` returns the pattern header features with following IDs for a pattern of pattern:

- 119
- 177
- 221
- 265

The properties `IpfcFeature.Pattern` and `IpfcFeatureGroup.Pattern` return the `IpfcFeaturePattern` object that contains the corresponding `IpfcFeature` or `IpfcFeatureGroup`. Use the method `IpfcSolid.CreateLocalGroup()` to take a sequence of



---

features and create a local group with the specified name. To delete a `IpfcFeaturePattern` object, call the method `IpfcFeaturePattern.Delete()`.

The method `IpfcFeature.GetPatternByType()` returns the pattern of the feature and is specified using the enumerated data type `EpfcPatternType`. The valid values are:

- `EpfcPATTERN_FEAT`—Feature pattern.
- `EpfcPATTERN_GROUP`—Group pattern.

The property `IpfcFeature.PatternStatus` returns the pattern status of the feature and is specified using the enumerated data type `EpfcPatternStatus`. The valid values are:

- `EpfcGRP_PATTERN_NONE`—Feature does not belong to any pattern.
- `EpfcGRP_PATTERN_LEADER`—Feature is leader of the pattern.
- `EpfcGRP_PATTERN_MEMBER`—Feature is member of the pattern.
- `EpfcGRP_PATTERN_HEADER`—Feature is header of the pattern.

The property `IpfcFeature.GroupStatus` returns the group status of the feature and is specified using the enumerated data type `EpfcGroupStatus`. The valid values are:

- `EpfcGROUP_NONE`—Feature does not belong to any group.
- `EpfcGROUP_MEMBER`—Feature is member of the group.

The property `IpfcFeature.GroupPatternStatus` returns the group pattern status of the feature and is specified using the enumerated data type `EpfcGroupPatternStatus`. The valid values are:

- `EpfcGRP_PATTERN_NONE`—Feature does not belong to any group pattern.
- `EpfcGRP_PATTERN_LEADER`—Feature is leader of the group pattern.
- `EpfcGRP_PATTERN_MEMBER`—Feature is member of the group pattern.
- `EpfcGRP_PATTERN_HEADER`—Feature is header of the group pattern.

## Notes On Feature Groups

Feature groups have a group header feature, which shows up in the model information and feature list for the model. This feature will be inserted in the regeneration list to a position just before the first feature in the group.

The results of the header feature are as follows:

- Models that contain groups will get one extra feature in the regeneration list, of type `EpfcFEATTYPE_GROUP_HEAD`. This affects the feature numbers of all subsequent features, including those in the group.
- Each group automatically contains the header feature in the list of features returned from `IpfcFeatureGroup.ListMembers()`.

- 
- Each group automatically gets the group head feature as the leader. This is returned from `IpfcFeatureGroup.GroupLeader`.
  - Each group pattern contains a series of groups, and each group in the pattern will be similarly constructed.

## User Defined Features

Groups in Creo Parametric represent sets of contiguous features that act as a single feature for specific operations. Individual features are affected by most operations while some operations apply to an entire group:

- Suppress
- Delete
- Layers
- Patterning

User defined Features (UDFs) are groups of features that are stored in a file. When a UDF is placed in a new model the created features are automatically assigned to a group. A local group is a set of features that have been specifically assigned to a group to make modifications and patterning easier.

---

### Note

All methods in this section can be used for UDFs and local groups.

---

## Read Access to Groups and User Defined Features

Methods and Properties Introduced:

- **`IpfcFeatureGroup.UDFName`**
- **`IpfcFeatureGroup.UDFInstanceName`**
- **`IpfcFeatureGroup.ListUDFDimensions()`**
- **`IpfcUDFDimension.GetUDFDimensionName`**

User defined features (UDF's) are groups of features that can be stored in a file and added to a new model. A local group is similar to a UDF except it is available only in the model in which it was created.

The property `IpfcFeatureGroup.UDFName` provides the name of the group for the specified group instance. A particular group definition can be used more than once in a particular model.

If the group is a family table instance, the property `IpfcFeatureGroup.UDFInstanceName` supplies the instance name.

---

The method `IpfcFeatureGroup.ListUDFDimensions()` traverses the dimensions that belong to the UDF. These dimensions correspond to the dimensions specified as variables when the UDF was created. Dimensions of the original features that were not variables in the UDF are not included unless the UDF was placed using the `Independent` option.

The property `IpfcUDFDimension.GetUDFDimensionName` provides access to the dimension name specified when the UDF was created, and not the name of the dimension in the current model. This name is required to place the UDF programmatically using the method `IpfcSolid.CreateUDFGroup()`.

## Creating Features from UDFs

Method Introduced:

- **`IpfcSolid.CreateUDFGroup()`**

The method `IpfcSolid.CreateUDFGroup()` is used to create new features by retrieving and applying the contents of an existing UDF file. It is equivalent to the Creo Parametric command `Feature, Create, User Defined`.

To understand the following explanation of this method, you must have a good knowledge and understanding of the use of UDF's in Creo Parametric. PTC recommends that you read about UDF's in the Creo Parametric help, and practice defining and using UDF's in Creo Parametric before you attempt to use this method.

When you create a UDF interactively, Creo Parametric prompts you for the information it needs to fix the properties of the resulting features. When you create a UDF from the VB API, you can provide some or all of this information programmatically by filling several compact data classes that are inputs to the method `IpfcSolid.CreateUDFGroup`.

During the call to `IpfcSolid.CreateUDFGroup()`, Creo Parametric prompts you for the following:

- Information required by the UDF that was not provided in the input data structures.
- Correct information to replace erroneous information

Such prompts are a useful way of diagnosing errors when you develop your application. This also means that, in addition to creating UDF's programmatically to provide automatic synthesis of model geometry, you can also use `IpfcSolid.CreateUDFGroup()` to create UDF's semi-interactively. This can simplify the interactions needed to place a complex UDF making it easier for the user and less prone to error.

### Creating UDFs

Creating a UDF requires the following information:

- 
- **Name**—The name of the UDF you are creating and the instance name if applicable.
  - **Dependency**—Specify if the UDF is independent of the UDF definition or is modified by the changes made to it.
  - **Scale**—How to scale the UDF relative to the placement model.
  - **Variable Dimension**—The new values of the variables dimensions and pattern parameters, those whose values can be modified each time the UDF is created.
  - **Dimension Display**—Whether to show or blank non-variable dimensions created within the UDF group.
  - **References**—The geometrical elements that the UDF needs in order to relate the features it contains to the existing models features. The elements correspond to the picks that Creo Parametric prompts you for when you create a UDF interactively using the prompts defined when the UDF was created. You cannot select an embedded datum as the UDF reference.
  - **Parts Intersection**—When a UDF that is being created in an assembly contains features that modify the existing geometry you must define which parts are affected or intersected. You also need to know at what level in an assembly each intersection is going to be visible.
  - **Orientations**—When a UDF contains a feature with a direction that is defined in respect to a datum plane Creo Parametric must know what direction the new feature will point to. When you create such a UDF interactively Creo Parametric prompt you for this information with a flip arrow.
  - **Quadrants**—When a UDF contains a linearly placed feature that references two datum planes to define its location in the new model Creo Parametric prompts you to pick the location of the new feature. This is determined by which side of each datum plane the feature must lie. This selection is referred to as the quadrant because there are four possible combinations for each linearly placed feature.

To pass all the above values to Creo Parametric, the VB API uses a special class that prepares and sets all the options and passes them to Creo Parametric.

## Creating Interactively Defined UDFs

Method Introduced:

- **CCpfcUDFPromptCreateInstructions.Create()**

This static method is used to create an instructions object that can be used to prompt a user for the required values that will create a UDF interactively.

## Creating a Custom UDF

Method Introduced:

---

- **CCpfcUDFCustomCreateInstructions.Create()**

This method creates a `IpfcUDFCustomCreateInstructions` object with a specified name. To set the UDF creation parameters programmatically you must modify this object as described below. The members of this class relate closely to the prompts Creo Parametric gives you when you create a UDF interactively. PTC recommends that you experiment with creating the UDF interactively using Creo Parametric before you write the the VB API code to fill the structure.

#### Setting the Family Table Instance Name

Property Introduced:

- **IpfcUDFCustomCreateInstructions.InstanceName**

If the UDF contains a family table, this field can be used to select the instance in the table. If the UDF does not contain a family table, or if the generic instance is to be selected, the do not set the string.

#### Setting Dependency Type

Property Introduced:

- **IpfcUDFCustomCreateInstructions.DependencyType**

The `EpfcUDFDependencyType` object represents the dependency type of the UDF. The choices correspond to the choices available when you create a UDF interactively. This enumerated type takes the following values:

- `EpfcUDFDEP_INDEPENDENT`
- `EpfcUDFDEP_DRIVEN`



#### Note

`EpfcUDFDEP_INDEPENDENT` is the default value, if this option is not set.

---

#### Setting Scale and Scale Type

Properties Introduced:

- **IpfcUDFCustomCreateInstructions.ScaleType**
- **IpfcUDFCustomCreateInstructions.Scale**

The property `IpfcUDFCustomCreateInstructions.ScaleType` specifies the length units of the UDF in the form of the `EpfcUDFScaleType` object. This enumerated type takes the following values:

- `EpfcUDFSCALE_SAME_SIZE`
- `EpfcUDFSCALE_SAME_DIMS`

- 
- `EpfcUDFSCALE_CUSTOM`
  - `EpfcUDFSCALE_nil`

---

 **Note**

The default value is `EpfcUDFSCALE_SAME_SIZE` if this option is not set.

---

The property `IpfcUDFCustomCreateInstructions.Scale` specifies the scale factor. If the *ScaleType* is set to `EpfcUDFSCALE_CUSTOM`, the property `IpfcUDFCustomCreateInstructions.Scale` assigns the user defined scale factor. Otherwise, this attribute is ignored.

### Setting the Appearance of the Non UDF Dimensions

Property Introduced:

- **`IpfcUDFCustomCreateInstructions.DimDisplayType`**

The object `EpfcUDFDimensionDisplayType` sets the options in Creo Parametric for determining the appearance in the model of UDF dimensions and pattern parameters that were not variable in the UDF, and therefore cannot be modified in the model. This enumerated type takes the following values:

- `EpfcUDFDISPLAY_NORMAL`
- `EpfcUDFDISPLAY_READ_ONLY`
- `EpfcUDFDISPLAY_BLANK`

---

 **Note**

The default value is `EpfcUDFDISPLAY_NORMAL` if this option is not set.

---

### Setting the Variable Dimensions and Parameters

Methods and Properties Introduced:

- **`IpfcUDFCustomCreateInstructions.VariantValues`**
- **`CCpfcUDFVariantDimension.Create()`**
- **`CCpfcUDFVariantPatternParam.Create()`**

`IpfcUDFVariantValues` class represents an array of variable dimensions and pattern parameters.

`CCpfcUDFVariantDimension.Create()` is a static method creating a `IpfcUDFVariantDimension`. It accepts the following parameters:

- *Name*—The symbol that the dimension had when the UDF was originally defined not the prompt that the UDF uses when it is created interactively. To make this name easy to remember, before you define the UDF that you plan to create with the VB API, you should modify the symbols of all the dimensions that you want to select to be variable. If you get the name wrong, `IpfcSolid.CreateUDFGroup()` will not recognize the dimension and prompts the user for the value in the usual way does not modify the value.
- *DimensionValue*—The new value.

If you do not remember the name, you can find it by creating the UDF interactively in a test model, then using the

`IpfcFeatureGroup.ListUDFDimensions` and

`IpfcUDFDimension.GetUDFDimensionName` to find out the name.

`CCpfcUDFVariantPatternParam.Create()` is a static method which creates a `IpfcUDFVariantPatternParam`. It accepts the following parameters:

- *name*—The string name that the pattern parameter had when the UDF was originally defined
- *patternparam*—The new value.

After the `IpfcUDFVariantValues` object has been compiled, use `IpfcUDFCustomCreateInstructions.VariantValues` to add the variable dimensions and parameters to the instructions.

## Setting the User Defined References

Methods and Properties Introduced:

- **`CCpfcUDFReference.Create()`**
- **`IpfcUDFReference.IsExternal`**
- **`IpfcUDFReference.ReferenceItem`**
- **`IpfcUDFCustomCreateInstructions.References`**

The method `CCpfcUDFReference.Create()` is a static method creating a `IpfcUDFReference` object. It accepts the following parameters:

- *PromptForReference*—The prompt defined for this reference when the UDF was originally set up. It indicates which reference this structure is providing. If you get the prompt wrong, `pfcSolid.CreateUDFGroup()` will not recognize it and prompts the user for the reference in the usual way.
- *ReferenceItem*—Specifies the `IpfcSelection` object representing the referenced element. You can set `Selection` programmatically or prompt the user for a selection separately. You cannot set an embedded datum as the UDF reference.

---

There are two types of reference:

- **Internal**—The referenced element belongs directly to the model that will contain the UDF. For an assembly, this means that the element belongs to the top level.
- **External**—The referenced element belongs to an assembly member other than the placement member.

To set the reference type, use the property  
`IpfcUDFReference.IsExternal`.

To set the item to be used for reference, use the property  
`IpfcUDFReference.ReferenceItem`.

After the `IpfcUDFReferences` object has been set, use  
`IpfcUDFCustomCreateInstructions.References` to add the  
program-defined references.

## Setting the Assembly Intersections

Methods and Properties Introduced:

- **`CCpfcUDFAssemblyIntersection.Create()`**
- **`IpfcUDFAssemblyIntersection.InstanceNames`**
- **`IpfcUDFCustomCreateInstructions.Intersections`**

`CCpfcUDFAssemblyIntersection.Create()` is a static method creating a `IpfcUDFReference` object. It accepts the following parameters:

- *ComponentPath*—Is an `intseq` type object representing the component path of the part to be intersected.
- *Visibility level*—The number that corresponds to the visibility level of the intersected part in the assembly. If the number is equal to the length of the component path the feature is visible in the part that it intersects. If *Visibility level* is 0, the feature is visible at the level of the assembly containing the UDF.

`IpfcUDFAssemblyIntersection.InstanceNames` sets an array of names for the new instances of parts created to represent the intersection geometry. This property accepts the following parameters:

- *instance names*—is a `com.ptc.cipjava.stringseq` type object representing the array of new instance names.

After the `IpfcUDFAssemblyIntersections` object has been set, use  
`IpfcUDFCustomCreateInstructions.Intersections` to add the  
assembly intersections.



---

## Setting Orientations

Properties Introduced:

- **IpfcUDFCustomCreateInstructions.Orientations**

`IpfcUDFOrientations` class represents an array of orientations that provide the answers to Creo Parametric prompts that use a flip arrow. Each term is a `EpfcUDFOrientation` object that takes the following values:

- `EpfcUDFORIENT_INTERACTIVE`—Prompt for the orientation using a flip arrow.
- `EpfcUDFORIENT_NO_FLIP`—Accept the default flip orientation.
- `EpfcUDFORIENT_FLIP`—Invert the orientation from the default orientation.

The order of orientations should correspond to the order in which Creo Parametric prompts for them when the UDF is created interactively. If you do not provide an orientation that Creo Parametric needs, it uses the default value `NO_FLIP`.

After the `IpfcUDFOrientations` object has been set use `IpfcUDFCustomCreateInstructions.Orientation` to add the orientations.

## Setting Quadrants

Property Introduced:

- **IpfcUDFCustomCreateInstructions.Quadrants**

The property `IpfcUDFCustomCreateInstructions.Quadrants` sets an array of points, which provide the X, Y, and Z coordinates that correspond to the picks answering the Creo Parametric prompts for the feature positions. The order of quadrants should correspond to the order in which Creo Parametric prompts for them when the UDF is created interactively.

## Setting the External References

Property Introduced:

- **IpfcUDFCustomCreateInstructions.ExtReferences**

The property `IpfcUDFCustomCreateInstructions.ExtReferences` sets an external reference assembly to be used when placing the UDF. This will be required when placing the UDF in the component using references outside of that component. References could be to the top level assembly of another component.

---

### Example Code 1

The sample code in the file `pfcFeaturesExamples.vb` located at `<creo_vbapi_loadpoint>/vbapi_examples` copies of a node UDF at a particular coordinate system location in a part. The node UDF is a spherical cut centered at the coordinate system whose diameter is driven by the 'diam' argument to the method..

### Example Code 2

The sample code in the file `pfcFeaturesExamples.vb` located at `<creo_vbapi_loadpoint>/vbapi_examples` places copies of a hole UDF at a particular location in an assembly. The hole is embedded in a surface of one of the assembly's components, and placed a particular location away from two normal datum planes (the default value for the dimension is used for this example). The UDF creation requires a quadrant determining the location for the UDF (since it could be one of four areas) and intersection instructions for the assembly members (this example makes the hole visible down to the part level). The method returns the `IpfcFeatureGroup` object created.

## Datum Features

### Datum Features

This section describes the VB API methods and properties that provide read access to the properties of datum features.

### Datum Plane Features

The properties of the Datum Plane feature are defined in the `IpfcDatumPlaneFeat` data object.

Methods and Properties Introduced:

- **`IpfcDatumPlaneFeat.Flip`**
- **`IpfcDatumPlaneFeat.Constraints`**
- **`IpfcDatumPlaneConstraint.ConstraintType`**
- **`IpfcDatumPlaneThroughConstraint.ThroughRef`**
- **`IpfcDatumPlaneNormalConstraint.NormalRef`**
- **`IpfcDatumPlaneParallelConstraint.ParallelRef`**
- **`IpfcDatumPlaneTangentConstraint.TangentRef`**
- **`IpfcDatumPlaneOffsetConstraint.OffsetRef`**
- **`IpfcDatumPlaneOffsetConstraint.OffsetValue`**

- **IpfcDatumPlaneOffsetCoordSysConstraint.CsysAxis**
- **IpfcDatumPlaneAngleConstraint.AngleRef**
- **IpfcDatumPlaneAngleConstraint.AngleValue**
- **IpfcDatumPlaneSectionConstraint.SectionRef**
- **IpfcDatumPlaneSectionConstraint.SectionIndex**

The properties of the `IpfcDatumPlaneFeat` object are described as follows:

- **Flip**—Specifies whether the datum plane was flipped during creation. Use the property `IpfcDatumPlaneFeat.Flip` to determine if the datum plane was flipped during creation.
- **Constraints**—Specifies a collection of constraints given by the `IpfcDatumPlaneConstraint` object. The property `IpfcDatumPlaneFeat.Constraints` obtains the collection of constraints defined for the datum plane.

Use the property `IpfcDatumPlaneConstraint.ConstraintType` to obtain the type of constraint. The type of constraint is given by the `IpfcDatumPlaneConstraintType` enumerated type. The available types are as follows:

- **EpfcDTMPLN\_THRU**—Specifies the Through constraint. The `IpfcDatumPlaneThroughConstraint` object specifies this constraint. Use the property `IpfcDatumPlaneThroughConstraint.ThroughRef` to get the reference selection handle for the Through constraint.
- **EpfcDTMPLN\_NORM**—Specifies the Normal constraint. The `IpfcDatumPlaneNormalConstraint` object specifies this constraint. Use the property `IpfcDatumPlaneNormalConstraint.NormalRef` to get the reference selection handle for the Normal constraint.
- **EpfcDTMPLN\_PRL**—Specifies the Parallel constraint. The `IpfcDatumPlaneParallelConstraint` object specifies this constraint. Use the property `IpfcDatumPlaneParallelConstraint.ParallelRef` to get the reference selection handle for the Parallel constraint.
- **EpfcDTMPLN\_TANG**—Specifies the Tangent constraint. The `IpfcDatumPlaneTangentConstraint` object specifies this constraint. Use the property `IpfcDatumPlaneTangentConstraint.TangentRef` to get the reference selection handle for the Tangent constraint.
- **EpfcDTMPLN\_OFFS**—Specifies the Offset constraint. The `IpfcDatumPlaneOffsetConstraint` object specifies this constraint. Use the property `IpfcDatumPlaneOffsetConstraint.OffsetRef`

---

to get the reference selection handle for the Offset constraint. Use the property `IpfcDatumPlaneOffsetConstraint.OffsetValue` to get the offset value.

An Offset constraint where the offset reference is a coordinate system is given by the `IpfcDatumPlaneOffsetCoordSysConstraint` object. Use the property

`IpfcDatumPlaneOffsetCoordSysConstraint.CsysAxis` to get the reference coordinate axis.

- `EpfcDTMPLN_ANG`—Specifies the Angle constraint. The `IpfcDatumPlaneAngleConstraint` object specifies this constraint. Use the property `IpfcDatumPlaneAngleConstraint.AngleRef` to get the reference selection handle for the Angle constraint. Use the property `IpfcDatumPlaneAngleConstraint.AngleValue` to get the angle value.
- `EpfcDTMPLN_SEC`—Specifies the Section constraint. The `IpfcDatumPlaneSectionConstraint` object specifies this constraint. Use the property `IpfcDatumPlaneSectionConstraint.SectionRef` to get the reference selection for the Section constraint. Use the property `IpfcDatumPlaneSectionConstraint.SectionIndex` to get the section index.

## Datum Axis Features

The properties of the Datum Axis feature are defined in the `IpfcDatumAxisFeat` data object.

Methods and Properties Introduced:

- **`IpfcDatumAxisFeat.Constraints`**
- **`IpfcDatumAxisConstraint.ConstraintType`**
- **`IpfcDatumAxisConstraint.ConstraintRef`**
- **`IpfcDatumAxisFeat.DimConstraints`**
- **`IpfcDatumAxisDimensionConstraint.DimOffset`**
- **`IpfcDatumAxisDimensionConstraint.DimRef`**

The properties of the `IpfcDatumAxisFeat` object are described as follows:

- `Constraints`—Specifies a collection of constraints given by the `IpfcDatumAxisConstraint` object. The property `IpfcDatumAxisFeat.Constraints` obtains the collection of constraints applied to the Datum Axis feature.

---

This object contains the following attributes:

- **ConstraintType**—Specifies the type of constraint in terms of the `EpfcDatumAxisConstraintType` enumerated type. The constraint type determines the type of datum axis. The constraint types are:
  - ◆ `EpfcDTMAXIS_NORMAL`—Specifies the Normal datum constraint.
  - ◆ `EpfcDTMAXIS_THRU`—Specifies the Through datum constraint.
  - ◆ `EpfcDTMAXIS_TANGENT`—Specifies the Tangent datum constraint.
  - ◆ `EpfcDTMAXIS_CENTER`—Specifies the Center datum constraint.Use the property `IpfcDatumAxisConstraint.ConstraintType` to get the constraint type.
- **ConstraintRef**—Specifies the reference selection for the constraint. Use the property `IpfcDatumAxisConstraint.ConstraintRef` to get the reference selection handle.
- **DimConstraints**—Specifies a collection of dimension constraints given by the `IpfcDatumAxisDimensionConstraint` object. The property `IpfcDatumAxisFeat.DimConstraints` obtains the collection of dimension constraints applied to the Datum Axis feature.

This `IpfcDatumAxisDimensionConstraint` object contains the following attributes:

- **DimOffset**—Specifies the offset value for the dimension constraint. Use the property `IpfcDatumAxisDimensionConstraint.DimOffset` to get the offset value.
- **DimRef**—Specifies the reference selection for the dimension constraint. Use the property `IpfcDatumAxisDimensionConstraint.DimRef` to get the reference selection handle.

## General Datum Point Features

The properties of the General Datum Point feature are defined in the `IpfcDatumPointFeat` data object.

Methods and Properties Introduced:

- **`IpfcDatumPointFeat.FeatName`**
- **`IpfcDatumPointFeat.GetPoints()`**
- **`IpfcGeneralDatumPoint.Name`**
- **`IpfcGeneralDatumPoint.PlaceConstraints`**

- 
- **IpfcGeneralDatumPoint.DimConstraints**
  - **IpfcDatumPointConstraint.ConstraintRef**
  - **IpfcDatumPointConstraint.ConstraintType**
  - **IpfcDatumPointConstraint.Value**

The properties of the `IpfcDatumPointFeat` object are described as follows:

- **FeatName**—Specifies the name of the General Datum Point feature. Use the property `IpfcDatumPointFeat.FeatName` to get the name.
- **GeneralDatumPoints**—Specifies a collection of general datum points given by the `IpfcGeneralDatumPoint` object. Use the method `IpfcDatumPointFeat.GetPoints()` to obtain the collection of general datum points. The `IpfcGeneralDatumPoint` object consists of the following attributes:
  - **Name**—Specifies the name of the general datum point. Use the property `IpfcGeneralDatumPoint.Name` to get the name.
  - **PlaceConstraints**—Specifies a collection of placement constraints given by the `IpfcDatumPointPlacementConstraint` object. Use the property `IpfcGeneralDatumPoint.PlaceConstraints` to obtain the collection of placement constraints.
  - **DimConstraints**—Specifies a collection of dimension constraints given by the `IpfcDatumPointDimensionConstraint` object. Use the property `IpfcGeneralDatumPoint.DimConstraints` to obtain the collection of dimension constraints.

The constraints for a datum point are given by the `IpfcDatumPointConstraint` object. This object contains the following attributes:

- **ConstraintRef**—Specifies the reference selection for the datum point constraint. Use the property `IpfcDatumPointConstraint.ConstraintRef` to get the reference selection handle.
- **ConstraintType**—Specifies the type of datum point constraint, in terms of the `IpfcDatumPointConstraintType` enumerated type. Use the property `IpfcDatumPointConstraint.ConstraintType` to get the constraint type.
- **Value**—Specifies the constraint reference value with respect to the datum point. Use the property `IpfcDatumPointConstraint.Value` to get the value of the constraint reference with respect to the datum point.

---

The `IpfcDatumPointPlacementConstraint` and `IpfcDatumPointDimensionConstraint` objects inherit from the `IpfcDatumPointConstraint` object. Use the methods of the `IpfcDatumPointFeat` object for the inherited objects.

## Datum Coordinate System Features

The properties of the Datum Coordinate System feature are defined in the `IpfcCoordSysFeat` object.

Methods and Properties Introduced:

- **`IpfcCoordSysFeat.OriginConstraints`**
- **`IpfcDatumCsysOriginConstraint.OriginRef`**
- **`IpfcCoordSysFeat.DimensionConstraints`**
- **`IpfcDatumCsysDimensionConstraint.DimRef`**
- **`IpfcDatumCsysDimensionConstraint.DimValue`**
- **`IpfcDatumCsysDimensionConstraint.DimConstraintType`**
- **`IpfcCoordSysFeat.OrientationConstraints`**
- **`IpfcDatumCsysOrientMoveConstraint.OrientMoveConstraintType`**
- **`IpfcDatumCsysOrientMoveConstraint.OrientMoveValue`**
- **`IpfcCoordSysFeat.IsNormalToScreen`**
- **`IpfcCoordSysFeat.OffsetType`**
- **`IpfcCoordSysFeat.OnSurfaceType`**
- **`IpfcCoordSysFeat.OrientByMethod`**

The properties of the `IpfcCoordSysFeat` object are described as follows:

- **`OriginConstraints`**—Specifies a collection of origin constraints given by the `IpfcDatumCsysOriginConstraint` object. Use the property `IpfcCoordSysFeat.OriginConstraints` to obtain the collection of origin constraints for the coordinate system. This object contains the following attribute:
  - **`OriginRef`**—Specifies the selection reference for the origin. Use the method property `IpfcDatumCsysOriginConstraint.OriginRef` to get the selection reference handle.
- **`DimensionConstraints`**—Specifies a collection of dimension constraints given by the `IpfcDatumCsysDimensionConstraint` object. Use the property `IpfcCoordSysFeat.DimensionConstraints` to obtain the

---

collection of dimension constraints for the coordinate system. This object contains the following attributes:

- **DimRef**—Specifies the reference selection for the dimension constraint. Use the property `IpfcDatumCsysDimensionConstraint.DimRef` to get the reference selection handle.
- **DimValue**—Specifies the value of the reference. Use the property `IpfcDatumCsysDimensionConstraint.DimValue` to get the value.
- **DimConstraintType**—Specifies the type of dimension constraint in terms of the `EpfcDatumCsysDimConstraintType` enumerated type. Use the property `IpfcDatumCsysDimensionConstraint.DimConstraintType` to get the constraint type. The constraint types are:
  - ◆ `EpfcDTMCSYS_DIM_OFFSET`—Specifies the offset type constraint.
  - ◆ `EpfcDTMCSYS_DIM_ALIGN`—Specifies the align type constraint.
- **OrientationConstraints**—Specifies a collection of orientation constraints given by the `IpfcDatumCsysOrientMoveConstraint` object. Use the property `IpfcCoordSysFeat.OrientationConstraints` to obtain the collection of orientation constraints for the coordinate system. This object contains the following attributes:
  - **OrientMoveConstraintType**—Specifies the type of orientation for the constraint. The orientation type is given by the `EpfcDatumCsysOrientMoveConstraintType` enumerated type. Use the property `IpfcDatumCsysOrientMoveConstraint.OrientMoveConstraintType` to get the orientation type.
  - **OrientMoveValue**—Specifies the reference value for the constraint. Use the property `IpfcDatumCsysOrientMoveConstraint.OrientMoveValue` to get the reference value.
- **IsNormalToScreen**—Specifies if the coordinate system is normal to screen. Use the property `IpfcCoordSysFeat.IsNormalToScreen` to determine if the coordinate system is normal to screen.
- **OffsetType**—Specifies the offset type of the coordinate system in terms of the `EpfcDatumCsysOffsetType` enumerated type. Use the property



---

`IpfcCoordSysFeat.OffsetType` to get the offset type. The offset types are:

- `EpfcDTMCSYS_OFFSET_CARTESIAN`—Specifies a cartesian coordinate system that has been defined by setting the values for the `EpfcDTMCSYS_MOVE_TRAN_X`, `EpfcDTMCSYS_MOVE_TRAN_Y`, and `EpfcDTMCSYS_MOVE_TRAN_Z` or `EpfcDTMCSYS_MOVE_ROT_X`, `EpfcDTMCSYS_MOVE_ROT_Y`, and `EpfcDTMCSYS_MOVE_ROT_Z` orientation constants.
- `EpfcDTMCSYS_OFFSET_CYLINDRICAL`—Specifies a cylindrical coordinate system that has been defined by setting the values for the `EpfcDTMCSYS_MOVE_RAD`, `EpfcDTMCSYS_MOVE_THETA`, and `EpfcDTMCSYS_MOVE_TRAN_ZI` orientation constants.
- `EpfcDTMCSYS_OFFSET_SPHERICAL`—Specifies a spherical coordinate system that has been defined by setting the values for the `EpfcDTMCSYS_MOVE_RAD`, `EpfcDTMCSYS_MOVE_THETA`, and `EpfcDTMCSYS_MOVE_TRAN_PHI` orientation constants.
- `OnSurfaceType`—Specifies the on surface type for the coordinate system in terms of the `EpfcDatumCsysOffsetType` enumerated type. Use the property `IpfcCoordSysFeat.OnSurfaceType` to get the on surface type property of the coordinate system. The on surface types are:
  - `EpfcDTMCSYS_ONSURF_LINEAR`—Specifies a coordinate system placed on the selected surface by using two linear dimensions.
  - `EpfcDTMCSYS_ONSURF_RADIAL`—Specifies a coordinate system placed on the selected surface by using a linear dimension and an angular dimension. The radius value is used to specify the linear dimension.
  - `EpfcDTMCSYS_ONSURF_DIAMETER`—This type is similar to the `EpfcDTMCSYS_ONSURF_RADIAL` type, except that the diameter value is used to specify the linear dimension. It is available only when planar surfaces are used as the reference.
- `OrientByMethod`—Specifies the orientation method in terms of the `EpfcDatumCsysOrientByMethod` enumerated type. Use the property `IpfcCoordSysFeat.OrientByMethod` to get the orientation method. The available orientation types are:
  - `EpfcDTMCSYS_ORIENT_BY_SEL_REFS`—Specifies the orientation by selected references.
  - `EpfcDTMCSYS_ORIENT_BY_SEL_CSYS_AXES`—Specifies the orientation by coordinate system axes.

---

# Geometry Evaluation

## Geometry Evaluation

This section describes geometry representation and discusses how to evaluate geometry using the VB API.

## Geometry Traversal

- A simple rectangular face has one contour and four edges.
- A contour will traverse a boundary so that the part face is always on the right-hand side (RHS). For an external contour the direction of traversal is clockwise. For an internal contour the direction of traversal is counterclockwise.
- If a part is extruded from a sketch that has a U-shaped cross section there will be separate surfaces at each leg of the U-channel.
- If a part is extruded from a sketch that has a square-shaped cross section, and a slot feature is then cut into the part to make it look like a U-channel, there will be one surface across the legs of the U-channel. The original surface of the part is represented as one surface with a cut through it.

## Geometry Terms

Following are definitions for some geometric terms:

- Surface—An ideal geometric representation, that is, an infinite plane.
- Face—A trimmed surface. A face has one or more contours.
- Contour—A closed loop on a face. A contour consists of multiple edges. A contour can belong to one face only.
- Edge—The boundary of a trimmed surface.

An edge of a solid is the intersection of two surfaces. The edge belongs to those two surfaces and to two contours. An edge of a datum surface can be either the intersection of two datum surfaces or the external boundary of the surface.

If the edge is the intersection of two datum surfaces it will belong to those two surfaces and to two contours. If the edge is the external boundary of the datum surface it will belong to that surface alone and to a single contour.

## Traversing the Geometry of a Solid Block

Methods Introduced:

- **IpfcModelItemOwner.ListItems()**
- **IpfcSurface.ListContours()**

- **IpfcContour.ListElements()**

To traverse the geometry, follow these steps:

1. Starting at the top-level model, use `IpfcModelItemOwner.ListItems()` with an argument of `ModelItemType.ITEM_SURFACE`.
2. Use `IpfcSurface.ListContours()` to list the contours contained in a specified surface.
3. Use `IpfcContour.ListElements()` to list the edges contained in the contour.

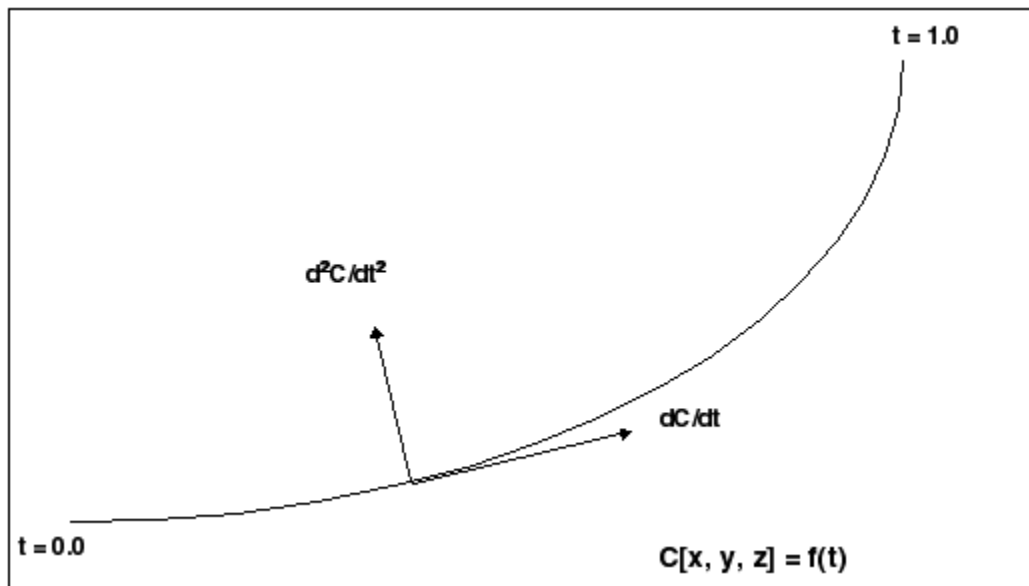
## Curves and Edges

Datum curves, surface edges, and solid edges are represented in the same way in the VB API. You can get edges through geometry traversal or get a list of edges using the methods presented in the section [ModelItem on page 166](#).

### The t Parameter

The geometry of each edge or curve is represented as a set of three parametric equations that represent the values of x, y, and z as functions of an independent parameter,  $t$ . The  $t$  parameter varies from 0.0 at the start of the curve to 1.0 at the end of it.

The following figure illustrates curve and edge parameterization.



### Curve and Edge Types

Solid edges and datum curves can be any of the following types:

- **LINE**—A straight line represented by the interface `IpfcLine`.
- **ARC**—A circular curve represented by the interface `IpfcArc`.
- **SPLINE**—A nonuniform cubic spline, represented by the interface `IpfcSpline`.
- **B-SPLINE**—A nonuniform rational B-spline curve or edge, represented by the interface `IpfcBSpline`.
- **COMPOSITE CURVE**—A combination of two or more curves, represented by the interface `IpfcCompositeCurve`. This is used for datum curves only.

See the appendix section [Geometry Representations on page 327](#) for the parameterization of each curve type. To determine what type of curve a `IpfcEdge` or `IpfcCurve` object represents, use the Java `instanceof` operator.

Because each curve class inherits from `IpfcGeomCurve`, you can use all the evaluation methods in `IpfcGeomCurve` on any edge or curve.

The following curve types are not used in solid geometry and are reserved for future expansion:

- **CIRCLE** (`IpfcCircle`)
- **ELLIPSE** (`IpfcEllipse`)
- **POLYGON** (`IpfcPolygon`)
- **ARROW** (`IpfcArrow`)
- **TEXT** (`IpfcText`)

## Evaluation of Curves and Edges

Methods Introduced:

- **`IpfcGeomCurve.Eval3DData()`**
- **`IpfcGeomCurve.EvalFromLength()`**
- **`IpfcGeomCurve.EvalParameter()`**
- **`IpfcGeomCurve.EvalLength()`**
- **`IpfcGeomCurve.EvalLengthBetween()`**

The methods in `IpfcGeomCurve` provide information about any curve or edge.

The method `IpfcGeomCurve.Eval3DData()` returns a `IpfcCurveXYZData` object with information on the point represented by the input parameter `t`. The method `IpfcGeomCurve.EvalFromLength()` returns a similar object with information on the point that is a specified distance from the starting point.

---

The method `IpfcGeomCurve.EvalParameter()` returns the `t` parameter that represents the input `IpfcPoint3D` object.

Both `IpfcGeomCurve.EvalLength()` and `IpfcGeomCurve.EvalLengthBetween()` return numerical values for the length of the curve or edge.

### Solid Edge Geometry

Methods and Properties Introduced:

- **`IpfcEdge.Surface1`**
- **`IpfcEdge.Surface2`**
- **`IpfcEdge.Edge1`**
- **`IpfcEdge.Edge2`**
- **`IpfcEdge.EvalUV()`**
- **`IpfcEdge.GetDirection()`**

---

#### Note

The methods in the interface `IpfcEdge` provide information only for solid or surface edges.

---

The properties `IpfcEdge.Surface1` and `Edge.Surface2` return the surfaces bounded by this edge. The properties `IpfcEdge.Edge1` and `IpfcEdge.Edge2` return the next edges in the two contours that contain this edge.

The method `IpfcEdge.EvalUV()` evaluates geometry information based on the UV parameters of one of the bounding surfaces.

The method `IpfcEdge.GetDirection()` returns a positive 1 if the edge is parameterized in the same direction as the containing contour, and -1 if the edge is parameterized opposite to the containing contour.

### Curve Descriptors

A curve descriptor is a data object that describes the geometry of a curve or edge. A curve descriptor describes the geometry of a curve without being a part of a specific model.

Methods Introduced:

- **`IpfcGeomCurve.GetCurveDescriptor()`**

- **IpfcGeomCurve.GetNURBSRepresentation()**

---

 **Note**

To get geometric information for an edge, access the `IpfcCurveDescriptor` object for one edge using `IpfcGeomCurve.GetCurveDescriptor()`.

---

The method `IpfcGeomCurve.GetCurveDescriptor()` returns a curve's geometry as a data object.

The method `IpfcGeomCurve.GetNURBSRepresentation()` returns a Non-Uniform Rational B-Spline Representation of a curve.

## Contours

Methods and Properties Introduced:

- **IpfcSurface.ListContours()**
- **IpfcContour.InternalTraversal**
- **IpfcContour.FindContainingContour()**
- **IpfcContour.EvalArea()**
- **IpfcContour.EvalOutline()**
- **IpfcContour.VerifyUV()**

Contours are a series of edges that completely bound a surface. A contour is not a `IpfcModelItem`. You cannot get contours using the methods that get different types of `IpfcModelItem`. Use the method `IpfcSurface.ListContours()` to get contours from their containing surfaces.

The property `IpfcContour.InternalTraversal` returns a `IpfcContourTraversal` enumerated type that identifies whether a given contour is on the outside or inside of a containing surface.

Use the method `IpfcContour.FindContainingContour()` to find the contour that entirely encloses the specified contour.

The method `IpfcContour.EvalArea()` provides the area enclosed by the contour.

The method `IpfcContour.EvalOutline()` returns the points that make up the bounding rectangle of the contour.

Use the method `IpfcContour.VerifyUV()` to determine whether the given `IpfcUVParams` argument lies inside the contour, on the boundary, or outside the contour.

---

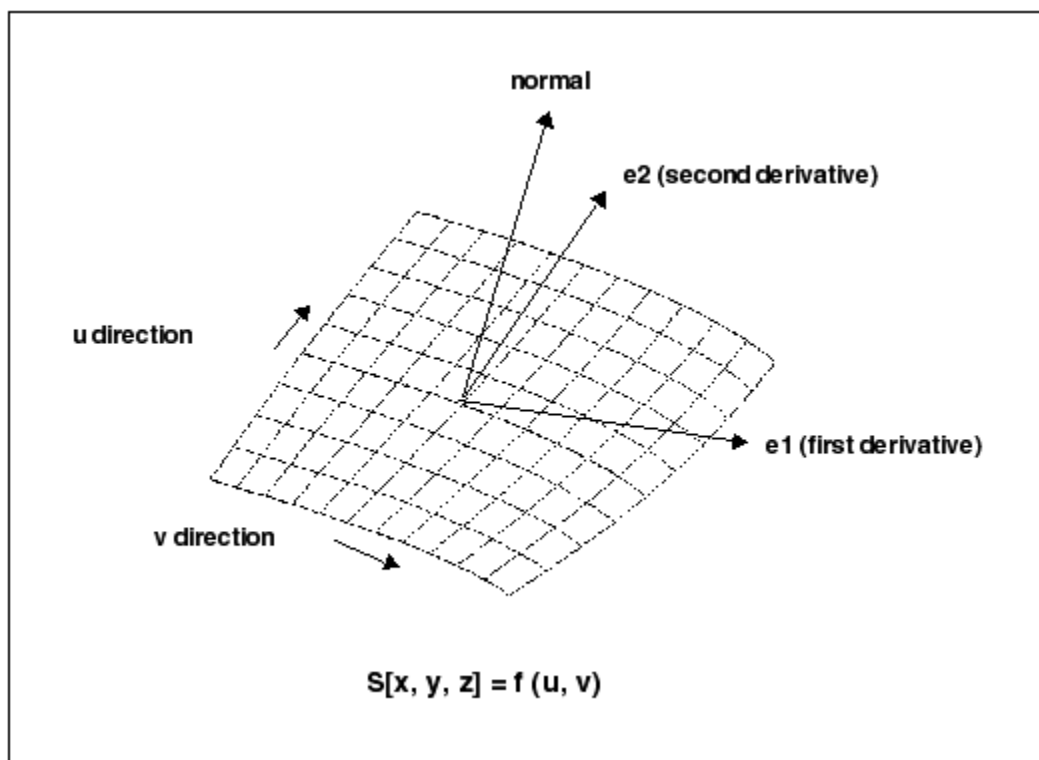
## Surfaces

Using the VB API you access datum and solid surfaces in the same way.

### UV Parameterization

A surface in Creo Parametric is described as a series of parametric equations where two parameters,  $u$  and  $v$ , determine the  $x$ ,  $y$ , and  $z$  coordinates. Unlike the edge parameter,  $t$ , these parameters need not start at 0.0, nor are they limited to 1.0.

The figure on the following page illustrates surface parameterization.



### Surface Types

Surfaces within Creo Parametric can be any of the following types:

- **PLANE**—A planar surface represented by the interface `IpfcPlane`.
- **CYLINDER**—A cylindrical surface represented by the interface `IpfcCylinder`.
- **CONE**—A conic surface region represented by the interface `IpfcCone`.
- **TORUS**—A toroidal surface region represented by the interface `IpfcTorus`.
- **REVOLVED SURFACE**—Generated by revolving a curve about an axis. This is represented by the interface `IpfcRevSurface`.

- RULED SURFACE—Generated by interpolating linearly between two curve entities. This is represented by the interface `IpfcRuledSurface`.
- TABULATED CYLINDER—Generated by extruding a curve linearly. This is represented by the interface `IpfcTabulatedCylinder`.
- COONS PATCH—A coons patch is used to blend surfaces together. It is represented by the interface `IpfcCoonsPatch`.
- FILLET SURFACE—A filleted surface is found where a round or fillet is placed on a curved edge or an edge with a non-consistant arc radii. On a straight edge a cylinder is used to represent a fillet. This is represented by the class interface `IpfcFilletedSurface`.
- SPLINE SURFACE— A nonuniform bicubic spline surface that passes through a grid with tangent vectors given at each point. This is represented by the interface `IpfcSplineSurface`.
- NURBS SURFACE—A NURBS surface is defined by basic functions (in  $u$  and  $v$ ), expandable arrays of knots, weights, and control points. This is represented by the class interface `IpfcNURBSSurface`.
- CYLINDRICAL SPLINE SURFACE— A cylindrical spline surface is a nonuniform bicubic spline surface that passes through a grid with tangent vectors given at each point. This is represented by the interface `IpfcCylindricalSplineSurface`.

To determine which type of surface a `IpfcSurface` object represents, access the surface type using `IpfcSurface.GetSurfaceType()`.

## Surface Information

Methods Introduced:

- **`IpfcSurface.GetSurfaceType()`**
- **`IpfcSurface.GetXYZExtents()`**
- **`IpfcSurface.GetUVExtents()`**
- **`IpfcSurface.GetOrientation()`**

The method `IpfcSurface.GetSurfaceType()` returns the type of the surface using the enumerated data type `EpfcSurfaceType` and the valid values are:

- `EpfcSURFACE_PLANE`
- `EpfcSURFACE_CYLINDER`
- `EpfcSURFACE_CONE`
- `EpfcSURFACE_TORUS`
- `EpfcSURFACE_RULED`



- `EpfcSURFACE_REVOLVED`
- `EpfcSURFACE_TABULATED_CYLINDER`
- `EpfcSURFACE_FILLET`
- `EpfcSURFACE_COONS_PATCH`
- `EpfcSURFACE_SPLINE`
- `EpfcSURFACE_NURBS`
- `EpfcSURFACE_CYLINDRICAL_SPLINE`
- `EpfcSURFACE_FOREIGN`
- `EpfcSURFACE_SPL2DER`

The method `IpfcSurface::GetXYZExtents()` returns the XYZ points at the corners of the surface.

The method `IpfcSurface::GetUVExtents()` returns the UV parameters at the corners of the surface.

The method `IpfcSurface::GetOrientation()` returns the orientation of the surface using the enumerated data type `EpfcSurfaceOrientation` and the valid values are:

- `EpfcSURFACEORIENT_NONE`—Surface that does not need orientation. For example, a solid surface needs orientation and therefore cannot be specified.
- `EpfcSURFACEORIENT_OUTWARD`—Surface that has oriented outward away from the solid model.  $du \times dv$  points outward.
- `EpfcSURFACEORIENT_INWARD`—Surface that has oriented inward toward the solid model.  $du \times dv$  points inward.

## Evaluation of Surfaces

Surface methods allow you to use multiple surface information to calculate, evaluate, determine, and examine surface functions and problems.

Methods and Properties Introduced:

- **`IpfcSurface.OwnerQuilt`**
- **`IpfcSurface.EvalClosestPoint()`**
- **`IpfcSurface.EvalClosestPointOnSurface()`**
- **`IpfcSurface.Eval3DData()`**
- **`IpfcSurfXYZData.FirstDerivative`**
- **`IpfcSurfXYZData.SecondDerivative`**
- **`IpfcSurface.EvalParameters()`**
- **`IpfcSurface.EvalArea()`**
- **`IpfcSurface.EvalDiameter()`**

- **IpfcSurface.EvalPrincipalCurv()**
- **IpfcSurface.VerifyUV()**
- **IpfcSurface.EvalMaximum()**
- **IpfcSurface.EvalMinimum()**
- **IpfcSurface.ListSameSurfaces**

The property `IpfcSurface.OwnerQuilt` returns the `Quilt` object that contains the datum surface.

The method `IpfcSurface.EvalClosestPoint()` projects a three-dimensional point onto the surface. Use the method `IpfcSurface.EvalClosestPointOnSurface()` to determine whether the specified three-dimensional point is on the surface, within the accuracy of the part. If it is, the method returns the point that is exactly on the surface. Otherwise the method returns null.

The method `IpfcSurface.Eval3DData()` returns a `IpfcSurfXYZData` object that contains information about the surface at the specified  $u$  and  $v$  parameters. The method `IpfcSurface.EvalParameters()` returns the  $u$  and  $v$  parameters that correspond to the specified three-dimensional point.

The property `IpfcSurfXYZData.FirstDerivative` returns the first partial derivatives of X, Y, and Z coordinates, with respect to the  $u$  and  $v$  parameters.

The property `IpfcSurfXYZData.SecondDerivative` returns the second partial derivatives of X, Y, and Z coordinates, with respect to the  $u$  and  $v$  parameters.

The method `IpfcSurface.EvalArea()` returns the area of the surface, whereas `IpfcSurface.EvalDiameter()` returns the diameter of the surface. If the diameter varies the optional `IpfcUVParams` argument identifies where the diameter should be evaluated.

The method `IpfcSurface.EvalPrincipalCurv()` returns a `IpfcCurvatureData` object with information regarding the curvature of the surface at the specified  $u$  and  $v$  parameters.

Use the method `IpfcSurface.VerifyUV()` to determine whether the `IpfcUVParams` are actually within the boundary of the surface.

The methods `IpfcSurface.EvalMaximum()` and `IpfcSurface.EvalMinimum()` return the three-dimensional point on the surface that is the furthest in the direction of (or away from) the specified vector.

The method `IpfcSurface.ListSameSurfaces()` identifies other surfaces that are tangent and connect to the given surface.

---

## Surface Descriptors

A surface descriptor is a data object that describes the shape and geometry of a specified surface. A surface descriptor allows you to describe a surface in 3D without an owner ID.

Methods Introduced:

- **IpfcSurface.GetSurfaceDescriptor()**
- **IpfcSurface.GetNURBSRepresentation()**

The method `IpfcSurface.GetSurfaceDescriptor()` returns a surfaces geometry as a data object.

The method `IpfcSurface.GetNURBSRepresentation()` returns a Non-Uniform Rational B-Spline Representation of a surface.

## Axes, Coordinate Systems, and Points

Coordinate axes, datum points, and coordinate systems are all model items. Use the methods that return `IpfcModelItems` to get one of these geometry objects. Refer to the section [ModelItem on page 166](#) for additional information.

### Evaluation of ModelItems

Properties Introduced:

- **IpfcAxis.Surf**
- **IpfcCoordSystem.CoordSys**
- **IpfcPoint.Point**

The property `IpfcAxis.Surf` returns the revolved surface that uses the axis.

The property `IpfcCoordSystem.CoordSys` returns the `IpfcTransform3D` object (which includes the origin and x-, y-, and z- axes) that defines the coordinate system.

The property `IpfcPoint.Point` returns the xyz coordinates of the datum point.

## Interference

Creo Parametric assemblies can contain interferences between components when constraint by certain rules defined by the user. The `IpfcInterference` module allows the user to detect and analyze any interferences within the assembly. The analysis of this functionality should be looked at from two standpoints: global and selection based analysis.

Methods and Properties Introduced:

- **CMpfcInterference.CreateGlobalEvaluator()**

- **IpfcGlobalEvaluator.ComputeGlobalInterference()**
- **IpfcGlobalEvaluator.Assem**
- **IpfcGlobalEvaluator.Assem**
- **IpfcGlobalInterference.Volume**
- **IpfcGlobalInterference.SelParts**

To compute all the interferences within an Assembly one has to call `CMpfcInterference.CreateGlobalEvaluator()` with a `IpfcAssembly` object as an argument. This call returns a `IpfcGlobalEvaluator` object.

The property `IpfcGlobalEvaluator.Assem` accesses the assembly to be evaluated.

The method

`IpfcGlobalEvaluator.ComputeGlobalInterference()` determines the set of all the interferences within the assembly.

This method will return a sequence of `IpfcGlobalInterference` objects or null if there are no interfering parts. Each object contains a pair of intersecting parts and an object representing the interference volume, which can be extracted by using `IpfcGlobalInterference.SelParts` and `IpfcGlobalInterference.Volume` respectively.

## Analyzing Interference Information

Methods and Properties Introduced:

- **CCpfcSelectionPair.Create()**
- **CCpfcInterference.CreateSelectionEvaluator()**
- **IpfcSelectionEvaluator.Selections**
- **IpfcSelectionEvaluator.ComputeInterference()**
- **IpfcSelectionEvaluator.ComputeClearance()**
- **IpfcSelectionEvaluator.ComputeNearestCriticalDistance()**

The method `CCpfcSelectionPair.Create()` creates a `IpfcSelectionPair` object using two `IpfcSelection` objects as arguments.

A return from this method will serve as an argument to `CMpfcInterference.CreateSelectionEvaluator()`, which will provide a way to determine the interference data between the two selections.

`IpfcSelectionEvaluator.Selections` will extract and set the object to be evaluated.

---

`IpfcSelectionEvaluator.ComputeInterference()` determines the interfering information about the provided selections. This method will return the `IpfcInterferenceVolume` object or null if the selections do not interfere.

`IpfcSelectionEvaluator.ComputeClearance()` computes the clearance data for the two selection. This method returns a `IpfcClearanceData` object, which can be used to obtain and set clearance distance, nearest points between selections, and a boolean `IsInterfering` variable.

`IpfcSelectionEvaluator.ComputeNearestCriticalDistance()` finds a critical point of the distance function between two selections.

This method returns a `IpfcCriticalDistanceData` object, which is used to determine and set critical points, surface parameters, and critical distance between points.

## Analyzing Interference Volume

Methods and Properties Introduced:

- **`IpfcInterferenceVolume.ComputeVolume()`**
- **`IpfcInterferenceVolume.Highlight()`**
- **`IpfcInterferenceVolume.Boundaries`**

The method `IpfcInterferenceVolume.ComputeVolume()` will calculate a value for interfering volume.

The method `IpfcInterferenceVolume.Highlight()` will highlight the interfering volume with the color provided in the argument to the function.

The property `IpfcInterferenceVolume.Boundaries` will return a set of boundary surface descriptors for the interference volume.

## Example Code

The sample code in the file `pfcGeometryExamples.vb` located at `<creo_vbapi_loadpoint>/vbapi_examples` finds the interference in an assembly, highlights the interfering surfaces, and highlights calculates the interference volume.

This application finds the interference in an assembly, highlights the interfering surfaces, and highlights calculates the interference volume.

---

# Dimensions and Parameters

## Dimensions and Parameters

This section describes the VB API methods and classes that affect dimensions and parameters.

### Overview

Dimensions and parameters in Creo Parametric have similar characteristics but also have significant differences. In the VB API, the similarities between dimensions and parameters are contained in the `IpfcBaseParameter` class. This class allows access to the parameter or dimension value and to information regarding a parameter's designation and modification. The differences between parameters and dimensions are recognizable because `IpfcDimension` inherits from the class `IpfcModelItem`, and can be assigned tolerances, whereas parameters are not `IpfcModelItems` and cannot have tolerances.

### The ParamValue Object

Both parameters and dimension objects contain an object of type `IpfcParamValue`. This object contains the integer, real, string, or Boolean value of the parameter or dimension. Because of the different possible value types that can be associated with a `IpfcParamValue` object there are different methods used to access each value type and some methods will not be applicable for some `IpfcParamValue` objects. If you try to use an incorrect method an exception will be thrown.

#### Accessing a ParamValue Object

Methods and Property Introduced:

- **`CMpfcModelItem.CreateIntParamValue()`**
- **`CMpfcModelItem.CreateDoubleParamValue()`**
- **`CMpfcModelItem.CreateStringParamValue()`**
- **`CMpfcModelItem.CreateBoolParamValue()`**
- **`CMpfcModelItem.CreateNoteParamValue()`**
- **`IpfcBaseParameter.Value`**

The `CMpfcModelItem` utility class contains methods for creating each type of `IpfcParamValue` object. Once you have established the value type in the object, you can change it. The property `IpfcBaseParameter.Value` returns the `IpfcParamValue` associated with a particular parameter or dimension.

---

A `NoteParamValue` is an integer value that refers to the ID of a specified note. To create a parameter of this type the identified note must already exist in the model.

### Accessing the ParamValue Value

Properties Introduced:

- **`IpfcParamValue.dscr`**
- **`IpfcParamValue.IntValue`**
- **`IpfcParamValue.DoubleValue`**
- **`IpfcParamValue.StringValue`**
- **`IpfcParamValue.BoolValue`**
- **`IpfcParamValue.NoteId`**

The property `IpfcParamValue.dscr` returns an enumeration object that identifies the type of value contained in the `IpfcParamValue` object. Use this information with the specified properties to access the value. If you use an incorrect property an exception of type `IpfcXBadGetParamValue` will be thrown.

## Parameter Objects

The following sections describe the VB API methods that access parameters. The topics are as follows:

- [Creating and Accessing Parameters on page 207](#)
- [Parameter Selection Options on page 208](#)
- [Parameter Information on page 210](#)
- [Parameter Restrictions on page 212](#)

### Creating and Accessing Parameters

Methods and Property Introduced:

- **`IpfcParameterOwner.CreateParam()`**
- **`IpfcParameterOwner.CreateParamWithUnits()`**
- **`IpfcParameterOwner.GetParam()`**
- **`IpfcParameterOwner.ListParams()`**
- **`IpfcParameterOwner.SelectParam()`**
- **`IpfcParameterOwner.SelectParameters()`**
- **`IpfcFamColParam.RefParam`**

---

In the VB API, models, features, surfaces, and edges inherit from the `IpfcParameterOwner` class, because each of the objects can be assigned parameters in Creo Parametric.

The method `IpfcParameterOwner.GetParam()` gets a parameter given its name.

The method `IpfcParameterOwner.ListParams()` returns a sequence of all parameters assigned to the object.

To create a new parameter with a name and a specific value, call the method `IpfcParameterOwner.CreateParam()`.

To create a new parameter with a name, a specific value, and units, call the method `IpfcParameterOwner.CreateParamWithUnits()`.

The method `IpfcParameterOwner.SelectParam()` allows you to select a parameter from the Creo Parametric user interface. The top model from which the parameters are selected must be displayed in the current window.

The method `IpfcParameterOwner.SelectParameters()` allows you to interactively select parameters from the Creo Parametric **Parameter** dialog box based on the parameter selection options specified by the `IpfcParameterSelectionOptions` object. The top model from which the parameters are selected must be displayed in the current window. Refer to the section [Parameter Selection Options on page 208](#) for more information.

The property `IpfcFamColParam.RefParam` returns the reference parameter from the parameter column in a family table.

## Parameter Selection Options

Parameter selection options in the VB API are represented by the `IpfcParameterSelectionOptions` class.

Methods and Properties Introduced:

- **`CCpfcParameterSelectionOptions.Create`**
- **`IpfcParameterSelectionOptions.AllowContextSelection`**
- **`IpfcParameterSelectionOptions.Contexts`**
- **`.IpfcParameterSelectionOptions.AllowMultipleSelections`**
- **`IpfcParameterSelectionOptions.SelectButtonLabel`**

The method `CCpfcParameterSelectionOptions.Create` creates a new instance of the `IpfcParameterSelectionOptions` object that is used by the method `IpfcParameterOwner.SelectParameters()`.

The parameter selection options are as follows:

- **`AllowContextSelection`**—This boolean attribute indicates whether to allow parameter selection from multiple contexts, or from the invoking



---

parameter owner. By default, it is false and allows selection only from the invoking parameter owner. If it is true and if specific selection contexts are not yet assigned, then you can select the parameters from any context.

Use the property

`IpfcParameterSelectionOptions.AllowContextSelection` to modify the value of this attribute.

- Contexts—The permitted parameter selection contexts in the form of the `IIfcParameterSelectionContexts` object. Use the property `IpfcParameterSelectionOptions.Contexts` to assign the parameter selection context. By default, you can select parameters from any context.
- The types of parameter selection contexts are as follows:
  - `EpfcPARAMSELECT_MODEL`—Specifies that the top level model parameters can be selected.
  - `EpfcPARAMSELECT_PART`—Specifies that any part's parameters (at any level of the top model) can be selected.
  - `EpfcPARAMSELECT_ASM`—Specifies that any assembly's parameters (at any level of the top model) can be selected.
  - `EpfcPARAMSELECT_FEATURE`—Specifies that any feature's parameters can be selected.
  - `EpfcPARAMSELECT_EDGE`—Specifies that any edge's parameters can be selected.
  - `EpfcPARAMSELECT_SURFACE`—Specifies that any surface's parameters can be selected.
  - `EpfcPARAMSELECT_QUILT`—Specifies that any quilt's parameters can be selected.
  - `EpfcPARAMSELECT_CURVE`—Specifies that any curve's parameters can be selected.
  - `EpfcPARAMSELECT_COMPOSITE_CURVE`—Specifies that any composite curve's parameters can be selected.
  - `EpfcPARAMSELECT_INHERITED`—Specifies that any inheritance feature's parameters can be selected.
  - `EpfcPARAMSELECT_SKELETON`—Specifies that any skeleton's parameters can be selected.
  - `EpfcPARAMSELECT_COMPONENT`—Specifies that any component's parameters can be selected.
- `AllowMultipleSelections`—This boolean attribute indicates whether or not to allow multiple parameters to be selected from the dialog box, or only

---

a single parameter. By default, it is true and allows selection of multiple parameters.

Use the property

`IpfcParameterSelectionOptions.AllowMultipleSelections` to modify this attribute.

- **SelectButtonLabel**—The visible label for the select button in the dialog box.

Use the property

`IpfcParameterSelectionOptions.SelectButtonLabel` to set the label. If not set, the default label in the language of the active Creo Parametric session is displayed.

## Parameter Information

Methods and Properties Introduced:

- **`IpfcBaseParameter.Value`**
- **`IpfcParameter.GetScaledValue()`**
- **`IpfcParameter.SetScaledValue()`**
- **`IpfcParameter.Units()`**
- **`IpfcBaseParameter.IsDesignated`**
- **`IpfcBaseParameter.IsModified`**
- **`IpfcBaseParameter.ResetFromBackup()`**
- **`IpfcParameter.Description`**
- **`IpfcParameter.GetRestriction()`**
- **`IpfcParameter.GetDriverType()`**
- **`IpfcParameter.Reorder()`**
- **`IpfcParameter.Delete()`**
- **`IpfcNamedModelItem.Name`**

Parameters inherit methods from the `IpfcBaseParameter`, `IpfcParameter` and `IpfcNamedModelItem` classes.

The property `IpfcBaseParameter.Value` returns the value of the parameter or dimension.

The method `IpfcParameter.GetScaledValue()` returns the parameter value in the units of the parameter, instead of the units of the owner model as returned by `IpfcBaseParameter.Value`.

---

The method `IpfcParameter.SetScaledValue()` assigns the parameter value in the units provided, instead of using the units of the owner model as assumed by `IpfcBaseParameter.Value`.

The method `IpfcParameter.Units()` returns the units assigned to the parameter.

You can access the designation status of the parameter using the property `IpfcBaseParameter.IsDesignated`.

The property `IpfcBaseParameter.IsModified` and the method `IpfcBaseParameter.ResetFromBackup()` enable you to identify a modified parameter or dimension, and reset it to the last stored value. A parameter is said to be "modified" when the value has been changed but the parameter's owner has not yet been regenerated.

The property `IpfcParameter.Description` returns the parameter description, or null, if no description is assigned.

The property `IpfcParameter.GetRestriction()` identifies if the parameter's value is restricted to a certain range or enumeration. It returns the `IpfcParameterRestriction` object. Refer to the section [Parameter Restrictions on page 212](#) for more information.

The property `IpfcParameter.GetDriverType()` returns the driver type for a material parameter. The driver types are as follows:

- `EpfcPARAMDRIVER_PARAM`—Specifies that the parameter value is driven by another parameter.
- `EpfcPARAMDRIVER_FUNCTION`—Specifies that the parameter value is driven by a function.
- `EpfcPARAMDRIVER_RELATION`—Specifies that the parameter value is driven by a relation. This is equivalent to the value obtained using `IpfcBaseParameter.IsRelationDriven` for a parameter object type.

The method `IpfcParameter.Reorder()` reorders the given parameter to come immediately after the indicated parameter in the **Parameter** dialog box and information files generated by Creo Parametric.

The method `IpfcParameter.Delete()` permanently removes a specified parameter.

The property `pfcModelItem.NamedModelItem.GetName` accesses the name of the specified parameter.

---

## Parameter Restrictions

Creo Parametric allows users to assign specified limitations to the value allowed for a given parameter (wherever the parameter appears in the model). You can only read the details of the permitted restrictions from the VB API, but not modify the permitted values or range of values. Parameter restrictions in the VB API are represented by the class `IpfcParameterRestriction`.

Method Introduced:

- **`IpfcParameterRestriction.Type`**

The method `IpfcParameterRestriction.Type` returns the `IpfcRestrictionType` object containing the types of parameter restrictions. The parameter restrictions are of the following types:

- `EpfcPARAMSELECT_ENUMERATION`—Specifies that the parameter is restricted to a list of permitted values.
- `EpfcPARAMSELECT_RANGE`—Specifies that the parameter is limited to a specified range of numeric values.

### Enumeration Restriction

The `EpfcPARAMSELECT_ENUMERATION` type of parameter restriction is represented by the interface `IpfcParameterEnumeration`. It is a child of the `IpfcParameterRestriction` interface.

Property Introduced:

- **`ParameterEnumeration.PermittedValues`**

The property `ParameterEnumeration.PermittedValues` returns a list of permitted parameter values allowed by this restriction in the form of a sequence of the `IpfcParamValue` objects.

### Range Restriction

The `EpfcPARAMSELECT_RANGE` type of parameter restriction is represented by the interface `IpfcParameterRange`. It is a child of the `IpfcParameterRestriction` interface.

Properties Introduced:

- **`IpfcParameterRange.Maximum`**
- **`IpfcParameterRange.Minimum`**
- **`IpfcParameterLimit.Type`**
- **`IpfcParameterLimit.Value`**

The property `IpfcParameterRange.Maximum` returns the maximum value limit for the parameter in the form of the `IpfcParameterLimit` object.

---

The property `IpfcParameterRange.Minimum` returns the minimum value limit for the parameter in the form of the `IpfcParameterLimit` object.

The property `IpfcParameterLimit.Type` returns the `IpfcParameterLimitType` containing the types of parameter limits. The parameter limits are of the following types:

- `EpfcPARAMLIMIT_LESS_THAN`—Specifies that the parameter must be less than the indicated value.
- `EpfcPARAMLIMIT_LESS_THAN_OR_EQUAL`—Specifies that the parameter must be less than or equal to the indicated value.
- `EpfcPARAMLIMIT_GREATER_THAN`—Specifies that the parameter must be greater than the indicated value.
- `EpfcPARAMLIMIT_GREATER_THAN_OR_EQUAL`—Specifies that the parameter must be greater than or equal to the indicated value.

The property `IpfcParameterLimit.Value` returns the boundary value of the parameter limit in the form of the `IpfcParamValue` object.

#### Example Code: Updating Model Parameters

The sample code in the file `pfcDimensionAndParameterExamples.vb` located at `<creo_vbapi_loadpoint>/vbapi_examples` contains a method that reads a "properties" file and creates or updates model parameters for each property which exists in the file. Since each property value is returned as a `String`, a utility method parses the `String` into `int`, `double`, or `boolean` values if possible.

## Dimension Objects

Dimension objects include standard Creo Parametric dimensions as well as reference dimensions. Dimension objects enable you to access dimension tolerances and enable you to set the value for the dimension. Reference dimensions allow neither of these actions.

### Getting Dimensions

Dimensions and reference dimensions are Creo Parametric model items. See the section [Getting ModelItem Objects on page 167](#) for methods that can return `IpfcDimension` and `IpfcRefDimension` objects.

### Dimension Information

Methods and Properties Introduced:

- **`IpfcBaseParameter.Value`**
- **`IpfcBaseDimension.DimValue`**

- 
- **IpfcBaseParameter.IsDesignated**
  - **IpfcBaseParameter.IsModified**
  - **IpfcBaseParameter.ResetFromBackup()**
  - **IpfcBaseParameter.IsRelationDriven**
  - **IpfcBaseDimension.DimType**
  - **IpfcBaseDimension.Symbol**
  - **IpfcBaseDimension.Texts**

See the section [Parameter Objects on page 207](#) for brief descriptions.

---

 **Note**

You cannot set the value or designation status of reference dimension objects.

---

The property `IpfcBaseDimension.DimValue` accesses the dimension value as a double. This property provides a shortcut for accessing the dimensions' values without using a `IpfcParamValue` object.

The property `IpfcBaseParameter.IsRelationDriven` identifies whether the part or assembly relations control a dimension.

The property `IpfcBaseDimension.DimType` returns an enumeration object that identifies whether a dimension is linear, radial, angular, or diametrical.

The property `IpfcBaseDimension.Symbol` returns the dimension or reference dimension symbol (that is, “d#” or “rd#”).

The property `IpfcBaseDimension.Texts` allows access to the text strings that precede or follow the dimension value.

## Dimension Tolerances

Methods and Properties Introduced:

- **IpfcDimension.Tolerance**
- **CCpfcDimTolPlusMinus.Create()**
- **CCpfcDimTolSymmetric.Create()**
- **CCpfcDimTolLimits.Create()**
- **CCpfcDimTolSymSuperscript.Create()**
- **CCpfcDimTolISODIN.Create()**

Only true dimension objects can have geometric tolerances.

The property `IpfcDimension.Tolerance` enables you to access the dimension tolerance. The object types for the dimension tolerance are:

- 
- `IpfcDimTolLimits`—Displays dimension tolerances as upper and lower limits.

---

 **Note**

This format is not available when only the tolerance value for a dimension is displayed.

---

- `IpfcDimTolPlusMinus`—Displays dimensions as nominal with plus-minus tolerances. The positive and negative values are independent.
- `IpfcDimTolSymmetric`—Displays dimensions as nominal with a single value for both the positive and the negative tolerance.
- `IpfcDimTolSymSuperscript`—Displays dimensions as nominal with a single value for positive and negative tolerance. The text of the tolerance is displayed in a superscript format with respect to the dimension text.
- `IpfcDimTolISODIN`—Displays the tolerance table type, table column, and table name, if the dimension tolerance is set to a hole or shaft table (DIN/ISO standard).

A null value is similar to the nominal option in Creo Parametric.

You can determine whether a given tolerance is plus/minus, symmetric, limits, or superscript using the following example code. `If TypeOf (tolerance) Is IpfcDimTolLimits.`

### Example Code: Setting Tolerances to a Specified Range

The sample code in the file `pfcdimensionAndParameterExamples.vb` located at `<creo_vbapi_loadpoint>/vbapi_examples` shows a utility method that sets angular tolerances to a specified range.

The example code shows a utility method that sets angular tolerances to a specified range. First, the program determines whether the dimension passed to it is angular. If it is, the method gets the dimension value and adds or subtracts the range to it to get the upper and lower limits.

## Relations

### Relations

This section describes how to access relations on all models and model items in Creo Parametric using the methods provided in the VB API.

---

## Accessing Relations

In the VB API, the set of relations on any model or model item is represented by the `IpfcRelationOwner` interface. Models, features, surfaces, and edges inherit from this interface, because each object can be assigned relations in Creo Parametric.

Methods and Properties Introduced:

- **`IpfcRelationOwner.RegenerateRelations()`**
- **`IpfcRelationOwner.DeleteRelations()`**
- **`IpfcRelationOwner.Relations`**
- **`IpfcRelationOwner.EvaluateExpression()`**

The method `IpfcRelationOwner.RegenerateRelations()` regenerates the relations assigned to the owner item. It also determines whether the specified relation set is valid.

The method `IpfcRelationOwner.DeleteRelations()` deletes all the relations assigned to the owner item.

The property `IpfcRelationOwner.Relations` returns the list of initial relations assigned to the owner item as a sequence of strings.

The method `IpfcRelationOwner.EvaluateExpression()` evaluates the given relations-based expression, and returns the resulting value in the form of the `IpfcParamValue` object. Refer to the section [The ParamValue Object on page 206](#) in the section [Dimensions and Parameters on page 206](#) for more information on this object.

### Example 1: Adding Relations between Parameters in a Solid Model

The sample code in the file `pfcRelationsExamples.vb` located at `<creo_vbapi_loadpoint>/vbapi_examples` demonstrates how to add relations between parameters in a solid model.

## Accessing Post Regeneration Relations

Method and Property Introduced:

- **`IpfcModel.PostRegenerationRelations`**
- **`IpfcModel.RegeneratePostRegenerationRelations()`**
- **`IpfcModel.DeletePostRegenerationRelations()`**

The property `IpfcModel.PostRegenerationRelations` lists the post-regeneration relations assigned to the model. It can be `NULL`, if not set.



---

 **Note**

To work with post-regeneration relations, use the post-regeneration relations attribute in the methods

`IpfcRelationOwner.RegenerateRelations()` and  
`IpfcRelationOwner.DeleteRelations()`.

---

You can regenerate the relation sets post-regeneration in a model using the method `IpfcModel.RegeneratePostRegenerationRelations()`.

To delete all the post-regeneration relations in the specified model, call the method `IpfcModel.DeletePostRegenerationRelations()`.

## Adding a Customized Function to the Relations Dialog Box in Creo Parametric

Methods Introduced:

- **`IpfcBaseSession.RegisterRelationFunction`**

The method `IpfcBaseSession.RegisterRelationFunction` registers a custom function that is included in the function list of the **Relations** dialog box in Creo Parametric. You can add the custom function to relations that are added to models, features, or other relation owners. The registration method takes the following input arguments:

- *Name*—The name of the custom function.
- `IpfcRelationFunctionOptions`—This object contains the options that determine the behavior of the custom relation function. Refer to the section [Relation Function Options on page 218](#) for more information.
- `IpfcRelationFunctionListener`—This object contains the action listener methods for the implementation of the custom function. Refer to the section [Relation Function Listeners on page 219](#) for more information.

---

 **Note**

The VB API relation functions are valid only when the custom function has been registered by the application. If the application is not running or not present, models that contain user-defined relations cannot evaluate these relations. In this situation, the relations are marked as errors. However, these errors can be commented until needed at a later time when the relations functions are reactivated in a Creo Parametric session.

---

---

## Relation Function Options

Methods and Properties Introduced:

- **CCpfcRelationFunctionOptions.Create()**
- **IpfcRelationFunctionOptions.ArgumentTypes**
- **CCpfcRelationFunctionArgument.Create()**
- **IpfcRelationFunctionArgument.Type**
- **.IpfcRelationFunctionArgument.IsOptional**
- **IpfcRelationFunctionOptions.EnableTypeChecking**
- **IpfcRelationFunctionOptions.EnableArgumentCheckMethod**
- **IpfcRelationFunctionOptions.EnableExpressionEvaluationMethod**
- **IpfcRelationFunctionOptions.EnableValueAssignmentMethod**

Use the method `CCpfcRelationFunctionOptions.Create()` to create the `IpfcRelationFunctionOptions` object containing the options to enable or disable various relation function related features. Use the methods listed above to access and modify the options. These options are as follows:

- *ArgumentTypes*—The types of arguments in the form of the `IpfcRelationFunctionArgument` object. By default, this parameter is null, indicating that no arguments are permitted.

Use the method `CCpfcRelationFunctionArgument.Create()` to create the `IpfcRelationFunctionArgument` object containing the attributes of the arguments passed to the custom relation function.

These attributes are as follows:

- *Type*—The type of argument value such as double, integer, and so on in the form of the `IpfcParamValueType` object.
- *IsOptional*—This boolean attribute specifies whether the argument is optional, indicating that it can be skipped when a call to the custom relation function is made. The optional arguments must fall at the end of the argument list. By default, this attribute is false.
- *EnableTypeChecking*—This boolean attribute determines whether or not to check the argument types internally. By default, it is false. If this attribute is set to false, Creo Parametric does not need to know the contents of the arguments array. The custom function must handle all user errors in such a situation.
- *EnableArgumentCheckMethod*—This boolean attribute determines whether or not to enable the arguments check listener function. By default, it is false.

- 
- *EnableExpressionEvaluationMethod*—This boolean attribute determines whether or not to enable the evaluate listener function. By default, it is true.
  - *EnableValueAssignmentMethod*—This boolean attribute determines whether or not to enable the value assignment listener function. By default, it is false.

## Relation Function Listeners

The interface `IpfcRelationFunctionListener` provides the method signatures to implement a custom relation function.

Methods Introduced:

- **`IpfcRelationFunctionListener.CheckArguments`**
- **`IpfcRelationFunctionListener.AssignValue`**
- **`IpfcRelationFunctionListener.EvaluateFunction`**

The method `IpfcRelationFunctionListener.CheckArguments` checks the validity of the arguments passed to the custom function. This listener method takes the following input arguments:

- The owner of the relation being evaluated
- The custom function name
- A sequence of arguments passed to the custom function

If the implementation of this method determines that the arguments are not valid for the custom function, then the listener method returns false. Otherwise, it returns true.

The method `IpfcRelationFunctionListener.EvaluateFunction` evaluates a custom relation function invoked on the right hand side of a relation. This listener method takes the following input arguments:

- The owner of the relation being evaluated
- The custom function name
- A sequence of arguments passed to the custom function

You must return the computed result of the custom relation function.

The method `IpfcRelationFunctionListener.AssignValue` evaluates a custom relation function invoked on the left hand side of a relation. It allows you to initialize properties to be stored and used by your application. This listener method takes the following input arguments:

- The owner of the relation being evaluated
- The custom function name
- A sequence of arguments passed to the custom function

- 
- The value obtained by Creo Parametric from evaluating the right hand side of the relation

## Example 2: Adding and Implementing a New Custom Relation Function

The sample code in the file `pfcRelationsExamples.vb` located at `<creo_vbapi_loadpoint>/vbapi_examples` has the function `addRelation` function, which defines the options for a new custom relation function and registers it in the current session. The `RelationListener` class contains the `CheckArguments`, `AssignValue` and `EvaluateFunction` listener methods that are called when the custom relation function is used.

# Assemblies and Components

## Assemblies and Components

This section describes the VB API functions that access the functions of a Creo Parametric assembly. You must be familiar with the following before you read this section:

- The Selection Object
- Coordinate Systems
- The Geometry section

## Structure of Assemblies and Assembly Objects

The object `IpfcAssembly` is an instance of `IpfcSolid`. The `IpfcAssembly` object can therefore be used as input to any of the `IpfcSolid` and `IpfcModel` methods applicable to assemblies. However assemblies do not contain solid geometry items. The only geometry in the assembly is datums (points, planes, axes, coordinate systems, curves, and surfaces). Therefore solid assembly features such as holes and slots will not contain active surfaces or edges in the assembly model.

The solid geometry of an assembly is contained in its components. A component is a feature of type `IpfcComponentFeat`, which is a reference to a part or another assembly, and a set of parametric constraints for determining its geometrical location within the parent assembly.

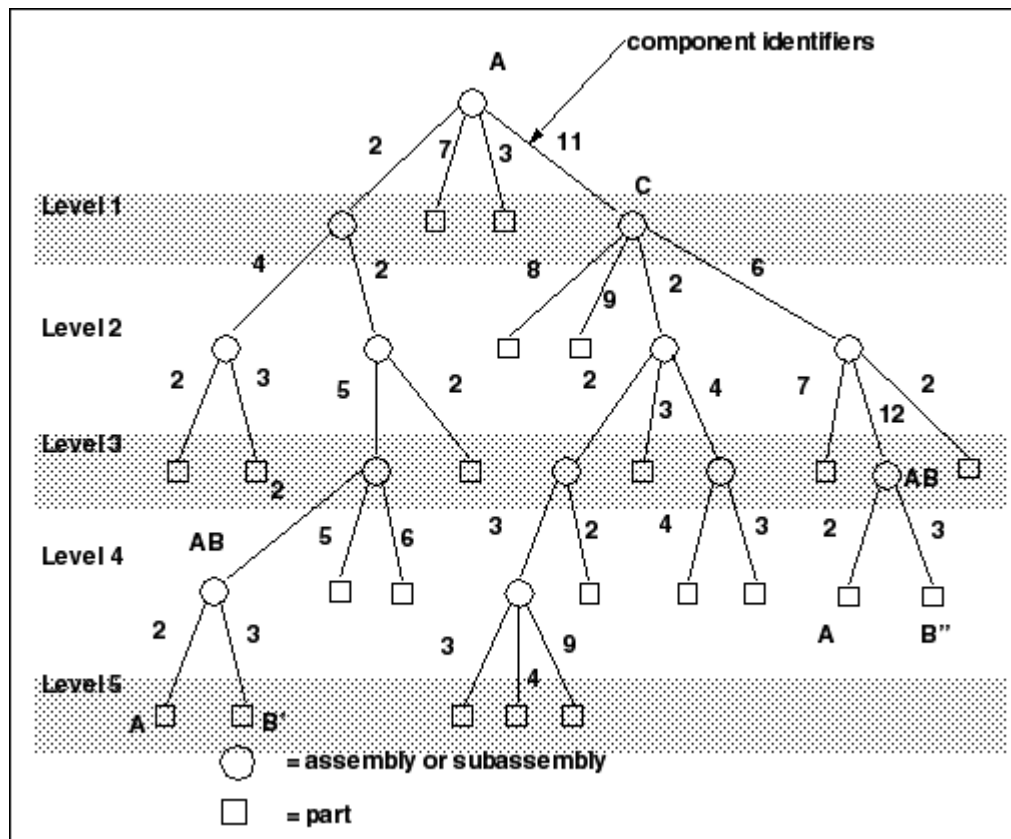
Assembly features that are solid, such as holes and slots, and therefore affect the solid geometry of parts in the assembly hierarchy, do not themselves contain the geometry items that describe those modifications. These items are always contained in the parts whose geometry is modified, within local features created for that purpose.

The important functions for assemblies are those that operate on the components of an assembly. The object `IpfcComponentFeat`, which is an instance of `IpfcFeature` is defined for that purpose. Each assembly component is treated as a variety of feature, and the integer identifier of the component is also the feature identifier.

An assembly can contain a hierarchy of assemblies and parts at many levels, in which some assemblies and parts may appear more than once. To identify the role of any database item in the context of the root assembly, it is not sufficient to have the integer identifier of the item and the handle to its owning part or assembly, as would be provided by its `IpfcFeature` description.

It is also necessary to give the full path of the assembly-component references down from the root assembly to the part or assembly that owns the database item. This is the purpose of the object `IpfcComponentPath`, which is used as the input to the VB API assembly functions.

The following figure shows an assembly hierarchy with two examples of the contents of a `IpfcComponentPath` object.



In the assembly shown in the figure, subassembly C is component identifier 11 within assembly A, Part B is component identifier 3 within assembly AB, and so on. The subassembly AB occurs twice. To refer to the two occurrences of part B, use the following:

---

| (?)Component B'          | Component B''             |
|--------------------------|---------------------------|
| ComponentIds.Item(0) = 2 | ComponentIds.Item(1) = 11 |
| ComponentIds.Item(1) = 2 | ComponentIds.Item(2) = 6  |
| ComponentIds.Item(2) = 5 | ComponentIds.Item(3) = 12 |
| ComponentIds.Item(3) = 2 | ComponentIds.Item(4) = 3  |
| ComponentIds.Item(4) = 3 |                           |

The object `IpfcComponentPath` is one of the main portions of the `IpfcSelection` object.

## Assembly Components

Methods and Properties Introduced:

- **`IpfcComponentFeat.IsBulkitem`**
- **`IpfcComponentFeat.GetIsSubstitute`**
- **`IpfcComponentFeat.CompType`**
- **`IpfcComponentFeat.ModelDescr`**
- **`IpfcComponentFeat.IsPlaced`**
- **`IpfcComponentFeat.IsPackaged`**
- **`IpfcComponentFeat.IsUnderconstrained`**
- **`IpfcComponentFeat.IsFrozen`**
- **`IpfcComponentFeat.Position`**
- **`IpfcComponentFeat.CopyTemplateContents()`**
- **`IpfcComponentFeat.CreateReplaceOp()`**

The property `IpfcComponentFeat.IsBulkitem` identifies whether an assembly component is a bulk item. A bulk item is a non-geometric assembly feature that should appear in an assembly bill of materials.

The property `IpfcComponentFeat.GetIsSubstitute` returns a true value if the component is substituted, else it returns a false. When you substitute a component in a simplified representation, you temporarily exclude the substituted component and superimpose the substituting component in its place.

The property `IpfcComponentFeat.CompType` returns the type of the assembly component. The component type identifies the purpose of the component in a manufacturing assembly.

The property `IpfcComponentFeat.ModelDescr` returns the model descriptor of the component part or subassembly.

---

### **Note**

From Pro/ENGINEER Wildfire 4.0 onwards, the property `IpfcComponentFeat.ModelDescr` throws an exception `IpfcXtoolkitCantOpen` if called on an assembly component whose immediate generic is not in session. Handle this exception and typecast the assembly component as `IpfcSolid`, which in turn can be typecast as `IpfcFamilyMember`, and use the method `IpfcFamilyMember.GetImmediateGenericInfo()` to get the model descriptor of the immediate generic model. If you wish to switch off this behavior and continue to run legacy applications in the pre-Wildfire 4.0 mode, set the configuration option `retrieve_instance_dependencies` to `instance_and_generic_deps`.

---

The property `IpfcComponentFeat.IsPlaced` determines whether the component is placed.

---

### **Note**

Once a component is constrained or packaged, it cannot be made unplaced again.

---

A component of an assembly that is either partially constrained or unconstrained is known as a packaged component. Use the property `IpfcComponentFeat.IsPackaged` to determine if the specified component is packaged.

The property `IpfcComponentFeat.IsUnderconstrained` determines if the specified component is underconstrained, that is, it possesses some constraints but is not fully constrained.

The property `IpfcComponentFeat.IsFrozen` determines if the specified component is frozen. The frozen component behaves similar to the packaged component and does not follow the constraints that you specify.

The property `IpfcComponentFeat.Position` retrieves the component's initial position before constraints and movements have been applied. If the component is packaged this position is the same as the constraint's actual position. This property modifies the assembly component data but does not regenerate the assembly component. To regenerate the component, use the method `IpfcComponentFeat.Regenerate`.

The method `IpfcComponentFeat.CopyTemplateContents()` copies the template model into the model of the specified component.

---

The method `IpfcComponentFeat.CreateReplaceOp()` creates a replacement operation used to swap a component automatically with a related component. The replacement operation can be used as an argument to `IpfcSolid.ExecuteFeatureOps`.

### Example Code: Replacing Instances

The sample code in the file `pfcAssembliesExamples.vb` located at `<creo_vbapi_loadpoint>/vbapi_examples` contains a single static utility method. This method takes an assembly for an argument. It searches through the assembly for all components that are instances of the model "bolt". It then replaces all such occurrences with a different instance of bolt.

### Regenerating an Assembly Component

Method Introduced:

- **`IpfcComponentFeat.Regenerate()`**

The method `IpfcComponentFeat.Regenerate()` regenerates an assembly component. The method regenerates the assembly component just as in an interactive Creo Parametric session.

### Creating a Component Path

Methods Introduced

- **`CMpfcAssembly.CreateComponentPath()`**

The method `CMpfcAssembly.CreateComponentPath()` returns a component path object, given the Assembly model and the integer id path to the desired component.

### Component Path Information

Methods and Properties Introduced:

- **`IpfcComponentPath.Root`**
- **`IpfcComponentPath.ComponentIds`**
- **`IpfcComponentPath.Leaf`**
- **`IpfcComponentPath.Transform()`**
- **`IpfcComponentPath.SetTransform()`**
- **`IpfcComponentPath.IsVisible()`**

The property `IpfcComponentPath.Root` returns the assembly at the head of the component path object.

The property `IpfcComponentPath.ComponentIds` returns the sequence of ids which is the path to the particular component.



---

The property `IpfcComponentPath.Leaf` returns the solid model at the end of the component path.

The method `IpfcComponentPath.Transform()` returns the coordinate system transformation between the assembly and the particular component. It has an option to provide the transformation from bottom to top, or from top to bottom. This method describes the current position and the orientation of the assembly component in the root assembly.

The method `IpfcComponentPath.SetTransform()` applies a temporary transformation to the assembly component, similar to the transformation that takes place in an exploded state. The transformation will only be applied if the assembly is using `DynamicPositioning`.

The method `IpfcComponentPath.IsVisible()` identifies if a particular component is visible in any simplified representation.

## Assembling Components

Methods Introduced:

- **`IpfcAssembly.AssembleComponent()`**
- **`IpfcAssembly.AssembleByCopy()`**
- **`IpfcComponentFeat.GetConstraints()`**
- **`IpfcComponentFeat.SetConstraints()`**
- **`IpfcComponentFeat.GetConstraintsWithCompPath()`**

The method `IpfcAssembly.AssembleComponent()` adds a specified component model to the assembly at the specified initial position. The position is specified in the format defined by the `IpfcTransform3D`. Specify the orientation of the three axes and the position of the origin of the component coordinate system, with respect to the target assembly coordinate system.



### Note

If the transform matrix passed as the initial position of the component is incorrect and non-orthonormal, the method `IpfcAssembly.AssembleComponent()` returns the error `pfcXToolkitBadInputs`. In such scenario, you can use the method `CMpfcBase.MakeMatrixOrthonormal()` to convert this non-orthonormal matrix to an orthonormal matrix.

---

The method `IpfcAssembly.AssembleByCopy()` creates a new component in the specified assembly by copying from the specified component. If no model is specified, then the new component is created empty. The input parameters for this method are:

- *LeaveUnplaced*—If true the component is unplaced. If false the component is placed at a default location in the assembly. Unplaced components belong to an assembly without being assembled or packaged. These components appear in the model tree, but not in the graphic window. Unplaced components can be constrained or packaged by selecting them from the model tree for redefinition. When its parent assembly is retrieved into memory, an unplaced component is also retrieved.
- *ModelToCopy*—Specify the model to be copied into the assembly
- *NewModelName*—Specify a name for the copied model

The method `IpfcComponentFeat.GetConstraints()` retrieves the constraints for a given assembly component.

The method `IpfcComponentFeat.SetConstraints()` allows you to set the constraints for a specified assembly component. The input parameters for this method are:

- *Constraints*—Constraints for the assembly component. These constraints are explained in detail in the later sections.
- *ReferenceAssembly*—The path to the owner assembly, if the constraints have external references to other members of the top level assembly. If the constraints are applied only to the assembly component then the value of this parameter should be null.

This method modifies the component feature data and regenerates the assembly component.

The method `IpfcComponentFeat.GetConstraintsWithCompPath()` retrieves the constraints for a given assembly component using the input argument *CompPath* which is the path to the owner assembly. Pass this input argument *CompPath*, if the constraints have references to other members of the top level assembly. Pass it as `Null`, if the constraints have references only to the owner assembly.

## Constraint Attributes

Methods and Properties Introduced:

- **CCpfcConstraintAttributes.Create()**
- **IpfcConstraintAttributes.Force**
- **IpfcConstraintAttributes.Ignore**

The method `CCpfcConstraintAttributes.Create()` returns the constraint attributes object based on the values of the following input parameters:

- *Ignore*—Constraint is ignored during regeneration. Use this capability to store extra constraints on the component, which allows you to quickly toggle between different constraints.

- *Force*—Constraint has to be forced for line and point alignment.
- *None*—No constraint attributes. This is the default value.

### Assembling a Component Parametrically

You can position a component relative to its neighbors (components or assembly features) so that its position is updated as its neighbors move or change. This is called parametric assembly. Creo Parametric allows you to specify constraints to determine how and where the component relates to the assembly. You can add as many constraints as you need to make sure that the assembly meets the design intent.

Methods and Properties Introduced:

- **CCpfcComponentConstraint.Create()**
- **IpfcComponentConstraint.Type**
- **IpfcComponentConstraint.AssemblyReference**
- **IpfcComponentConstraint.AssemblyDatumSide**
- **IpfcComponentConstraint.ComponentReference**
- **IpfcComponentConstraint.ComponentDatumSide**
- **IpfcComponentConstraint.Offset**
- **IpfcComponentConstraint.Attributes**
- **IpfcComponentConstraint.UserDefinedData**

The method `CCpfcComponentConstraint.Create()` returns the component constraint object having the following parameters:

- *EpfcComponentConstraintType*—Using the TYPE options, you can specify the placement constraint types. They are as follows:
  - `EpfcASM_CONSTRAINT_MATE`—Use this option to make two surfaces touch one another, that is coincident and facing each other.
  - `EpfcASM_CONSTRAINT_MATE_OFF`—Use this option to make two planar surfaces parallel and facing each other.
  - `EpfcASM_CONSTRAINT_ALIGN`—Use this option to make two planes coplanar, two axes coaxial and two points coincident. You can also align revolved surfaces or edges.
  - `EpfcASM_CONSTRAINT_ALIGN_OFF`—Use this option to align two planar surfaces at an offset.
  - `EpfcASM_CONSTRAINT_INSERT`—Use this option to insert a "male" revolved surface into a "female" revolved surface, making their respective axes coaxial.

- 
- `EpfcASM_CONSTRAINT_ORIENT`—Use this option to make two planar surfaces to be parallel in the same direction.
  - `EpfcASM_CONSTRAINT_CSYS`—Use this option to place a component in an assembly by aligning the coordinate system of the component with the coordinate system of the assembly.
  - `EpfcASM_CONSTRAINT_TANGENT`—Use this option to control the contact of two surfaces at their tangents.
  - `EpfcASM_CONSTRAINT_PNT_ON_SRF`—Use this option to control the contact of a surface with a point.
  - `EpfcASM_CONSTRAINT_EDGE_ON_SRF`—Use this option to control the contact of a surface with a straight edge.
  - `EpfcASM_CONSTRAINT_DEF_PLACEMENT`—Use this option to align the default coordinate system of the component to the default coordinate system of the assembly.
  - `EpfcASM_CONSTRAINT_SUBSTITUTE`—Use this option in simplified representations when a component has been substituted with some other model
  - `EpfcASM_CONSTRAINT_PNT_ON_LINE`—Use this option to control the contact of a line with a point.
  - `EpfcASM_CONSTRAINT_FIX`—Use this option to force the component to remain in its current packaged position.
  - `EpfcASM_CONSTRAINT_AUTO`—Use this option in the user interface to allow an automatic choice of constraint type based upon the references.
  - `EpfcASM_CONSTRAINT_EXPLICIT`
  - *AssemblyReference*—A reference in the assembly.
  - *AssemblyDatumSide*—Orientation of the assembly. This can have the following values:
    - `Yellow`—The primary side of the datum plane which is the default direction of the arrow.
    - `Red`—The secondary side of the datum plane which is the direction opposite to that of the arrow.
  - *ComponentReference*—A reference on the placed component.
  - *ComponentDatumSide*—Orientation of the assembly component. This can have the following values:
    - `Yellow`—The primary side of the datum plane which is the default direction of the arrow.

- *Red*—The secondary side of the datum plane which is the direction opposite to that of the arrow.
- *Offset*—The mate or align offset value from the reference.
- *Attributes*—Constraint attributes for a given constraint
- *UserDefinedData*—A string that specifies user data for the given constraint.

Use the properties listed above to access the parameters of the component constraint object.

## Redefining and Rerouting Assembly Components

These functions enable you to reroute previously assembled components, just as in an interactive Creo Parametric session.

Methods Introduced:

- **IpfcComponentFeat.RedefineThroughUI()**
- **IpfcComponentFeat.MoveThroughUI()**

The method `IpfcComponentFeat.RedefineThroughUI()` must be used in interactive VB API applications. This method displays the Creo Parametric **Constraint** dialog box. This enables the end user to redefine the constraints interactively. The control returns to the VB API application when the user selects **OK** or **Cancel** and the dialog box is closed.

The method `IpfcComponentFeat.MoveThroughUI()` invokes a dialog box that prompts the user to interactively reposition the components. This interface enables the user to specify the translation and rotation values. The control returns to the VB API application when the user selects **OK** or **Cancel** and the dialog box is closed.

### Example: Component Constraints

The sample code in the file `pfcAssembliesExamples.vb` located at `<creo_vbapi_loadpoint>/vbapi_examples` displays each constraint of the component visually on the screen, and includes a text explanation for each constraint.

### Example: Assembling Components

The sample code in the file `pfcAssembliesExamples.vb` located at `<creo_vbapi_loadpoint>/vbapi_examples` demonstrates how to assemble a component into an assembly, and how to constrain the component by aligning datum planes. If the complete set of datum planes is not found, the function will show the component constraint dialog to the user to allow them to adjust the placement.

---

## Exploded Assemblies

These methods enable you to determine and change the explode status of the assembly object.

Methods and Properties Introduced:

- **`IpfcAssembly.IsExploded`**
- **`IpfcAssembly.Explode()`**
- **`IpfcAssembly.UnExplode()`**
- **`IpfcAssembly.GetActiveExplodedState()`**
- **`IpfcAssembly.GetDefaultExplodedState()`**
- **`IpfcExplodedState.Activate()`**

The methods `IpfcAssembly.Explode()` and `IpfcAssembly.UnExplode()` enable you to determine and change the explode status of the assembly object.

The method `IpfcAssembly.GetActiveExplodedState()` reports whether the specified assembly is currently exploded. Use this method in the assembly mode only. The exploded status of an assembly depends on the mode. If an assembly is opened in the drawing mode, the state of the assembly in the drawing view is displayed. The drawing view does not represent the actual exploded state of the assembly.

The method `IpfcAssembly.GetActiveExplodedState()` returns the current active explode state.

The method `IpfcAssembly.GetDefaultExplodedState()` returns the default explode state.

The method `IpfcExplodedState.Activate()` activates the specified explode state representation.

## Skeleton Models

Skeleton models are a 3-dimensional layout of the assembly. These models are holders or distributors of critical design information, and can represent space requirements, important mounting locations, and motion.

Methods and Properties Introduced:

- **`IpfcAssembly.AssembleSkeleton()`**
- **`IpfcAssembly.AssembleSkeletonByCopy()`**
- **`IpfcAssembly.GetSkeleton()`**
- **`IpfcAssembly.DeleteSkeleton()`**
- **`IpfcSolid.IsSkeleton`**

---

The method `IpfcAssembly.AssembleSkeleton()` adds an existing skeleton model to the specified assembly.

The method `IpfcAssembly.GetSkeleton()` returns the skeleton model of the specified assembly.

The method `IpfcAssembly.DeleteSkeleton()` deletes a skeleton model component from the specified assembly.

The method `IpfcAssembly.AssembleSkeletonByCopy()` adds a specified skeleton model to the assembly. The input parameters for this method are:

- *SkeletonToCopy*—Specify the skeleton model to be copied into the assembly
- *NewSkeletonName*—Specify a name for the copied skeleton model

The property `IpfcSolid.IsSkeleton` determines if the specified part model is a skeleton model or a concept model. It returns a true if the model is a skeleton else it returns a false.

## Family Tables

### Family Tables

This section describes how to use the VB API classes and methods to access and manipulate family table information.

### Working with Family Tables

The VB API provides several methods for accessing family table information. Because every model inherits from the interface `IpfcFamilyMember`, every model can have a family table associated with it.

#### Accessing Instances

Methods and Properties Introduced:

- **`IpfcFamilyMember.Parent`**
- **`IpfcFamilyMember.GetImmediateGenericInfo()`**
- **`IpfcFamilyMember.GetTopGenericInfo()`**
- **`IpfcFamilyTableRow.CreateInstance()`**
- **`IpfcFamilyMember.ListRows()`**
- **`IpfcFamilyMember.GetRow()`**
- **`IpfcFamilyMember.RemoveRow()`**
- **`IpfcFamilyTableRow.InstanceName`**

---

- **IpfcFamilyTableRow.IsLocked**

To get the generic model for an instance, call the property `IpfcFamilyMember.Parent`.

From Pro/ENGINEER Wildfire 4.0 onwards, the behavior of the property `IpfcFamilyMember.Parent` has changed as a result of performance improvement in family table retrieval mechanism. When you now call the property `IpfcFamilyMember.Parent`, it throws an exception `IpfcXToolkitCantOpen`, if the immediate generic of a model instance in a nested family table is currently not in session. Handle this exception and use the method `IpfcFamilyMember.GetImmediateGenericInfo()` to get the model descriptor of the immediate generic model. This information can be used to retrieve the immediate generic model.

If you wish to switch off the above behavior and continue to run legacy applications in the pre-Wildfire 4.0 mode, set the configuration option `retrieve_instance_dependencies` to `instance_and_generic_deps`.

To get the model descriptor of the top generic model, call the method `IpfcFamilyMember.GetTopGenericInfo()`.

Similarly, the method `IpfcFamilyTableRow.CreateInstance()` returns an instance model created from the information stored in the `IpfcFamilyTableRow` object.

The method `IpfcFamilyMember.ListRows()` returns a sequence of all rows in the family table, whereas `IpfcFamilyMember.GetRow()` gets the row object with the name you specify.

Use the method `IpfcFamilyMember.RemoveRow()` to permanently delete the row from the family table.

The property `IpfcFamilyTableRow.InstanceName` returns the name that corresponds to the invoking row object.

To control whether the instance can be changed or removed, call the property `IpfcFamilyTableRow.IsLocked`.

## Accessing Columns

Methods and Properties Introduced:

- **IpfcFamilyMember.ListColumns()**
- **IpfcFamilyMember.GetColumn()**
- **IpfcFamilyMember.RemoveColumn**
- **IpfcFamilyTableColumn.Symbol**
- **IpfcFamilyTableColumn.Type**
- **IpfcFamColModelItem.ReflItem**



---

- **IpfcFamColParam.RefParam**

The method `IpfcFamilyMember.ListColumns()` returns a sequence of all columns in the family table.

The method `IpfcFamilyMember.GetColumn()` returns a family table column, given its symbolic name.

To permanently delete the column from the family table and all changed values in all instances, call the method `IpfcFamilyMember.RemoveColumn`.

The property `IpfcFamilyTableColumn.Symbol` returns the string symbol at the top of the column, such as D4 or F5.

The property `IpfcFamilyTableColumn.Type` returns an enumerated value indicating the type of parameter governed by the column in the family table.

The property `IpfcFamColModelItem.RefItem` returns the `IpfcModelItem` (Feature or Dimension) controlled by the column, whereas `IpfcFamColParam.RefParam` returns the `Parameter` controlled by the column.

## Accessing Cell Information

Methods and Properties Introduced:

- **IpfcFamilyMember.GetCell()**
- **IpfcFamilyMember.GetCellIsDefault()**
- **IpfcFamilyMember.SetCell()**
- **IpfcParamValue.StringValue**
- **IpfcParamValue.IntValue**
- **IpfcParamValue.DoubleValue**
- **IpfcParamValue.BoolValue**

The method `IpfcFamilyMember.GetCell()` returns a string `IpfcParamValue` that corresponds to the cell at the intersection of the row and column arguments. Use the method `IpfcFamilyMember.GetCellIsDefault()` to check if the value of the specified cell is the default value, which is the value of the specified cell in the generic model.

The method `IpfcFamilyMember.SetCell()` assigns a value to a column in a particular family table instance.

The `IpfcParamValue.StringValue`, `IpfcParamValue.IntValue`, `IpfcParamValue.DoubleValue`, and `IpfcParamValue.BoolValue` properties are used to get the different types of parameter values.

---

## Creating Family Table Instances

Methods Introduced:

- **`IpfcFamilyMember.AddRow()`**
- **`CMpfcModelItem.CreateStringParamValue()`**
- **`CMpfcModelItem.CreateIntParamValue()`**
- **`CMpfcModelItem.CreateDoubleParamValue()`**
- **`CMpfcModelItem.CreateBoolParamValue()`**

Use the method `IpfcFamilyMember.AddRow()` to create a new instance with the specified name, and, optionally, the specified values for each column. If you do not pass in a set of values, the value `*` will be assigned to each column. This value indicates that the instance uses the generic value.

## Creating Family Table Columns

Methods Introduced:

- **`IpfcFamilyMember.CreateDimensionColumn()`**
- **`IpfcFamilyMember.CreateParamColumn()`**
- **`IpfcFamilyMember.CreateFeatureColumn()`**
- **`IpfcFamilyMember.CreateComponentColumn()`**
- **`IpfcFamilyMember.CreateCompModelColumn()`**
- **`IpfcFamilyMember.CreateGroupColumn()`**
- **`IpfcFamilyMember.CreateMergePartColumn()`**
- **`IpfcFamilyMember.CreateColumn()`**
- **`IpfcFamilyMember.AddColumn()`**
- **`IpfcpfcModelItem.CreateStringParamValue()`**

The above methods initialize a column based on the input argument. These methods assign the proper symbol to the column header.

The method `IpfcFamilyMember.CreateColumn()` creates a new column given a properly defined symbol and column type. The results of this call should be passed to the method `IpfcFamilyMember.AddColumn()` to add the column to the model's family table.

The method `IpfcFamilyMember.AddColumn()` adds the column to the family table. You can specify the values; if you pass nothing for the values, the method assigns the value `*` to each instance to accept the column's default value.

---

## Example Code: Adding Dimensions to a Family Table

The sample code in the file `pfcFamilyTablesExamples.vb` located at `<creo_vbapi_loadpoint>/vbapi_examples` adds all the dimensions to a family table. The program lists the dependencies of the assembly and loops through each dependency, assigning the model to a new `FamColDimension` column object. All the dimensions, parameters, features, and components could be added to the family table using a similar method.

# Action Listeners

## Action Listeners

This section describes the VB API methods that enable you to use action listeners.

### The VB API Action Listeners

An `ActionListener` is a class that is assigned to respond to certain events. In the VB API, you can assign action listeners to respond to events involving the following tasks:

- Changing windows
- Changing working directories
- Model operations
- Regenerating
- Creating, deleting, and redefining features
- Checking for regeneration failures

All action listeners in the VB API are defined by these classes:

- Interface—Named `<Object>ActionListener`. This interface defines the methods that can respond to various events.
- Default class—Named `Default<Object>ActionListener`. This class has every available method overridden by an empty implementation. You create your own action listeners by extending the default class and overriding the methods for events that interest you.

---

## Note

When notifications are set in VB API applications, every time an event is triggered, notification messages are added to the trail files. From Creo Parametric 2.0 M210 onward, a new environment variable `PROTK_LOG_DISABLE` enables you to disable this behavior. When set to `true`, the notifications messages are not added to the trail files.

---

## Action Sources

Methods introduced:

- **`IpfcActionSource.AddActionListener`**
- **`IpfcActionSource.AddActionListenerWithType`**
- **`IpfcActionSource.RemoveActionListener`**

Many VB API classes inherit the `IpfcActionSource` interface, but only the following classes currently make calls to the methods of registered `IpfcActionListeners`:

- `IpfcSession`
  - Session Action Listener
  - Model Action Listener
  - Solid Action Listener
  - Model Event Action Listener
  - Feature Action Listener
- `IpfcUICommand`
  - UI Action Listener
- `IpfcModel` (and its subclasses)
  - Model Action Listener
  - Parameter Action Listener
- `IpfcSolid` (and its subclasses)
  - Solid Action Listener
  - Feature Action Listener
- `IpfcFeature` (and its subclasses)
  - Feature Action Listener

---

### Note

- Assigning an action listener to a source not related to it will not cause an error but the listener method will never be called.
  - The method `IpfcActionSource.AddActionListener` adds action listeners to notify you of all the events.
  - The method `IpfcActionSource.AddActionListenerWithType` adds action listeners to notify you about events that are specified in the input argument *ATypes*.
- 

## Types of Action Listeners

The following sections describe the different kinds of action listeners: session, UI command, solid, and feature.

### Session Level Action Listeners

Methods introduced:

- **`IpfcSessionActionListener.OnAfterDirectoryChange()`**
- **`IpfcSessionActionListener.OnAfterWindowChange()`**
- **`IpfcSessionActionListener.OnAfterModelDisplay()`**
- **`IpfcSessionActionListener.OnBeforeModelErase()`**
- **`IpfcSessionActionListener.OnBeforeModelDelete()`**
- **`IpfcSessionActionListener.OnBeforeModelRename()`**
- **`IpfcSessionActionListener.OnBeforeModelSave()`**
- **`IpfcSessionActionListener.OnBeforeModelPurge()`**
- **`IpfcSessionActionListener.OnBeforeModelCopy()`**
- **`IpfcSessionActionListener.OnAfterModelPurge()`**

The `IpfcSessionActionListener.OnAfterDirectoryChange()` method activates after the user changes the working directory. This method takes the new directory path as an argument.

The `IpfcSessionActionListener.OnAfterWindowChange()` method activates when the user activates a window other than the current one. Pass the new window to the method as an argument.

The `IpfcSessionActionListener.OnAfterModelDisplay()` method activates every time a model is displayed in a window.



## Note

Model display events happen when windows are moved, opened and closed, repainted, or the model is regenerated. The event can occur more than once in succession.

---

### The methods

`IpfcSessionActionListener.OnBeforeModelErase()`,  
`IpfcSessionActionListener.OnBeforeModelRename`,  
`IpfcSessionActionListener.OnBeforeModelSave`, and  
`IpfcSessionActionListener.OnBeforeModelCopy` take special arguments. They are designed to allow you to fill in the arguments and pass this data back to Creo Parametric. The model names placed in the descriptors will be used by Creo Parametric as the default names in the user interface.

### UI Command Action Listeners

Methods introduced:

- **`IpfcSession.UICreateCommand()`**
- **`IpfcUICommandActionListener.OnCommand()`**

The `IpfcSession.UICreateCommand()` method takes a `IpfcUICommandActionListener` argument and returns a `IpfcUICommand` action source with that action listener already registered. This `IpfcUICommand` object is subsequently passed as an argument to the `IpfcSession.UIAddButton()` method that adds a command button to a Creo Parametric menu. The `IpfcUICommandActionListener.OnCommand` method of the registered `IpfcUICommandActionListener` is called whenever the command button is clicked.

### Model Level Action listeners

Methods introduced:

- **`IpfcModelActionListener.OnAfterModelSave()`**
- **`IpfcModelEventActionListener.OnAfterModelCopy()`**
- **`IpfcModelEventActionListener.OnAfterModelRename()`**
- **`IpfcModelEventActionListener.OnAfterModelErase()`**
- **`IpfcModelEventActionListener.OnAfterModelDelete()`**
- **`IpfcModelActionListener.OnAfterModelRetrieve()`**
- **`IpfcModelActionListener.OnBeforeModelDisplay()`**

- 
- **`IpfcModelActionListener.OnAfterModelCreate()`**
  - **`IpfcModelActionListener.OnAfterModelSaveAll()`**
  - **`IpfcModelEventActionListener.OnAfterModelCopyAll()`**
  - **`IpfcModelActionListener.OnAfterModelEraseAll()`**
  - **`IpfcModelActionListener.OnAfterModelDeleteAll()`**
  - **`IpfcModelActionListener.OnAfterModelRetrieveAll()`**

Methods ending in `All` are called after any event of the specified type. The call is made even if the user did not explicitly request that the action take place. Methods that do not end in `All` are only called when the user specifically requests that the event occurs.

The method `IpfcModelActionListener.OnAfterModelSave()` is called after successfully saving a model.

The method `IpfcModelEventActionListener.OnAfterModelCopy()` is called after successfully copying a model.

The method `IpfcModelEventActionListener.OnAfterModelRename()` is called after successfully renaming a model.

The method `IpfcModelEventActionListener.OnAfterModelErase()` is called after successfully erasing a model.

The method `IpfcModelEventActionListener.OnAfterModelDelete()` is called after successfully deleting a model.

The method `IpfcModelActionListener.OnAfterModelRetrieve()` is called after successfully retrieving a model.

The method `IpfcModelActionListener.OnBeforeModelDisplay()` is called before displaying a model.

---

 **Note**

The method `IpfcModelActionListener.OnBeforeModelDisplay()` is not supported in asynchronous mode.

---

The method `IpfcModelActionListener.OnAfterModelCreate()` is called after the successful creation of a model.

---

## Solid Level Action Listeners

Methods introduced:

- **`IpfcSolidActionListener.OnBeforeRegen()`**
- **`IpfcSolidActionListener.OnAfterRegen()`**
- **`IpfcSolidActionListener.OnBeforeUnitConvert()`**
- **`IpfcSolidActionListener.OnAfterUnitConvert()`**
- **`IpfcSolidActionListener.OnBeforeFeatureCreate()`**
- **`IpfcSolidActionListener.OnAfterFeatureCreate()`**
- **`IpfcSolidActionListener.OnAfterFeatureDelete()`**

The `IpfcSolidActionListener.OnBeforeRegen()` and `IpfcSolidActionListener.OnAfterRegen()` methods occur when the user regenerates a solid object within the `IpfcActionSource` to which the listener is assigned. These methods take the first feature to be regenerated and a handle to the `IpfcSolid` object as arguments. In addition, the method `IpfcSolidActionListener.OnAfterRegenerate()` includes a Boolean argument that indicates whether regeneration was successful.

---

### **Note**

- It is not recommended to modify geometry or dimensions using the `IpfcSolidActionListener.OnBeforeRegenerate()` method call.
- A regeneration that did not take place because nothing was modified is identified as a regeneration failure.

---

The methods `IpfcSolidActionListener.OnBeforeUnitConvert()` and `IpfcSolidActionListener.OnAfterUnitConvert()` activate when a user modifies the unit scheme (by selecting the Creo Parametric command **Set Up, Units**). The methods receive the `Solid` object to be converted and a Boolean flag that identifies whether the conversion changed the dimension values to keep the object the same size.

---

### **Note**

`IpfcSolidActionListeners` can be registered with the session object so that its methods are called when these events occur for any solid model that is in session.

---



---

The `IpfcSolidActionListener.OnBeforeFeatureCreate()` method activates when the user starts to create a feature that requires the **Feature Creation** dialog box. Because this event occurs only after the dialog box is displayed, it will not occur at all for datums and other features that do not use this dialog box. This method takes two arguments: the solid model that will contain the feature and the `IpfcModelItem` identifier.

The `IpfcSolidActionListener.OnAfterFeatureCreate()` method activates after any feature, including datums, has been created. This method takes the new `IpfcFeature` object as an argument.

The `IpfcSolidActionListener.OnAfterFeatureDelete()` method activates after any feature has been deleted. The method receives the solid that contained the feature and the (now defunct) `IpfcModelItem` identifier.

### Selection Level Action Listeners

Method introduced:

- **`IpfcSelectionBufferListener.OnAfterSelBufferChange()`**

The method **`IpfcSelectionBufferListener.OnAfterSelBufferChange()`** is called after the selection buffer is changed.

### Feature Level Action Listeners

Methods introduced:

- **`IpfcFeatureActionListener.OnBeforeDelete()`**
- **`IpfcFeatureActionListener.OnBeforeSuppress()`**
- **`IpfcFeatureActionListener.OnAfterSuppress()`**
- **`IpfcFeatureActionListener.OnBeforeRegen()`**
- **`IpfcFeatureActionListener.OnAfterRegen()`**
- **`IpfcFeatureActionListener.OnRegenFailure()`**
- **`IpfcFeatureActionListener.OnBeforeRedefine()`**
- **`IpfcFeatureActionListener.OnAfterCopy()`**
- **`IpfcFeatureActionListener.OnBeforeParameterDelete()`**

Each method in `IpfcFeatureActionListener` takes as an argument the feature that triggered the event.

`IpfcFeatureActionListeners` can be registered with the object so that the action listener's methods are called whenever these events occur for any feature that is in session or with a solid model to react to changes only in that model.

The method `IpfcFeatureActionListener.OnBeforeDelete()` is called before a feature is deleted.

---

The method `IpfcFeatureActionListener.OnBeforeSuppress()` is called before a feature is suppressed.

The method `IpfcFeatureActionListener.OnAfterSuppressv` is called after a successful feature suppression.

The method `IpfcFeatureActionListener.OnBeforeRegen()` is called before a feature is regenerated.

The method `IpfcFeatureActionListener.OnAfterRegenv` is called after a successful feature regeneration.

The method `IpfcFeatureActionListener.OnRegenFailure()` is called when a feature fails regeneration.

The method `IpfcFeatureActionListener.OnBeforeRedefine()` is called before a feature is redefined.

The method `IpfcFeatureActionListener.OnAfterCopy()` is called after a feature has been successfully copied.

The method `IpfcFeatureActionListener.OnBeforeParameterDelete()` is called before a feature parameter is deleted.

## Cancelling an ActionListener Operation

The VB API allows you to cancel certain notification events, registered by the action listeners.

Methods Introduced:

- **CCpfcXCancelProEAction.Throw()**

The static method `CCpfcXCancelProEAction.Throw()` must be called from the body of an action listener to cancel the impending Creo Parametric operation. This method will throw a the VB API exception signalling to Creo Parametric to cancel the listener event.

Note: Your application should not catch the VB API exception, or should rethrow it if caught, so that Creo Parametric is forced to handle it.

The following events can be cancelled using this technique:

- `IpfcSessionActionListener.OnBeforeModelErase()`
- `IpfcSessionActionListener.OnBeforeModelDelete()`
- `IpfcSessionActionListener.OnBeforeModelRename()`
- `IpfcSessionActionListener.OnBeforeModelSave()`
- `IpfcSessionActionListener.OnBeforeModelPurge()`
- `IpfcSessionActionListener.OnBeforeModelCopy()`
- `IpfcModelActionListener.OnBeforeParameterCreate()`

- 
- `IpfcModelActionListener.OnBeforeParameterDelete()`
  - `IpfcModelActionListener.OnBeforeParameterModify()`
  - `IpfcFeatureActionListener.OnBeforeDelete()`
  - `IpfcFeatureActionListener.OnBeforeSuppress()`
  - `IpfcFeatureActionListener.OnBeforeParameterDelete()`
  - `IpfcFeatureActionListener.OnBeforeParameterCreate()`
  - `IpfcFeatureActionListener.OnBeforeRedefine()`

## Interface

### Interface

This section describes various methods of importing and exporting files in the VB API.

### Exporting Files and 2D Models

Method Introduced:

- **`IpfcModel.Export()`**

The method `IpfcModel.Export()` exports model data to a file. The exported files are placed in the current Creo Parametric working directory. The input parameters are:

- *filename*—Output file name including extensions
- *exportdata*—The `IpfcExportInstructions` object that controls the export operation. The type of data that is exported is given by the `IpfcExportType` object.

There are four general categories of files to which you can export models:

- File types whose instructions inherit from `IpfcGeomExportInstructions`.

These instructions export files that contain precise geometric information used by other CAD systems.

- File types whose instructions inherit from `IpfcCoordSysExportInstructions`.

These instructions export files that contain coordinate information describing faceted, solid models (without datums and surfaces).

- File types whose instructions inherit from `IpfcFeatIdExportInstructions`.

---

These instructions export information about a specific feature.

- General file types that inherit only from `IpfcExportInstructions`.

These instructions provide conversions to file types such as BOM (bill of materials).

For information on exporting to a specific format, see the VB API APIWizard and online help for the Creo Parametric interface.

## Export Instructions

Methods Introduced:

- **`CCpfcRelationExportInstructions.Create()`**
- **`CCpfcModelInfoExportInstructions.Create()`**
- **`CCpfcProgramExportInstructions.Create()`**
- **`CCpfcIGESFileExportInstructions.Create()`**
- **`CCpfcDXFExportInstructions.Create()`**
- **`CCpfcRenderExportInstructions.Create()`**
- **`CCpfcSTLASCIExportInstructions.Create()`**
- **`CCpfcSTLBinaryExportInstructions.Create()`**
- **`CCpfcBOMExportInstructions.Create()`**
- **`CCpfcDWGSetupExportInstructions.Create()`**
- **`CCpfcFeatInfoExportInstructions.Create()`**
- **`CCpfcMFGFeatCLExportInstructions.Create()`**
- **`CCpfcMFGOperCLExportInstructions.Create()`**
- **`CCpfcMaterialExportInstructions.Create()`**
- **`CCpfcCGMFILEExportInstructions.Create()`**
- **`CCpfcInventorExportInstructions.Create()`**
- **`CCpfcFIATExportInstructions.Create()`**
- **`CCpfcConnectorParamExportInstructions.Create()`**
- **`CCpfcCableParamsFileInstructions.Create()`**
- **`CCpfcCATIAFacetsExportInstructions.Create()`**
- **`CCpfcVRMLModelExportInstructions.Create`**
- **`CCpfcSTEP2DExportInstructions.Create()`**
- **`CCpfcMedusaExportInstructions.Create()`**
- **`CCpfcCADDSExportInstructions.Create()`**
- **`CCpfcSliceExportData.Create()`**

- **CCpfcNEUTRALFileExportInstructions.Create()**
- **CCpfcProductViewExportInstructions.Create()**
- **IpfcBaseSession.ExportDirectVRML()**

Export Instructions Table

| Interface                            | Used to Export                                                                   |
|--------------------------------------|----------------------------------------------------------------------------------|
| IpfcRelationExportInstructions       | A list of the relations and parameters in a part or assembly                     |
| IpfcModelInfoExportInstructions      | Information about a model, including units information, features, and children   |
| IpfcProgramExportInstructions        | A program file for a part or assembly that can be edited to change the model     |
| IpfcIGESEExportInstructions          | A drawing in IGES format                                                         |
| IpfcDXFExportInstructions            | A drawing in DXF format                                                          |
| IpfcRenderExportInstructions         | A part or assembly in RENDER format                                              |
| IpfcSTLASCIIExportInstructions       | A part or assembly to an ASCII STL file                                          |
| IpfcSTLBinaryExportInstructions      | A part or assembly in a binary STL file                                          |
| IpfcBOMExportInstructions            | A BOM for an assembly                                                            |
| IpfcDWGSetupExportInstructions       | A drawing setup file                                                             |
| IpfcFeatInfoExportInstructions       | Information about one feature in a part or assembly                              |
| IpfcMfgFeatCLEExportInstructions     | A cutter location (CL) file for one NC sequence in a manufacturing assembly      |
| IpfcMfgOperCLEExportInstructions     | A cutter location (CL) file for all the NC sequences in a manufacturing assembly |
| IpfcMaterialExportInstructions       | A material from a part                                                           |
| IpfcCGMFILEExportInstructions        | A drawing in CGM format                                                          |
| IpfcInventorExportInstructions       | A part or assembly in Inventor format                                            |
| IpfcFIATExportInstructions           | A part or assembly in FIAT format                                                |
| IpfcConnectorParamExportInstructions | The parameters of a connector to a text file                                     |
| IpfcCableParamsFileInstructions      | Cable parameters from an assembly                                                |
| IpfcCATIAFacetsExportInstructions    | A part or assembly in CATIA format (as a faceted model)                          |
| IpfcVRMLModelExportInstructions      | A part or assembly in VRML format                                                |
| IpfcSTEP2DExportInstructions         | A two-dimensional STEP format file                                               |
| IpfcMedusaExportInstructions         | A drawing in MEDUSA file                                                         |
| IpfcCADDSEExportInstructions         | A CADD5 solid model                                                              |
| IpfcNEUTRALFileExportInstructions    | A Creo Parametric part to neutral format                                         |
| IpfcProductViewExportInstructions    | A part, assembly, or drawing in Creo View format                                 |
| IpfcExport.SliceExportData           | A slice export format                                                            |



## Note

The New Instruction Classes replace the following Deprecated Classes:

| Deprecated Classes                        | New Instruction Classes                      |
|-------------------------------------------|----------------------------------------------|
| <code>IpfcSTEPExportInstructions</code>   | <code>IpfcSTEP3DExportInstructions</code>    |
| <code>IpfcVDAExportInstructions</code>    | <code>IpfcVDA3DExportInstructions</code>     |
| <code>IpfcIGES3DExportInstructions</code> | <code>IpfcIGES3DNewExportInstructions</code> |

## Exporting Drawing Sheets

The options required to export multiple sheets of a drawing are given by the `IpfcExport2DOption` object.

Methods and Properties Introduced:

- **`CCpfcExport2DOption.Create()`**
- **`IpfcExport2DOption.ExportSheetOption`**
- **`IpfcExport2DOption.ModelSpaceSheet`**
- **`IpfcExport2DOption.Sheets`**

The method `CCpfcExport2DOptions.Create()` creates a new instance of the `IpfcExport2DOption` object. This object contains the following options:

- *ExportSheetOption*—Specifies the option for exporting multiple drawing sheets. Use the property `IpfcExport2DOption.ExportSheetOption` to set the option for exporting multiple drawing sheets. The options are given by the `EpfcExport2DSheetOption` class and can be of the following types:
  - `EpfcEXPORT_CURRENT_TO_MODEL_SPACE`—Exports only the drawing's current sheet as model space to a single file. This is the default type.
  - `EpfcEXPORT_CURRENT_TO_PAPER_SPACE`—Exports only the drawing's current sheet as paper space to a single file. This type is the same as `EpfcEXPORT_CURRENT_TO_MODEL_SPACE` for formats that do not support the concept of model space and paper space.
  - `EpfcEXPORT_ALL`—Exports all the sheets in a drawing to a single file as paper space, if applicable for the format type.
  - `EpfcEXPORT_SELECTED`—Exports selected sheets in a drawing as paper space and one sheet as model space.
- *ModelSpaceSheet*—Specifies the sheet number that needs to be exported as model space. This option is applicable only if the export formats support the

---

concept of model space and paper space and if *ExportSheetOption* is set to `EpfcEXPORT_SELECTED`. Use the property `IpfcExport2DOption.ModelSpaceSheet` to set this option.

- *Sheets*—Specifies the sheet numbers that need to be exported as paper space. This option is applicable only if *ExportSheetOption* is set to `EpfcEXPORT_SELECTED`. Use the property `IpfcExport2DOption.Sheets` to set this option.

## Exporting to Faceted Formats

The methods and properties described in this section support the export of Creo Parametric drawings and solid models to faceted formats like CATIA CGR.

Methods and Properties Introduced:

- **`IpfcTriangulationInstructions.AngleControl`**
- **`IpfcTriangulationInstructions.ChordHeight`**
- **`IpfcTriangulationInstructions.StepSize`**
- **`IpfcTriangulationInstructions.FacetControlOptions`**

The property `IpfcTriangulationInstructions.AngleControl` allow you to control the angle control for the exported facet drawings and models. You can set the value between 0.0 to 1.0.

Use the property `IpfcTriangulationInstructions.ChordHeight` allow you to control the chord height for the exported facet drawings and models.

The property `IpfcTriangulationInstructions.StepSize` allows you to control the step size for the exported files. The default value is 0.0.



### Note

You must pass the value of Step Size value as NULL, if you specify the Quality value.

---

The property `IpfcCoordSysExportInstructions.StepSize` control the step size for the exported files. The default value is 0.0.



### Note

You must pass the value of Step Size value as NULL, if you specify the Quality value.

---

---

### The property

`IpfcTriangulationInstructions.FacetControlOptions` control the facet export options using bit flags. You can set the bit flags using the `IpfcFacetControlFlag` object. It has the following values:

- `EpfcFACET_STEP_SIZE_ADJUST`—Adjusts the step size according to the component size.
- `EpfcFACET_CHORD_HEIGHT_ADJUST`—Adjusts the chord height according to the component size.
- `EpfcFACET_USE_CONFIG`—If this flag is set, values of the flags `EpfcFACET_STEP_SIZE_OFF`, `EpfcFACET_STEP_SIZE_ADJUST`, and `EpfcFACET_CHORD_HEIGHT_ADJUST` are ignored and the configuration settings from the Creo Parametric user interface are used during the export operation.
- `EpfcFACET_CHORD_HEIGHT_DEFAULT`—Uses the default value set in the Creo Parametric user interface for the chord height.
- `EpfcFACET_ANGLE_CONTROL_DEFAULT`—Uses the default value set in the Creo Parametric user interface for the angle control.
- `EpfcFACET_STEP_SIZE_DEFAULT`—Uses the default value set in the Creo Parametric user interface for the step size.
- `EpfcFACET_STEP_SIZE_OFF`—Switches off the step size control.
- `EpfcFACET_FORCE_INTO_RANGE`—Forces the out-of-range parameters into range. If any of the `EpfcFACET_*_DEFAULT` option is set, then the option `EpfcFACET_FORCE_INTO_RANGE` is not applied on that parameter.
- `EpfcFACET_STEP_SIZE_FACET_INCLUDE_QUILTS`—Includes quilts in the export of Creo Parametric model to the specified format.
- `EpfcEXPORT_INCLUDE_ANNOTATIONS`—Includes annotations in the export of Creo Parametric model to the specified format.

---

### Note

To include annotations, during the export of Creo Parametric model, you must call the method `IpfcModel.Display()` before calling `IpfcModel.Export()`.

---

- `EpfcFACET_VISIBLE_MODELS`—Exports models which have their visibility status set to **Show** in Creo Parametric. Models that are hidden are not exported.



---

## Exporting Using Coordinate System

The methods and properties described in this section support the export of files with information about the faceted solid models (without datums and surfaces). The files are exported in reference to the coordinate-system feature in the model being exported.

Methods and Properties Introduced:

- **`IpfcCoordSysExportInstructions.CsysName`**
- **`IpfcCoordSysExportInstructions.Quality`**
- **`IpfcCoordSysExportInstructions.MaxChordHeight`**
- **`IpfcCoordSysExportInstructions.AngleControl`**
- **`IpfcCoordSysExportInstructions.GetSliceExportData()`**
- **`IpfcCoordSysExportInstructions.SetSliceExportData()`**
- **`IpfcCoordSysExportInstructions.StepSize`**
- **`IpfcCoordSysExportInstructions.FacetControlOptions`**
- **`IpfcSelection.SetIntf3DCsys()`**

The property `IpfcCoordSysExportInstructions.CsysName` returns the name of the the name of a coordinate system feature in the model being exported. It is recommended to use the coordinate system that places the part or assembly in its upper-right quadrant, so that all position and distance values of the exported assembly or part are positive.

The property `IpfcCoordSysExportInstructions.Quality` can be used instead of

`IpfcCoordSysExportInstructions.MaxChordHeight` and `IpfcCoordSysExportInstructions.AngleControl`. You can set the value between 1 and 10. The higher the value you pass, the lower is the Maximum Chord Height setting and higher is the Angle Control setting the method uses. The default Quality value is 1.0.



### Note

You must pass the value of Quality as `NULL`, if you use Maximum Chord Height and Angle Control values. If Quality, Maximum Chord Height, and Angle Control are all `NULL`, then the Quality setting of 3 is used.

---

---

Use the property

`IpfcCoordSysExportInstructions.MaxChordHeight` to work with the maximum chord height for the exported files. The default value is 0.1.

---

 **Note**

You must pass the value of Maximum Chord Height as NULL, if you specify the Quality value.

---

The property `IpfcCoordSysExportInstructions.AngleControl` allow you to work with the angle control setting for the exported files. The default value is 0.1.

---

 **Note**

You must pass the value of Angle Control value as NULL, if you specify the Quality value.

---

The methods

`IpfcCoordSysExportInstructions.GetSliceExportData()` and `IpfcCoordSysExportInstructions.SetSliceExportData()` get and set the `IpfcSliceExportData` data object that specifies data for the slice export. The options in this object are described as follows:

- *CompIds*—Specifies the sequence of integers that identify the components that form the path from the root assembly down to the component part or assembly being referred to. Use the property `IpfcSliceExportData.CompIds` to work with the component IDs.

The property `IpfcCoordSysExportInstructions.StepSize` controls the step size for the exported files. The default value is 0.0.

---

 **Note**

You must pass the value of Step Size value as NULL, if you specify the Quality value.

---

The property

`IpfcCoordSysExportInstructions.FacetControlOptions` control the facet export options using bit flags. You can set the bit flags using the `IpfcFacetControlFlag` object. For more information on the bit flag values, please refer to the section [Exporting to Faceted Formats on page 247](#).

---

The function `IpfcSelection.SetIntf3DCsys()` sets the reference coordinate system for the export. The input argument *ReferenceCsys* is the reference coordinate system selection. Call this method without any argument to set default coordinate system. Reference coordinate system is not supported for CADDs and NEUTRAL file types.

## Exporting to PDF and U3D

The methods and properties described in this section support the export of Creo Parametric drawings and solid models to Portable Document Format (PDF) and U3D format. You can export a drawing or a 2D model as a 2D raster image embedded in a PDF file. You can export Creo Parametric solid models in the following ways:

- As a U3D model embedded in a one-page PDF file
- As 2D raster images embedded in the pages of a PDF file representing saved views
- As a standalone U3D file

While exporting multiple sheets of a Creo Parametric drawing to a PDF file, you can choose to export all sheets, the current sheet, or selected sheets.

These methods also allow you to insert a variety of non-geometric information to improve document content, navigation, and search.

### Methods and Properties Introduced:

- **CCpfcPDFExportInstructions.Create()**
- **IpfcPDFExportInstructions.FilePath**
- **IpfcPDFExportInstructions.Options**
- **IpfcPDFExportInstructions.ProfilePath**
- **CCpfcPDFOption.Create()**
- **IpfcPDFOption.OptionType**
- **PDFOption.OptionValue**

The method `CCpfcPDFExportInstructions.Create()` creates a new instance of the `IpfcPDFExportInstructions` data object that describes how to export Creo Parametric drawings or solid models to the PDF and U3D formats. The options in this object are described as follows:

- *FilePath*—Specifies the name of the output file. Use the property `IpfcPDFExportInstructions.FilePath` to set the name of the output file.
- *Options*—Specifies a collection of PDF export options of the type `IpfcPDFOption`. Create a new instance of this object using the method `IpfcPDFOption.Create()`. This object contains the following attributes:

- *OptionType*—Specifies the type of option in terms of the `EpfcPDFOptionType` enumerated class. Set this option using the property `IpfcPDFOption.OptionType`.
- *OptionValue*—Specifies the value of the option in terms of the `IpfcArgValue` object. Set this option using the property `IpfcPDFOption.OptionValue`.

Use the property `IpfcPDFExportInstructions.Options` to set the collection of PDF export options.

- *ProfilePath*—Specifies the export profile path. Use the property `IpfcPDFExportInstructions.ProfilePath` to set the profile path. When you set the profile path, the PDF export options set in the data object `IpfcPDFExportInstructions` data object are ignored when the method `IpfcModel.Export()` is called. You can set the profile path as `NULL`.

---

#### **Note**

You can specify the profile path only for drawings.

---

The types of options (given by the `EpfcPDFOptionType` enumerated class) available for export to PDF and U3D formats are described as follows:

- `EpfcPDFOPT_FONT_STROKE`—Allows you to switch between using TrueType fonts or “stroking” text in the resulting document. This option is given by the `EpfcPDFFontStrokeMode` enumerated class and takes the following values:
  - `EpfcPDF_USE_TRUE_TYPE_FONTS`—Specifies TrueType fonts. This is the default type.
  - `EpfcPDF_STROKE_ALL_FONTS`—Specifies the option to stroke all fonts.
- `EpfcPDFOPT_COLOR_DEPTH`—Allows you to choose between color, grayscale, or monochrome output. This option is given by the `EpfcPDFColorDepth` enumerated class and takes the following values:
  - `EpfcPDF_CD_COLOR`—Specifies color output. This is the default value.
  - `EpfcPDF_CD_GRAY`—Specifies grayscale output.
  - `EpfcPDF_CD_MONO`—Specifies monochrome output.
- `EpfcPDFOPT_HIDDENLINE_MODE`—Enables you to set the style for hidden lines in the resulting PDF document. This option is given by the `EpfcPDFHiddenLineMode` enumerated class and takes the following values:

- 
- `EpfcPDF_HLM_SOLID`—Specifies solid hidden lines.
  - `EpfcPDF_HLM_DASHED`—Specifies dashed hidden lines. This is the default type.
  - `EpfcPDFOPT_SEARCHABLE_TEXT`—If true, stroked text is searchable. The default value is true.
  - `EpfcPDFOPT_RASTER_DPI`—Allows you to set the resolution for the output of any shaded views in DPI. It can take a value between 100 and 600. The default value is 300.
  - `EpfcPDFOPT_LAUNCH_VIEWER`—If true, launches the Adobe Acrobat Reader. The default value is true.
  - `EpfcPDFOPT_LAYER_MODE`—Enables you to set the availability of layers in the document. It is given by the `EpfcPDFLayerMode` enumerated class and takes the following values:
    - `EpfcPDF_LAYERS_ALL`—Exports the visible layers and entities. This is the default.
    - `EpfcPDF_LAYERS_VISIBLE`—Exports only visible layers in a drawing.
    - `EpfcPDF_LAYERS_NONE`—Exports only the visible entities in the drawing, but not the layers on which they are placed.
  - `EpfcPDFOPT_PARAM_MODE`—Enables you to set the availability of model parameters as searchable metadata in the PDF document. It is given by the `EpfcPDFParameterMode` enumerated class and takes the following values:
    - `EpfcPDF_PARAMS_ALL`—Exports the drawing and the model parameters to PDF. This is the default.
    - `EpfcPDF_PARAMS_DESIGNATED`—Exports only the specified model parameters in the PDF metadata.
    - `EpfcPDF_PARAMS_NONE`—Exports the drawing to PDF without the model parameters.
  - `EpfcPDFOPT_HYPERLINKS`—Sets hyperlinks to be exported as label text only or sets the underlying hyperlink URLs as active. The default value is true, specifying that the hyperlinks are active.
  - `EpfcPDFOPT_BOOKMARK_ZONES`—If true, adds bookmarks to the PDF showing zoomed in regions or zones in the drawing sheet. The zone on an A4-size drawing sheet is ignored.
  - `EpfcPDFOPT_BOOKMARK_VIEWS`—If true, adds bookmarks to the PDF document showing zoomed in views on the drawing.

- 
- `EpfcPDFOPT_BOOKMARK_SHEETS`—If true, adds bookmarks to the PDF document showing each of the drawing sheets.
  - `EpfcPDFOPT_BOOKMARK_FLAG_NOTES`—If true, adds bookmarks to the PDF document showing the text of the flag note.
  - `EpfcPDFOPT_TITLE`—Specifies a title for the PDF document.
  - `EpfcPDFOPT_AUTHOR`—Specifies the name of the person generating the PDF document.
  - `EpfcPDFOPT_SUBJECT`—Specifies the subject of the PDF document.
  - `EpfcPDFOPT_KEYWORDS`—Specifies relevant keywords in the PDF document.
  - `EpfcPDFOPT_PASSWORD_TO_OPEN`—Sets a password to open the PDF document. By default, this option is NULL, which means anyone can open the PDF document without a password.
  - `EpfcPDFOPT_MASTER_PASSWORD`—Sets a password to restrict or limit the operations that the viewer can perform on the opened PDF document. By default, this option is NULL, which means you can make any changes to the PDF document regardless of the settings of the modification flags `EpfcPDFOPT_ALLOW_*`.
  - `EpfcPDFOPT_RESTRICT_OPERATIONS`—If true, enables you to restrict or limit operations on the PDF document. By default, is false.
  - `EpfcPDFOPT_ALLOW_MODE`—Enables you to set the security settings for the PDF document. This option must be set if `PDFOPT_RESTRICT_OPERATIONS` is set to true. It is given by the `EpfcPDFRestrictOperationsMode` enumerated class and takes the following values:
    - `EpfcPDF_RESTRICT_NONE`—Specifies that the user can perform any of the permitted viewer operations on the PDF document. This is the default value.
    - `EpfcPDF_RESTRICT_FORMS_SIGNING`—Restricts the user from adding digital signatures to the PDF document.
    - `EpfcPDF_RESTRICT_INSERT_DELETE_ROTATE`—Restricts the user from inserting, deleting, or rotating the pages in the PDF document.
    - `EpfcPDF_RESTRICT_COMMENT_FORM_SIGNING`—Restricts the user from adding or editing comments in the PDF document.
    - `EpfcPDF_RESTRICT_EXTRACTING`—Restricts the user from extracting pages from the PDF document.
  - `EpfcPDFOPT_ALLOW_PRINTING`—If true, allows you to print the PDF document. By default, it is true.

- 
- `EpfcPDFOPT_ALLOW_PRINTING_MODE`—Enables you to set the print resolution. It is given by the `EpfcPDFPrintingMode` enumerated class and takes the following values:
    - `EpfcPDF_PRINTING_LOW_RES`—Specifies low resolution for printing.
    - `EpfcPDF_PRINTING_HIGH_RES`—Specifies high resolution for printing. This is the default value.
  - `EpfcPDFOPT_ALLOW_COPYING`—If true, allows you to copy content from the PDF document. By default, it is true.
  - `EpfcPDFOPT_ALLOW_ACCESSIBILITY`—If true, enables visually-impaired screen reader devices to extract data independent of the value given by the `EpfcPDFRestrictOperationsMode` enumerated class. The default value is true.
  - `EpfcPDFOPT_PENTABLE`—If true, uses the standard Creo Parametric pentable to control the line weight, line style, and line color of the exported geometry. The default value is false.
  - `EpfcPDFOPT_LINECAP`—Enables you to control the treatment of the ends of the geometry lines exported to PDF. It is given by the `EpfcPDFLinecap` enumerated class and takes the following values:
    - `EpfcPDF_LINECAP_BUTT`—Specifies the butt cap square end. This is the default value.
    - `EpfcPDF_LINECAP_ROUND`—Specifies the round cap end.
    - `EpfcPDF_LINECAP_PROJECTING_SQUARE`—Specifies the projecting square cap end.
  - `EpfcPDFOPT_LINEJOIN`—Enables you to control the treatment of the joined corners of connected lines exported to PDF. It is given by the `EpfcPDFLinejoin` enumerated class and takes the following values:
    - `EpfcPDF_LINEJOIN_MITER`—Specifies the miter join. This is the default.
    - `EpfcPDF_LINEJOIN_ROUND`—Specifies the round join.
    - `EpfcPDF_LINEJOIN_BEVEL`—Specifies the bevel join.
  - `EpfcPDFOPT_SHEETS`—Allows you to specify the sheets from a Creo Parametric drawing that are to be exported to PDF. It is given by the `EpfcPrintSheets` enumerated class and takes the following values:
    - `EpfcPRINT_CURRENT_SHEET`—Only the current sheet is exported to PDF

- `EpfcPRINT_ALL_SHEETS`—All the sheets are exported to PDF. This is the default value.
- `EpfcPRINT_SELECTED_SHEETS`—Sheets of a specified range are exported to PDF. If this value is assigned, then the value of the option `EpfcPDFOPT_SHEET_RANGE` must also be known.
- `EpfcPDFOPT_SHEET_RANGE`—Specifies the range of sheets in a drawing that are to be exported to PDF. If this option is set, then the option `EpfcPDFOPT_SHEETS` must be set to the value `EpfcPRINT_SELECTED_SHEETS`.
- `EpfcPDFOPT_EXPORT_MODE`—Enables you to select the object to be exported to PDF and the export format. It is given by the `EpfcPDFExportMode` enumerated class and takes the following values:
  - `EpfcPDF_2D_DRAWING`—Only drawings are exported to PDF. This is the default value.
  - `EpfcPDF_3D_AS_NAMED_VIEWS`—3D models are exported as 2D raster images embedded in PDF files.
  - `EpfcPDF_3D_AS_U3D_PDF`—3D models are exported as U3D models embedded in one-page PDF files.
  - `EpfcPDF_3D_AS_U3D`—A 3D model is exported as a U3D (.u3d) file. This value ignores the options set for the `EpfcPDFOptionType` enumerated class.
- `EpfcPDFOPT_LIGHT_DEFAULT`—Enables you to set the default lighting style used while exporting 3D models in the U3D format to a one-page PDF file, that is when the option `EpfcPDFOPT_EXPORT_MODE` is set to `EpfcPDF_3D_AS_U3D`. The values for this option are given by the `EpfcPDFU3DLightingMode` enumerated class.
- `EpfcPDFOPT_RENDER_STYLE_DEFAULT`—Enables you to set the default rendering style used while exporting Creo Parametric models in the U3D format to a one-page PDF file, that is when the option `EpfcPDFOPT_EXPORT_MODE` is set to `EpfcPDF_3D_AS_U3D`. The values for this option are given by the `EpfcPDFU3DRenderMode` enumerated class.
- `EpfcPDFOPT_SIZE`—Allows you to specify the page size of the exported PDF file. The values for this option are given by the `EpfcPlotPaperSize` enumerated class. If the value is set to `EpfcVARIABLESIZEPLOT`, you also need to set the options `EpfcPDFOPT_HEIGHT` and `EpfcPDFOPT_WIDTH`.
- `EpfcPDFOPT_HEIGHT`—Enables you to set the height for a user-defined page size of the exported PDF file. The default value is 0.0.
- `EpfcPDFOPT_WIDTH`—Enables you to set the width for a user-defined page size of the exported PDF file. The default value is 0.0.



- 
- `EpfcPDFOPT_ORIENTATION`—Enables you to specify the orientation of the pages in the exported PDF file. It is given by the `EpfcSheetOrientation` enumerated class.
    - `EpfcORIENT_PORTRAIT`—Exports the pages in portrait orientation. This is the default value.
    - `EpfcORIENT_LANDSCAPE`—Exports the pages in landscape orientation.
  - `EpfcPDFOPT_TOP_MARGIN`—Allows you to specify the top margin of the view port. The default value is 0.0.
  - `EpfcPDFOPT_LEFT_MARGIN`—Allows you to specify the left margin of the view port. The default value is 0.0.
  - `EpfcPDFOPT_BACKGROUND_COLOR_RED`—Specifies the default red background color that appears behind the U3D model. You can set any value within the range of 0.0 to 1.0. The default value is 1.0.
  - `EpfcPDFOPT_BACKGROUND_COLOR_GREEN`—Specifies the default green background color that appears behind the U3D model. You can set any value within the range of 0.0 to 1.0. The default value is 1.0.
  - `EpfcPDFOPT_BACKGROUND_COLOR_BLUE`—Specifies the default blue background color that appears behind the U3D model. You can set any value within the range of 0.0 to 1.0. The default value is 1.0.
  - `EpfcPDFOPT_ADD_VIEWS`—If true, allows you to add view definitions to the U3D model from a file. By default, it is true.
  - `EpfcPDFOPT_VIEW_TO_EXPORT`—Specifies the view or views to be exported to the PDF file. It is given by the `EpfcPDFSelectedViewMode` enumerated class and takes the following values:
    - `EpfcPDF_VIEW_SELECT_CURRENT`—Exports the current graphical area to a one-page PDF file.
    - `EpfcPDF_VIEW_SELECT_ALL`—Exports all the views to a multi-page PDF file. Each page contains one view with the view name displayed at the bottom center of the view port.
    - `EpfcPDF_VIEW_SELECT_BY_NAME`—Exports the selected view to a one-page PDF file with the view name printed at the bottom center of the view port. If this value is assigned, then the option `EpfcPDFOPT_SELECTED_VIEW` must also be set.
  - `EpfcPDFOPT_SELECTED_VIEW`—Sets the option `EpfcPDFOPT_VIEW_TO_EXPORT` to the value `EpfcPDF_VIEW_SELECT_BY_NAME`, if the corresponding view is successfully found.

- `EpfcPDFOPT_PDF_SAVE`—Specifies the PDF save options. It is given by the `EpfcPDFSaveMode` enumerated class and takes the following values:
  - `EpfcPDF_ARCHIVE_1`—Applicable only for the value `EpfcPDF_2D_DRAWING`. Saves the drawings as PDF with the following conditions:
    - ◆ The value of `EpfcPDFLayerMode` is set to `EpfcPDF_LAYERS_NONE`.
    - ◆ The value of `EpfcPDFOPT_HYPERLINKS` is set to `FALSE`.
    - ◆ The shaded views in the drawings will not have transparency and may overlap other data in the PDF.
    - ◆ The value of `EpfcPDFOPT_PASSWORD_TO_OPEN` is set to `NULL`.
    - ◆ The value of `EpfcPDFOPT_MASTER_PASSWORD` is set to `NULL`.
  - `EpfcPDF_FULL`—Saves the PDF with the values set by you. This is the default value.

## Exporting 3D Geometry

The VB API allows you to export three dimensional geometry to various formats.

### Export Instructions

Methods and Properties Introduced:

- **`IpfcModel.ExportIntf3D()`**
- **`IpfcBaseSession.ExportProfileLoad()`**
- **`CCpfcGeometryFlags.Create()`**
- **`CCpfcInclusionFlags.Create()`**
- **`CCpfcLayerExportOptions.Create()`**
- **`CCpfcTriangulationInstructions.Create()`**

From Creo Parametric 5.0 F000 onward, the following interfaces along with their methods have been deprecated. Use the method

`IpfcModel.ExportIntf3D()` instead to export Creo Parametric models to other file formats. All the options that can be set with these interfaces and methods, can also be set using the export profile option in Creo Parametric. Refer to the Creo Parametric Data Exchange Online Help for more information.

- `Export3DInstructions`
- `ACIS3DExportInstructions`
- `CATIAModel3DExportInstructions`
- `CATIASession3DExportInstructions`
- `CatiaPart3DExportInstructions`

- 
- `CatiaProduct3DExportInstructions`
  - `CatiaCGR3DExportInstructions`
  - `DXF3DExportInstructions`
  - `DWG3DExportInstructions`
  - `IGES3DNewExportInstructions`
  - `JT3DExportInstructions`
  - `ParaSolid3DExportInstructions`
  - `STEP3DExportInstructions`
  - `SWPart3DExportInstructions`
  - `SWAsm3DExportInstructions`
  - `UG3DExportInstructions`
  - `VDA3DExportInstructions`

The method `IfcModel.ExportIntf3D()` exports a Creo Parametric model to the specified output format using the default export profile. The export options must be set using the export profile option in Creo Parametric.

The method `IpfcBaseSession.ExportProfileLoad()` loads the specified profile for export. You can use this function when you want to use the export profile of your choice instead of the default export profile in a particular Creo Parametric session. The input argument *ProfileFile* is the full path to the profile along with the profile name and extension.



#### Note

Once the export profile file is loaded in a Creo Parametric session, it will be active in the interactive mode as well.

---

The method `CCpfcTriangulationInstructions.Create()` creates a object that will be used to define the parameters for faceted exports.

### Export Utilities

Methods Introduced:

- **`IpfcBaseSession.IsConfigurationSupported()`**
- **`IpfcBaseSession.IsGeometryRepSupported()`**

The method `IpfcBaseSession.IsConfigurationSupported()` checks whether the specified assembly configuration is valid for a particular model and the specified export format. The input parameters for this method are:

- *Configuration*—Specifies the structure and content of the output files.

- *Type*—Specifies the output file type to create.

The method returns a true value if the configuration is supported for the specified export type.

The method `IpfcBaseSession.IsGeometryRepSupported()` checks whether the specified geometric representation is valid for a particular export format. The input parameters are :

- *Flags*—The type of geometry supported by the export operation.
- *Type*—The output file type to create.

The method returns a true value if the geometry combination is valid for the specified model and export type.

The methods `IpfcBaseSession.IsConfigurationSupported()` and `IpfcBaseSession.IsGeometryRepSupported()` must be called before exporting an assembly to the specified export formats except for the CADDs and STEP2D formats. The return values of both the methods must be true for the export operation to be successful.

Use the method `IpfcModel.Export()` to export the assembly to the specified output format.

## Shrinkwrap Export

To improve performance in a large assembly design, you can export lightweight representations of models called shrinkwrap models. A shrinkwrap model is based on the external surfaces of the source part or assembly model and captures the outer shape of the source model.

You can create the following types of nonassociative exported shrinkwrap models:

- **Surface Subset**—This type consists of a subset of the original model's surfaces.
- **Faceted Solid**—This type is a faceted solid representing the original solid.
- **Merged Solid**—The external components from the reference assembly model are merged into a single part representing the solid geometry in all collected components.

Methods Introduced:

- **`IpfcSolid.ExportShrinkwrap()`**

You can export the specified solid model as a shrinkwrap model using the method `IpfcSolid.ExportShrinkwrap()`. This method takes the `IpfcShrinkwrapExportInstructions` object as an argument.

Use the appropriate interface given in the following table to create the required type of shrinkwrap. All the interfaces have their own static method to create an object of the specified type. The object created by these interfaces can be used as an object of type `IpfcShrinkwrapExportInstructions` or `IpfcShrinkwrapModelExportInstructions`.

| Type of Shrinkwrap Model | Interface to Use                                     |
|--------------------------|------------------------------------------------------|
| Surface Subset           | <code>IpfcShrinkwrapSurfaceSubsetInstructions</code> |
| Faceted Part             | <code>IpfcShrinkwrapFacetedPartInstructions</code>   |
| Faceted VRML             | <code>IpfcShrinkwrapFacetedVRMLInstructions</code>   |
| Faceted STL              | <code>IpfcShrinkwrapFacetedSTLInstructions</code>    |
| Merged Solid             | <code>IpfcShrinkwrapMergedSolidInstructions</code>   |

### Setting Shrinkwrap Options

The interface `IpfcShrinkwrapModelExportInstructions` contains the general methods available for all the types of shrinkwrap models. The object created by any of the interfaces specified in the preceding table can be used with these methods.

Properties Introduced:

- **`IpfcShrinkwrapModelExportInstructions.Method`**
- **`IpfcShrinkwrapModelExportInstructions.Quality`**
- **`IpfcShrinkwrapModelExportInstructions.AutoHoleFilling`**
- **`IpfcShrinkwrapModelExportInstructions.IgnoreSkeleton`**
- **`IpfcShrinkwrapModelExportInstructions.IgnoreQuilts`**
- **`IpfcShrinkwrapModelExportInstructions.AssignMassProperties`**
- **`IpfcShrinkwrapModelExportInstructions.IgnoreSmallSurfaces`**
- **`IpfcShrinkwrapModelExportInstructions.SmallSurfPercentage`**
- **`IpfcShrinkwrapModelExportInstructions.DatumReferences`**

The property `IpfcShrinkwrapModelExportInstructions.Method` returns the method used to create the shrinkwrap. The types of shrinkwrap methods are:

- `SWCREATE_SURF_SUBSET`—Surface Subset
- `SWCREATE_FACETED_SOLID`—Faceted Solid
- `SWCREATE_MERGED_SOLID`—Merged Solid

---

The property `IpfcShrinkwrapModelExportInstructions.Quality` specifies the quality level for the system to use when identifying surfaces or components that contribute to the shrinkwrap model. Quality ranges from 1 which produces the coarsest representation of the model in the fastest time, to 10 which produces the most exact representation. The default value is 1.

The property

`IpfcShrinkwrapModelExportInstructions.AutoHoleFilling` sets a flag that forces Creo Parametric to identify all holes and surfaces that intersect a single surface and fills those holes during shrinkwrap. The default value is true.

The property

`IpfcShrinkwrapModelExportInstructions.IgnoreSkeleton` determines whether the skeleton model geometry must be included in the shrinkwrap model.

The property

`IpfcShrinkwrapModelExportInstructions.IgnoreQuilts` and determines whether external quilts must be included in the shrinkwrap model.

The property

`IpfcShrinkwrapModelExportInstructions.AssignMassProperties` assigns mass properties to the shrinkwrap model. The default value is false and the mass properties of the original model is assigned to the shrinkwrap model. If the value is set to true, the user must assign a value for the mass properties.

The property

`IpfcShrinkwrapModelExportInstructions.IgnoreSmallSurfaces` sets a flag that forces Creo Parametric to skip surfaces smaller than a certain size. The default value is false. The size of the surface is specified as a percentage of the model's size. This size can be modified using the property `IpfcShrinkwrapModelExportInstructions.SmallSurfPercentage`.

The property

`IpfcShrinkwrapModelExportInstructions.DatumReferences` specifies and selects the datum planes, points, curves, axes, and coordinate system references to be included in the shrinkwrap model.

## Surface Subset Options

Methods and Properties Introduced:

- **`CCpfcShrinkwrapSurfaceSubsetInstructions.Create()`**
- **`IpfcShrinkwrapSurfaceSubsetInstructions.AdditionalSurfaces`**
- **`IpfcShrinkwrapSurfaceSubsetInstructions.OutputModel`**

---

The static method

`CCpfcShrinkwrapSurfaceSubsetInstructions.Create()` returns an object used to create a shrinkwrap model of surface subset type. Specify the name of the output model in which the shrinkwrap is to be created as an input to this method.

The property

`IpfcShrinkwrapSurfaceSubsetInstructions.AdditionalSurfaces` selects individual surfaces to be included in the shrinkwrap model.

The property

`IpfcShrinkwrapSurfaceSubsetInstructions.OutputModel` sets the template model.

### Faceted Solid Options

The `IpfcShrinkwrapFacetedFormatInstructions` interface consists of the following types:

- `SWFACETED_PART`—Creo Parametric part with normal geometry. This is the default format type.
- `SWFACETED_STL`—An STL file.
- `SWFACETED_VRML`—A VRML file.

Use the `Create` method to create the object of the specified type. Upcast the object to use the general methods available in this interface.

Properties Introduced:

- **`IpfcShrinkwrapFacetedFormatInstructions.Format`**
- **`IpfcShrinkwrapFacetedFormatInstructions.FrameFile`**

The property `IpfcShrinkwrapFacetedFormatInstructions.Format` returns the output file format of the shrinkwrap model.

The property

`IpfcShrinkwrapFacetedFormatInstructions.FrameFile` enables you to select a frame file to create a faceted solid motion envelope model that represents the full motion of the mechanism captured in the frame file. Specify the name and complete path of the frame file.

### Faceted Part Options

Methods and Properties Introduced:

- **`CCpfcShrinkwrapFacetedPartInstructions.Create()`**
- **`IpfcShrinkwrapFacetedPartInstructions.Lightweight`**

---

The static method

`CCpfcShrinkwrapFacetedPartInstructions.Create()` returns an object used to create a shrinkwrap model of shrinkwrap faceted type. The input parameters of this method are:

- *OutputModel*—Specify the output model where the shrinkwrap must be created.
- *Lightweight*—Specify this value as True if the shrinkwrap model is a Lightweight Creo Parametric part.

The property

`IpfcShrinkwrapFacetedPartInstructions.Lightweight` specifies if the Creo Parametric part is exported as a light weight faceted geometry.

## VRML Export Options

Methods and Properties Introduced:

- **`CCpfcShrinkwrapVRMLInstructions.Create()`**
- **`IpfcShrinkwrapVRMLInstructions.OutputFile`**

The static method `CCpfcShrinkwrapVRMLInstructions.Create()` returns an object used to create a shrinkwrap model of shrinkwrap VRML format. Specify the name of the output model as an input to this method.

The property `IpfcShrinkwrapVRMLInstructions.OutputFile` specifies the name of the output file to be created.

## STL Export Options

Methods and Properties Introduced:

- **`CCpfcShrinkwrapVRMLInstructions.Create()`**
- **`IpfcShrinkwrapVRMLInstructions.OutputFile`**

The static method `CCpfcShrinkwrapVRMLInstructions.Create()` returns an object used to create a shrinkwrap model of shrinkwrap STL format. Specify the name of the output model as an input to this method.

The property `IpfcShrinkwrapSTLInstructions.OutputFile` specifies the name of the output file to be created.

## Merged Solid Options

Methods and Properties Introduced:

- **`CCpfcShrinkwrapMergedSolidInstructions.Create()`**
- **`IpfcShrinkwrapMergedSolidInstructions.AdditionalComponents`**



---

The static method

`CCpfcShrinkwrapMergedSolidInstructions.Create()` returns an object used to create a shrinkwrap model of merged solids format. Specify the name of the output model as an input to this method.

The property

`IpfcShrinkwrapMergedSolidInstructions.AdditionalComponents` specifies individual components of the assembly to be merged into the shrinkwrap model.

## VRML Representation

### Example Code

The sample code in the file `pfcInterfaceExamples.vb` located at `<creo_vbapi_loadpoint>/vbapi_examples` demonstrates the use of `IpfcDisplay` methods with 3D display lists. The static method `AddCircleDisplay()` leverages the fact that when a model with a model program attached is erased or deleted the stop method of the model program is called. This example code uses the stop method to produce a VRML representation of the model in a standard directory for Web publishing.

## Importing Files

Method Introduced:

- **`IpfcModel.Import()`**

The method `IpfcModel.Import()` reads a file into Creo Parametric. The format must be the same as it would be if these files were created by Creo Parametric. The parameters are:

- *FilePath*—Absolute path of the file to be imported along with its extension.
- *ImportData*—The `IpfcImportInstructions` object that controls the import operation.

### Import Instructions

Methods Introduced:

- **`CCpfcRelationImportInstructions.Create()`**
- **`CCpfcIGESSectionImportInstructions.Create()`**
- **`CCpfcProgramImportInstructions.Create()`**
- **`CCpfcConfigImportInstructions.Create()`**
- **`CCpfcDWGSetupImportInstructions.Create()`**
- **`CCpfcSpoolImportInstructions.Create()`**

- **CCpfcConnectorParamsImportInstructions.Create()**
- **CCpfcASSEMBTreeCFGImportInstructions.Create()**
- **CCpfcWireListImportInstructions.Create()**
- **CCpfcCableParamsImportInstructions.Create()**
- **CCpfcSTEPImport2DInstructions.Create()**
- **CCpfcIGESImport2DInstructions.Create()**
- **CCpfcDXFImport2DInstructions.Create()**
- **CCpfcDWGImport2DInstructions.Create()**

The methods described in this section create an instructions data object to import a file of a specified type into Creo Parametric. The details are as shown in the table below:

| Interface                             | Used to Import                                                                |
|---------------------------------------|-------------------------------------------------------------------------------|
| IpfcRelationImportInstructions        | A list of relations and parameters in a part or assembly.                     |
| IpfcIGESSectionImportInstructions     | A section model in IGES format.                                               |
| IpfcProgramImportInstructions         | A program file for a part or assembly that can be edited to change the model. |
| IpfcConfigImportInstructions          | Configuration instructions.                                                   |
| IpfcDWGSetupImportInstructions        | A drawing s/u file.                                                           |
| IpfcSpoolImportInstructions           | Spool instructions.                                                           |
| IpfcConnectorParamsImportInstructions | Connector parameter instructions.                                             |
| IpfcASSEMBTreeCFGImportInstructions   | Assembly tree CFG instructions.                                               |
| IpfcWireListImportInstructions        | Wirelist instructions.                                                        |
| IpfcCableParamsImportInstructions     | Cable parameters from an assembly.                                            |
| IpfcSTEPImport2DInstructions          | A part or assembly in STEP format.                                            |
| IpfcIGESImport2DInstructions          | A part or assembly in IGES format.                                            |
| IpfcDXFImport2DInstructions           | A drawing in DXF format.                                                      |
| IpfcDWGImport2DInstructions           | A drawing in DWG format.                                                      |

### **Note**

- The method `IpfcModel.Import()` does not support importing of CADAM type of files.
- If a model or the file type STEP, IGES, DWX, or SET already exists, the imported model is appended to the current model. For more information on methods that return models of the types STEP, IGES, DWX, and SET, refer to [Getting a Model Object on page 72](#).

---

## Importing 2D Models

Method Introduced:

- **IpfcBaseSession.Import2DModel()**

The method `IpfcBaseSession.Import2DModel()` imports a two dimensional model based on the following parameters:

- *NewModelName*—Specifies the name of the new model.
- *Type*—Specifies the type of the model. The type can be one of the following:
  - STEP
  - IGES
  - DXF
  - DWG
  - SET
- *FilePath*—Specifies the location of the file to be imported along with the file extension
- *Instructions*—Specifies the `IpfcImport2DInstructions` object that controls the import operation.

The interface `IpfcImport2DInstructions` contains the following attributes:

- *Import2DViews*—Defines whether to import 2D drawing views.
- *ScaleToFit*—If the current model has a different sheet size than that specified by the imported file, set the parameter to true to retain the current sheet size. Set the parameter to false to retain the sheet size of the imported file.
- *FitToLeftCorner*—If this parameter is set to true, the bottom left corner of the imported file is adjusted to the bottom left corner of the current model. If it is set to false, the size of imported file is retained.



### Note

The method `IpfcBaseSession.Import2DModel()` does not support importing of CADAM type of files.

---

## Importing 3D Geometry

Methods Introduced:

- **IpfcBaseSession.GetImportSourceType()**

---

- **IpfcBaseSession.ImportNewModel()**

For some input formats, the method

`IpfcBaseSession.GetImportSourceType()` returns the type of model that can be imported using a designated file. The input parameters of this method are:

- *FileToImport*—Specifies the path of the file along with its name and extension.
- *NewModelImportType*—Specifies the type of model to be imported.

The method `IpfcBaseSession.ImportNewModel()` is used to import an external 3D format file and creates a new model or set of models of type `IpfcModel`. The input parameters of this method are:

- *FileToImport*—Specifies the path to the file along with its name and extension
- *EpfcNewModelImportType*—Specifies the type of model to be imported. The types of models that can be imported are as follows:

- `EpfcIMPORT_NEW_IGES`
- `EpfcIMPORT_NEW_VDA`
- `EpfcIMPORT_NEW_NEUTRAL`
- `EpfcIMPORT_NEW_CADDs`
- `EpfcIMPORT_NEW_STEP`
- `EpfcIMPORT_NEW_STL`
- `EpfcIMPORT_NEW_VRML`
- `EpfcIMPORT_NEW_POLTXT`
- `EpfcIMPORT_NEW_CATIA_SESSION`
- `EpfcIMPORT_NEW_CATIA_MODEL`
- `EpfcIMPORT_NEW_DXF`
- `EpfcIMPORT_NEW_ACIS`
- `EpfcIMPORT_NEW_PARASOLID`
- `EpfcIMPORT_NEW_ICEM`
- `EpfcIMPORT_NEW_DESKTOP`
- `EpfcIMPORT_NEW_CATIA_PART`
- `EpfcIMPORT_NEW_CATIA_PRODUCT`
- `EpfcIMPORT_NEW_UG`
- `EpfcIMPORT_NEW_PRODUCTVIEW`
- `EpfcIMPORT_NEW_CATIA_CGR`
- `EpfcIMPORT_NEW_JT`

- 
- `EpfcIMPORT_NEW_SW_PART`
  - `EpfcIMPORT_NEW_SW_ASSEM`
  - `EpfcIMPORT_NEW_INVENTOR_PART`
  - `EpfcIMPORT_NEW_INVENTOR_ASSEM`
  - `EpfcModelType`—Specifies the type of the model. It can be a part, assembly or drawing.
  - *NewModelName*—Specifies a name for the imported model.
  - `IpfcLayerImportFilter`—Specifies the layer filter. This parameter is optional.

### Modifying the Imported Layers

Layers help you organize model items so that you can perform operations on those items collectively. These operations primarily include ways of showing the items in the model, such as displaying or blanking, selecting, and suppressing. The methods described in this section modify the attributes of the imported layers.

Methods and Properties Introduced:

- **`IpfcImportedLayer.Name`**
- **`IpfcImportedLayer.SetNewName()`**
- **`IpfcImportedLayer.SurfaceCount`**
- **`IpfcImportedLayer.CurveCount`**
- **`IpfcImportedLayer.TrimmedSurfaceCount`**
- **`IpfcImportedLayer.SetAction()`**

Layers are identified by their names. The property `IpfcImportedLayer.Name` returns the name of the layer while the method `IpfcImportedLayer.SetNewName()` can be used to set the name of the layer. The name can be numeric or alphanumeric.

The property `IpfcImportedLayer.SurfaceCount` returns the number of curves on the layer.

The property `IpfcImportedLayer.TrimmedSurfaceCount` returns the number of trimmed surfaces on the layer and the method `IpfcImportedLayer.CurveCount` returns the number of curves on the layer.

The method `IpfcImportedLayer.SetAction()` sets the display of the imported layers. The input parameter for this method is `ImportAction`. The types of actions that can be performed on the imported layers are:

- `IMPORT_LAYER_DISPLAY`—Displays the imported layer.
- `IMPORT_LAYER_SKIP`—Does not import entities on this layer.

- 
- `IMPORT_LAYER_BLANK`—Blanks the selected layer.
  - `IMPORT_LAYER_IGNORE`—Imports only entities on this layer but not the layer.

The default action type is `IMPORT_LAYER_DISPLAY`.

## Plotting Files

From Pro/ENGINEER Wildfire 5.0 onwards, the `IpfcPlotInstructions` object containing the instructions for plotting files has been deprecated. All the methods listed below for creating and accessing the instruction attributes in `IpfcPlotInstructions` have also been deprecated. Use the new interface type `IpfcPrinterInstructions` and its methods described in the next section.

Methods and Properties Deprecated:

- **`CCpfcPlotInstructions.Create()`**
- **`IpfcPlotInstructions.PlotterName`**
- **`IpfcPlotInstructions.OutputQuality`**
- **`IpfcPlotInstructions.UserScale`**
- **`IpfcPlotInstructions.PenSlew`**
- **`IpfcPlotInstructions.PenVelocityX`**
- **`IpfcPlotInstructions.PenVelocityY`**
- **`IpfcPlotInstructions.SegmentedOutput`**
- **`IpfcPlotInstructions.LabelPlot`**
- **`IpfcPlotInstructions.SeparatePlotFiles`**
- **`IpfcPlotInstructions.PaperSize`**
- **`IpfcPlotInstructions.PageRangeChoice`**
- **`IpfcPlotInstructions.PaperSizeX`**
- **`IpfcPlotInstructions.FirstPage`**
- **`IpfcPlotInstructions.LastPage`**

## Printing Files

The printer instructions for printing a file are defined in `IpfcPrinterInstructions` data object.

Methods and Properties Introduced:

- **`CCpfcPrinterInstructions.Create()`**
- **`IpfcPrinterInstructions.PrinterOption`**

- **IpfcPrinterInstructions.PlacementOption**
- **IpfcPrinterInstructions.ModelOption**
- **pfcExport.IpfcPrinterInstructions.WindowId**

The method `CCpfcPrinterInstructions.Create()` creates a new instance of the `IpfcPrinterInstructions` object. The object contains the following instruction attributes:

- *PrinterOption*—Specifies the printer settings for printing a file in terms of the `IpfcPrintPrinterOption` object. Set this attribute using the property `IpfcPrinterInstructions.PrinterOption`.
- *PlacementOption*—Specifies the placement options for printing purpose in terms of the `IpfcPrintMdlOption` object. Set this attribute using the property `IpfcPrinterInstructions.PlacementOption`.
- *ModelOption*—Specifies the model options for printing purpose in terms of the `IpfcPrintPlacementOption` object. Set this attribute using the property `IpfcPrinterInstructions.ModelOption`.
- *WindowId*—Specifies the current window identifier. Set this attribute using the property `IpfcPrinterInstructions.WindowId`.

## Printer Options

The printer settings for printing a file are defined in the `IpfcPrintPrinterOption` object.

Methods and Properties Introduced:

- **CCpfcPrintPrinterOption.Create()**
- **IpfcBaseSession.GetPrintPrinterOptions()**
- **IpfcPrintPrinterOption.DeleteAfter**
- **IpfcPrintPrinterOption.FileName**
- **IpfcPrintPrinterOption.PaperSize**
- **CCpfcPrintSize.Create()**
- **IpfcPrintSize.Height**
- **IpfcPrintSize.Width**
- **IpfcPrintSize.PaperSize**
- **IpfcPrintPrinterOption.PenTable**
- **IpfcPrintPrinterOption.PrintCommand**
- **IpfcPrintPrinterOption.PrinterType**
- **IpfcPrintPrinterOption.Quantity**
- **IpfcPrintPrinterOption.RollMedia**

- 
- **IpfcPrintPrinterOption.RotatePlot**
  - **IpfcPrintPrinterOption.SaveMethod**
  - **IpfcPrintPrinterOption.SaveToFile**
  - **IpfcPrintPrinterOption.SendToPrinter**
  - **IpfcPrintPrinterOption.Slew**
  - **IpfcPrintPrinterOption.SwHandshake**
  - **IpfcPrintPrinterOption.UseTtf**

The method `CCpfcPrintPrinterOption.Create()` creates a new instance of the `IpfcPrintPrinterOption` object.

The method `IpfcBaseSession.GetPrintPrinterOptions()` retrieves the printer settings.

The `IpfcPrintPrinterOption` object contains the following options:

- *DeleteAfter*—Determines if the file is deleted after printing. Set it to true to delete the file after printing. Use the property `IpfcPrintPrinterOption.DeleteAfter` to assign this option.
- *FileName*—Specifies the name of the file to be printed. Use the property `IpfcPrintPrinterOption.FileName` to set the name.

---

#### **Note**

If the method `IpfcModel.Export()` is called for `IpfcExportType` object, then the argument *FileName* is ignored, and can be passed as NULL. You must use the method `IpfcModel.Export()` to set the *FileName*.

---

- *PaperSize*—Specifies the parameters of the paper to be printed in terms of the `IpfcPrintSize` object. The property `IpfcPrintPrinterOption.PaperSize` assigns the *PaperSize* option. Use the method `CCpfcPrintSize.Create()` to create a new instance of the `IpfcPrintSize` object. This object contains the following options:
  - *Height*—Specifies the height of paper. Use the property `IpfcPrintSize.Height` to set the paper height.
  - *Width*—Specifies the width of paper. Use the property `IpfcPrintSize.Width` to set the paper width.
  - *PaperSize*—Specifies the size of the paper used for the plot in terms of the `EpfcPlotPaperSize` object. Use the property `IpfcPrintSize.PaperSize` to set the paper size.



---

 **Note**

If you want to plot a layout without adding a border on the paper, use the following paper sizes defined in the enumerated data type `EpfcPlotPaperSize`:

- ◆ `EpfcCEEMPTYPLOT`—The paper size is 22.5 x 36 in
  - ◆ `EpfcCEEMPTYPLOT_MM`—The paper size is 625 x 1000 mm
- 

- *PenTable*—Specifies the file containing the pen table. Use the property `IpfcPrintPrinterOption.PenTable` to set this option.
- *PrintCommand*—Specifies the command to be used for printing. Use the property `IpfcPrintPrinterOption.PrintCommand` to set the command.
- *PrinterType*—Specifies the printer type. Use the property `IpfcPrintPrinterOption.PrinterType` to assign the type.
- *Quantity*—Specifies the number of copies to be printed. Use the property `IpfcPrintPrinterOption.Quantity` to assign the quantity.
- *RollMedia*—Determines if roll media is to be used for printing. Set it to true to use roll media. Use the property `IpfcPrintPrinterOption.RollMedia` to assign this option.
- *RotatePlot*—Determines if the plot is rotated by 90 degrees. Set it to true to rotate the plot. Use the property `IpfcPrintPrinterOption.SetRotatePlot` to set this option.
- *SaveMethod*—Specifies the save method in terms of the `EpfcPrintSaveMethod` enumerated class. Use the property `IpfcPrintPrinterOption.SaveMethod` to specify the save method. The available methods are as follows:
  - `EpfcPRINT_SAVE_SINGLE_FILE`—Plot is saved to a single file.
  - `EpfcPRINT_SAVE_MULTIPLE_FILE`—Plot is saved to multiple files.
  - `EpfcPRINT_SAVE_APPEND_TO_FILE`—Plot is appended to a file.
- *SaveToFile*—Determines if the file is saved after printing. Set it to true to save the file after printing. Use the property `IpfcPrintPrinterOption.SaveToFile` to assign this option.
- *SendToPrinter*—Determines if the plot is directly sent to the printer. Set it to true to send the plot to the printer. Use the property `IpfcPrintPrinterOption.SendToPrinter` to set this option.

- *Slew*—Specifies the speed of the pen in centimeters per second in X and Y direction. Use the property `IpfcPrintPrinterOption.Slew` to set this option.
- *SwHandshake*—Determines if the software handshake method is to be used for printing. Set it to true to use the software handshake method. Use the property `IpfcPrintPrinterOption.SwHandshake` to set this option.
- *UseTtf*—Specifies whether TrueType fonts or stroked text is used for printing. Set this option to true to use TrueType fonts and to false to stroke all text. Use the property `IpfcPrintPrinterOption.UseTtf` to set this option.

## Placement Options

The placement options for printing purpose are defined in the `IpfcPrintPlacementOption` object.

Methods and Properties Introduced:

- **`CCpfcPrintPlacementOption.Create()`**
- **`IpfcBaseSession.GetPrintPlacementOptions()`**
- **`IpfcPrintPlacementOption.BottomOffset`**
- **`IpfcPrintPlacementOption.ClipPlot`**
- **`IpfcPrintPlacementOption.KeepPanzoom`**
- **`IpfcPrintPlacementOption.LabelHeight`**
- **`IpfcPrintPlacementOption.PlaceLabel`**
- **`IpfcPrintPlacementOption.Scale`**
- **`IpfcPrintPlacementOption.ShiftAllCorner`**
- **`IpfcPrintPlacementOption.SideOffset`**
- **`IpfcPrintPlacementOption.X1ClipPosition`**
- **`IpfcPrintPlacementOption.X2ClipPosition`**
- **`IpfcPrintPlacementOption.Y1ClipPosition`**
- **`IpfcPrintPlacementOption.Y2ClipPosition`**

The method `CCpfcPrintPlacementOption.Create()` creates a new instance of the `IpfcPrintPlacementOption` object.

The method `IpfcBaseSession.GetPrintPlacementOptions()` retrieves the placement options.

The `IpfcPrintPlacementOption` object contains the following options:

- 
- *BottomOffset*—Specifies the offset from the lower-left corner of the plot. Use the property `IpfcPrintPlacementOption.BottomOffset` to set this option.
  - *ClipPlot*—Specifies whether the plot is clipped. Set this option to true to clip the plot or to false to avoid clipping of plot. Use the property `IpfcPrintPlacementOption.ClipPlot` to set this option.
  - *KeepPanzoom*—Determines whether pan and zoom values of the window are used. Set this option to true use pan and zoom and false to skip them. Use the property `IpfcPrintPlacementOption.KeepPanzoom` to set this option.
  - *LabelHeight*—Specifies the height of the label in inches. Use the property `IpfcPrintPlacementOption.LabelHeight` to set this option.
  - *PlaceLabel*—Specifies whether you want to place the label on the plot. Use the property `IpfcPrintPlacementOption.PlaceLabel` to set this option.
  - *Scale*—Specifies the scale used for the plot. Use the property `IpfcPrintPlacementOption.Scale` to set this option.
  - *ShiftAllCorner*—Determines whether all corners are shifted. Set this option to true to shift all corners or to false to skip shifting of corners. Use the property `IpfcPrintPlacementOption.ShiftAllCorner` to set this option.
  - *SideOffset*—Specifies the offset from the sides. Use the property `IpfcPrintPlacementOption.SideOffset` to set this option.
  - *X1ClipPosition*—Specifies the first X parameter for defining the clip position. Use the property `IpfcPrintPlacementOption.X1ClipPosition` to set this option.
  - *X2ClipPosition*—Specifies the second X parameter for defining the clip position. Use the property `IpfcPrintPlacementOption.X2ClipPosition` to set this option.
  - *Y1ClipPosition*—Specifies the first Y parameter for defining the clip position. Use the property `IpfcPrintPlacementOption.Y1ClipPosition` to set this option.
  - *Y2ClipPosition*—Specifies the second Y parameter for defining the clip position. Use the property `IpfcPrintPlacementOption.Y2ClipPosition` to set this option.

## Model Options

The model options for printing purpose are defined in the `IpfcPrintMdlOption` object.

Methods and Properties Introduced:

- 
- **CCpfcPrintMdlOption.Create()**
  - **IpfcBaseSession.GetPrintMdlOptions()**
  - **IpfcPrintMdlOption.DrawFormat**
  - **IpfcPrintMdlOption.FirstPage**
  - **IpfcPrintMdlOption.LastPage**
  - **IpfcPrintMdlOption.LayerName**
  - **IpfcPrintMdlOption.LayerOnly**
  - **IpfcPrintMdlOption.Mdl**
  - **IpfcPrintMdlOption.Quality**
  - **IpfcPrintMdlOption.Segmented**
  - **IpfcPrintMdlOption.Sheets**
  - **IpfcPrintMdlOption.UseDrawingSize**
  - **IpfcPrintMdlOption.UseSolidScale**

The method `CCpfcPrintMdlOption.Create()` creates a new instance of the `IpfcPrintMdlOption` object.

The method `IpfcBaseSession.GetPrintMdlOptions()` retrieves the model options.

The `IpfcPrintMdlOption` object contains the following options:

- *DrawFormat*—Displays the drawing format used for printing. Use the property `IpfcPrintMdlOption.DrawFormat` to set this option.
- *FirstPage*—Specifies the first page number. Use the property `IpfcPrintMdlOption.FirstPage` to set this option.
- *LastPage*—Specifies the last page number. Use the property `IpfcPrintMdlOption.LastPage` to set this option.
- *LayerName*—Specifies the name of the layer. Use the property `IpfcPrintMdlOption.LayerName` to set the name.
- *LayerOnly*—Prints the specified layer only. Set this option to `true` to print the specified layer. Use the property `IpfcPrintMdlOption.LayerOnly` to set this option.
- *Mdl*—Specifies the model to be printed. Use the property `IpfcPrintMdlOption.Mdl` to set this option.
- *Quality*—Determines the quality of the model to be printed. It checks for no line, no overlap, simple overlap, and complex overlap. Use the property `IpfcPrintMdlOption.Quality` to set this option.
- *Segmented*—If set to `true`, the printer prints the drawing in full size, but in segments that are compatible with the selected paper size. This option is

---

available only if you are plotting a single page. Use the property `IpfcPrintMdlOption.Segmented` to set this option.

- *Sheets*—Specifies the sheets that need to be printed in terms of the `EpfcPrintSheets` class. Use the property `IpfcPrintMdlOption.Sheets` to specify the sheets. The sheets can be of the following types:
  - `EpfcPRINT_CURRENT_SHEET`—Only the current sheet is printed.
  - `EpfcPRINT_ALL_SHEETS`—All the sheets are printed.
  - `EpfcPRINT_SELECTED_SHEETS`—Sheets of a specified range are printed.
- *UseDrawingSize*—Overrides the paper size specified in the printer options with the drawing size. Set this option to `true` to use the drawing size. Use the property `IpfcPrintMdlOption.UseDrawingSize` to set this option.
- *UseSolidScale*—Prints with the scale used in the solid model. Set this option to `true` to use solid scale. Use the property `IpfcPrintMdlOption.UseSolidScale` to set this option.

### Plotter Configuration File (PCF) Options

The printing options for PCF file are defined in the `IpfcPrinterPCFOptions` object.

Methods and Properties Introduced:

- **`CCpfcPrinterPCFOptions.Create()`**
- **`IpfcPrinterPCFOptions.PrinterOption`**
- **`IpfcPrinterPCFOptions.PlacementOption`**
- **`IpfcPrinterPCFOptions.ModelOption`**

The method `CCpfcPrinterPCFOptions.Create()` creates a new instance of the `IpfcPrinterPCFOptions` object.

The `IpfcPrinterPCFOptions` object contains the following options:

- *PrinterOption*—Specifies the printer settings for printing a file in terms of the `IpfcPrintPrinterOption` object. Set this attribute using the property `IpfcPrinterPCFOptions.PrinterOption`.
- *PlacementOption*—Specifies the placement options for printing purpose in terms of the `IpfcPrintMdlOption` object. Set this attribute using the property `IpfcPrinterPCFOptions.SetPlacementOption`.
- *ModelOption*—Specifies the model options for printing purpose in terms of the `IpfcPrintPlacementOption` object. Set this attribute using the property `IpfcPrinterPCFOptions.ModelOption`.

---

## Solid Operations

Method Introduced:

- **IpfcSolid.CreateImportFeat()**

The method `IpfcSolid.CreateImportFeat()` creates a new import feature in the solid and takes the following input arguments:

- *IntfData*—The source of data from which to create the import feature. It is given by the `IpfcIntfDataSource` object. The type of source data that can be imported is given by the `EpfcIntfType` class and can be of the following types:
  - `EpfcINTF_NEUTRAL`
  - `EpfcINTF_NEUTRAL_FILE`
  - `EpfcINTF_IGES`
  - `EpfcINTF_STEP`
  - `EpfcINTF_VDA`
  - `EpfcINTF_ICEM`
  - `EpfcINTF_ACIS`
  - `EpfcINTF_DXF`
  - `EpfcINTF_CDRS`
  - `EpfcINTF_STL`
  - `EpfcINTF_VRML`
  - `EpfcINTF_PARASOLID`
  - `EpfcINTF_AI`
  - `EpfcINTF_CATIA_PART`
  - `EpfcINTF_UG`
  - `EpfcINTF_PRODUCTVIEW`
  - `EpfcINTF_CATIA_CGR`
  - `EpfcINTF_JT`
- *CoordSys*—The pointer to a reference coordinate system. If this is NULL, the function uses the default coordinate system.
- *FeatAttr*—The attributes for creation of the new import feature given by the `IpfcImportFeatAttr` object. If this pointer is NULL, the function uses the default attributes.

---

### Example Code: Returning a Feature Object

The sample code in the file `pfcInterfaceExamples.vb` located at `<creo_vbapi_loadpoint>/vbapi_examples` return a feature object when provided with a solid coordinate system name and an import feature's file name. The method will find the coordinate system in the model, set the Import Feature Attributes, and create an import feature. The feature is then returned.

## Window Operations

Method Introduced:

- **`IpfcWindow.ExportRasterImage()`**

The method `IpfcWindow.ExportRasterImage()` outputs a standard Creo Parametric raster output file.

### Example Code: Generating Raster Files

The sample code in the file `pfcInterfaceExamples.vb` located at `<creo_vbapi_loadpoint>/vbapi_examples` generates raster image files using a specified window and file type.

# Simplified Representations

## Simplified Representations

The VB API gives programmatic access to all the simplified representation functionality of Creo Parametric. Create simplified representations either permanently or on the fly and save, retrieve, or modify them by adding or deleting items.

### Overview

Using the VB API, you can create and manipulate assembly simplified representations just as you can using Creo Parametric interactively.



#### Note

the VB API supports simplified representation of assemblies only, not parts.

---

Simplified representations are identified by the `IpfcSimRep` class. This class is a child of `IpfcModelItem`, so you can use the methods dealing with `IpfcModelItems` to collect, inspect, and modify simplified representations.

The information required to create and modify a simplified representation is stored in a class called `IpfcSimpRepInstructions` which contains several data objects and fields, including:

- `String`—The name of the simplified representation
- `IpfcSimpRepAction`—The rule that controls the default treatment of items in the simplified representation.
- `IpfcSimpRepItem`—An array of assembly components and the actions applied to them in the simplified representation.

A `IpfcSimpRepItem` is identified by the assembly component path to that item. Each `IpfcSimpRepItem` has its own `IpfcSimpRepAction` assigned to it. `IpfcSimpRepAction` is a visible data object that includes a field of type `IpfcSimpRepActionType`. You can use the method `IpfcSimpRepAction.GetType()` to set the actions. To delete an existing item, you must set the action as `NULL`.

`EpfcSimpRepActionType` is an enumerated type that specifies the possible treatment of items in a simplified representation. The possible values are as follows

| Values                               | Action                                                                                                |
|--------------------------------------|-------------------------------------------------------------------------------------------------------|
| <code>EpfcSIMPREP_NONE</code>        | No action is specified.                                                                               |
| <code>EpfcSIMPREP_REVERSE</code>     | Reverse the default rule for this component (for example, include it if the default rule is exclude). |
| <code>EpfcSIMPREP_INCLUDE</code>     | Include this component in the simplified representation.                                              |
| <code>EpfcSIMPREP_EXCLUDE</code>     | Exclude this component from the simplified representation.                                            |
| <code>EpfcSIMPREP_SUBSTITUTE</code>  | Substitute the component in the simplified representation.                                            |
| <code>EpfcSIMPREP_GEOM</code>        | Use only the geometrical representation of the component.                                             |
| <code>EpfcSIMPREP_GRAPHICS</code>    | Use only the graphics representation of the component.                                                |
| <code>EpfcSIMPREP_SYMB</code>        | Use the symbolic representation of the component.                                                     |
| <code>EpfcSIMPREP_BOUNDBOX</code>    | Use the boundary box representation of the component.                                                 |
| <code>EpfcSIMPREP_DEFENV</code>      | Use the default envelope representation of the component.                                             |
| <code>EpfcSIMPREP_LIGHT_GRAPH</code> | Use the light weight graphics representation of the component.                                        |
| <code>EpfcSIMPREP_AUTO</code>        | Use the automatic representation of the component.                                                    |

## Retrieving Simplified Representations

Methods Introduced:

- **`IpfcBaseSession.RetrieveAssemSimpRep()`**



- **IpfcBaseSession.RetrieveGeomSimpRep()**
- **IpfcBaseSession.RetrieveGraphicsSimpRep()**
- **IpfcBaseSession.RetrieveSymbolicSimpRep()**
- **CCpfcRetrieveExistingSimpRepInstructions.Create()**

You can retrieve a named simplified representation from a model using the method `IpfcBaseSession.RetrieveAssemSimpRep()`, which is analogous to the Assembly mode option **Retrieve Rep** in the **SIMPLFD REP** menu. This method retrieves the object of an existing simplified representation from an assembly without fetching the generic representation into memory. The method takes two arguments, the name of the assembly and the simplified representation data.

To retrieve an existing simplified representation, pass an instance of `CCpfcRetrieveExistingSimpRepInstructions.Create()` and specify its name as the second argument to this method. Creo Parametric retrieves that representation and any active submodels and returns the object to the simplified representation as a `IpfcAssembly.Assembly` object.

You can retrieve geometry, graphics, and symbolic simplified representations into session using the methods `IpfcBaseSession.RetrieveGeomSimpRep()`, `IpfcBaseSession.RetrieveGraphicsSimpRep()`, and `IpfcBaseSession.RetrieveSymbolicSimpRep()` respectively. Like `IpfcBaseSession.RetrieveAssemSimpRep()`, these methods retrieve the simplified representation without bringing the master representation into memory. Supply the name of the assembly whose simplified representation is to be retrieved as the input parameter for these methods. The methods output the assembly. They do not display the simplified representation.

## Creating and Deleting Simplified Representations

Methods Introduced:

- **CCpfcCreateNewSimpRepInstructions.Create()**
- **IpfcSolid.CreateSimpRep()**
- **IpfcSolid.DeleteSimpRep()**

To create a simplified representation, you must allocate and fill a `IpfcSimpRepInstructions` object by calling the method `CCpfcCreateNewSimpRepInstructions.Create()`. Specify the name of the new simplified representation as an input to this method. You should also set the default action type and add `SimpRepItems` to the object.

To generate the new simplified representation, call `IpfcSolid.CreateSimpRep()`. This method returns the `IpfcSimpRep` object for the new representation.

---

The method `IpfcSolid.DeleteSimpRep()` deletes a simplified representation from its model owner. The method requires only the `IpfcSimpRep` object as input.

## Extracting Information About Simplified Representations

Methods and Properties Introduced:

- **`IpfcSimpRep.GetInstructions()`**
- **`IpfcSimpRepInstructions.DefaultAction()`**
- **`IpfcCreateNewSimpRepInstructions.NewSimpName`**
- **`IpfcSimpRepInstructions.IsTemporary`**
- **`IpfcSimpRepInstructions.Items`**

Given the object to a simplified representation, `IpfcSimpRep.GetInstructions()` fills out the `IpfcSimpRepInstructions` object.

The properties `IpfcSimpRepInstructions.DefaultAction`, `IpfcCreateNewSimpRepInstructions.NewSimpName`, and `IpfcSimpRepInstructions.IsTemporary` return the associated values contained in the `IpfcSimpRepInstructions` object.

The property `IpfcSimpRepInstructions.Items` returns all the items that make up the simplified representation.

### Example 1: Working with Simplified Representation

The sample code in the file `pfcsimplifiedrepresentationexamples.vb` located at `<creo_vbapi_loadpoint>/vbapi_examples` demonstrates the functionality used when working with existing simplified representations in Creo Parametric. This function `matchSimpRepItem` returns an array of simplified representation matching a `ComponentPath` for a certain feature as well as the `IpfcSimpRepActionType` for that item's action in the representation. If none are found the method prints the `<NOT FOUND>` message and returns null.

## Modifying Simplified Representations

Methods and Properties Introduced:

- **`IpfcSimpRep.GetInstructions()`**
- **`IpfcSimpRep.SetInstructions()`**
- **`IpfcSimpRepInstructions.DefaultAction`**
- **`IpfcCreateNewSimpRepInstructions.NewSimpName`**
- **`IpfcSimpRepInstructions.IsTemporary`**

---

Using the VB API, you can modify the attributes of existing simplified representations. After you create or retrieve a simplified representation, you can make calls to the methods listed in this section to designate new values for the fields in the `IpfcSimpRepInstructions` object.

To modify an existing simplified representation retrieve it and then get the `IpfcSimpRepInstructions` object by calling `IpfcSimpRep.GetInstructions()`. If you created the representation programmatically within the same application, the `IpfcSimpRepInstructions` object is already available. Once you have modified the data object, reassign it to the corresponding simplified representation by calling the method `IpfcSimpRep.SetInstructions()`.

## **Adding Items to and Deleting Items from a Simplified Representation**

Methods and Properties Introduced:

- **`CCpfcSimpRepReverse.Create()`**
- **`CCpfcSimpRepInclude.Create()`**
- **`CCpfcSimpRepExclude.Create()`**
- **`CCpfcSimpRepSubstitute.Create()`**
- **`CCpfcSimpRepGeom.Create()`**
- **`CCpfcSimpRepGraphics.Create()`**
- **`CCpfcSimpRepDefaultEnvelope.Create()`**
- **`CCpfcSimpRepBoundingBox.Create()`**
- **`CCpfcSimpRepLightWeightGraphics.Create()`**
- **`CCpfcSimpRepItem.Create()`**
- **`CCpfcSimpRep.SetInstructions()`**
- **`CCpfcSimpRepInstructions.Items()`**

The method `CCpfcSimpRepReverse.Create()` creates an object that reverses the default rule for the assembly component in the simplified representation.

The method `CCpfcSimpRepInclude.Create()` creates an object that includes the component in the simplified representation.

The method `CCpfcSimpRepExclude.Create()` creates an object that excludes the component from the simplified representation.

The method `CCpfcSimpRepSubstitute.Create()` creates an object that substitutes the component with a component from the `IpfcSubstitution` class in the simplified representation.

---

The method `CCpfcSimpRepGeom.Create()` creates an object that represents the component that contains complete geometric information in the simplified representation.

The method `CCpfcSimpRepGraphics.Create()` creates an object that represents the component that contains only display information in the simplified representation. These graphic representations cannot be modified or referenced, they can only be browsed through.

---

 **Note**

As compared to graphics representations, geometry representations take longer to retrieve and require more memory.

---

The method `CCpfcSimpRepDefaultEnvelope.Create()` creates an object that represents the component in default envelope type of simplified representation.

The method `CCpfcSimpRepBoundingBox.Create()` creates an object that represents the component in boundary box type of simplified representation.

The method `CCpfcSimpRepLightWeightGraphics.Create()` creates an object that represents the component in lightweight type of simplified representation. Light graphics representations of assemblies contain assembly information and 3D thumbnail graphics representations of assembly components.

The method `CCpfcSimpRepItem.Create()` creates an object that defines the status of the component or feature of the simplified representation.

The method `CCpfcSimpRep.SetInstructions()` sets or changes the instructions that specify the internal data required to define the simplified representation.

The method `CCpfcSimpRepInstructions.Items` applies an array of actions to the components or features in the simplified representation.

You can add and delete items from the list of components in a simplified representation using the VB API. If you created a simplified representation using the option **Exclude** as the default rule, you would generate a list containing the items you want to include. Similarly, if the default rule for a simplified representation is **Include**, you can add the items that you want to be excluded from the simplified representation to the list, setting the value of the `EpfcSimpRepActionType` to `EpfcSIMPREP_EXCLUDE`.

## How to Add Items

1. Get the `IpfcSimpRepInstructions` object, as described in the previous section.

- 
2. Specify the action to be applied to the item with a call to one of following methods.
  3. Initialize a `IpfcSimpRepItem` object for the item by calling the method `IpfcSimpRepItem.Create()`.
  4. Add the item to the `IpfcSimpRepItem` sequence. Put the new `IpfcSimpRepInstructions` using `IpfcSimpRepInstructions.Items`.
  5. Reassign the `IpfcSimpRepInstructions` object to the corresponding `IpfcSimpRep` object by calling `IpfcSimpRep.SetInstructions()`

#### How to Remove Items

Follow the procedure above, except remove the unwanted `IpfcSimpRepItem` from the sequence.

## Simplified Representation Utilities

Methods Introduced:

- **`IpfcModelItemOwner.ListItems()`**
- **`IpfcModelItemOwner.GetItemById()`**
- **`IpfcSolid.GetSimpRep()`**
- **`IpfcSolid.SelectSimpRep()`**
- **`IpfcSolid.ActivateSimpRep()`**
- **`IpfcSolid.GetActiveSimpRep()`**

This section describes the utility methods that relate to simplified representations.

The method `IpfcModelItemOwner.ListItems()` can list all of the simplified representations in a `Solid`.

The method `IpfcModelItemOwner.GetItemById()` initializes a `IpfcSimpRep` object. It takes an integer `id`.

---

#### **Note**

the VB API supports simplified representation of Assemblies only, not Parts.

---

The method `IpfcSolid.GetSimpRep()` initializes a `IpfcSimpRep` object. The method takes *SimpRepname* as the input argument. *SimpRepname* is the name of the simplified representation in the solid. If you specify this argument, the method ignores the *rep\_id*.

---

The method `IpfcSolid.SelectSimpRep()` creates a Creo Parametric menu to enable interactive selection. The method takes the owning solid as input, and outputs the object to the selected simplified representation. If you choose the **Quit** menu button, the method throws an exception `XToolkitUserAbort`

The methods `IpfcSolid.GetActiveSimpRep()` and `IpfcSolid.ActivateSimpRep()` enable you to find and get the currently active simplified representation, respectively. Given an assembly object, `IpfcSolid.GetActiveSimpRep()` returns the object to the currently active simplified representation. If the current representation is the master representation, the return is null.

The method `IpfcSolid.ActivateSimpRep()` activates the requested simplified representation.

To set a simplified representation to be the currently displayed model, you must also call `IpfcModel.Display()`.

## Task Based Application Libraries

### Task Based Application Libraries

Applications created using different Creo Parametric API products are interoperable. These products use Creo Parametric as the medium of interaction, eliminating the task of writing native-platform specific interactions between different programming languages.

Application interoperability allows the VB API applications to call into Creo TOOLKIT from areas not covered in the native interface. It allows you to put a VBA or VB.NET front end on legacy Creo TOOLKIT applications, and also allows you to use applications and listeners in conjunction with a or asynchronous application.

### Managing Application Arguments

The VB API passes application data to and from tasks in other applications as members of a sequence of `IpfcArgument` objects. Application arguments consist of a label and a value. The value may be of any one of the following types:

- Integer
- Double
- Boolean
- ASCII string (a non-encoded string, provided for compatibility with arguments provided from C applications)
- String (a fully encoded string)

- `IpfcSelection` (a selection of an item in a Creo Parametric session)
- `IpfcTransform3D` (a coordinate system transformation matrix)

Methods and Properties Introduced:

- **`CMpfcArgument.CreateIntArgValue()`**
- **`CMpfcArgument.CreateDoubleArgValue()`**
- **`CMpfcArgument.CreateBoolArgValue()`**
- **`CMpfcArgument.CreateASCIIStringArgValue()`**
- **`CMpfcArgument.CreateStringArgValue()`**
- **`CMpfcArgument.CreateSelectionArgValue()`**
- **`CMpfcArgument.CreateTransformArgValue()`**
- **`IpfcArgValue.dscr`**
- **`IpfcArgValue.IntValue`**
- **`IpfcArgValue.DoubleValue`**
- **`IpfcArgValue.BoolValue`**
- **`IpfcArgValue.ASCIIStringValue`**
- **`IpfcArgValue.StringValue`**
- **`IpfcArgValue.SelectionValue`**
- **`IpfcArgValue.TransformValue`**

The class `IpfcArgValue` contains one of the seven types of values. the VB API provides different methods to create each of the seven types of argument values.

The property `IpfcArgValue.dscr` returns the type of value contained in the argument value object.

Use the methods listed above to access and modify the argument values.

## Modifying Arguments

Methods and Properties Introduced:

- **`CCpfcArgument.Create()`**
- **`IpfcpfcArgument.Label`**
- **`IpfcpfcArgument.Value`**

The method `CCpfcArgument.Create()` creates a new argument. Provide a name and value as the input arguments of this method.

The property `IpfcpfcArgument.Label` returns the label of the argument.

The property `IpfcpfcArgument.Value` returns the value of the argument.

---

## Launching a Creo TOOLKIT DLL

The methods described in this section enable a the VB API user to register and launch a Creo TOOLKIT DLL from an application. The ability to launch and control a Creo TOOLKIT application enables the following:

- Reuse of existing Creo TOOLKIT code with the VB API applications.
- ATB operations.

Methods and Properties Introduced:

- **IpfcBaseSession.LoadProToolkitDll()**
- **IpfcBaseSession.LoadProToolkitLegacyDll()**
- **IpfcBaseSession.GetProToolkitDll()**
- **IpfcDll.ExecuteFunction()**
- **IpfcDll.Id**
- **IpfcDll.IsActive()**
- **IpfcDll.Unload()**

Use the method `IpfcBaseSession.LoadProToolkitDll()` to register and start a Creo TOOLKIT DLL. The input parameters of this method are similar to the fields of a registry file and are as follows:

- *ApplicationName*—The name of the application to initialize.
- *DllPath*—The full path to the DLL binary file.
- *TextPath*—The path to the application's message and user interface text files.
- *UserDisplay*—Set this parameter to `true` to register the application in the Creo Parametric user interface and to see error messages if the application fails. If this parameter is `false`, the application will be invisible to the user.

The application's `user_initialize()` function is called when the application is started. The method returns a handle to the loaded Creo TOOLKIT DLL.

In order to register and start a legacy Pro/TOOLKIT DLL that is not Unicode-compliant, use the method

`IpfcBaseSession.LoadProToolkitLegacyDll()`. This method conveys to Creo Parametric that the loaded DLL application is not Unicode-compliant and built in the pre-Wildfire 4.0 environment. It takes the same input parameters as the earlier method

`IpfcBaseSession.LoadProToolkitDll()`.



---

## Note

The method `IpfcBaseSession.LoadProToolkitLegacyDll()` must be used only by a pre-Wildfire 4.0 application to load a pre-Wildfire 4.0 Pro/TOOLKIT DLL.

---

Use the method `IpfcBaseSession.GetProToolkitDll()` to obtain a Creo TOOLKIT DLL handle. Specify the *Application\_Id*, that is, the DLL's identifier string as the input parameter of this method. The method returns the DLL object or null if the DLL was not in session. The *Application\_Id* can be determined as follows:

- Use the function `ProToolkitDllIdGet()` within the DLL application to get a string representation of the DLL application. Pass `NULL` to the first argument of `ProToolkitDllIdGet()` to get the string identifier for the calling application.
- Use the `Get` method for the `Id` attribute in the DLL interface. The method `IpfcDll.Id` returns the DLL identifier string.

Use the method `IpfcDll.ExecuteFunction()` to call a properly designated function in the Creo TOOLKIT DLL library. The input parameters of this method are:

- *FunctionName*—Name of the function in the Creo TOOLKIT DLL application.
- *InputArguments*—Input arguments to be passed to the library function.

The method returns an object of `IpfcFunctionReturn`. This interface contains data returned by a Creo TOOLKIT function call. The object contains the return value, as integer, of the executed function and the output arguments passed back from the function call.

The method `IpfcDll.IsActive()` determines whether a Creo TOOLKIT DLL previously loaded by the method `IpfcBaseSession.LoadProToolkitDll()` is still active.

The method `IpfcDll.Unload()` is used to shutdown a Creo TOOLKIT DLL previously loaded by the method `IpfcBaseSession.LoadProToolkitDll()` and the application's `user_terminate()` function is called.

## Launching Tasks from J-Link Task Libraries

The methods described in this section allow you to launch tasks from a predefined J-Link task library.

Methods Introduced:

- **IpfcBaseSession.StartJLinkApplication()**
- **IpfcJLinkApplication.ExecuteTask()**
- **IpfcJLinkApplication.IsActive()**
- **IpfcJLinkApplication.Stop()**

Use the method `IpfcBaseSession.StartJLinkApplication()` to start a application. The input parameters of this method are similar to the fields of a registry file and are as follows:

- *ApplicationName*—Assigns a unique name to this J-Link application.
- *ClassName*—Specifies the name of the Java class that contains the application's start and stop method. This should be a fully qualified Java package and class name.
- *StartMethod*—Specifies the start method of the J-Link application.
- *StopMethod*—Specifies the stop method of the J-Link application.
- *AdditionalClassPath*—Specifies the locations of packages and classes that must be loaded when starting this J-Link application. If this parameter is specified as null, the default classpath locations are used.
- *TextPath*—Specifies the application text path for menus and messages. If this parameter is specified as null, the default text locations are used.
- *UserDisplay*—Specifies whether to display the application in the **Auxiliary Applications** dialog box in Creo Parametric.

Upon starting the application, the static `start()` method is invoked. The method returns a `IpfcJLinkApplication` referring to the J-Link application.

The method `IpfcJLinkApplication.ExecuteTask()` calls a registered task method in the J-Link application. The input parameters of this method are:

- Name of the task to be executed.
- A sequence of name value pair arguments contained by the interface `IpfcArguments`.

The method outputs an array of output arguments.

The method `JLinkApplication.IsActive` returns a `True` value if the application specified by the `JLink.IpfcJLinkApplication` object is active.

The method `IpfcJLinkApplication.Stop()` stops the application specified by the `IpfcJLinkApplication` object. This method activates the application's static `Stop()` method.

---

# Graphics

## Graphics

This section covers the VB API Graphics including displaying lists, displaying text and using the mouse.

### Overview

The methods described in this section allow you to draw temporary graphics in a display window. Methods that are identified as 2D are used to draw entities (arcs, polygons, and text) in screen coordinates. Other entities may be drawn using the current model's coordinate system or the screen coordinate system's lines, circles, and polylines. Methods are also included for manipulating text properties and accessing mouse inputs.

### Getting Mouse Input

The following methods are used to read the mouse position in screen coordinates with the mouse button depressed. Each method outputs the position and an enumerated type description of which mouse button was pressed when the mouse was at that position. These values are contained in the interface `IpfcMouseStatus`.

The enumerated values are defined in `EpfcMouseButton` and are as follows:

- `EpfcMOUSE_BTN_LEFT`
- `EpfcMOUSE_BTN_RIGHT`
- `EpfcMOUSE_BTN_MIDDLE`
- `EpfcMOUSE_BTN_LEFT_DOUBLECLICK`

Methods Introduced:

- **`IpfcSession.UIGetNextMousePick()`**
- **`IpfcSession.UIGetCurrentMouseStatus()`**

The method `IpfcSession.UIGetNextMousePick()` returns the mouse position when you press a mouse button. The input argument is the mouse button that you expect the user to select.

The method `IpfcSession.UIGetCurrentMouseStatus()` returns a value whenever the mouse is moved or a button is pressed. With this method a button does not have to be pressed for a value to be returned. You can use an input argument to flag whether or not the returned positions are snapped to the window grid.

---

## Drawing a Mouse Box

This method allows you to draw a mouse box.

Method Introduced:

- **`IpfcSession.UIPickMouseBox()`**

The method `IpfcSession.UIPickMouseBox()` draws a dynamic rectangle from a specified point in screen coordinates to the current mouse position until the user presses the left mouse button. The return value for this method is of the type `IpfcOutline3D`.

You can supply the first corner location programmatically or you can allow the user to select both corners of the box.

## Displaying Graphics

All the methods in this section draw graphics in the Creo Parametric current window and use the color and linestyle set by calls to `IpfcBaseSession.SetStdColorFromRGB()` and `IpfcBaseSession.SetLineStyle()`. The methods draw the graphics in the Creo Parametric graphics color. The default graphics color is white.

The methods in this section are called using the interface `IpfcDisplay`. This interface is extended by the `IpfcBaseSession` interface. This architecture allows you to call all these methods on any `IpfcSession` object.

Methods Introduced:

- **`IpfcDisplay.SetPenPosition()`**
- **`IpfcDisplay.DrawLine()`**
- **`IpfcDisplay.DrawPolyline()`**
- **`IpfcDisplay.DrawCircle()`**
- **`IpfcDisplay.DrawArc2D()`**
- **`IpfcDisplay.DrawPolygon2D()`**

The method `IpfcDisplay.SetPenPosition()` sets the point at which you want to start drawing a line. The function `IpfcDisplay.DrawLine()` draws a line to the given point from the position given in the last call to either of the two functions. Call `IpfcDisplay.SetPenPosition()` for the start of the polyline, and `IpfcDisplay.DrawLine()` for each vertex. If you use these methods in two-dimensional modes, use screen coordinates instead of solid coordinates.

The method `IpfcDisplay.DrawCircle()` uses solid coordinates for the center of the circle and the radius value. The circle will be placed to the XY plane of the model.

---

The method `IpfcDisplay.DrawPolyline()` also draws polylines, using an array to define the polyline.

In two-dimensional models the Display Graphics methods draw graphics at the specified screen coordinates.

The method `IpfcDisplay.DrawPolygon2D()` draws a polygon in screen coordinates. The method `IpfcDisplay.DrawArc2D()` draws an arc in screen coordinates.

### Controlling Graphics Display

Properties Introduced:

- **`IpfcDisplay.CurrentGraphicsColor`**
- **`IpfcDisplay.CurrentGraphicsMode`**

The property `IpfcDisplay.CurrentGraphicsColor` returns the Creo Parametric standard color used to display graphics. The Creo Parametric default is `EpfcCOLOR_DRAWING` (white).

The property `IpfcDisplay.CurrentGraphicsMode` returns the mode used to draw graphics:

- `EpfcDRAW_GRAPHICS_NORMAL`—Creo Parametric draws graphics in the required color in each invocation.
- `EpfcDRAW_GRAPHICS_COMPLEMENT`—Creo Parametric draws graphics normally, but will erase graphics drawn a second time in the same location. This allows you to create rubber band lines.

### Example Code: Creating Graphics On Screen

The sample code in the file `pfcGraphicsExamples.vb` located at `<creo_vbapi_loadpoint>/vbapi_examples` demonstrates the use of mouse-tracking methods to draw graphics on the screen. The static method `DrawRubberbandLine` prompts the user to pick a screen point. The example uses the 'complement mode' to cause the line to display and erase as the user moves the mouse around the window.

---

#### **Note**

This example uses the method `transformPosition` to convert the coordinates into the 3D coordinate system of a solid model, if one is displayed.

---

---

## Displaying Text in the Graphics Window

Method Introduced:

- **IpfcDisplay.DrawText2D()**

The method `IpfcDisplay.DrawText2D()` places text at a position specified in screen coordinates. If you want to add text to a particular position on the solid, you must transform the solid coordinates into screen coordinates by using the view matrix.

Text items drawn are not known to Creo Parametric and therefore are not redrawn when you select **View, Repaint**. To notify the Creo Parametric of these objects, create them inside the `OnDisplay()` method of the Display Listener.

## Controlling Text Attributes

Properties Introduced:

- **IpfcDisplay.extHeight**
- **IpfcDisplay.WidthFactor**
- **IpfcDisplay.RotationAngle**
- **IpfcDisplay.SlantAngle**

These properties control the attributes of text added by calls to `IpfcDisplay.DrawText2D()`.

You can access the following information:

- Text height (in screen coordinates)
- Width ratio of each character, including the gap, as a proportion of the height
- Rotation angle of the whole text, in counterclockwise degrees
- Slant angle of the text, in clockwise degrees

## Controlling Text Fonts

Methods and Properties Introduced:

- **IpfcDisplay.DefaultFont**
- **IpfcDisplay.CurrentFont**
- **IpfcDisplay.GetFontById()**
- **IpfcDisplay.GetFontByName()**

The property `IpfcDisplay.DefaultFont` returns the default Creo Parametric text font. The text fonts are identified in Creo Parametric by names and by integer identifiers. To find a specific font, use the methods `IpfcDisplay.GetFontById()` or `IpfcDisplay.GetFontByName()`.

---

## Display Lists and Graphics

When generating a display of a solid in a window, Creo Parametric maintains two display lists. A display list contains a set of vectors that are used to represent the shape of the solid in the view. A 3D display list contains a set of three-dimensional vectors that represent an approximation to the geometry of the edges of the solid. This list gets rebuilt every time the solid is regenerated.

A 2D display list contains the two-dimensional projections of the edges of the solid 3D display list onto the current window. It is rebuilt from the 3D display list when the orientation of the solid changes. The methods in this section enable you to add your own vectors to the display lists, so that the graphics will be redisplayed automatically by Creo Parametric until the display lists are rebuilt.

When you add graphics items to the 2D display list, they will be regenerated after each repaint (when zooming and panning) and will be included in plots created by Creo Parametric. When you add graphics to the 3D display list, you get the further benefit that the graphics survive a change to the orientation of the solid and are displayed even when you spin the solid dynamically.

Methods Introduced:

- **`IpfcDisplayListener.OnDisplay()`**
- **`IpfcDisplay.CreateDisplayList2D()`**
- **`IpfcDisplay.CreateDisplayList3D()`**
- **`IpfcDisplayList2D.Display()`**
- **`IpfcDisplayList3D.Display()`**
- **`IpfcDisplayList2D.Delete()`**
- **`IpfcDisplayList3D.Delete()`**

A display listener is a class that acts similarly to an action listener. You must implement the method inherited from the `IpfcDisplayListener` interface. The implementation should provide calls to methods on the provided `IpfcDisplay` object to produce 2D or 3D graphics.

In order to create a display list in Creo Parametric, you call `IpfcDisplay.CreateDisplayList2D` or `IpfcDisplay.CreateDisplayList3D()` to tell Creo Parametric to use your listener to create the display list vectors.

`IpfcDisplayList2D.Display()` or `IpfcDisplayList3D.Display()` will display or redisplay the elements in your display list. The application should delete the display list data when it is no longer needed.

The method `IpfcDisplayList2D.Delete()` and the method `IpfcDisplayList3D.Delete()` will remove both the specified display list from a session.

---

## Note

The method `IpfcWindow.Refresh()` does not cause either of the display lists to be regenerated, but simply repaints the window using the 2D display list.

---

## Exceptions

Possible exceptions that might be thrown by displaying graphics methods are shown in the following table:

| Exception                             | Reason                                                                                                                                       |
|---------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------|
| <code>IpfcXToolkitNotExist</code>     | The display list is empty.                                                                                                                   |
| <code>IpfcXToolkitNotFound</code>     | The function could not find the display list or the font specified in a previous call to <code>IpfcDisplay.CurrentFont</code> was not found. |
| <code>IpfcXToolkitCantOpen</code>     | The use of display lists is disabled.                                                                                                        |
| <code>IpfcXToolkitAbort</code>        | The display was aborted.                                                                                                                     |
| <code>IpfcXToolkitNotValid</code>     | The specified display list is invalid.                                                                                                       |
| <code>IpfcXToolkitInvalidItem</code>  | There is an invalid item in the display list.                                                                                                |
| <code>IpfcXToolkitGeneralError</code> | The specified display list is already in the process of being displayed.                                                                     |

## Example Code

The sample code in the file `pfcGraphicsExamples.vb` located at `<creo_vbapi_loadpoint>/vbapi_examples` demonstrates the use of `IpfcDisplay` methods with 3D display lists. The static method `AddCircleDisplay()` creates a new 3D display list whose graphics are generated by the code in the `OnDisplay()` method of the `Display Circles` class. This display list places circles at all of the vertices of a part model on the screen.

# External Data

## External Data

This section explains using External Data in the VB API.



---

## External Data

This section describes how to store and retrieve external data. External data enables a the VB API application to store its own data in a Creo Parametric database in such a way that it is invisible to the Creo Parametric user. This method is different from other means of storage accessible through the Creo Parametric user interface.

### Introduction to External Data

External data provides a way for the Creo Parametric application to store its own private information about a Creo Parametric model within the model file. The data is built and interrogated by the application as a workspace data structure. It is saved to the model file when the model is saved, and retrieved when the model is retrieved. The external data is otherwise ignored by Creo Parametric; the application has complete control over form and content.

The external data for a specific Creo Parametric model is broken down into classes and slots. A class is a named “bin” for your data, and identifies it as yours so no other Creo Parametric API application (or other classes in your own application) will use it by mistake. An application usually needs only one class. The class name should be unique for each application and describe the role of the data in your application.

Each class contains a set of data slots. Each slot is identified by an identifier and optionally, a name. A slot contains a single data item of one of the following types:

| The VB API Type     | Data    |
|---------------------|---------|
| EpfcEXTDATA_INTEGER | integer |
| EpfcEXTDATA_DOUBLE  | double  |
| EpfcEXTDATA_STRING  | string  |

The the VB API interfaces used to access external data in Creo Parametric are:

| The VB API Type        | Data Type                                                                                 |
|------------------------|-------------------------------------------------------------------------------------------|
| IpfcExternalDataAccess | This is the top level object and is created when attempting to access external data.      |
| IpfcExternalDataClass  | This is a class of external data and is identified by a unique name.                      |
| IpfcExternalDataSlot   | This is a container for one item of data. Each slot is stored in a class.                 |
| IpfcExternalData       | This is a compact data structure that contains either an integer, double or string value. |

---

## Compatibility with Creo TOOLKIT

the VB API and Creo TOOLKIT share external data in the same manner. the VB API external data is accessible by Creo TOOLKIT and the reverse is also true. However, an error will result if the VB API attempts to access external data previously stored by Creo TOOLKIT as a stream.

## Accessing External Data

Methods Introduced:

- **`IpfcModel.AccessExternalData()`**
- **`IpfcModel.TerminateExternalData()`**
- **`IpfcExternalDataAccess.IsValid()`**

The method `IpfcModel.AccessExternalData()` prepares Creo Parametric to read external data from the model file. It returns the `IpfcExternalDataAccess` object that is used to read and write data. This method should be called only once for any given model in session.

The method `IpfcModel.TerminateExternalData()` stops Creo Parametric from accessing external data in a model. When you use this method all external data in the model will be removed. Permanent removal will occur when the model is saved.

---

### Note

If you need to preserve the external data created in session, you must save the model before calling this function. Otherwise, your data will be lost.

---

The method `IpfcExternalDataAccess.IsValid()` determines if the `IpfcExternalDataAccess` object can be used to read and write data.

## Storing External Data

Methods and Properties Introduced:

- **`IpfcExternalDataAccess.CreateClass()`**
- **`IpfcExternalDataClass.CreateSlot()`**
- **`IpfcExternalDataSlot.Value`**

The first step in storing external data in a new class and slot is to set up a class using the method `IpfcExternalDataAccess.CreateClass()`, which provides the class name. The method outputs `IpfcExternalDataClass`, used by the application to reference the class.

---

The next step is to use `IpfcExternalDataClass.CreateSlot()` to create an empty data slot and input a slot name. The method outputs a `IpfcExternalDataSlot` object to identify the new slot.



### Note

Slot names cannot begin with a number.

---

The property `IpfcExternalDataSlot.Value` specifies the data type of a slot and writes an item of that type to the slot. The input is a `IpfcExternalData` object that you can create by calling any one of the methods in the next section.

### Example code

The sample code in the file `pfExternalDataExamples.vb` located at `<creo_vbapi_loadpoint>/vbapi_examples` demonstrates the usage of external data. It provides utility methods to convert a VB hashtable to a model's external data.

## Initializing Data Objects

Methods Introduced:

- **`CMpfExternal.CreateIntExternalData()`**
- **`CMpfExternal.CreateDoubleExternalData()`**
- **`CMpfExternal.CreateStringExternalData()`**

These methods initialize a `IpfcExternalData` object with the appropriate data inputs.

## Retrieving External Data

Methods and Properties Introduced:

- **`IpfcExternalDataAccess.LoadAll()`**
- **`IpfcExternalDataAccess.ListClasses()`**
- **`IpfcExternalDataClass.ListSlots()`**
- **`IpfcExternalData.discr`**
- **`IpfcExternalData.IntegerValue`**
- **`IpfcExternalData.DoubleValue`**
- **`IpfcExternalData.StringValue`**

For improved performance, external data is not loaded automatically into memory with the model. When the model is in session, call the method `IpfcExternalDataAccess.LoadAll()` to retrieve all the external data for the specified model from the Creo Parametric model file and put it in the workspace. The method needs to be called only once to retrieve all the data.

The method `IpfcExternalDataAccess.ListClasses()` returns a sequence of `IpfcExternalDataClasses` registered in the model. The method `IpfcExternalDataClass.ListSlots()` provide a sequence of `IpfcExternalDataSlots` existing for each class.

To find out a data type of a `IpfcExternalData`, call `IpfcExternalData.dscr` and then call one of these properties to get the data, depending on the data type:

- `IpfcExternalData.IntegerValue`
- `IpfcExternalData.DoubleValue`
- `IpfcExternalData.StringValue`

## Example Code

The sample code in the file `pfExternalDataExamples.vb` located at `<creo_vbapi_loadpoint>/vbapi_examples` demonstrates the usage of external data. It provides utility methods to convert a VB hashtable to a model's external data.

## Exceptions

Most exceptions thrown by external data methods in the VB API extend `IpfcXExternalDataError`, which is a subclass of `IpfcXToolkitError`.

An additional exception thrown by external data methods is `IpfcXBadExternalData`. This exception signals an error accessing data. For example, external data access might have been terminated or the model might contain stream data from Creo TOOLKIT.

The following table lists these exceptions.

| Exception                                       | Cause                                                                                  |
|-------------------------------------------------|----------------------------------------------------------------------------------------|
| <code>IpfcXExternalDataInvalidObject</code>     | Generated when a model or class is invalid.                                            |
| <code>IpfcXExternalDataClassOrSlotExists</code> | Generated when creating a class or slot and the proposed class or slot already exists. |
| <code>IpfcXExternalDataNamesTooLong</code>      | Generated when a class or slot name is too long.                                       |
| <code>IpfcXExternalDataSlotNotFound</code>      | Generated when a specified class or slot does not exist.                               |
| <code>IpfcXExternalDataEmptySlot</code>         | Generated when the slot you are                                                        |

| Exception                                     | Cause                                                                                              |
|-----------------------------------------------|----------------------------------------------------------------------------------------------------|
|                                               | attempting to read is empty.                                                                       |
| <code>IpfcXExternalDataInvalidSlotName</code> | Generated when a specified slot name is invalid.                                                   |
| <code>IpfcXBadGetExternalData</code>          | Generated when you try to access an incorrect data type in a <code>IpfcExternalData</code> object. |

## Windchill Connectivity APIs

### Windchill Connectivity APIs

Creo Parametric has the capability to be directly connected to Windchill solutions, including WindchillProjectLink and PDMLink servers. This access allows users to manage and control the product data seamlessly from within Creo Parametric.

This section lists the VB APIs that support Windchill servers and server operations in a connected Creo Parametric session.

### Introduction

The methods introduced in this section provide support for the basic Windchill server operations from within Creo Parametric. With these methods, operations such as registering a Windchill server, managing workspaces, and check in or check out of objects will be possible via the VB API. The capabilities of these APIs are similar to the operations available from within the Creo Parametric client, with some restrictions.

Some of these APIs are supported from a non-interactive, that is, batch mode application or asynchronous application. For more information about batch mode and asynchronous mode, refer to the section on Asynchronous mode.

### Accessing a Windchill Server from a Creo Parametric Session

Creo Parametric allows you to register Windchill servers as a connection between the Windchill database and Creo Parametric. Although the represented Windchill database can be from WindchillProjectLink or Windchill PDMLink all types of databases are represented in the same way.

You can use the following identifiers when referring to Windchill servers in the VB API:

- **Codebase URL**—This is the root portion of the URL that is used to connect to a Windchill server. For example `http://wcserver.company.com/Windchill`.

- 
- **Server Alias**—A server alias is used to refer to the server after it has been registered. The alias is also used to construct paths to files in the server workspaces and commonspace. The server alias is chosen by the user or application and it need not have any direct relationship to the codebase URL. An alias can be any normal name, such as `my_alias`.

### Accessing Information Before Registering a Server

To start working with a Windchill server, you must establish a connection by registering the server in Creo Parametric. The methods described in this section allow you to connect to a Windchill server and access information related to the server.

Methods and Properties Introduced:

- **`IpfcBaseSession.AuthenticateBrowser()`**
- **`IpfcBaseSession.GetServerLocation()`**
- **`IpfcServerLocation.Class`**
- **`IpfcServerLocation.Location`**
- **`IpfcServerLocation.Version`**
- **`IpfcServerLocation.ListContexts()`**
- **`IpfcServerLocation.CollectWorkspaces()`**

Use the method `IpfcBaseSession.AuthenticateBrowser()` to set the authentication context using a valid username and password. A successful call to this method allows the Creo Parametric session to register with any server that accepts the username and password combination. A successful call to this method also ensures that an authentication dialog box does not appear during the registration process. You can call this method any number of times to set the authentication context for any number of Windchill servers, provided that you register the appropriate servers or servers immediately after setting the context.

The property `IpfcServerLocation.Location` specifies a `IpfcServerLocation` object representing the codebase URL for a possible server. The server may not have been registered yet, but you can use this object and the methods it contains to gather information about the server prior to registration.

The property `IpfcServerLocation.Class` specifies the class of the server or server location. The values are:

- **Windchill**—Denotes a Windchill PDMLink server.
- **ProjectLink**—Denotes Windchill ProjectLink type of servers.

The property `IpfcServerLocation.Version` specifies the version of Windchill that is configured on the server or server location, for example, 9.0 or 10.0. This method accepts the server codebase URL as the input.

---

## Note

`IpfcServerLocation.Version` works only for Windchill servers and throws the `IpfcXToolkitUnsupported` exception, if the server is not a Windchill server.

---

The method `IpfcServerLocation.ListContexts()` gives a list of all the available contexts for a specified server. A context is used to associate a workspace with a product, project, or library.

The method `IpfcServerLocation.CollectWorkspaces()` returns the list of available workspaces for the specified server. The workspace objects returned contain the name of each workspace and its context.

## Registering and Activating a Server

From Creo Parametric 2.0 onward, the the VB API methods call the same underlying API as Creo Parametric to register and unregister servers. Hence, registering the servers using the VB API methods is similar to registering the servers using the Creo Parametric user interface. Therefore, the servers registered by the VB API are available in the Creo Parametric Server Registry. The servers are also available in other locations in the Creo Parametric user interface such as, the **Folder Navigator** and the embedded browser.

Methods Introduced:

- **`IpfcBaseSession.RegisterServer()`**
- **`IpfcServer.Activate()`**
- **`IpfcServer.Unregister()`**

The method `IpfcBaseSession.RegisterServer()` registers the specified server with the codebase URL. You can automate the registration of servers in interactive mode. To preregister the servers use the standard `config.fld` setup. If you do not want the servers to be preregistered in batch mode, set the environment variable `PTC_WF_ROOT` to an empty directory before starting Creo Parametric.

A successful call to `IpfcBaseSession.AuthenticateBrowser()` with a valid username and password is essential for

`IpfcBaseSession.RegisterServer()` to register the server without launching the authentication dialog box. Registration of the server establishes the server alias. You must designate an existing workspace to use when registering the server. After the server has been registered, you may create a new workspace.

The method `IpfcServer.Activate()` sets the specified server as the active server in the Creo Parametric session.

---

The method `IpfcServer.Unregister()` unregisters the specified server. This is similar to **Server Registry ► Delete** through the user interface.

### Accessing Information From a Registered Server

Properties Introduced:

- **`IpfcServer.IsActive`**
- **`IpfcServer.Alias`**
- **`IpfcServer.Context`**

The property `IpfcServer.IsActive` specifies if the server is active.

The property `IpfcServer.Alias` returns the alias of a server if you specify the codebase URL.

The property `IpfcServer.Context` returns the active context of the active server.

### Information on Servers in Session

Methods Introduced:

- **`IpfcBaseSession.GetActiveServer()`**
- **`IpfcBaseSession.GetServerByAlias()`**
- **`IpfcBaseSession.GetServerByUrl()`**
- **`IpfcBaseSession.ListServers()`**

The method `IpfcBaseSession.GetActiveServer()` returns the active server handle.

The method `IpfcBaseSession.GetServerByAlias()` returns the handle to the server matching the given server alias, if it exists in session.

The method `IpfcBaseSession.GetServerByUrl()` returns the handle to the server matching the given server URL and workspace name, if it exists in session.

The method `IpfcBaseSession.ListServers()` returns a list of servers registered in this session.

## Accessing Workspaces

For every workspace, a new distinct storage location is maintained in the user's personal folder on the server (server-side workspace) and on the client (client-side workspace cache). Together, the server-side workspace and the client-side workspace cache make up the workspace.

Methods and Properties Introduced:

- **`CCpfcWorkspaceDefinition.Create()`**



- 
- **IpfcWorkspaceDefinition.WorkspaceName**
  - **IpfcWorkspaceDefinition.WorkspaceContext**

The class `IpfcWorkspaceDefinition` contains the name and context of the workspace. The method `IpfcServerLocation.CollectWorkspaces()` returns an array of workspace data. Workspace data is also required for the method `IpfcServer.CreateWorkspace()` to create a workspace with a given name and a specific context.

The method `CCpfcWorkspaceDefinition.Create()` creates a new workspace definition object suitable for use when creating a new workspace on the server.

The property `IpfcWorkspaceDefinition.WorkspaceName` retrieves the name of the workspace.

The property `IpfcWorkspaceDefinition.WorkspaceContext` retrieves the context of the workspace.

## Creating and Modifying the Workspace

Methods and Properties Introduced:

- **IpfcServer.CreateWorkspace()**
- **IpfcServer.ActiveWorkspace**
- **IpfcServerLocation.DeleteWorkspace()**

The method `IpfcServer.CreateWorkspace()` creates and activates a new workspace.

The property `IpfcServer.ActiveWorkspace` retrieves the name of the active workspace.

The method `IpfcServerLocation.DeleteWorkspace()` deletes the specified workspace. The method deletes the workspace only if the following conditions are met:

- The workspace is not the active workspace.
- The workspace does not contain any checked out objects.

Use one of the following techniques to delete an active workspace:

- Make the required workspace inactive using `IpfcServer.ActiveWorkspace` with the name of some other workspace and then call `IpfcServerLocation.DeleteWorkspace()`.
- Unregister the server using `IpfcServer.Unregister()` and delete the workspace.

---

## Workflow to Register a Server

### To Register a Server with an Existing Workspace

Perform the following steps to register a Windchill server with an existing workspace:

1. Set the appropriate authentication context using the method `IpfcBaseSession.AuthenticateBrowser()` with a valid username and password.
2. Look up the list of workspaces using the method `IpfcServerLocation.CollectWorkspaces()`. If you already know the name of the workspace on the server, then ignore this step.
3. Register the workspace using the method `IpfcBaseSession.RegisterServer()` with an existing workspace name on the server.
4. Activate the server using the method `IpfcServer.Activate()`.

### To Register a Server with a New Workspace

Perform the following steps to register a Windchill server with a new workspace:

1. Perform steps 1 to 4 in the preceding section to register the Windchill server with an existing workspace.
2. Use the method `IpfcServerLocation.ListContexts()` to choose the required context for the server.
3. Create a new workspace with the required context using the method `IpfcServer.CreateWorkspace()`. This method automatically makes the created workspace active.

---

#### Note

You can create a workspace only after the server is registered.

---

## Aliased URL

An aliased URL serves as a handle to the server objects. You can access the server objects in the commonspace (shared folders) and the workspace using an aliased URL. An aliased URL is a unique identifier for the server object and its format is as follows:

- Object in workspace has a prefix `wtws`  
`wtws://<server_alias>/<workspace_name>/<object_server_name>`

---

where <object\_server\_name> includes <object\_name>.<object\_extension>

For example,

wtws://my\_server/my\_workspace/abcd.prt,  
wtws://my\_server/my\_workspace/intf\_file.igs

where

<server\_alias> is my\_server

<workspace\_name> is my\_workspace

- Object in commonspace has a prefix wtpub  
wtpub://<server\_alias>/<folder\_location>/<object\_server\_name>

For example,

wtpub://my\_server/path/to/cs\_folder/abcd.prt

where

<server\_alias> is my\_server

<folder\_location> is path/to/cs\_folder



#### Note

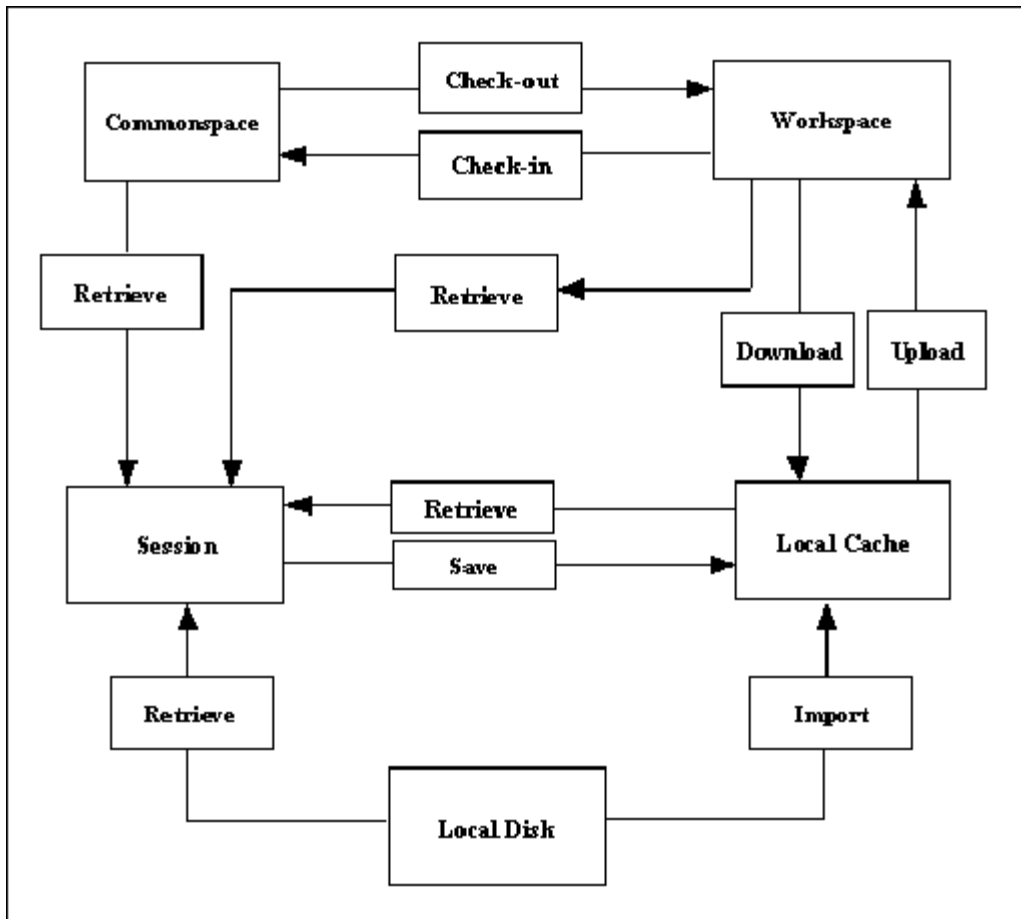
- object\_server\_name must be in lowercase.
  - The APIs are case-sensitive to the aliased URL.
  - <object\_extension> should not contain Creo Parametric versions, for example, .1 or .2, and so on.
- 

## Server Operations

After registering the Windchill server with Creo Parametric, you can start accessing the data on the Windchill servers. The Creo Parametric interaction with Windchill servers leverages the following locations:

- Commonsplace (Shared folders)
- Workspace (Server-side workspace)
- Workspace local cache (Client-side workspace)
- Creo Parametric session
- Local disk

The methods described in this section enable you to perform the basic server operations. The following illustration shows how data is transferred among these locations.



## Save

Method Introduced:

- **IpfcModel.Save()**

The method `IpfcModel.Save()` stores the object from the session in the local workspace cache, when a server is active.

## Upload

An upload transfers Creo Parametric files and any other dependencies from the local workspace cache to the server-side workspace.

Methods Introduced:

- **IpfcServer.UploadObjects()**
- **IpfcServer.UploadObjectsWithOptions()**
- **CCpfcUploadOptions.Create()**

---

The method `IpfcServer.UploadObjects()` uploads the object to the workspace. The object to be uploaded must be present in the current Creo Parametric session. You must save the object to the workspace using `IpfcModel.Save()` `IpfcBaseSession.ImportToCurrentWS()` before attempting to upload it.

The method `IpfcServer.UploadObjectsWithOptions()` uploads objects to the workspace using the options specified in the `IpfcIpfcUploadOptions` interface. These options allow you to upload the entire workspace, auto-resolve missing references, and indicate the target folder location for the new content during the upload. You must save the object to the workspace using `IpfcModel.Save()`, or import it to the workspace using `IpfcBaseSession.ImportToCurrentWS()` before attempting to upload it.

Create the `IpfcUploadOptions` object using the method `CCpfcUploadOptions.Create()`.

The methods available for setting the upload options are described in the following section.

## CheckIn

After you have finished working on objects in your workspace, you can share the design changes with other users. The checkin operation copies the information and files associated with all changed objects from the workspace to the Windchill database.

Methods and Properties Introduced:

- **`IpfcServer.CheckinObjects()`**
- **`CCpfcCheckinOptions.Create()`**
- **`IpfcUploadBaseOptions.DefaultFolder`**
- **`IpfcUploadBaseOptions.NonDefaultFolderAssignments`**
- **`IpfcUploadBaseOptions.AuteresolveOption`**
- **`IpfcCheckinOptions.BaselineName`**
- **`IpfcCheckinOptions.BaselineNumber`**
- **`IpfcCheckinOptions.BaselineLocation`**
- **`IpfcCheckinOptions.BaselineLifecycle`**
- **`IpfcCheckinOptions.KeepCheckedout`**

The method `IpfcServer.CheckinObjects()` checks in an object into the database. The object to be checked in must be present in the current Creo Parametric session. Changes made to the object are not included unless you save the object to the workspace using the method `IpfcModel.Save()` before you check it in.

---

If you pass `NULL` as the value of the *options* parameter, the checkin operation is similar to the **Auto Check-In** option in Creo Parametric. For more details on **Auto Check-In**, refer to the online help for Creo Parametric.

Use the method `CCpfcCheckinOptions.Create()` to create a new `IpfcCheckinOptions` object.

By using an appropriately constructed *options* argument, you can control the checkin operation. Use the APIs listed above to access and modify the checkin options. The checkin options are as follows:

- *DefaultFolder*—Specifies the default folder location on the server for the automatic checkin operation.
- *NonDefaultFolderAssignment*—Specifies the folder location on the server to which the objects will be checked in.
- *AutoresolveOption*—Specifies the option used for auto-resolving missing references. These options are defined in the `EpfcServerAutoresolveOption` enumerated type, and are as follows:
  - `EpfcSERVER_DONT_AUTORESOLVE`—Model references missing from the workspace are not automatically resolved. This may result in a conflict upon checkin. This option is used by default.
  - `EpfcSERVER_AUTORESOLVE_IGNORE`—Missing references are automatically resolved by ignoring them.
  - `EpfcSERVER_AUTORESOLVE_UPDATE_IGNORE`—Missing references are automatically resolved by updating them in the database and ignoring them if not found.
- *Baseline*—Specifies the baseline information for the objects upon checkin. The baseline information for a checkin operation is as follows:
  - *BaselineName*—Specifies the name of the baseline.
  - *BaselineNumber*—Specifies the number of the baseline.

The default format for the baseline name and baseline number is `Username + time (GMT) in milliseconds`.

- *BaselineLocation*—Specifies the location of the baseline.
  - *BaselineLifecycle*—Specifies the name of the lifecycle.
- *KeepCheckedout*—If the value specified is `true`, then the contents of the selected object are checked into the Windchill server and automatically checked out again for further modification.

---

## Retrieval

Standard VB API provides several methods that are capable of retrieving models. When using these methods with Windchill servers, remember that these methods do not check out the object to allow modifications.

Methods Introduced:

- **IpfcBaseSession.RetrieveModel()**
- **IpfcBaseSession.RetrieveModelWithOpts()**
- **IpfcBaseSession.OpenFile()**

The methods `IpfcBaseSession.RetrieveModel()`, `IpfcBaseSession.RetrieveModelWithOpts()`, and `IpfcBaseSession.OpenFile()` load an object into a session given its name and type. The methods search for the object in the active workspace, the local directory, and any other paths specified by the `search_path` configuration option.

## Checkout and Download

To modify an object from the commonspace, you must check out the object. The process of checking out communicates your intention to modify a design to the Windchill server. The object in the database is locked, so that other users can obtain read-only copies of the object, and are prevented from modifying the object while you have checked it out.

Checkout is often accompanied by a download action, where the objects are brought from the server-side workspace to the local workspace cache. In the VB API, both operations are covered by the same set of methods.

Methods and Properties Introduced:

- **IpfcServer.CheckoutObjects()**
- **IpfcServer.CheckoutMultipleObjects()**
- **CCpfcCheckoutOptions.Create**
- **IpfcCheckoutOptions.Dependency**
- **IpfcCheckoutOptions.SelectedIncludes**
- **IpfcCheckoutOptions.IncludeInstances**
- **IpfcCheckoutOptions.Version**
- **IpfcCheckoutOptions.Download**
- **IpfcCheckoutOptions.Readonly**

The method `IpfcServer.CheckoutObjects()` checks out and optionally downloads the object to the workspace based on the configuration specifications of the workspace. The input arguments of this method are as follows:

- 
- *Mdl*—Specifies the object to be checked out. This is applicable if the model has already been retrieved without checking it out.
  - *File*—Specifies the top-level object to be checked out.
  - *Checkout*—The checkout flag. If you specify the value of this argument as `true`, the selected object is checked out. Otherwise, the object is downloaded without being checked out. The download action enables you to bring read-only copies of objects into your workspace. This allows you to examine the object without locking it.
  - *Options*—Specifies the checkout options object. If you pass `NULL` as the value of this argument,, then the default Creo Parametric checkout rules apply. Use the method `CCpfcCheckoutOptions.Create()` to create a new `IpfcCheckoutOptions` object.

Use the method `IpfcServer.CheckoutMultipleObjects()` to check out and download multiple objects to the workspace based on the configuration specifications of the workspace. This method takes the same input arguments as listed above, except for *Mdl* and *File*. Instead it takes the argument *Files* that specifies the sequence of the objects to check out or download.

---

 **Note**

VB API methods do not support `AS_STORED` configuration.

---

By using an appropriately constructed *options* argument in the above functions, you can control the checkout operation. Use the APIs listed above to modify the checkout options. The checkout options are as follows:

- *Dependency*—Specifies the dependency rule used while checking out dependents of the object selected for checkout. The types of dependencies given by the `EpfcServerDependency` enumerated type are as follows:
  - `EpfcSERVER_DEPENDENCY_ALL`—All the objects that are dependent on the selected object are downloaded, that is, they are added to the workspace.
  - `EpfcSERVER_DEPENDENCY_REQUIRED`—All the objects that are required to successfully retrieve the selected object in the CAD application are downloaded, that is, they are added to workspace.
  - `EpfcSERVER_DEPENDENCY_NONE`—None of the dependent objects from the selected object are downloaded, that is, they are not added to workspace.



- *IncludeInstances*—Specifies the rule for including instances from the family table during checkout. The type of instances given by the `EpfcServerIncludeInstances` enumerated type are as follows:
  - `EpfcSERVER_INCLUDE_ALL`—All the instances of the selected object are checked out.
  - `EpfcSERVER_INCLUDE_SELECTED`—The application can select the family table instance members to be included during checkout.
  - `EpfcSERVER_INCLUDE_NONE`—No additional instances from the family table are added to the object list.
- *SelectedIncludes*—Specifies the sequence of URLs to the selected instances, if *IncludeInstances* is of type `EpfcSERVER_INCLUDE_SELECTED`.
- *Version*—Specifies the version of the checked out object. If this value is set to `NULL`, the object is checked out according to the current workspace configuration.
- *Download*—Specifies the checkout type as `download` or `link`. The value `download` specifies that the object content is downloaded and checked out, while `link` specifies that only the metadata is downloaded and checked out.
- *Readonly*—Specifies the checkout type as a read-only checkout. This option is applicable only if the checkout type is `link`.

The following truth table explains the dependencies of the different control factors in the method `IpfcServer.CheckoutObjects()` and the effect of different combinations on the end result.

| Argument <i>checkout</i> in <code>IpfcServer.CheckoutObjects()</code> | <code>IpfcCheckoutOptions.Download</code> | <code>IpfcCheckoutOptions.Readonly</code> | Result                                                  |
|-----------------------------------------------------------------------|-------------------------------------------|-------------------------------------------|---------------------------------------------------------|
| true                                                                  | true                                      | NA                                        | Object is checked out and its content is downloaded.    |
| true                                                                  | false                                     | NA                                        | Object is checked out but content is not downloaded.    |
| false                                                                 | NA                                        | true                                      | Object is downloaded without checkout and as read-only. |
| false                                                                 | NA                                        | false                                     | Not supported                                           |

## Undo Checkout

Method Introduced:

- **`IpfcServer.UndoCheckout()`**

---

Use the method `IpfcServer.UndoCheckout()` to undo a checkout of the specified object. When you undo a checkout, the changes that you have made to the content and metadata of the object are discarded and the content, as stored in the server, is downloaded to the workspace. This method is applicable only for the model in the active Creo Parametric session.

## Import and Export

The VB API provides you with the capability of transferring specified objects to and from a workspace. Import and export operations must take place in a session with no models. An import operation transfers a file from the local disk to the workspace.

Methods and Properties Introduced:

- **`IpfcBaseSession.ExportFromCurrentWS()`**
- **`IpfcBaseSession.ImportToCurrentWS()`**
- **`IpfcWSImportExportMessage.Description`**
- **`IpfcWSImportExportMessage.FileName`**
- **`IpfcWSImportExportMessage.MessageType`**
- **`IpfcWSImportExportMessage.Resolution`**
- **`IpfcWSImportExportMessage.Succeeded`**
- **`IpfcBaseSession.SetWSExportOptions()`**
- **`CCpfcWSExportOptions.Create()`**
- **`IpfcWSExportOptions.IncludeSecondaryContent`**

The method `IpfcBaseSession.ExportFromCurrentWS()` exports the specified objects from the current workspace to a disk in a linked session of Creo Parametric.

The method `IpfcBaseSession.ImportToCurrentWS()` imports the specified objects from a disk to the current workspace in a linked session of Creo Parametric.

Both `IpfcBaseSession.ExportFromCurrentWS()` and `IpfcBaseSession.ImportToCurrentWS()` allow you to specify a dependency criterion to process the following items:

- All external dependencies
- Only required dependencies
- No external dependencies

Both `IpfcBaseSession.ExportFromCurrentWS()` and `IpfcBaseSession.ImportToCurrentWS()` return the messages generated during the export or import operation in the form of the

---

`IpfcWSImportExportMessages` object. Use the APIs listed above to access the contents of a message. The message specified by the `IpfcWSImportExportMessage` object contains the following items:

- **Description**—Specifies the description of the problem or the message information.
- **FileName**—Specifies the object name or the name of the object path.
- **MessageType**—Specifies the severity of the message in the form of the `EpfcWSImportExportMessageType` enumerated type. The severity is one of the following types:
  - `EpfcWSIMPEX_MSG_INFO`—Specifies an informational type of message.
  - `EpfcWSIMPEX_MSG_WARNING`—Specifies a low severity problem that can be resolved according to the configured rules.
  - `EpfcWSIMPEX_MSG_CONFLICT`—Specifies a conflict that can be overridden.
  - `EpfcWSIMPEX_MSG_ERROR`—Specifies a conflict that cannot be overridden or a serious problem that prevents processing of an object.
- **Resolution**—Specifies the resolution applied to resolve a conflict that can be overridden. This is applicable when the message is of the type `EpfcWSIMPEX_MSG_CONFLICT`.
- **Succeeded**—Determines whether the resolution succeeded or not. This is applicable when the message is of the type `EpfcWSIMPEX_MSG_CONFLICT`.

The method `IpfcBaseSession.SetWSExportOptions()` sets the export options used while exporting the objects from a workspace in the form of the `IpfcWSExportOptions` object. Create this object using the method `CCpfcWSExportOptions.Create()`. The export options are as follows:

- ***Include Secondary Content***—Indicates whether or not to include secondary content while exporting the primary Creo Parametric model files. Use the property `IpfcWSExportOptions.IncludeSecondaryContent` to set this option.

## File Copy

The VB API provides you with the capability of copying a file from the workspace or target folder to a location on the disk and vice-versa.

Methods Introduced:

- **`IpfcBaseSession.CopyFileToWS()`**
- **`IpfcBaseSession.CopyFileFromWS()`**

---

Use the method `IpfcBaseSession.CopyFileToWS()` to copy a file from the disk to the workspace. The file can optionally be added as secondary content to a given workspace file. If the viewable file is added as secondary content, a dependency is created between the Creo Parametric model and the viewable file.

Use the method `IpfcBaseSession.CopyFileFromWS()` to copy a file from the workspace to a location on disk.

When importing or exporting Creo Parametric models, PTC recommends that you use methods `IpfcBaseSession.ImportToCurrentWS()` and `IpfcBaseSession.ExportFromCurrentWS()`, respectively, to perform the import or export operation. Methods that copy individual files do not traverse Creo Parametric model dependencies, and therefore do not copy a fully retrievable set of models at the same time.

Additionally, only the methods

`IpfcBaseSession.ImportToCurrentWS()` and `IpfcBaseSession.ExportFromCurrentWS()` provide full metadata exchange and support. That means

`IpfcBaseSession.ImportToCurrentWS()` can communicate all the Creo Parametric designated parameters, dependencies, and family table information to a PDM system while

`IpfcBaseSession.ExportFromCurrentWS()` can update exported Creo Parametric data with PDM system changes to designated and system parameters, dependencies, and family table information. Hence PTC recommends the use of `IpfcBaseSession.CopyFileToWS()` and `IpfcBaseSession.CopyFileFromWS()` to process only non-Creo Parametric files.

## Server Object Status

Methods Introduced:

- **`IpfcServer.IsObjectCheckedOut()`**
- **`IpfcServer.IsObjectModified()`**

The methods described in this section verify the current status of the object in the workspace. The method `IpfcServer.IsObjectCheckedOut()` specifies whether the object is checked out for modification. The value `true` indicates that the specified object is checked out to the active workspace.

The value `false` indicates one of the following statuses:

- The specified object is not checked out
- The specified object is only uploaded to the workspace, but was never checked in
- The specified object is only saved to the local workspace cache, but was never uploaded

---

The method `IpfcServer.IsObjectModified()` specifies whether the object has been modified in the workspace. This method returns `true` if the object was modified locally.

## Delete Objects

Method Introduced:

- **`IpfcServer.RemoveObjects()`**

The method `IpfcServer.RemoveObjects()` deletes the array of objects from the workspace. When passed with the *ModelNames* array as `NULL`, this method removes all the objects in the active workspace.

## Conflicts During Server Operations

An exception is provided to capture the error condition while performing the following server operations using the specified APIs:

| Operation                      | API                                                                             |
|--------------------------------|---------------------------------------------------------------------------------|
| Checkin an object or workspace | <code>IpfcServer.CheckinObjects()</code>                                        |
| Checkout an object             | <code>IpfcServer.CheckoutObjects()</code>                                       |
| Undo checkout of an object     | <code>IpfcServer.UndoCheckout()</code>                                          |
| Upload object                  | <code>IpfcServer.UploadObjects()</code>                                         |
| Download object                | <code>IpfcServer.CheckoutObjects()</code> (with <code>download as true</code> ) |
| Delete workspace               | <code>IpfcServerLocation.DeleteWorkspace()</code>                               |
| Remove object                  | <code>IpfcServer.RemoveObjects()</code>                                         |

These APIs throw a common exception `IpfcXToolkitCheckoutConflict` if an error is encountered during server operations. The exception description will include the details of the error condition. This description is similar to the description displayed by the Creo Parametric HTML user interface in the conflict report.

## Example Code: Server APIs

The sample code in the file `pfcServerExamples.vb` located at `<creo_vbapi_loadpoint>/vbapi_examples` demonstrates the implementation of the Windchill server APIs described in the previous sections.

## Utility APIs

The methods specified in this section enable you to obtain the handle to the server objects to access them. The handle may be the aliased URL or the model name of the http URL. These utilities enable the conversion of one type of handle to another.

Methods Introduced:

- **IpfcServer.GetAliasedUrl()**
- **IpfcBaseSession.GetModelNameFromAliasedUrl()**
- **IpfcBaseSession.GetAliasFromAliasedUrl()**
- **IpfcBaseSession.GetUrlFromAliasedUrl()**

The method `IpfcServer.GetAliasedUrl()` enables you to search for a server object by its name. Specify the complete filename of the object as the input, for example, `test_part.prt`. The method returns the aliased URL for a model on the server. For more information regarding the aliased URL, refer to the section [Aliased URL on page 306](#). During the search operation, the workspace takes precedence over the shared space.

You can also use this method to search for files that are not in the Creo Parametric format. For example, `my_text.txt`, `intf_file.stp`, and so on.

The method `IpfcBaseSession.GetModelNameFromAliasedUrl()` returns the name of the object from the given aliased URL on the server.

The method `IpfcBaseSession.GetUrlFromAliasedUrl()` converts an aliased URL to a standard URL for the objects on the server.

For example, `wtws://my_alias/Creo Parametric/abcd.prt` is converted to an appropriate URL on the server as `<server.mycompany.com>/Windchill`.

The method `IpfcBaseSession.GetAliasFromAliasedUrl()` returns the server alias from aliased URL.

## Sample Batch Workflow

A typical workflow using the Windchill APIs for an asynchronous non-graphical application is as follows:

1. Start a Creo Parametric session using the method `CCpfcAsyncConnection.Connect()`.
2. Authenticate the browser using the method `IpfcBaseSession.AuthenticateBrowser()`.
3. Register the server with the new workspace using the method `IpfcBaseSession.RegisterServer()`.
4. Activate the server using the method `IpfcServer.Activate()`.
5. Check out and retrieve the model from the vault URL using the method `IpfcServer.CheckoutObjects()` followed by `IpfcBaseSession.RetrieveModel()`.
6. Modify the model according to the application logic.
7. Save the model to the workspace using the method `IpfcModel.Save()`.

8. Check in the modified model back to the server using the method `pfcServer.Server.CheckinObjects()`.
9. After processing all models, unregister from the server using the method `IpfcServer.Unregister()`.
10. Delete the workspace using `IpfcServerLocation.DeleteWorkspace()`.
11. Stop Creo Parametric using the method `IpfcAsyncConnection.End()`.

## Sample Applications

### Sample Applications

This appendix section lists the sample applications provided with the VB API.

#### Installing the VB API

The VB API is available on the same CD as Creo Parametric. When Creo Parametric is installed using PTC.SetUp, one of the optional components is API Toolkits. This includes Creo TOOLKIT, J-Link, Web.Link, and Visual Basic API.

If you select Visual Basic API, a directory called `vbapi` is created under the Creo Parametric loadpoint and the VB API is automatically installed in this directory. This directory contains all the libraries, example applications, and documentation specific to the VB API.

#### Sample Applications

The VB API sample applications are available in the directories `vbapi_examples` and `vbapi_appls` under the root directory `vbapi`.

##### VBAPIExamples

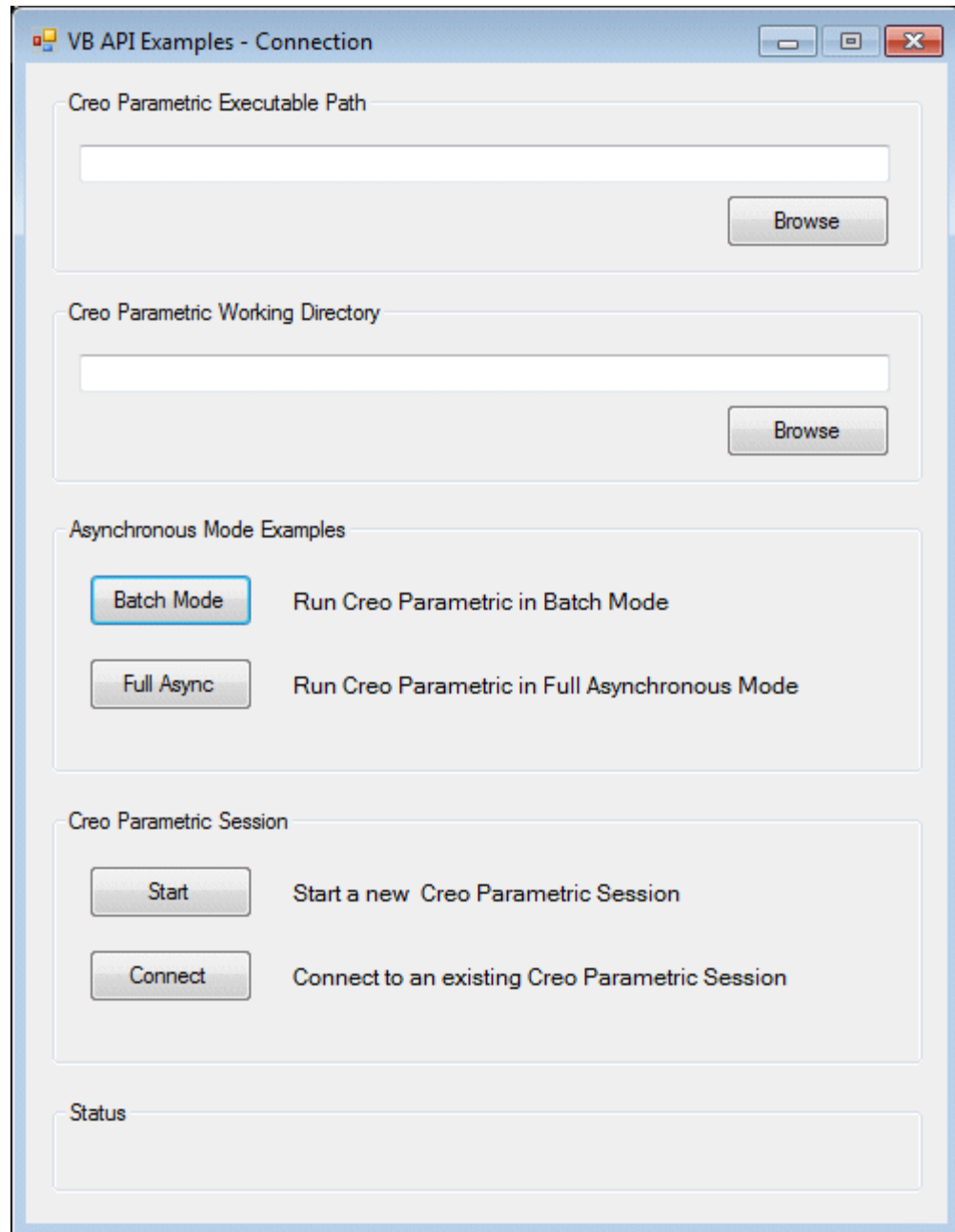
| Location                                                 | Main Classes                                              |
|----------------------------------------------------------|-----------------------------------------------------------|
| <code>&lt;creo_vbapi_loadpoint&gt;/vbapi_examples</code> | <code>formConnection</code> and <code>formExamples</code> |

The `vbapi_examples` directory is a collection of all the VB.NET example source files present in the *VB API User's Guide*. All the example source files are also available along with a VB.NET solution called `VB API Examples.sln` and a project file called `VB API Examples.vbproj` as a single `VBAPIExamples.zip` file in the same directory.

Set up and run the examples using the following procedure:

- 
1. Set the `PRO_COMM_MSG_EXE` environment variable to the full path of the executable `pro_comm_msg.exe` for your application to communicate with Creo Parametric. Typically, the path to the executable is `<creo_loadpoint>\<datecode>\Common Files\<machine_type>\obj\pro_comm_msg.exe`, where `machine_type` is `i486_nt` for 32-bit Windows and `x86_win64` for 64-bit Windows installations.
  2. Register the COM server by running the `vb_api_register.bat` file located at `<creo_loadpoint>/Parametric/bin`.
  3. Unzip the `VBAPIExamples.zip` file in a local folder on your machine and open the `VB API Examples.sln` solution in Microsoft Visual Studio.
  4. Set the COM reference for your project to `Creo Parametric VB API Type Library for Creo Parametric`.
  5. Build the solution and execute the `VB API Examples.exe` created in your local folder. The **VB API Examples - Connection** form as shown in the following figure is loaded.





6. Click **Start** to start a new Creo Parametric session in the simple asynchronous mode. You must specify the Creo Parametric working directory and executable path before attempting to start a new session. You can also connect to an existing session in the simple asynchronous mode by clicking **Connect**. Click **Batch Mode** to start a new Creo Parametric session in the batch mode, or click **Full Async** to start a new Creo Parametric session in the full asynchronous mode. Refer to the section [Asynchronous ModeVB API Fundamentals](#):

[Controlling Creo Parametric on page 27](#) for more information on the modes of communication.

7. Once you are connected to a Creo Parametric session, the VB API Examples form is loaded. You can execute all the examples available in the `vbapi_examples` directory from this form.

### Parameters and Dimensions

| Location                                   | Main Class |
|--------------------------------------------|------------|
| <creo_vbapi_loadpoint>/vbapi_appls/vbparam | FormPD     |

The parameters and dimensions example is an asynchronous mode VB.NET application that allows you to access and modify the parameters and dimensions of a Creo Parametric model. All the VB source files for this application are available along with a VB.NET solution called `ParameterAndDimension.sln` and a project file called `ParameterAndDimension.vbproj` as a single `VBParam.zip` file in the `vbparam` directory.

Set up and run this application using the following procedure:

1. Set the `PRO_COMM_MSG_EXE` environment variable to the full path of the executable `pro_comm_msg.exe` for your application to communicate with Creo Parametric. Typically, the path to the executable is `<creo_loadpoint>\<datecode>\Common Files\<machine type>\obj\pro_comm_msg.exe`, where `machine_type` is `i486_nt` for 32-bit Windows and `x86_win64` for 64-bit Windows installations.
2. Register the COM server by running the `vb_api_register.bat` file located at `<creo_loadpoint>/Parametric/bin`
3. Unzip the `VBParam.zip` file in a local folder on your machine and open the `ParameterAndDimension.sln` solution in Microsoft Visual Studio.
4. Set the COM reference for your project to `Creo Parametric VB API Type Library for Creo Parametric`.
5. Build the solution and execute the `ParameterAndDimension.exe` created in your local folder. The **Parameters and Dimensions** form is loaded.
6. Start Creo Parametric and open a PART model containing parameters and dimensions.
7. Click the **Connect** button in the form to connect to the active Creo Parametric session in the simple asynchronous mode. Click the **Add** button to connect in the full asynchronous mode, wherein a new **PDMenu** menu gets added to the menubar in the Creo Parametric user interface.

---

You can perform the same set of operations on parameters and dimensions from the **Parameters and Dimensions** form in the simple asynchronous mode and from the **PDMenu** menu in the full asynchronous mode.

8. Click **Disconnect** to disconnect from the current Creo Parametric session.

You can perform the following operations on parameters from the **Parameters and Dimensions** form:

- Retrieve all the parameters of a PART model in the current Creo Parametric session inside the parameter table in the **Parameters and Dimensions** form.
- Modify the unit, value, designated status, and description, except name and type for each parameter.
- Delete a parameter and all the values associated with it.
- Save the updated list of parameters back in the model.
- Save the list of parameters retrieved from the model in an XML file, or read the parameters from a previously saved XML file in the form.

You can perform the following operations on dimensions from the **Parameters and Dimensions** form:

- Retrieve all the dimensions of a PART model in the current Creo Parametric session inside the dimensions table in the **Parameters and Dimensions** form.
- Modify the name, nominal value, tolerance type, tolerance value 1, and tolerance value 2, except ID and type for each dimension.
- Save the updated list of dimensions back in the model.
- Save the list of dimensions retrieved from the model in an XML file or read the dimensions from a previously saved XML file in the form.

The **Parameters and Dimensions** form containing the parameters retrieved from a PART model is shown in the following figure.

Parameters And Dimensions

Parameter
Dimensions

Model

Retrieve
Retrieve Parameter from current Model
Save
Save Parameters to current Model

XML File

Path to XML File
Browse

Read
Read Parameter from XML File
Save
Save Parameters to XML File

Auto Delete
False

|   | NAME          | TYPE    | UNIT     | VALUE         | DESIG STATUS | DESCRIPTION |
|---|---------------|---------|----------|---------------|--------------|-------------|
| ▶ | BEARB         | String  | No Units | Angela Simson | False        |             |
|   | ZEICHNUNGSNR  | String  | No Units | 300-44-201    | False        |             |
|   | PROI_REVISION | String  | No Units |               | False        |             |
|   | PROI_VERSION  | String  | No Units |               | False        |             |
|   | PROI_BRANCH   | String  | No Units |               | False        |             |
|   | PROI_RELEASE  | String  | No Units |               | False        |             |
|   | PROI_MODIFIED | Integer | No Units | 0             | False        |             |
|   | WERKSTOFF     | String  | No Units | -             | True         |             |
|   | BENENNUNG_3   | String  | No Units | -             | True         |             |
|   | BENENNUNG_2   | String  | No Units | niedrige Fom  | True         |             |

Validate
Delete Row
Clear

Status

Parameter information obtained from Model 300442011\_1

Add
Add Import / Export functionality to Creo Parametric Menu

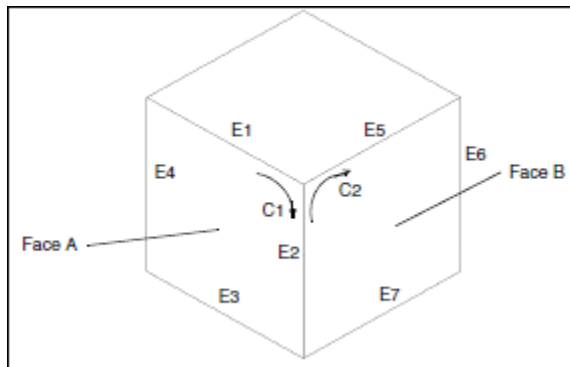
Connect
Disconnect

## Geometry Traversal

### Geometry Traversal

This appendix section illustrates the relationships between faces, contours, and edges. Examples E-1 through E-5 show some sample parts and list the information about their surfaces, faces, contours, and edges.

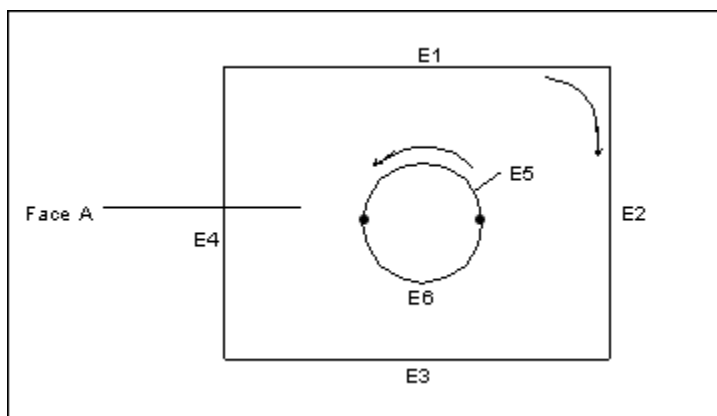
## Example 1



This part has 6 faces.

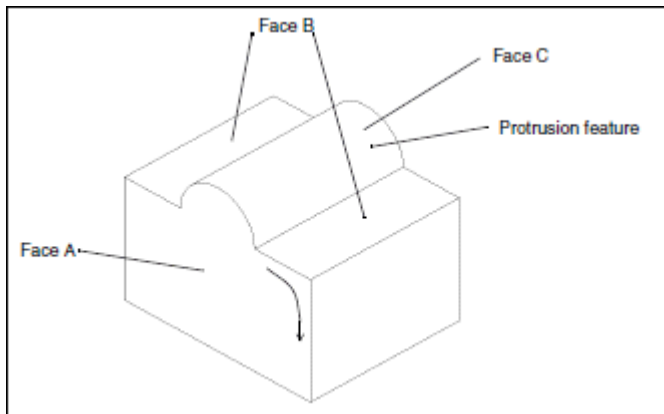
- Face A has 1 contour and 4 edges.
- Edge E2 is the intersection of faces A and B.
- Edge E2 is a component of contours C1 and C2.

## Example 2



Face A has 2 contours and 6 edges.

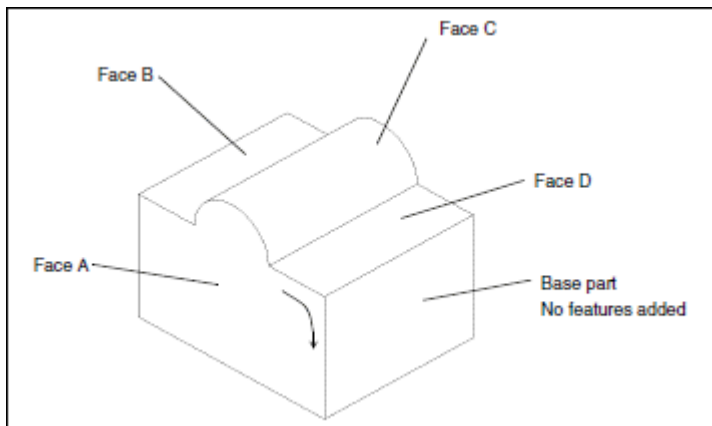
### Example 3



This part was extruded from a rectangular cross section. The feature on the top was added later as an extruded protrusion in the shape of a semicircle.

- Face A has 1 contour and 6 edges.
- Face B has 2 contours and 8 edges.
- Face C has 1 contour and 4 edges.

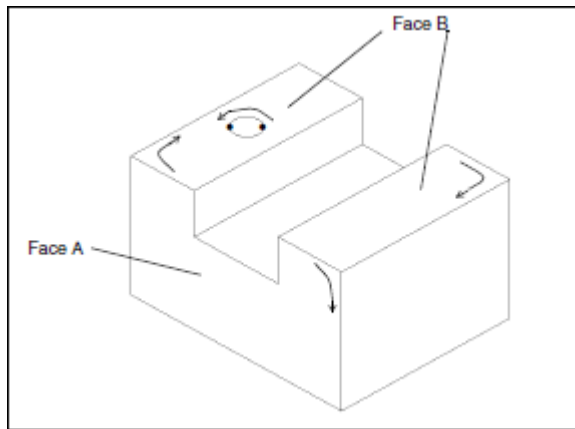
### Example 4



This part was extruded from a cross section identical to Face A. In the Sketcher, the top boundary was sketched with two lines and an arc. The sketch was then extruded to form the base part, as shown.

- Face A has 1 contour and 6 edges.
- Face B has 1 contour and 4 edges.
- Face C has 1 contour and 4 edges.
- Face D has 1 contour and 4 edges.

## Example 5



This part was extruded from a rectangular cross section. The slot and hole features were added later.

- Face A has 1 contour and 8 edges.
- Face B has 3 contours and 10 edges.

# Geometry Representations

## Geometry Representations

This appendix section describes the geometry representations of the data used by the VB API.

### Surface Parameterization

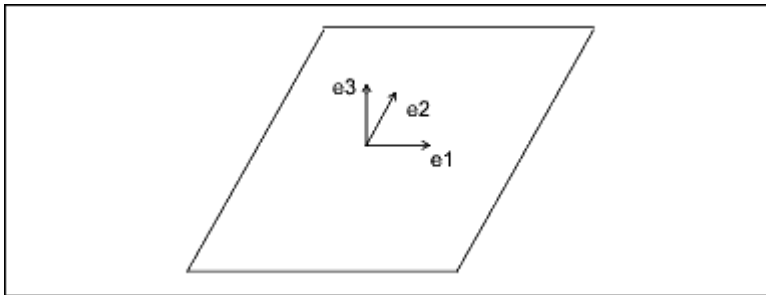
A surface in Creo Parametric contains data that describes the boundary of the surface, and a pointer to the primitive surface on which it lies. The primitive surface is a three-dimensional geometric surface parameterized by two variables ( $u$  and  $v$ ). The surface boundary consists of closed loops (contours) of edges. Each edge is attached to two surfaces, and each edge contains the  $u$  and  $v$  values of the portion of the boundary that it forms for both surfaces. Surface boundaries are traversed clockwise around the outside of a surface, so an edge has a direction in each surface with respect to the direction of traversal.

This section describes the surface parameterization. The surfaces are listed in order of complexity. For ease of use, the alphabetical listing of the data structures is as follows:

- [Cone on page 329](#)
- [Coons Patch on page 332](#)
- [Cylinder on page 328](#)

- Cylindrical Spline Surface on page 335
- Fillet Surface on page 333
- General Surface of Revolution on page 330
- NURBS on page 338
- Plane on page 328
- Ruled Surface on page 331
- Spline Surface on page 333
- Tabulated Cylinder on page 332
- Torus on page 330

## Plane



The plane entity consists of two perpendicular unit vectors ( $e1$  and  $e2$ ), the normal to the plane ( $e3$ ), and the origin of the plane.

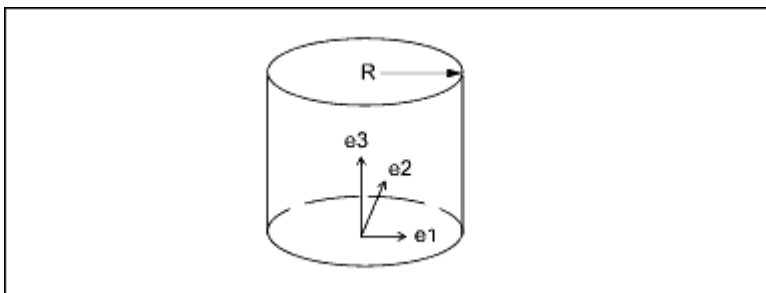
### Data Format:

```
e1[3] Unit vector, in the u direction
e2[3] Unit vector, in the v direction
e3[3] Normal to the plane
origin[3] Origin of the plane
```

### Parameterization:

$$(x, y, z) = u * e1 + v * e2 + \text{origin}$$

## Cylinder





The generating curve of a cylinder is a line, parallel to the axis, at a distance  $R$  from the axis. The radial distance of a point is constant, and the height of the point is  $v$ .

**Data Format:**

```
e1[3] Unit vector, in the u direction
e2[3] Unit vector, in the v direction
e3[3] Normal to the plane
origin[3] Origin of the cylinder
radius Radius of the cylinder
```

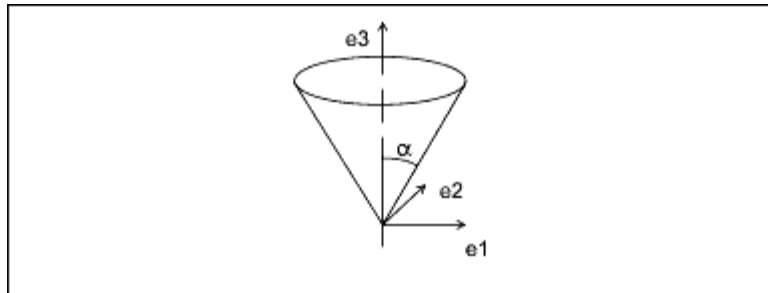
**Parameterization:**

$$(x, y, z) = \text{radius} * [\cos(u) * e1 + \sin(u) * e2] + v * e3 + \text{origin}$$

**Engineering Notes:**

For the cylinder, cone, torus, and general surface of revolution, a local coordinate system is used that consists of three orthogonal unit vectors ( $e1$ ,  $e2$ , and  $e3$ ) and an origin. The curve lies in the plane of  $e1$  and  $e3$ , and is rotated in the direction from  $e1$  to  $e2$ . The  $u$  surface parameter determines the angle of rotation, and the  $v$  parameter determines the position of the point on the generating curve.

## Cone



The generating curve of a cone is a line at an angle  $\alpha$  to the axis of revolution that intersects the axis at the origin. The  $v$  parameter is the height of the point along the axis, and the radial distance of the point is  $v * \tan(\alpha)$ .

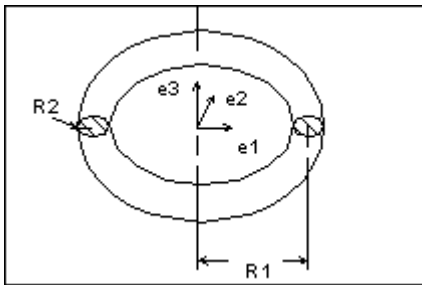
**Data Format:**

```
e1[3] Unit vector, in the u direction
e2[3] Unit vector, in the v direction
e3[3] Normal to the plane
origin[3] Origin of the cone
alpha Angle between the axis of the cone
 and the generating line
```

**Parameterization:**

$$(x, y, z) = v * \tan(\alpha) * [\cos(u) * e1 + \sin(u) * e2] + v * e3 + \text{origin}$$

## Torus



The generating curve of a torus is an arc of radius  $R2$  with its center at a distance  $R1$  from the origin. The starting point of the generating arc is located at a distance  $R1 + R2$  from the origin, in the direction of the first vector of the local coordinate system. The radial distance of a point on the torus is  $R1 + R2 * \cos(v)$ , and the height of the point along the axis of revolution is  $R2 * \sin(v)$ .

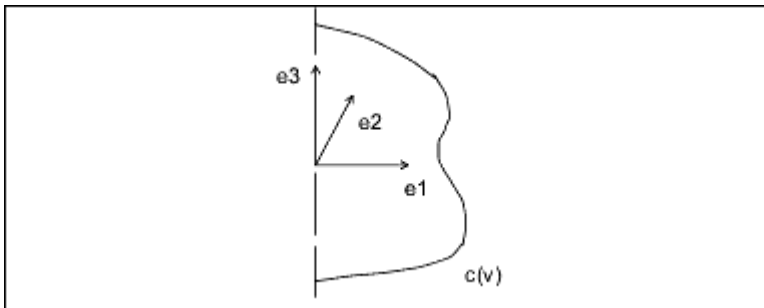
### Data Format:

```
e1[3] Unit vector, in the u direction
e2[3] Unit vector, in the v direction
e3[3] Normal to the plane
origin[3] Origin of the torus
radius1 Distance from the center of the
 generating arc to the axis of
 revolution
radius2 Radius of the generating arc
```

### Parameterization:

$$(x, y, z) = (R1 + R2 * \cos(v)) * [\cos(u) * e1 + \sin(u) * e2] + R2 * \sin(v) * e3 + \text{origin}$$

## General Surface of Revolution



A general surface of revolution is created by rotating a curve entity, usually a spline, around an axis. The curve is evaluated at the normalized parameter  $v$ , and the resulting point is rotated around the axis through an angle  $u$ . The surface of revolution data structure consists of a local coordinate system and a curve structure.

### Data Format:

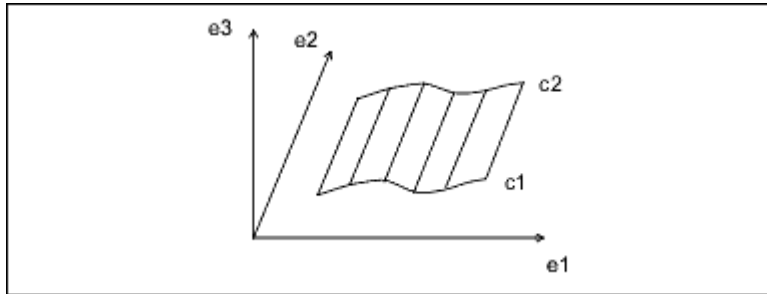
e1[3]            Unit vector, in the u direction  
 e2[3]            Unit vector, in the v direction  
 e3[3]            Normal to the plane  
 origin[3]        Origin of the surface of revolution  
 curve            Generating curve

**Parameterization:**

curve(v) = (c1, c2, c3) is a point on the curve.

$$\begin{aligned}
 (x, y, z) = & [c1 * \cos(u) - c2 * \sin(u)] * e1 + \\
 & [c1 * \sin(u) + c2 * \cos(u)] * e2 + \\
 & c3 * e3 + \text{origin}
 \end{aligned}$$

**Ruled Surface**



A ruled surface is the surface generated by interpolating linearly between corresponding points of two curve entities. The u coordinate is the normalized parameter at which both curves are evaluated, and the v coordinate is the linear parameter between the two points. The curves are not defined in the local coordinate system of the part, so the resulting point must be transformed by the local coordinate system of the surface.

**Data Format:**

e1[3]            Unit vector, in the u direction  
 e2[3]            Unit vector, in the v direction  
 e3[3]            Normal to the plane  
 origin[3]        Origin of the ruled surface  
 curve\_1        First generating curve  
 curve\_2        Second generating curve

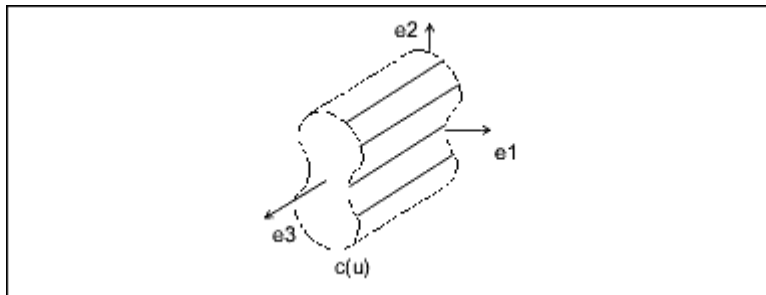
**Parameterization:**

(x', y', z') is the point in local coordinates.

$$(x', y', z') = (1 - v) * C1(u) + v * C2(u)$$

$$(x, y, z) = x' * e1 + y' * e2 + z' * e3 + \text{origin}$$

## Tabulated Cylinder



A tabulated cylinder is calculated by projecting a curve linearly through space. The curve is evaluated at the  $u$  parameter, and the  $z$  coordinate is offset by the  $v$  parameter. The resulting point is expressed in local coordinates and must be transformed by the local coordinate system to be expressed in part coordinates.

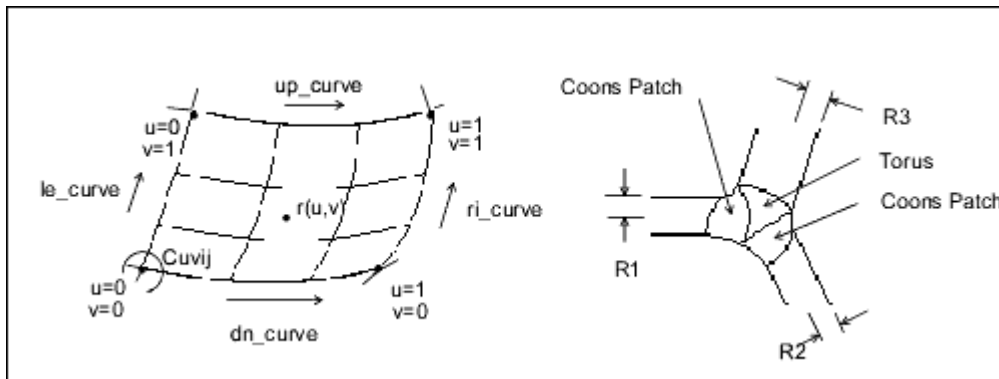
### Data Format:

e1[3] Unit vector, in the  $u$  direction  
 e2[3] Unit vector, in the  $v$  direction  
 e3[3] Normal to the plane  
 origin[3] Origin of the tabulated cylinder  
 curve Generating curve

### Parameterization:

$(x', y', z')$  is the point in local coordinates.  
 $(x', y', z') = C(u) + (0, 0, v)$   
 $(x, y, z) = x' * e1 + y' * e2 + z' * e3 + \text{origin}$

## Coons Patch



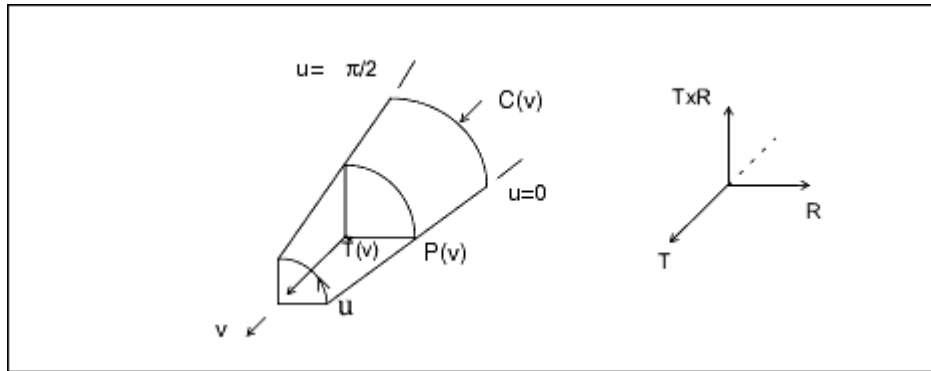
A Coons patch is used to blend surfaces together. For example, you would use a Coons patch at a corner where three fillets (each of a different radius) meet.

### Data Format:

le\_curve  $u = 0$  boundary  
 ri\_curve  $u = 1$  boundary  
 dn\_curve  $v = 0$  boundary  
 up\_curve  $v = 1$  boundary

point\_matrix[2][2]      Corner points  
uvder\_matrix[2][2]      Corner mixed derivatives

## Fillet Surface



A fillet surface is found where a round or a fillet is placed on a curved edge, or on an edge with non-constant arc radii. On a straight edge, a cylinder would be used to represent the fillet.

### Data Format:

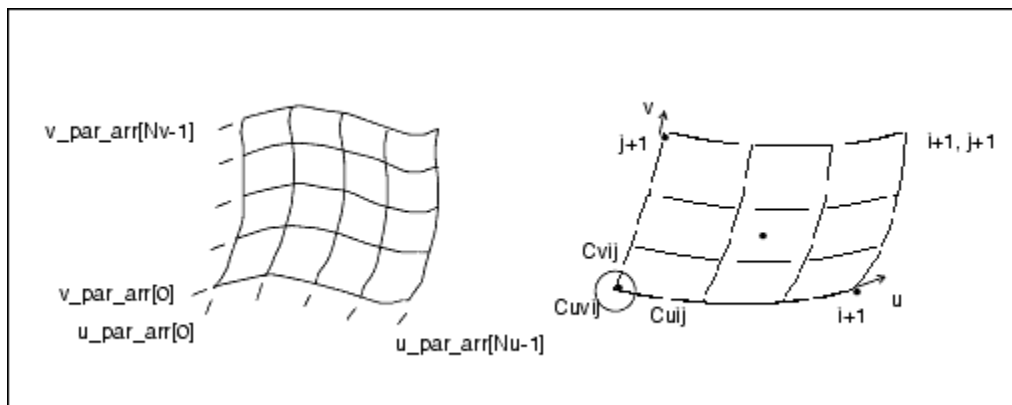
pnt\_spline       $P(v)$  spline running along the  $u = 0$  boundary  
ctr\_spline       $C(v)$  spline along the centers of the fillet arcs  
tan\_spline       $T(v)$  spline of unit tangents to the axis of the fillet arcs

### Parameterization:

$$R(v) = P(v) - C(v)$$

$$(x, y, z) = C(v) + R(v) * \cos(u) + T(v) \times R(v) * \sin(u)$$

## Spline Surface



The parametric spline surface is a nonuniform bicubic spline surface that passes through a grid with tangent vectors given at each point. The grid is curvilinear in  $uv$  space. Use this for bicubic blending between corner points.

#### Data Format:

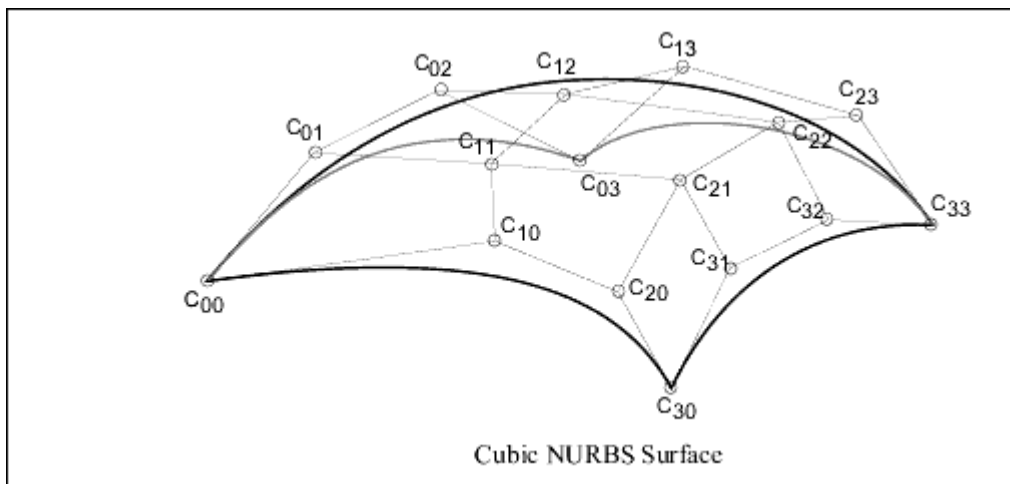
|                             |                                                                   |
|-----------------------------|-------------------------------------------------------------------|
| <code>u_par_arr[]</code>    | Point parameters, in the u direction, of size Nu                  |
| <code>v_par_arr[]</code>    | Point parameters, in the v direction, of size Nv                  |
| <code>point_arr[][3]</code> | Array of interpolant points, of size Nu x Nv                      |
| <code>u_tan_arr[][3]</code> | Array of u tangent vectors at interpolant points, of size Nu x Nv |
| <code>v_tan_arr[][3]</code> | Array of v tangent vectors at interpolant points, of size Nu x Nv |
| <code>uvder_arr[][3]</code> | Array of mixed derivatives at interpolant points, of size Nu x Nv |

#### Engineering Notes:

- Allows for a unique 3x3 polynomial around every patch.
- There is second order continuity across patch boundaries.
- The point and tangent vectors represent the ordering of an array of `[i][j]`, where u varies with i, and v varies with j. In walking through the `point_arr[][3]`, you will find that the innermost variable representing `v(j)` varies first.

#### NURBS Surface

The NURBS surface is defined by basis functions (in u and v), expandable arrays of knots, weights, and control points.



#### Data Format:

|                     |                                            |
|---------------------|--------------------------------------------|
| <code>deg[2]</code> | Degree of the basis functions (in u and v) |
|---------------------|--------------------------------------------|

---

```

u_par_arr[] Array of knots on the
 parameter line u
v_par_arr[] Array of knots on the
 parameter line v
wghts[] Array of weights for
 rational NURBS, otherwise
 NULL
c_point_arr[][3] Array of control points

```

**Definition:**

$$R(u, v) = \frac{\sum_{i=0}^{N1} \sum_{j=0}^{N2} C_{i,j} \times B_{i,k}(u) \times B_{j,l}(v)}{\sum_{i=0}^{N1} \sum_{j=0}^{N2} w_{i,j} \times B_{i,k}(u) \times B_{j,l}(v)}$$

```

k = degree in u
l = degree in v
N1 = (number of knots in u) - (degree in u) - 2
N2 = (number of knots in v) - (degree in v) - 2
Bi,k = basis function in u
Bj,l = basis function in v
wij = weights
Ci,j = control points (x,y,z) * wi,j

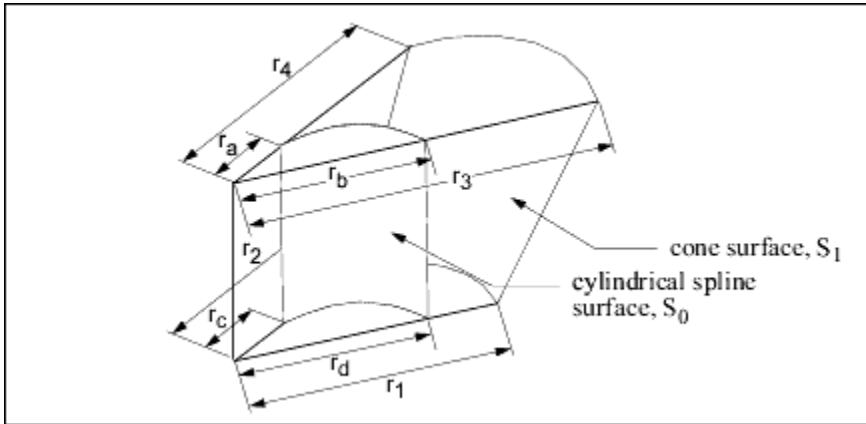
```

**Engineering Notes:**

The weights and c\_points\_arr arrays represent matrices of size wghts[N1+1] [N2+1] and c\_points\_arr [N1+1] [N2+1]. Elements of the matrices are packed into arrays in row-major order.

### Cylindrical Spline Surface

The cylindrical spline surface is a nonuniform bicubic spline surface that passes through a grid with tangent vectors given at each point. The grid is curvilinear in modeling space.



#### Data Format:

```
e1[3] x' vector of the local coordinate
 system
e2[3] y' vector of the local coordinate
 system
e3[3] z' vector of the local coordinate
 system, which corresponds to the
 axis of revolution of the surface
origin[3] Origin of the local coordinate
 system
splsrfs Spline surface data structure
```

#### The spline surface data structure contains the following fields:

```
u_par_arr[] Point parameters, in the
 u direction, of size Nu
v_par_arr[] Point parameters, in the
 v direction, of size Nv
point_arr[][3] Array of points, in
 cylindrical coordinates,
 of size Nu x Nv. The array
 components are as follows:
 point_arr[i][0] - Radius
 point_arr[i][1] - Theta
 point_arr[i][2] - Z
u_tan_arr[][3] Array of u tangent vectors.
 in cylindrical coordinates,
 of size Nu x Nv
v_tan_arr[][3] Array of v tangent vectors,
 in cylindrical coordinates,
 of size Nu x Nv
uvder_arr[][3] Array of mixed derivatives,
 in cylindrical coordinates,
 of size Nu x Nv
```

#### Engineering Notes:

If the surface is represented in cylindrical coordinates ( $r$ ,  $\theta$ ,  $z$ ), the local coordinate system values ( $x'$ ,  $y'$ ,  $z'$ ) are interpreted as follows:



---

```

x' = r cos (theta)
y' = r sin (theta)
z' = z

```

A cylindrical spline surface can be obtained, for example, by creating a smooth rotational blend (shown in the figure on the previous page).

In some cases, you can replace a cylindrical spline surface with a surface such as a plane, cylinder, or cone. For example, in the figure, the cylindrical spline surface  $S_1$  was replaced with a cone ( $r_1 = r_2$ ,  $r_3 = r_4$ , and  $r_1 \neq r_3$ ).

If a replacement cannot be done (such as for the surface  $S_0$  in the figure ( $r_a \neq r_b$  or  $r_c \neq r_d$ )), leave it as a cylindrical spline surface representation.

## Edge and Curve Parameterization

This parameterization represents edges (line, arc, and spline) as well as the curves (line, arc, spline, and NURBS) within the surfaces.

This section describes edges and curves, arranged in order of complexity. For ease of use, the alphabetical listing is as follows:

- [Arc on page 337](#)
- [Line on page 337](#)
- [NURBS on page 338](#)
- [Spline on page 338](#)

### Line

#### Data Format:

```

end1[3] Starting point of the line
end2[3] Ending point of the line

```

#### Parameterization:

```

(x, y, z) = (1 - t) * end1 + t * end2

```

### Arc

The arc entity is defined by a plane in which the arc lies. The arc is centered at the origin, and is parameterized by the angle of rotation from the first plane unit vector in the direction of the second plane vector. The start and end angle parameters of the arc and the radius are also given. The direction of the arc is counterclockwise if the start angle is less than the end angle, otherwise it is clockwise.

#### Data Format:

```

vector1[3] First vector that defines the
 plane of the arc
vector2[3] Second vector that defines the
 plane of the arc

```

---

```

origin[3] Origin that defines the plane
 of the arc
start_angle Angular parameter of the starting
 point
end_angle Angular parameter of the ending
 point
radius Radius of the arc.

```

#### Parameterization:

```

t' (the unnormalized parameter) is
 (1 - t) * start_angle + t * end_angle
(x, y, z) = radius * [cos(t') * vector1 +
 sin(t') * vector2] + origin

```

### Spline

The spline curve entity is a nonuniform cubic spline, defined by a series of three-dimensional points, tangent vectors at each point, and an array of unnormalized spline parameters at each point.

#### Data Format:

```

par_arr[] Array of spline parameters
 (t) at each point.
pnt_arr[][3] Array of spline interpolant points
tan_arr[][3] Array of tangent vectors at
 each point

```

#### Parameterization:

$x$ ,  $y$ , and  $z$  are a series of unique cubic functions, one per segment, fully determined by the starting and ending points, and tangents of each segment.

Let  $p\_max$  be the parameter of the last spline point. Then,  $t$ , the unnormalized parameter, is  $t * p\_max$ .

Locate the  $i$ th spline segment such that:

```
par_arr[i] < t' < par_arr[i+1]
```

(If  $t < 0$  or  $t > +1$ , use the first or last segment.)

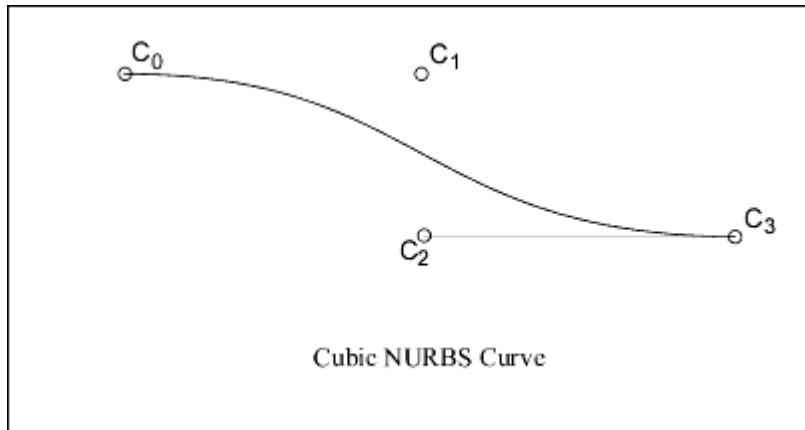
```

t0 = (t' - par_arr[i]) / (par_arr[i+1] - par_arr[i])
t1 = (par_arr[i+1] - t') / (par_arr[i+1] - par_arr[i])

```

### NURBS

The NURBS (nonuniform rational B-spline) curve is defined by expandable arrays of knots, weights, and control points.



#### Data Format:

degree                      Degree of the basis function  
 params[]                    Array of knots  
 weights[]                   Array of weights for rational  
                                  NURBS, otherwise NULL.  
 c\_pnts[][3]                Array of control points

#### Definition:

$$R(t) = \frac{\sum_{i=0}^N C_i \times B_{i,k}(t)}{\sum_{i=0}^N w_i \times B_{i,k}(t)}$$

k            = degree of basis function  
 N           = (number of knots) - (degree) - 2  
 wi          = weights  
 Ci          = control points (x, y, z) \* wi  
 Bi,k       = basis functions

By this equation, the number of control points equals N+1.

#### References:

Faux, I.D., M.J. Pratt. Computational Geometry for Design and Manufacture. Ellis Harwood Publishers, 1983.

Mortenson, M.E. Geometric Modeling. John Wiley & Sons, 1985.

---

# Index

## A

- Accuracy
  - getting and setting, 138
- ActionListener
  - description of the class, 16
  - events, 235
  - feature-level, 241
  - session-level, 237
  - solid-level, 240
  - types, 235
  - UI command, 238
- ActionSource interface, 236
- Activate
  - window, 158
- Add
  - items to a layer, 169
- Allocate
  - simplified representations, 281
- Applications
  - hierarchy, 18
- Arcs
  - description, 195
  - representation, 337
- Area
  - surface, 201
- Arrows, 195
- Assemblies
  - coordinate systems, 162
  - creating, 138
  - hierarchy, 220
  - structure of, 220
- Axes, 203
  - evaluating, 203

## B

- B-splines
  - description, 195

## C

- Cells
  - accessing, 233
- Children, 170
- Circles, 195
- Clear
  - window, 158
- Close
  - window, 158
- Colors, 38
- Commands
  - designating, 67
- compatibility of deprecated methods, 35
- Composite curves
  - description, 195
- Cones
  - class representation, 199
  - geometry representation, 329
- Configuration options, 37
- Contours
  - evaluating, 198
  - traversing, 194
- Contours, locating in a model, 198
- Coons patches
  - geometry representation, 332
- Coordinate systems, 203
  - assemblies, 162
  - datum, 163
  - drawing, 162
  - Drawing View, 162

---

- evaluating, 203
- screen, 161
- section, 163
- solid, 161
- window, 162

Coordinate transformations, 161

Copy

- models, 79

Create

- assembly, 138
- buttons, 64
- family table columns, 234
- family table instance, 231
- layer, 169
- local group, 174
- material, 150
- menus, 64
- parameters, 207
- part, 138
- simplified representations, 281
- UDFs, 179
- window, 157

Create Interactively Defined UDFs, 180

Creating UDFs, 179

Creo Parametric

- accessing, 39

Curves, 195

- data structures, 337
- determining the type, 195
- t parameter, 195
- types, 195
  - reserved, 195

Cylinders, 199

- geometry representation, 328
- spline surfaces, 335
- tabulated, 199
  - geometry representation, 332

## D

Delete

- feature pattern, 174
- models, 79
- simplified representations, 281

deprecated methods compatibility, 35

Depth

- selection, 50

Descriptors

- model, 73

Designating

- command, 68
- commands, 67
- icon, 67

Designating commands, 67

Detail.DetailEntityInstructions

- interface
  - description, 116

Dimension2D.Dimension2D interface

- description, 101

Dimensions, 213

- information, 213
- tolerances, 214

Display

- model in window, 157
- models, 79
- selection, 51

Display status

- of layers, 169

Drawing

- models
  - code example, 98
- sheets
  - example, 96
- transformations, 166
- views
  - code example, 98

## E

Edges, 195

- determining the type, 195
- evaluating, 196
- t parameter, 195

- traversing, 194
- types, 195
  - reserved, 195
- Ellipses, 195
- Erase
  - models, 79
- Evaluation
  - axes, 203
  - contour, 198
  - coordinate system, 203
  - edge, 196
  - point, 203
  - surface, 200
- Examples
  - creating drawing views, 98
  - listing views, 100
  - normalizing a coordinate transformation matrix, 166
  - retrieving a model, 75
  - setting angular tolerances, 215
  - using drawing sheets, 96
  - visiting the items in a simplified representation, 282
- Exceptions
  - ActionListener, 18
- Export
  - files, 243
- F**
- Faces
  - traversing, 194
- Family tables, 231
  - cells, 233
  - columns
    - accessing, 232
  - instances
    - accessing, 231
  - symbols, 232
- Features
  - accessing, 170
  - creating, 172

- failed, 170
- groups, 170, 174
- identifiers, 170
- information, 171
- operations, 172
- parents, 170
- patterns, 174
- read-only, 171
- resuming, 172
- suppressing, 172
- user-defined, 178
- Files
  - exporting, 243
  - message, 39
    - contents, 40
  - naming restrictions, 39
  - plotting, 270
- Fillet surfaces
  - geometry representation, 333

## G

- General surface of revolution, 330
- Generic model
  - getting, 231
- Geometry
  - solid edge, 197
  - terms, 194
  - traversal, 194
- Groups, 174, 178

## H

- Hierarchy
  - application, 18
- Highlight
  - selections, 51

## I

- Information
  - Drawing, 91
- Initialize

---

- ActionListeners, 16
- Install
  - See J-Link Installing, 319
- Installation
  - See J-Link Installing, 319
- Interactive selection, 49
- Interactively Defined UDFs
  - create, 180

## K

- Keywords
  - instanceof
  - using, 195

## L

- Layers, 169
  - operations, 169
- Lines
  - description, 195
  - representation, 337
  - styles, 38
- Lists
  - of children, 170
  - of current windows, 157
  - of layer items, 169
  - of materials, 150
  - of ModelItems, 167
  - of pattern members, 170, 174
  - of rows in a family table, 231
  - of subitems, 167
  - of views, 160
  - of windows, 157
- Local groups
  - creating, 174
- Locks, 231

## M

- Macros, 37
- Mass properties, 147
- Materials, 150

- Matrix
  - code example, 166
- Matrix3D object, 166
- Message files, 39
  - contents, 40
  - restrictions, 39
- Message window
  - reading from, 41
  - writing to, 41
- ModelItems
  - duplicating, 168
  - evaluating, 203
  - getting, 167
  - information, 168
  - types, 166
- Models
  - descriptors, 73
  - Drawing
    - Obtaining, 91
  - exporting, 243
  - getting, 72
  - operations, 79
  - retrieving, 74
    - code example, 75
- Modify
  - simplified representations, 282

## N

- Normalize
  - matrix, 166
- NURBS
  - representation, 338
  - surface, 334

## O

- Open
  - file, 74
- Operations
  - Drawing, 92
  - feature, 172
  - layer, 169

---

- model, 79
- solid, 139, 278
- view, 160
- window, 158, 279

- Outlines
  - contour, 198

## P

- Parameters, 207
  - information, 210
- ParamValue objects, 206
- Parents, 170
- Parts, 150
  - creating, 138
- Pattern leaders, 170, 174
- Patterns, 174
- pfcExceptions.
  - XToolkitDrawingCreateErrors
    - description, 89
- pfcModel.Models
  - information, 75
- pfcSelect.Selection.GetSelItem method
  - getting a ModelItem, 167
- pfcSelect.SelectionOptions argument
  - table of strings, 49
- Planes, 199
  - geometry representation, 328
- Plot, 270
- Points, 203
  - evaluating, 203
- Polygons, 195
- Popup Menu
  - Adding to the Graphics Window, 70
  - Using Trail files to determine names, 70
- Popup menus
  - Adding, 71
- Popup Menus, 69
  - Accessing, 70
- Principal curve, 201

## R

- Refresh
  - window, 158, 279
- Regenerate
  - events, 237
  - solids, 139, 278
- Remove
  - items from a layer, 169
- Rename
  - models, 79
- Repaint
  - events, 237
  - window, 158, 279
- Reset
  - view, 160
- Restrictions
  - on text message files, 39
- Retrieve
  - geometry of a simplified representation, 280
  - graphics of a simplified representation, 280
  - material, 150
  - model
    - code example, 75
  - simplified representations, 280
  - view, 160
- Revolved surfaces, 199
- Rotate
  - view, 161
- Ruled surfaces, 199
  - geometry representation, 331

## S

- Save
  - models, 79
  - view, 160
- Screen coordinate system, 161
- Selection, 49
  - accessing data, 50
  - controlling display, 51



- 
- Sheets
    - Drawing, 93
  - Simplified representations
    - adding items, 283
    - creating, 281
    - deleting, 281
      - items, 283
    - extracting information from, 282
    - modifying, 282
    - retrieving
      - geometry, 280
      - graphics, 280
      - utilities, 285
  - Solids
    - accuracy, 138
    - coordinate system, 161
    - geometry traversal, 166
    - getting a solid object, 138
    - information, 138
    - mass properties, 147
    - operations, 139, 278
  - Splines
    - cylindrical spline surface, 335
    - description, 195
    - representation, 338
    - surface, 333
  - Status
    - feature, 171
    - layer, 169
  - Surfaces, 199
    - cylindrical spline, 335
    - data structures, 327
    - evaluating, 200
      - area, 201
    - evaluating parameters, 201
    - fillet
      - geometry representation, 333
    - general surface of revolution, 330
  - NURBS
    - geometry representation, 334
  - revolved, 199
  - ruled, 199, 331
  - spline, 333
  - traversing, 194
  - types, 199
  - UV parameterization, 199
- ## T
- t parameter
    - description, 195
  - Table
    - creating, 107-108
    - drawing cells, 107-108
    - retrieving, 109
    - selecting, 108
  - Table.Table interface
    - description, 107
  - Tabulated cylinders, 199
    - geometry representation, 332
  - Text, 195
    - message files, 39
  - Tolerance, 214
  - Tolerances
    - setting
      - example, 215
  - Torii, 199
  - Torus, 330
  - Transformations, 161, 163
    - solid to coordinate system datum
      - coordinates, 165
    - solid to screen coordinates, 165
      - in a drawing, 166
  - Traversal
    - of a solid block, 194
    - of geometry, 194
- ## U
- UDFs, 178
    - creating, 179
  - Utilities
    - simplified representations, 285

---

## V

### Values

- ParamValue, 207

### View2D.View2D interface

- description, 96

### Views

- Drawing, 96

- getting a view object, 160

- list of, 160

- listing

  - example, 100

- operations, 160

- retrieving, 160

- saving, 160

### Visibility, 171

### Visit

- simplified representations, 282

## W

### Window coordinate system, 162

### Windows, 156

- activating, 158

- clearing, 158

- closing, 158

- creating, 157

- flush, 158

- operations, 158, 279

- repainting, 158

### Write

- to the message window, 41